

Plutus Pioneer Program

3rd iteration - textbook



IOG

Plutus Pioneer Program 3rd iteration – textbook

Copyright © April 2022, IOG (Input Output Global)

This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc/4.0/> or send a letter to Creative Commons, 559 Nathan Abbott Way, Stanford, California 94305, USA.

This book was prepared by Luka Kurnjek, a Plutus community member. Majority of the text is taken from the Plutus pioneer program 3rd iteration that was presented by Lars Brünjes. All program code in this book is the copyright of IOG. The code and videos of the 3rd iteration for the Plutus pioneer program are freely available at IOG GitHub page:

<https://github.com/input-output-hk/plutus-pioneer-program/tree/third-iteration>

All pictures in this book are the copyright of IOG if not otherwise noted.

Table of Contents

1 Plutus introduction	8
1.1 Installation of Haskell and Plutus	8
1.2 Running the Plutus playground	10
1.3 The EUTxO model	13
1.4 Plutus code	13
2 Simple validation scripts	19
2.1 Low-level untyped validation scripts.....	19
2.2 High Level typed validation scripts.....	25
2.3 Homework	28
3 Time, parameterized contracts and the cardano testnet.....	30
3.1 Time.....	31
3.2 Parameterized Contracts.....	37
3.3 The Cardano testnet.....	41
3.4 Homework	46
4 Monads, Traces & Contracts	53
4.1 Monads.....	53
4.2 The emulator trace monad.....	60
4.3 The contract monad	63
4.4 Homework	67
5 Native tokens	70
5.1 Simple Minting Policy	72
5.2 More Realistic Minting Policy.....	74
5.3 NFT-s.....	76
5.4 Homework	81
6 Deployment.....	86
6.1 The minting policy	86
6.2 Minting with the CLI	87
6.3 Deployment Scenarios	89
6.4 The Contracts	90

6.5 Minting with the PAB	96
7 State Machines	105
7.1 Commit schemes	105
7.2 Implementation without State Machines	106
7.3 State Machines	121
7.4 Homework	136
8 Testing	143
8.1 State Machine Example: Token Sale	143
8.2 Automatic testing using emulator traces	152
8.3 Test Coverage	155
8.4 Interlude optics	156
8.5 Property based testing with QuickCheck	158
8.6 Property based testing of Plutus contracts	160
8.7 Homework	172
9 Oracles	177
9.1 Workflow	177
9.2 Code examples	178
9.3 Using the PAB	198
Appendix A. Code examples	209
A.1. Token sale	209
A.2. Uniswap contract	222
10 Resources	239

Table of Figures

Figure 1 - Plutus playground editor	11
Figure 2 - Putus playground simulation	12
Figure 3 - Putus playground transactions	12
Figure 4 - Data type.....	19
Figure 5 – TxOutRef type	23
Figure 6 – ChainIndexTxOut type.....	23
Figure 7 - Grab action.....	29
Figure 8 - ScriptContext type	30
Figure 9 – TxInfo type	30
Figure 10 – ScriptPurpose type.....	31
Figure 11 - Cardano node.....	41
Figure 12 - runEmulatorTrace function	60
Figure 13 – EmulatorConfig type	60
Figure 14 - SlotConfig type.....	60
Figure 15 - FeeConfig type	61
Figure 16 - EmulatorTrace type	61
Figure 17 - TraceConfig type	62
Figure 18 - Value type	70
Figure 19 – AssetClass type.....	70
Figure 20 – TxInInfo type	78
Figure 21 – TxOut type.....	79
Figure 22 - Address type	79
Figure 23 - Credential type	79
Figure 24 – StakingCredential type.....	80
Figure 25 – MintingPolicy type	88
Figure 26 - dApp schema	90

Figure 27 - findOwnInput function	115
Figure 28 - findDatum function.....	115
Figure 29 - valuePaidTo function	116
Figure 30 - Game state diagram	121
Figure 31 - State type.....	122
Figure 32 - StateMachine type.....	122
Figure 33 – ThreadToken type	122
Figure 34 – StateMachineClient type.....	130
Figure 35 - State machine instance.....	131
Figure 36 – getOnchainState function	131
Figure 37 – OnChainState type	132
Figure 38 – TypedScriptTxOutput type	132
Figure 39 – TypedScriptTxOutputRef type.....	132
Figure 40 - Run step	133
Figure 41 - Token sale workflow	143
Figure 42 - checkPredicate function	153
Figure 43 – checkPredicateOptions function.....	153
Figure 44 - checkOptions type	154
Figure 45 – TracePredicate type	154
Figure 46 – walletFundsChange function	154
Figure 47 – checkPredicateCoverage function	155
Figure 48 – CoverageRef type	155
Figure 49 - Testing schema	160
Figure 50 – ModelState type.....	168
Figure 51 - contractState function.....	168
Figure 52 – askModelState function	169
Figure 53 – propRunActionsWithOptions function	170

Figure 54 - Oracle workflow.....	178
Figure 55 – TxOutTx type	185
Figure 56 – Tx type.....	185
Figure 57 – Auction workflow	209
Figure 58 – Wallets	220
Figure 59 - Playground workflow.....	221
Figure 60 - Auction transactions	222
Figure 61 - Uniswap workflow	224

1 Plutus introduction

Plutus is the native smart contract language for Cardano. It is a Turing-complete language written in Haskell, and Plutus smart contracts are effectively Haskell programs. By using Plutus, you can be confident in the correct execution of your smart contracts. It draws from modern language research to provide a safe, full-stack programming environment based on Haskell, the leading purely-functional programming language. [2]

So, in order to understand this book and its code examples one has to understand the basics of the Haskell programming language. There is a free self-paced Haskell online course provided by IOG at: <https://github.com/input-output-hk/haskell-course>. Other learning resources for Haskell can be found at: <https://www.haskell.org/documentation/>.

1.1 Installation of Haskell and Plutus

The installation instructions are written for a Unix style OS (e.g. Mac OS or Linux). In order to install the Haskell tool chain, which is composed of GHC (Glasgow Haskell compiler), Cabal and some other tools, you can use GHCup: <https://www.haskell.org/ghcup/>. You will need curl installed on your OS, before you can use the GHCup installation.

After the installation is completed check from a terminal if you can run GHCi, the GHC Repl:

```
[user@fedora ~]$ ghci
GHCi, version 8.10.7: https://www.haskell.org/ghc/  :? for help
Prelude>
```

The GHCi version you will see can of course be different and should not matter as long as it is above or the same as the version shown in this example.

To run Plutus code examples from this book, you will need the following GIT repositories:

- <https://github.com/input-output-hk/plutus-apps>
- <https://github.com/lukakurnjek/plutus-pioneer-program>

You will have to clone them with the git tool that you will also need to install on your OS. Roughly speaking the plutus-apps repository contains the “off-chain” code for Plutus that runs in a Wallet. It also references another repository called plutus, which contains the “on-chain” code for Plutus that runs on the Cardano blockchain.

To be able to build the code you will need to install the nix command line toolset. To install nix you can follow the instructions from their web-page: <https://nixos.org/download.html>.

Once nix is installed, you have to add the IOHK binary caches. You can do this by editing the `/etc/nix/nix.conf`, where you add the following lines:

```
substituters = https://hydra.iohk.io https://iohk.cachix.org https://cache.nixos.org/  
trusted-public-keys = hydra.iohk.io:f/Ea+s+dFdN+3Y/G+FDgSq+a5NEWhJGzdjvKNGv0/EQ=  
iohk.cachix.org-1:DpRUyj7h7V830dp/i6Nti+NE02/nhblbov/8MW7Rqoo= cache.nixos.org-  
1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY=
```

If you don't have an `/etc/nix/nix.conf` or don't want to edit it, you may add the `nix.conf` lines to `~/.config/nix/nix.conf` instead. You must be a trusted user to do this. If you are running NixOS you can, set the following NixOS options:

```
nix = {  
  binaryCaches = [ "https://hydra.iohk.io" "https://iohk.cachix.org" ];  
  binaryCachePublicKeys = [ "hydra.iohk.io:f/Ea+s+dFdN+3Y/G+FDgSq+a5NEWhJGzdjvKNGv0/EQ=" "iohk.cachix.org-1:DpRUyj7h7V830dp/i6Nti+NE02/nhblbov/8MW7Rqoo=" ];  
};
```

The above nix configuration instructions are kept up-to-date in the `plutus-apps` repository [4].



A lot of dependencies are cached there and it will make the Plutus builds much faster.

The example code in this book comes directly from the `plutus-pioneer-program` git repository mentioned earlier. To build the code that is contained in the week folders follow these steps:

1. Open up a terminal and `cd` into a week folder of the `plutus-pioneer-program` repo, e.g. `week01`. You will need to figure out which commit of the `plutus-apps` repository this week uses. To do this open the `cabal.project` file, which contains various dependencies and scroll to the section *source-repository-package*:

```
source-repository-package  
  type: git  
  location: https://github.com/input-output-hk/plutus-apps.git  
  tag: 41149926c108c71831cfe8d244c83b0ee4bf5c8a
```

2. Copy the commit under the tag section. `cd` into the `plutus-apps` repository and checkout the commit you just have copied:

```
[user@fedora ~/plutus-apps]$ git checkout 41149926c108c71831cfe8d244c83b0ee4bf5c8a
```

3. Now run the command `nix-shell`. When you do this for the first time it can take a while until everything has built. After the build your command prompt will change to the nix shell. In this shell `cd` back into the `week01` folder and run the `cabal build` command:

```
[nix-shell: ~/plutus-pioneer-program/code/week01]$ cabal build
```

4. This can also take some time if you build it the first time. When the build has finished you can start the GHC Repl with the *cabal repl* command:

```
[nix-shell: ~/plutus-pioneer-program/code/week01]$ cabal repl
Build profile: -w ghc-8.10.4.20210212 -01
Preprocessing library for plutus-pioneer-program-week01-0.1.0.0..
GHCi, version 8.10.4.20210212: https://www.haskell.org/ghc/  :? for help
Ok, one module loaded.
Prelude Week01.EnglishAuction>
```

5. To leave the cabal repl simply type “:q”.

Another useful thing to have is the documentation for various Plutus libraries. You can build the documentation yourself. From inside the nix-shell in the plutus-apps folder run:

```
[nix-shell:~/plutus-apps]$ build-and-serve-docs
Serving HTTP on 0.0.0.0 port 8002 (http://0.0.0.0:8002/) ...
```

You can open the displayed address and port in your web-browser and will see the high-level documentation for Plutus. A more useful thing is the Plutus library documentation which you will find at this location: <http://0.0.0.0:8002/haddock>. Because is not just a static file but and actual web-server you can search through it by pressing Ctrl+S.

1.2 Running the Plutus playground

The Plutus playground is an interactive environment where you can compile your Plutus code and simulate it. In the simulation you can specify:

- how many wallets with how many ADA (native currency of Cardano) you want to have
- which wallet actions also called a transaction; you want to perform at which timeslots
- what are the input parameters for your transactions if there are any input fields present

These transactions will then be executed on the playground and you can view them interactively. To set up the playground perform the following steps:

1. cd into the plutus-apps repository and start the nix-shell.
2. Then cd into *plutus-playground-client/* folder and start the *plutus-playground-server*:

```
[nix-shell:~/plutus-apps/plutus-playground-client]$ plutus-playground-server
```

NOTE: The default server timeout is set to 80 seconds. If you want to increase the server timeout to e.g. 120 seconds you can pass a “-i 120s” argument to the command.

3. Open up another nix shell at the same location and start the playground client:

```
[nix-shell:~/plutus-apps/plutus-playground-client]$ npm start
...
Project is running at https://localhost:8009/
```

If you then go to your web-browser and open <https://localhost:8009/> you will see the Plutus playground (Figure 1). In the middle you will see the editor window, where you can copy paste the code from the plutus-pioneer-program repository. You can delete the default example code. On the right side you see the compile and simulate buttons. With the compile button you compile the code and a status bar at the bottom of the editor will tell you if the compilation succeeded. If not, it will throw an error pointing to the line that makes trouble.

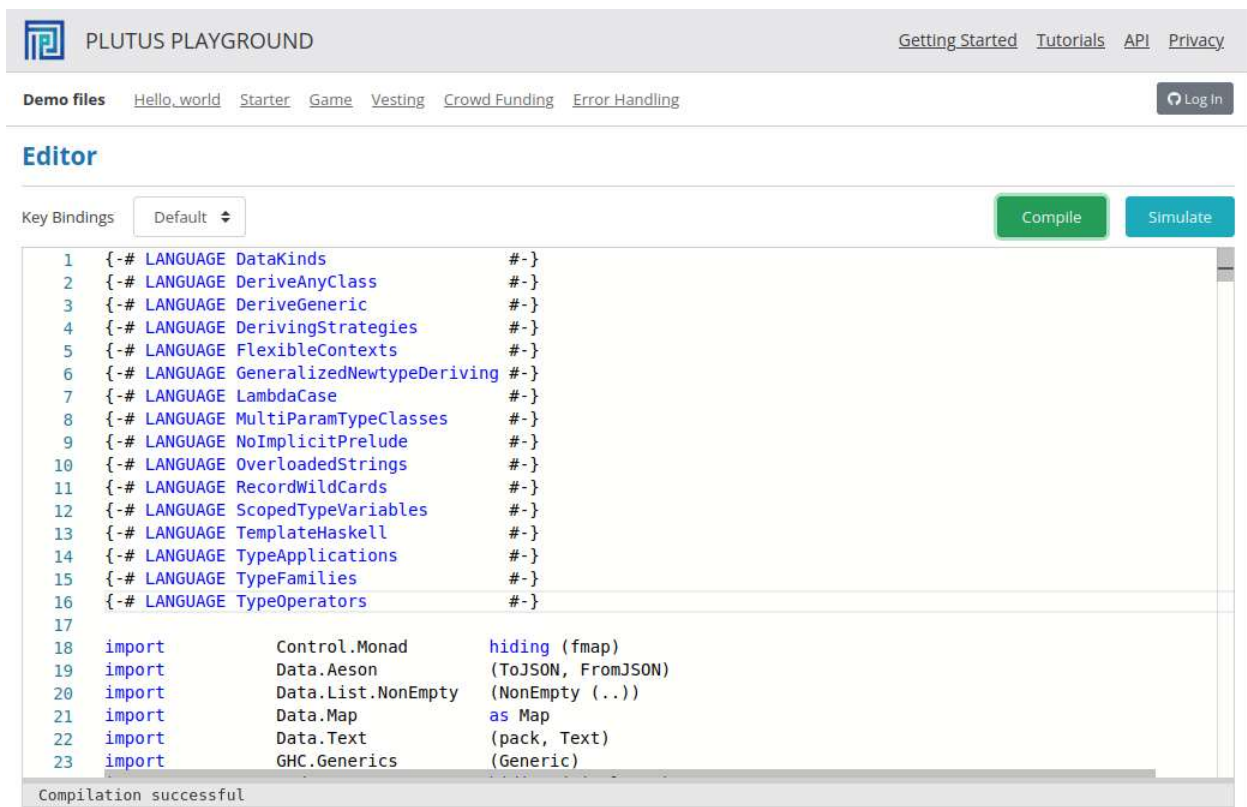


Figure 1 - Plutus playground editor

After the compilation successfully succeeded you can use the simulate button to open up the Simulate view (Figure 2). There you can add wallets, set amounts of Lovelace (= 0.000001 ADA) for initial funds of the wallets, trigger the available wallet functions and add wait actions. Once you are finished with defining your actions you can click on the Evaluate button. The transaction window will appear (Figure 3). There you will always see a genesis transaction. If there were actions defined, there will also be other transactions. You can then click on different slots in the upper row and the transactions for these slots will appear.

Simulator

[< Return to Editor](#)

Simulation 1



Wallets

Add some initial wallets, then click one of your function calls inside the wallet to begin a chain of actions.

Evaluate

Transactions

Wallet 1



Opening Balances

Lovelace 100000000

T 100000000

Available functions

bid + close + start +

Pay to Wallet +

Wallet 2



Opening Balances

Lovelace 100000000

T 100000000

Available functions

bid + close + start +

Pay to Wallet +

Add Wallet

Actions

This is your action sequence. Click 'Evaluate' to run these actions against a simulated blockchain.



Add Wait Action

Evaluate

Transactions

Figure 2 - Putus playground simulation

Transactions

Blockchain

Click a transaction for details

Slot 0, Tx 0

Inputs

Transaction

Slot 0, Tx 0

Tx: d8e1263889cf12a70c756adc03c7fd14437eec240f2b8e8f...

Validity: All time

Signatures:None

Forge

Ada Lovelace 200000000

66 T 200000000

Outputs

Wallet 2

PubKeyHash 80a4f45b...

Ada Lovelace 100000000

66 T 100000000

Unspent

Wallet 1

PubKeyHash a2c20c77...

Ada Lovelace 100000000

66 T 100000000

Unspent

Figure 3 - Putus playground transactions

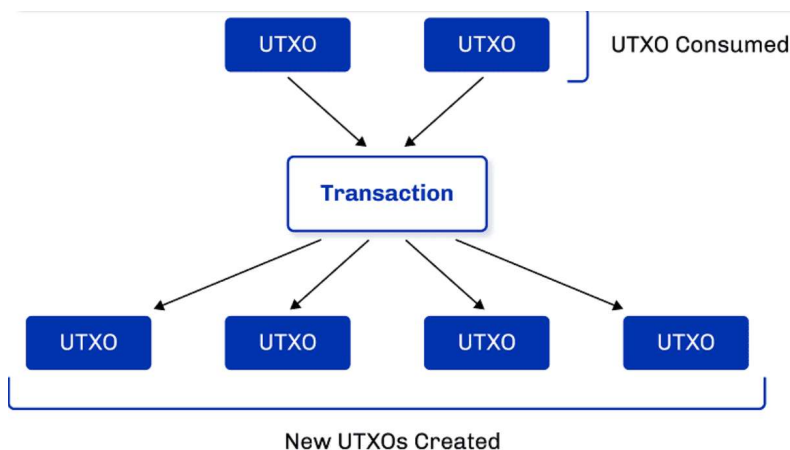


You can also use the online Plutus playground (<https://playground.plutus.iohkdev.io>) to compile your code. But it is not necessarily up-to-date with the code from the git repositories, so some Plutus code from the examples in this book may not compile.

The Auction example code in the folder week01 will not be covered here, since it is a to advanced example to start with. It is covered in the Appendix chapter A.1 at the end of this book. But you can look at the demonstration of this code in the videos “Auction Contract in the EUTxO-Model” and “Auction Contract on the Playground” that can both be found in the plutus-pioneer-program git repository under the Lecture #1 chapter.

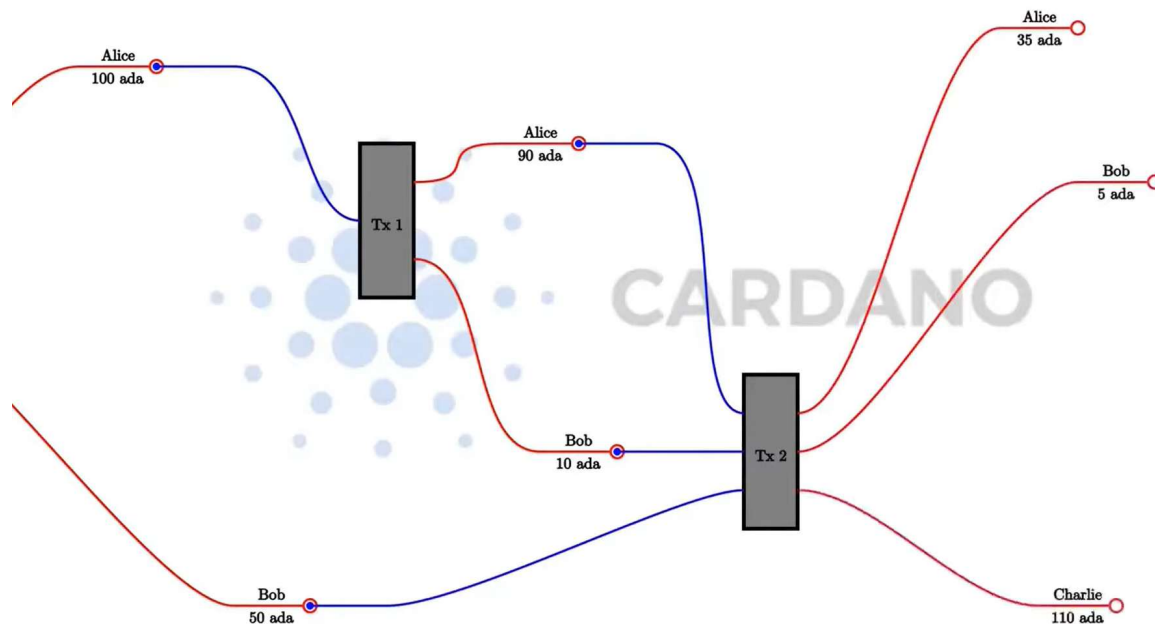
1.3 The EUTxO model

The cardano blockchain uses the extended UTXO model (EUTXO) that is a variant of the Unspent Transaction Output (UTXO) model used by Bitcoin. Transactions consume unspent outputs (UTXOs) from previous transactions and produce new outputs, which can be used as inputs to later transactions. Unspent outputs are the liquid funds on the blockchain. Users do not have individual accounts, but rather have a software wallet on a smartphone or PC which manages UTXOs on the blockchain. It can initiate transactions involving UTXOs owned by the user. Every core node on the blockchain maintains a record of all the currently unspent outputs, the UTXO set. When outputs are spent, they are removed from the UTXO set.



There are models other than UTXO. Ethereum, for example, uses an account-based model, which is what a normal bank uses. There everybody has an account, and each account has a balance. If you transfer money from one account to another, then the balances get updated accordingly. But in the UTXO model, the input is always the entire balance of an UTXO, and the outputs are newly created UTXOs from which one of them could belong to the user that provided his UTXO as input and would represent his change amount. Let us say Alice has 100

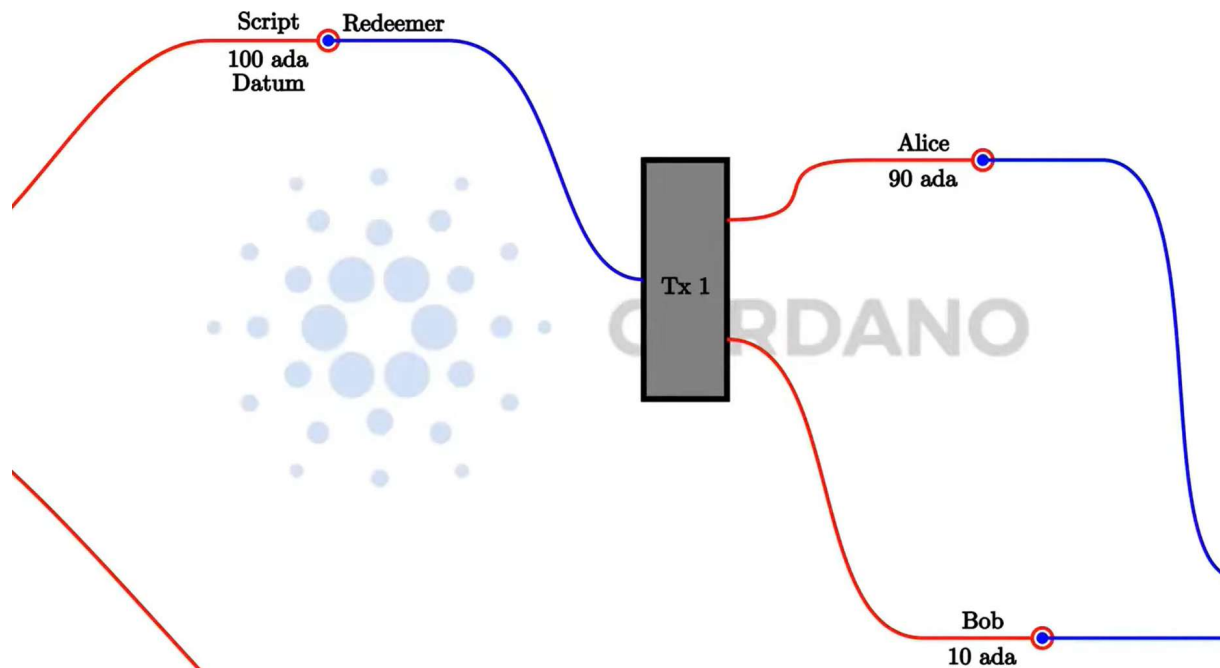
ada and wants to send Bob 10 ada. After that she wants to send to Charlie 55 ada and in the same transaction also Bob wants to send Charlie 55 ada.



In the picture above you can see that the inputs are always entire UTXOs, and outputs are newly created UTXOs that can belong to different participants, depending on the transaction.

As soon as an output is used as input in a transaction, it becomes spent and can never be used again. The UTXO output is associated with an address which is represented by a public key hash. We call them public key addresses. The ada amount and optionally native tokens of a public key address is the sum of ada and native tokens from all UTXOs belonging to this address. A transaction must be signed by the owner of the private key corresponding to the address that defines the input UTXO. Think of an address as a 'lock' that can only be 'unlocked' by the right 'key' – the correct signature. The user which controls a private key of an address can create transactions and use the ada or native tokens sitting at the UTXOs of this address. For every transaction there is also a fee to pay denominated in ada for the Cardano blockchain.

The extended UTXO model introduces in addition to public key addresses also script addresses that can contain some logic. That logic defines under which conditions the UTXOs sitting at this address can be spent. The address is unlocked by a piece of data called the *redeemer*, which in the conventional UTXO model would be a private key. A UTXO also contains some data called the *datum*, beside the amount of ada sitting at the address. The datum together with the redeemer and the transaction context are the input information for a script logic that then chooses whether this transaction is valid and can be processed by a node on the network.



You can check the validity of a transaction in your wallet. If it is valid, you can be sure it will be processed on the network, given the condition that all the UTXO inputs are still present at processing time. If they are not the transaction will simply fail and no fee will be charged to the user that sent the transaction. We call the script that validates a transaction the validator. The script address is defined as a hash of the validator code written in Plutus core language. The script addresses are publicly known. We will talk about Plutus core in the next chapter.

As said the validator script takes the datum, the redeemer and the transaction context as input information. The input for the datum is collected from each UTXO individually that is sitting at a script address. That means if there are multiple UTXOs specified to be consumed in the transaction, the validation logic is checked for each of them separately. So, in each validation there is only one datum as input coming from one UTXO.

This limited view of the validator script that can see only inputs, outputs and the transaction context that will be processed, has a security advantage compared to the Ethereum model, where the script can see the whole state of the blockchain. That enables Ethereum's scripts to be much more powerful but for this reason it's also very difficult to predict what a given script will do. That opens the door to all sorts of security issues. It can be mathematically proven that every logic you can express in Ethereum you can also express in the extended UTXO model. And that makes it a much safer and reliable transaction model compared to Ethereum.

A transaction in the EUTXO model can be classified as a producing transaction that produces a script UTXOs or as a spending transaction that spends a script UTXOs. In general, every

transaction except the genesis transaction takes at least one UTXO as input and produces at least one UTXO as output. So, we use the terms spending and producing only when we talk about script addresses. If we first send ada to a script address, we call it a producing transaction. After that if we try to collect that ada from the script address, we call this transaction a spending transaction.

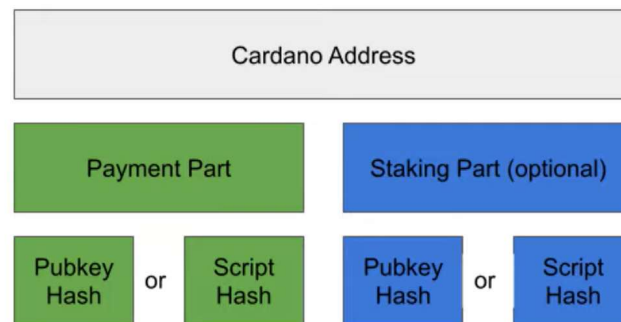
The producing transaction must include this address, and it must include the datum or the hash of the datum that will be attached to the UTXO created at script address. If it includes the hash of the datum, only a person that knows the datum by some other means not by looking at the blockchain is able to ever spend such an UTXO. The spending transaction is responsible for providing the redeemer, the transaction context and optionally also the datum. It also must provide the validator script. If we construct a transaction where the funds go to a public key address only a signature with the private key of the sending address is needed.

Let us look now in more detail how the datum or its hash gets provided by the producing transaction. There are three options to do this. The first option is that the output contains only the hash of the datum. Then the consuming transaction which wants to spend this script output must contain the actual datum. This option is the cheapest for the producing transaction because a hash is small and the transaction costs less fees. In this case the creator of the consuming transaction must know the value of the datum.

Producer Provides		Consumer Provides
In Output	In Tx Body	
Datum Hash	-	Datum
Datum Hash	Datum	Datum
(Inline) Datum	-	-

The second option is that beside the datum hash we include the actual datum in the transaction body, which means that people can look up the datum on the blockchain and then include it in their spending transaction. Such a producing transaction costs more than the previous one because it stores the datum in the transaction. However, it's not so convenient because the cardano node sees the statement value during validation but the forgets its value. So, for somebody else to later discover the datum on the blockchain another tool is needed like a chain indexer or dbsync. Since the Vasil hard fork there is also the third option where the output itself contains the datum. In this case we call it an inline datum and then it does not have to provide it in the body of the producing transaction. Additionally, the consuming transaction does not have to provide the datum and so it becomes smaller and cheaper.

Let us also now look at Cardano addresses in detail. A Cardano address has two parts. The payment part and optionally the staking part.



The payment part is responsible for deciding under which conditions an UTXO sitting at such an address can be spent. It is defined either by the hash of a public key or the hash of a plutus script. In cardano we also call the private key the signing key and the public key the verification key. If it contains the public key hash UTXOs sitting at such an address can only be spent if the transaction is signed with the corresponding private key pair (signing key). But if it contains the script hash, the corresponding script is executed during validation and its output determines if one or more UTXOs sitting at the script address can be spent.

The optional staking part of an address decides who is entitled to staking rewards and is in control of delegation. Also, here you have two options. If the staking part is specified with a public key hash, then the owner of the corresponding private key is entitled to staking rewards. If it is a script hash, then the corresponding plutus script is executed for transactions that try to withdraw staking rewards for example.

Finally let us look at how the validation script can be referenced by the spending transaction. The spending or also called consuming transaction must provide the validation script as an input to the transaction. Because some scripts are used very often since the Vasil hard fork there is another way which is called reference scripts. In this case a plutus script can be attached to the datum of a UTXO. Usually, you do not want this UTXO to be consumed, so you can send it to a script address where the validation logic fails no matter the inputs. Then a spending transaction can simply reference this script sitting at a permanent UTXO instead of providing it as input. In either case a cardano node checks if the hash of the script equals the script address name which a spending transaction is processing. In the end we state that the redeemer is always supplied by the consuming transaction.

The EUTXO model is not tied to a specific programming language. What we have in cardano is Plutus which is based on Haskell but you could use the same model with a different

programming language. There are other blockchains also using EUTXO which are not using Plutus, as the Ergo blockchain for example.

1.4 Plutus code

The code for Plutus smart contracts is separated into two. First is the “on-chain” code, which consists of the validator function and some additional declarations and variables as the script address. This code gets compiled to Plutus Core language. It runs on the cardano blockchain and once submitted it cannot be changed.

From the official documentation [2] we get the following description for Plutus Core:

Plutus Core is the scripting language used by cardano to implement the EUTXO model. It is a simple, functional language similar to Haskell, and a large subset of Haskell can be used to write Plutus Core scripts. As a smart contract author, you don’t write any Plutus Core; rather, all Plutus Core scripts are generated by a Haskell compiler plugin called Plutus Tx.

The “off-chain” code is written in Haskell, just like the on-chain code, unlike Ethereum where the on-chain code is written in Solidity, but the off-chain code is written in JavaScript. That way, the business logic only needs to be written once. This logic can then be used in the validator script and in the code that builds the transactions that run the validator script. [2]

The off-chain code basically constructs the transaction and submits it to the blockchain. Since both the on-chain and off-chain code are written in Haskell they can reside in one Haskell file while testing your code, which allows them to share code between them.

2 Simple validation scripts

We said earlier that a script sitting on a UTXO address takes in 3 parameters: the datum, the redeemer and the transaction context, which is the submitted transaction with all the inputs and outputs. In the low-level implementation of Plutus these 3 parameters are represented with the same data type. In the high-level implementation you can use custom Haskell data types for datum and redeemer and a predefined type for the transaction context. You can use both of the implementations in your smart-contract code. The difference between them is code performance which is better for the low-level implementation data types.

The data type for the low-level implementation is called *BuiltinData*. It contains two conversion functions *builtinDataToData* and *dataToBuiltinData*, that can convert back and forth to the *Data* type. The *Data* type has its constructor exposed and has the following definition:

Constructors

```
Constr Integer [Data]
Map [(Data, Data)]
List [Data]
I Integer
B ByteString
```

Figure 4 - Data type

Both data types are contained in the *PlutusTx* module. To be able to assign a string value to the B ByteString constructor you need to import the language extension *OverloadedStrings*.

2.1 Low-level untyped validation scripts

Let's look at the code from the *Gift.hs* file in the week02 folder.

```
1 {-# LANGUAGE DataKinds           #-}
2 {-# LANGUAGE FlexibleContexts     #-}
3 {-# LANGUAGE NoImplicitPrelude    #-}
4 {-# LANGUAGE ScopedTypeVariables #-}
5 {-# LANGUAGE TemplateHaskell      #-}
6 {-# LANGUAGE TypeApplications     #-}
7 {-# LANGUAGE TypeFamilies         #-}
8 {-# LANGUAGE TypeOperators        #-}
9
```

```

10 module Week02.Gift where
11
12 import          Control.Monad      hiding (fmap)
13 import          Data.Map           as Map
14 import          Data.Text          (Text)
15 import          Data.Void          (Void)
16 import          Plutus.Contract
17 import          PlutusTx           (Data (..))
18 import qualified PlutusTx
19 import qualified PlutusTx.Builtins as Builtins
20 import          PlutusTx.Prelude   hiding (Semigroup(..), unless)
21 import          Ledger             hiding (singleton)
22 import          Ledger.Constraints as Constraints
23 import qualified Ledger.Scripts    as Scripts
24 import          Ledger.Ada         as Ada
25 import          Playground.Contract (printJson, printSchemas,
                                     ensureKnownCurrencies, stage)
26 import          Playground.TH      (mkKnownCurrencies, mkSchemaDefinitions)
27 import          Playground.Types    (KnownCurrency (..))
28 import          Prelude             (IO, Semigroup (..), String)
29 import          Text.Printf         (printf)
30
31 {-# OPTIONS_GHC -fno-warn-unused-imports #-}
32
33 {-# INLINABLE mkValidator #-}
34 mkValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
35 mkValidator _ _ _ = ()
36
37 validator :: Validator
38 validator = mkValidatorScript $(PlutusTx.compile [| mkValidator |])
39
40 valHash :: Ledger.ValidatorHash
41 valHash = Scripts.validatorHash validator
42
43 scrAddress :: Ledger.Address
44 scrAddress = scriptAddress validator
45
46 type GiftSchema =
47     Endpoint "give" Integer
48     .\ Endpoint "grab" ()
49
50 give :: AsContractError e => Integer -> Contract w s e ()
51 give amount = do
52     let tx = mustPayToOtherScript valHash (Datum $ Builtins.mkI 0) $
53         Ada.lovelaceValueOf amount

```

```

53     ledgerTx <- submitTx tx
54     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
55     logInfo @String $ printf "made a gift of %d lovelace" amount
56
57     grab :: forall w s e. AsContractError e => Contract w s e ()
58     grab = do
59         utxos <- utxosAt scrAddress
60         let orefs = fst <$> Map.toList utxos
61             lookups = Constraints.unspentOutputs utxos      <>
62                     Constraints.otherScript validator
63         tx :: TxConstraints Void Void
64         tx = mconcat [mustSpendScriptOutput oref $ Redeemer $
65                     Builtins.mkI 17 | oref <- orefs]
66         ledgerTx <- submitTxConstraintsWith @Void lookups tx
67         void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
68         logInfo @String $ "collected gifts"
69
70     endpoints :: Contract () GiftSchema Text ()
71     endpoints = awaitPromise (give' `select` grab') >> endpoints
72     where
73         give' = endpoint @"give" give
74         grab' = endpoint @"grab" $ const grab
75
76     mkSchemaDefinitions ''GiftSchema
77     mkKnownCurrencies []

```

There are various language pragmas added in the beginning. One that is worth of noticing is the *NoImplicitPrelude* extension which allows you to use a custom prelude. That being said some standard Haskell functions you are used to may not work since we are importing *Plutus.Prelude* in the import section. To get a list of all functions of the custom prelude you can search the Plutus library documentation for the keyword *Plutus.Prelude* that you have learned to build.

In our code we first define the validator function called *mkValidator*. It accepts the datum, the redeemer and the context. We chose to use the *BuiltinData* datatype. Because of that the output of the validator is the unit *()*. We said earlier the validator only validates a given transaction and for that reason you would expect a *Bool* value as return type. This is the case if we use the high-level implementation. For the low-level we have instead the unit, which is returned if the transaction passed or the error type that gets returned if it fails.

In our case the validation passes no matter the input arguments, which means that everyone can process this UTXO and take some ADA from it if it contains any. For this reason, we call this

example gift. Next, we want to define our *validator*. To get it we need to compile the validator to Plutus Core script. This is done in line 38.

To make use of the *mkValidatorScript* function we need to import *Ledger.Script*. The *compile* function from the *PlutusTx* module takes as input a syntax tree of a function which we can get if we put the oxford brackets `[| mkValidator |]` around our desired function. The *compile* function produces another syntax tree that is written in the Plutus core language. Then the `$$` symbol called *splice*, takes a syntax tree and splices it back to Haskell source code, which is what we need for input to our *mkValidatorScript* function. If you want to be able to call external helper functions from the validator function you need to add the *INLINABLE* pragma statement before the validator function (33).

Next, we create the validator hash and the script address which contains the validator hash and staking information that is used when we stake to a stake pool. The *scriptAddress* function is contained in the module *Ledger.Address*. The definitions until now represent the on-chain code. Everything after that is the off-chain code.

First, we define the endpoints that the user can use to interact with the blockchain from within his wallet. Our *give* endpoint takes an integer parameter, which will represent the amount of ADA we are willing to give and *grab* does not take any parameter because it just spends funds. Then we define the *give* and *grab* functions. For the *give* function we first define the transaction (52). The transaction says a certain amount of lovelace should be paid to the specified address and with this datum. Line 53 submits the transaction, line 54 awaits confirmation of the transaction and line 55 logs the provided information which can be seen on the playground.

For the *grab* function we first lookup all the UTXOs sitting at a given address (59). The *utxosAt* function takes an address and returns a contract with the last parameter being the result.

```
utxosAt :: Address -> Contract w s e (Map TxOutRef ChainIndexTxOut)
```

Contract is a member of the monad type class and because we assign the *utxos* value with the `<-` operator we get back only the result of the contract which is a *Map* of *TxOutRef* and *ChainIndexTxOut*. Each entry in the *Map* represents a UTXO which is identified with the *TxOutRef* type and described with the *ChainIndexTxOut* type that holds information about the value, address, validator and datum. The types can be seen in Figure 5 and Figure 6 below. Figure 5 shows us that a UTXO is defined by the transaction ID of type *TxId* and an index which gets assigned to each of the UTXOs a transaction produces. The *TxId* type is just a newtype wrapper around the *BuiltinByteString* type and represents the transaction hash. The *TxId* type is also an instance of the *IsString* type class that implements the function *fromString* which you can use to convert between string and the type in question.

data **TxOutRef**

Source

A reference to a transaction output. This is a pair of a transaction reference, and an index indicating which of the outputs of that transaction we are referring to.

Constructors

TxOutRef

txOutRefId :: **TxId**

txOutRefIdx :: **Integer** Index into the referenced transaction's outputs

Figure 5 – TxOutRef type

data **ChainIndexTxOut**

Source

Transaction output that comes from a chain index query.

It is defined here instead of the `plutus-chain-index` because `plutus-ledger` uses that datatype, and `plutus-ledger` can't depend on `plutus-chain-index` because of a cyclic dependency.

This datatype was created in order to be used in `processConstraint`, specifically with the constraints `MustSpendPubKeyOutput` and `MustSpendScriptOutput`.

Constructors

PublicKeyChainIndexTxOut

_ciTxOutAddress :: **Address**

_ciTxOutValue :: **Value**

ScriptChainIndexTxOut

_ciTxOutAddress :: **Address**

_ciTxOutValidator :: **Either ValidatorHash Validator**

_ciTxOutDatum :: **Either DatumHash Datum**

_ciTxOutValue :: **Value**

Figure 6 – ChainIndexTxOut type

Then we get all the references of the UTXOs (60). The references are composed of a transaction index and an index number that gets assigned to every UTXO a transaction produces. Together they uniquely identify a UTXO. Next, we define lookups in order to tell the wallet how to construct the transaction (61). First, we say that we are looking for unspent outputs where also potentially the datums of the UTXOs are included in the *ChainIndexTxOut* data type. And second, we provide the validator script.

Then we define the transaction so that it consumes all the UTXOs with a specific redeemer (64). When we submit the transaction, we provide the lookups so that we include the validator data and datums of the UTXOs. Next, we wait for confirmation and after that we log a message.

In the endpoint function we give the user the choice of selecting between the 2 endpoints and then recourse to do the same again. In line 75 we generate the schema definition and in line 76 we call the *mkKnownCurrencies* function so that in the playground we have ADA available.

Let's look now at an example where the validator function fails. In order that we can use the *traceError* function we need also to import the *OverloadedStrings* language extension.

```
{-# LANGUAGE OverloadedStrings #-}
mkValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkValidator _ _ _ = traceError "BURNT!"
```

Now the validation will fail and the Grab transaction will not be processed. For our next example we will take into account the redeemer (example *FortyTwo.hs*).

```
mkValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkValidator _ r _
  | r == Builtins.mkI 42 = ()
  | otherwise             = traceError "wrong redeemer!"
```

We will also need to modify the off-chain code. The grab function will take now an input parameter that will be of type Integer.

```
1  type GiftSchema =
2      Endpoint "give" Integer
3      .\ / Endpoint "grab" Integer
4
5  grab :: forall w s e. AsContractError e => Integer -> Contract w s e ()
6  grab n = do
7      utxos <- utxosAt scrAddress
8      let orefs = fst <$> Map.toList utxos
9          lookups = Constraints.unspentOutputs utxos      <>
10                 Constraints.otherScript validator
11      tx :: TxConstraints Void Void
12      tx = mconcat [mustSpendScriptOutput oref $ Redeemer $
13                  Builtins.mkI n | oref <- orefs]
14      ledgerTx <- submitTxConstraintsWith @Void lookups tx
15      void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
16      logInfo @String $ "collected gifts"
17
18  endpoints :: Contract () GiftSchema Text ()
19  endpoints = awaitPromise (give' `select` grab') >> endpoints
```



```

20     where
21         give' = endpoint @"give" give
22         grab' = endpoint @"grab" grab

```

From the original code in *Gift.hs* what changes are lines 3, 5, 6, 12-13 and 22. These examples were for the low-level data types in the validator function.

2.2 High Level typed validation scripts

Now let's look at an example where we use the high-level data types. We call it a typed validation script. The code can be found in *Typed.hs*. Here we provide only the significant parts.

```

1  import qualified Ledger.Typed.Scripts as Scripts
2
3  {-# INLINABLE mkValidator #-}
4  mkValidator :: () -> Integer -> ScriptContext -> Bool
5  mkValidator _ r _ = traceIfFalse "wrong redeemer" $ r == 42
6
7  data Typed
8  instance Scripts.ValidatorTypes Typed where
9      type instance DatumType Typed = ()
10     type instance RedeemerType Typed = Integer
11
12  typedValidator :: Scripts.TypedValidator Typed
13  typedValidator = Scripts.mkTypedValidator @Typed
14      $(PlutusTx.compile [| mkValidator |])
15      $(PlutusTx.compile [| wrap |])
16  where
17      wrap = Scripts.wrapValidator @() @Integer
18
19  validator :: Validator
20  validator = Scripts.validatorScript typedValidator
21
22  valHash :: Ledger.ValidatorHash
23  valHash = Scripts.validatorHash typedValidator
24
25  scrAddress :: Ledger.Address
26  scrAddress = scriptAddress validator
27
28  type GiftSchema =
29      Endpoint "give" Integer
30      .\ Endpoint "grab" Integer
31
32  give :: AsContractError e => Integer -> Contract w s e ()

```

```

33 give amount = do
34     let tx = mustPayToTheScript () $ Ada.lovelaceValueOf amount
35     ledgerTx <- submitTxConstraints typedValidator tx
36     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
37     logInfo @String $ printf "made a gift of %d lovelace" amount

```

First we need to import *Ledger.Typed.Scripts* instead of *Ledger.Scripts*. We mentioned we can use arbitrary data types for the datum and redeemer. Since we don't care for the datum, we just use unit. But for the script we need to use the *ScriptContext* data type (4). And the result will now be of type *Bool*. To compile the validator function to Plutus core we need to introduce a new type that encodes the information of the datum and the redeemer (7-10). We can pick an arbitrary name for this type. It doesn't need any constructors we just need to make it an instance of *Scripts.ValidatorTypes*. Next, we compile our validator function but because we need a translation from our custom types to the low-level types, we need to add the *wrap* function (14-16). This gives us a typed validator and we turn it into an un-typed validator (19-20). When computing the validator hash, we use the typed validator as input (22-23). But for the script address we use the validator as input (25-26). In the off-chain code we redefine our *give* function. When constructing the transaction, we use now the function *mustPayToTheScript* that is a typed version for the case that the transaction we are constructing only involves one script. We now only provide as input the datum and the amount of ADA in lovelace. Then we also need to use the function *submitTxConstraints* to submit the transaction where we additionally provide as input our typed validator function. We also need the code for *grab* function introduced in the previous example and some code from the initial example, for the playground to work. Now let's look at the functions that allow us to convert between the low-level and high-level data types. For this we need to import the *PlutusTx.IsData.Class* module which contains the functions *toData* and *fromData*.

```

ghci> import PlutusTx.IsData.Class
ghci> toData ()
Constr 0 []
ghci> fromData (Constr 0 []) :: Maybe ()
Just ()
ghci> fromData (Constr 1 []) :: Maybe ()
Nothing
ghci> toData (42 :: Integer)
I 42
ghci> fromData (I 42) :: Maybe Integer
Just 42
ghci> fromData (List []) :: Maybe Integer
Nothing

```

This works only for predefined instances of the `ToData` class, which you can get with the command `":i ToData"`. If you want another data type you need to make it an instance of this class. But Plutus provides a mechanism that automatically does that for you. We look at this in our next example *IsData.hs* where we use custom defined data types for our redeemer.

```
1  newtype MySillyRedeemer = MySillyRedeemer Integer
2
3  PlutusTx.unstableMakeIsData 'MySillyRedeemer
4
5  {-# INLINEABLE mkValidator #-}
6  mkValidator :: () -> MySillyRedeemer -> ScriptContext -> Bool
7  mkValidator _ (MySillyRedeemer r) _ = traceIfFalse "wrong redeemer" $ r == 42
8
9  data Typed
10 instance Scripts.ValidatorTypes Typed where
11     type instance DatumType Typed = ()
12     type instance RedeemerType Typed = MySillyRedeemer
13
14 typedValidator :: Scripts.TypedValidator Typed
15 typedValidator = Scripts.mkTypedValidator @Typed
16     $$ (PlutusTx.compile [| mkValidator |])
17     $$ (PlutusTx.compile [| wrap |])
18 where
19     wrap = Scripts.wrapValidator @() @MySillyRedeemer
```

We first define our custom data type *MySillyRedeemer*. To make our data an instance of the *ToData* type class we can use a template Haskell function called *unstableMakeIsData* which does that for us. The syntax to provide a type is to use 2 single quotes in front of the type. If we manually try to convert it in the Repl we get the following result:

```
ghci> :l src/Week02/IsData.hs
ghci> import PlutusTx.IsData.Class
ghci> toData (MySillyRedeemer 42)
Constr 0 [I 42]
```

There is also a stable version of the template function which is more commonly used in production code. In our case we had only one data constructor but if there are many it's not clear how they will be ordered. The unstable version does not make any guarantees that between different Plutus versions the constructor number corresponding to a given constructor will be preserved. Next the validator function now changes, where we pattern match the redeemer data type. Also, the type instance and the wrapper function get updated. For the off-chain code only the transaction in the `grab` function has to be updated where we use the *PlutusTx.toBuiltinData* function that takes a custom data type and converts it to *BuiltinData*.

```
tx = mconcat [mustSpendScriptOutput oref $ Redeemer $ PlutusTx.toBuiltinData
              (MySillyRedeemer r) | oref <- orefs]
```

2.3 Homework

Let's look now at an example where your custom data type is defined in record syntax. We will have two Booleans as input and the validator function should return True if both parameters are equal. You can find such an example in the *Solution2.hs* file from week02 examples.

```
1  {-# LANGUAGE DeriveAnyClass      #-}
2  {-# LANGUAGE DeriveGeneric      #-}
3
4  import           Data.Aeson      (FromJSON, ToJSON)
5  import           GHC.Generics    (Generic)
6  import           Playground.Contract (printJson, printSchemas,
7                                     ensureKnownCurrencies, stage, ToSchema)
8
9  data MyRedeemer = MyRedeemer
10     { flag1 :: Bool
11     , flag2 :: Bool
12     } deriving (Generic, FromJSON, ToJSON, ToSchema)
13
14  PlutusTx.unstableMakeIsData ''MyRedeemer
15
16  -- This should validate if the two Booleans in the redeemer are equal!
17  mkValidator :: () -> MyRedeemer -> ScriptContext -> Bool
18  mkValidator () (MyRedeemer b c) _ = traceIfFalse "wrong redeemer" $ b == c
19
20  grab :: forall w s e. AsContractError e => MyRedeemer -> Contract w s e ()
21  grab r = do
22      utxos <- utxosAt scrAddress
23      let orefs = fst <$> Map.toList utxos
24          lookups = Constraints.unspentOutputs utxos      <>
25                  Constraints.otherScript validator
26      tx :: TxConstraints Void Void
27      tx = mconcat [mustSpendScriptOutput oref $ Redeemer $
28                  PlutusTx.toBuiltinData r | oref <- orefs]
29      ledgerTx <- submitTxConstraintsWith @Void lookups tx
30      void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
31      logInfo @String $ "collected gifts"
```

Compared to our *Typed.hs* examples we need to add two new language extensions. After that we import 2 new libraries and from the *Playground.Contract* library we additionally import the *ToSchema* type class. Then we define our custom data type using record syntax where we

derive the generic, JSON and schema type classes. Next, we write our validator function where we can use pattern matching. And for the rest of the code the only thing that changes is the grab function. There we use now the *MyRedeemer* data type in the type signature and in the transaction definition (28) we provide the input parameter *r* that represents the data type *MyRedeemer*. In the Plutus playground if we define a grab action then we will see we have now two Boolean checkboxes with the above description of the record syntax functions (Figure 7). If both fields are checked or unchecked the transaction should be valid. Otherwise, it should fail.



The image shows a window titled "3" with a close button (X) in the top right corner. The main content area is titled "Wallet 2: grab". Below the title, there are two Boolean checkboxes. The first checkbox is labeled "flag1" and is currently unchecked, with the word "False" in green text next to it. The second checkbox is labeled "flag2" and is also currently unchecked, with the word "False" in green text next to it.

Figure 7 - Grab action

3 Time, parameterized contracts and the cardano testnet

In this chapter we will first time look at the script context. If we want to work with it, we need to import the module *Ledger.Contexts* or just *Ledger* module which contains the data type *ScriptContext*. Script context defines two constructors that represent the transaction information and the script purpose (Figure 8). The script purpose constructor can be defined with four different parameters depending for what we use a script (Figure 10).

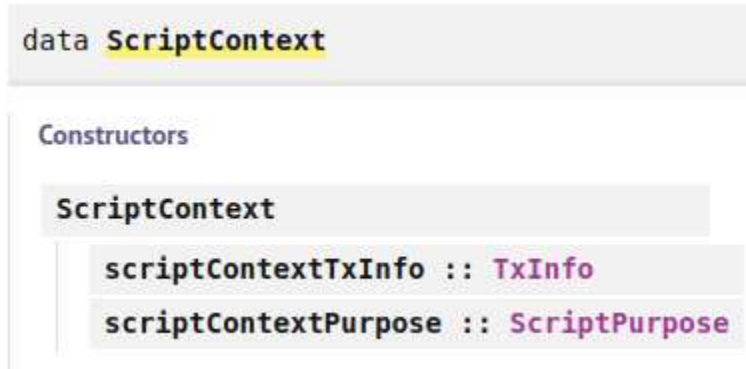


Figure 8 - ScriptContext type

The transaction info data type is defined with various parameters that define properties of the transaction which are described in Figure 9. The *txInfoSignatories* parameter contains a list of public addresses which have signed this transaction. For producing transactions, the *txInfoData* parameter is optional, whereas for spending transactions it is mandatory.

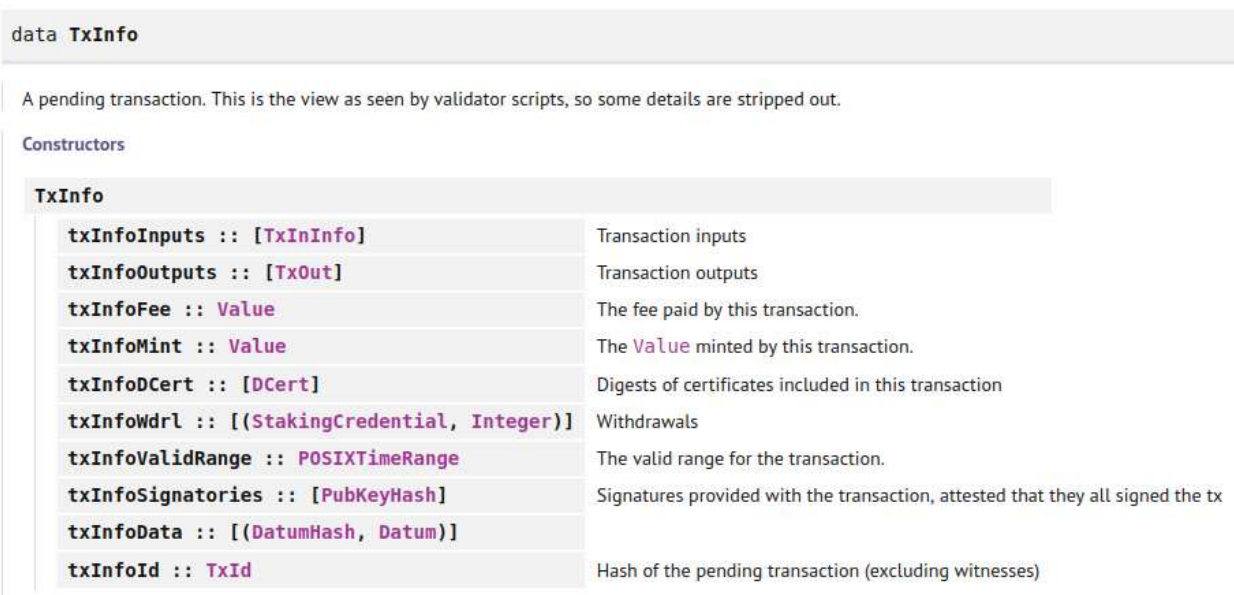


Figure 9 – TxInfo type



Figure 10 – ScriptPurpose type

3.1 Time

If we can validate transactions in the wallet, we should have a mechanism that prevents a transaction being executed on a node if it does not fall in a certain time range. And for this we have the *txInfoValidRange* field that is part of the transaction info and specifies the time range in which a transaction is valid. This is part of the general check procedure that happens before a transaction is executed. Part of this procedure is also checking that all the inputs are present, that the balances add up and that the fees are included. If this pre-check succeeds, we can be sure that the validation will succeed if it was also successfully validated in the wallet. By default, all transactions use an infinite time range.

The consensus protocol of Cardano called Ouroboros uses slots instead of using POSIX time. Currently a slot equals to 1 second but this can change in the future. A hard fork which would change the slot interval is known around 36 hours in advance. For this reason, slot intervals should not have an upper bound that is too further in the future as 36 hours. If we look at the *POSIXTimeRange* data type in depth we get the following information:

```
type POSIXTimeRange = Interval POSIXTime
```

```
data Interval a
```

```
Constructors:
```

```
  ivFrom :: LowerBound a
```

```
  ivTo :: UpperBound a
```

```
data LowerBound a
```

```
Constructors:
```

```
  (Extended a) Closure
```

```
data UpperBound a
```

```
Constructors:
```

```

(Extended a) Closure
-----
type Closure = Bool
-----
data Extended a
Constructors:
    NegInf
    Finite a
    PosInf
-----
newtype POSIXTime
Constructors:
    getPOSIXTime :: Integer

```

Closure specifies whether the boundary is included or not. POSIX time is measured as the number of milliseconds since 1970-01-01 00:00:00. There are also some functions available associated with the *Interval* data type:

- *member*: Checks whether a value is in an interval.
- *interval*: takes in two parameters and constructs an *Interval a* with boundaries included.
- *from*: gives an interval *a* that includes all values that are greater than or equal to *a*.
- *to*: gives an interval *a* that includes all values that are smaller than or equal to *a*.
- *always*: An interval *a* that covers every slot.
- *never*: An interval *a* that is empty.
- *singleton*: An interval *a* that only contains the one *a*.
- *hull*: 'hull a b' is the smallest interval containing *a* and *b* intervals.
- *intersection*: 'intersection a b' is the largest interval that is contained in *a* and in *b* intervals, if it exists.
- *overlap*: checks whether two intervals have a value in common and returns a Boolean
- *contains*: checks whether the second interval is contained in the first one
- *isEmpty*: checks whether an interval is empty
- *before*: checks whether a given time is before the given interval
- *after*: checks whether a given time is after the given interval

All of these functions and data types are included in the *Ledger.Interval* module which we need to import to work with them. Let's look at some code examples that use these functions:

```

ghci> interval (10 :: Integer) 20
Interval {ivFrom = LowerBound (Finite 10) True, ivTo = UpperBound (Finite 20) True}
ghci> member 9 $ interval (10 :: Integer) 20
False
ghci> member 10 $ interval (10 :: Integer) 20
True
ghci> member 29 $ from (30 :: Integer)
False
ghci> member 30 $ from (30 :: Integer)

```



```

True
ghci> member 31 $ to (30 :: Integer)
False
ghci> member 30 $ to (30 :: Integer)
True
ghci> intersection (interval (10 :: Integer) 20) $ interval 18 30
Interval {ivFrom = LowerBound (Finite 18) True, ivTo = UpperBound (Finite 20) True}
ghci> contains (to (100 :: Integer)) $ interval 30 80
True
ghci> contains (to (100 :: Integer)) $ interval 30 101
False
ghci> overlaps (to (100 :: Integer)) $ interval 30 101
True

```

Next let's look at the *Vesting.hs* example found in week03 folder where we create a script address with some ADA that can be redeemed after a certain date has passed.

```

1  import           Ledger.Constraints    (TxConstraints)
2  import qualified Ledger.Constraints    as Constraints
3  import           Prelude                (IO, Semigroup (..), Show (..), String)
4
5  data VestingDatum = VestingDatum
6    { beneficiary :: PaymentPubKeyHash
7    , deadline    :: POSIXTime
8    } deriving Show
9
10 PlutusTx.unstableMakeIsData ''VestingDatum
11
12 {-# INLINABLE mkValidator #-}
13 mkValidator :: VestingDatum -> () -> ScriptContext -> Bool
14 mkValidator dat () ctx = traceIfFalse "beneficiary's signature missing"
15                           signedByBeneficiary &&
16                           traceIfFalse "deadline not reached" deadlineReached
17
18 where
19   info :: TxInfo
20   info = scriptContextTxInfo ctx
21
22   signedByBeneficiary :: Bool
23   signedByBeneficiary = txSignedBy info $ unPaymentPubKeyHash $
24     beneficiary dat
25
26   deadlineReached :: Bool
27   deadlineReached = contains (from $ deadline dat) $ txInfoValidRange info
28
29 data Vesting
30 instance Scripts.ValidatorTypes Vesting where
31   type instance DatumType Vesting = VestingDatum

```

```

29     type instance RedeemerType Vesting = ()
30
31     typedValidator :: Scripts.TypedValidator Vesting
32     typedValidator = Scripts.mkTypedValidator @Vesting
33         $$ (PlutusTx.compile [| mkValidator |])
34         $$ (PlutusTx.compile [| wrap |])
35     where
36         wrap = Scripts.wrapValidator @VestingDatum @()
37
38     validator :: Validator
39     validator = Scripts.validatorScript typedValidator
40
41     valHash :: Ledger.ValidatorHash
42     valHash = Scripts.validatorHash typedValidator
43
44     scrAddress :: Ledger.Address
45     scrAddress = scriptAddress validator
46
47     data GiveParams = GiveParams
48         { gpBeneficiary :: !PaymentPubKeyHash
49         , gpDeadline     :: !POSIXTime
50         , gpAmount       :: !Integer
51         } deriving (Generic, ToJSON, FromJSON, ToSchema)
52
53     type VestingSchema =
54         Endpoint "give" GiveParams
55         .\ Endpoint "grab" ()
56
57     give :: AsContractError e => GiveParams -> Contract w s e ()
58     give gp = do
59         let dat = VestingDatum
60             { beneficiary = gpBeneficiary gp
61             , deadline    = gpDeadline gp
62             }
63         tx = Constraints.mustPayToTheScript dat $ Ada.lovelaceValueOf $
64             gpAmount gp
65         ledgerTx <- submitTxConstraints typedValidator tx
66         void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
67         logInfo @String $ printf "made a gift of %d lovelace to %s
68                                 with deadline %s"
69                                 (gpAmount gp)
70                                 (show $ gpBeneficiary gp)
71                                 (show $ gpDeadline gp)
72
73     grab :: forall w s e. AsContractError e => Contract w s e ()

```

```

72 grab = do
73     now <- currentTime
74     pkh <- ownPaymentPubKeyHash
75     utxos <- Map.filter (isSuitable pkh now) <$> utxosAt scrAddress
76     if Map.null utxos
77     then logInfo @String $ "no gifts available"
78     else do
79         let orefs = fst <$> Map.toList utxos
80             lookups = Constraints.unspentOutputs utxos <>
81                     Constraints.otherScript validator
82             tx :: TxConstraints Void Void
83             tx = mconcat [Constraints.mustSpendScriptOutput oref
84                           unitRedeemer | oref <- orefs] <>
85                     Constraints.mustValidateIn (from now)
86             ledgerTx <- submitTxConstraintsWith @Void lookups tx
87             void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
88             logInfo @String $ "collected gifts"
89 where
90     isSuitable :: PaymentPubKeyHash -> POSIXTime -> ChainIndexTxOut -> Bool
91     isSuitable pkh now o = case _ciTxOutDatum o of
92         Left _ -> False
93         Right (Datum e) -> case PlutusTx.fromBuiltinData e of
94             Nothing -> False
95             Just d -> beneficiary d == pkh && deadline d <= now
96 endpoints :: Contract () VestingSchema Text ()
97 endpoints = awaitPromise (give' `select` grab') >> endpoints
98 where
99     give' = endpoint @"give" give
100    grab' = endpoint @"grab" $ const grab
101
102 mkSchemaDefinitions ''VestingSchema
103
104 mkKnownCurrencies []

```

Compared to the last code example *Solution2.hs* from the week02 folder, we make a different import of *Ledger.Constraints* one qualified and one normal. And from the standard Prelude we also import the *Show* type class. Then we define the vesting datum, which contains the hash of the payment public key from the user that will retrieve the funds and the deadline after which the funds can be retrieved. We derive the *Show* type class so we can display our information in the console. Next, we write our validator function where we look at the datum and the context. In the *where* clause we first define the transaction info. After that we implement our two conditions. For the first condition *signedByBeneficiary* we use the *txSignedBy* function.

```
txSignedBy :: TxInfo -> PubKeyHash -> Bool
```

It takes a transaction info and a public key hash and returns True if the transaction was signed with this key. The function expects a *PubKeyHash* variable which we get with the record syntax function *unPaymentPubKeyHash*.

```
Prelude Ledger> :i PaymentPubKeyHash
type PaymentPubKeyHash :: *
newtype PaymentPubKeyHash
  = PaymentPubKeyHash {unPaymentPubKeyHash :: PubKeyHash}
```

For the second condition *deadlineReached* we construct an interval that stretches from the deadline to infinity and if the validity interval *txInfoValidRange* from the transaction info object is contained in this interval the deadline is definitely reached. Next follows the code where we compile our validator and create the script address.

After that follows the off-chain code. First, we define a data type called *GiveParams* that includes information necessary to construct the script address. We add an exclamation mark to the parameters so they get strictly evaluated. Then we write our give function where we first define the datum which we use in the next step when constructing the transaction.

For the grab function we can have more UTXOs sitting at the vesting address. First, we get the current time and lookup our own payment public key hash. Then we get all suitable UTXOs, where we use the helper function *isSuitable*. We use *Map.filter* that filters through the values.

```
filter :: (a -> Bool) -> Map k a -> Map k a
```

We have to apply the functor operator *<\$>* because the *utxosAt* functions returns a contract monad. The *isSuitable* function takes in our payment key, the current time and the Map keys which are of type *ChainIndexTxOut* and returns a Bool. Inside the body of the helper function, we check the datum of the UTXO with the record syntax function *_ciTxOutDatum* which can be seen in Figure 6. If it does not exist and only the hash exists, we will get a Left type and can return false. If it exists, we check the content of the datum. If there is a content present, we can validate our two conditions. Else we also just return False.

After we filtered out the valid UTXOs we check if there are any. If there aren't we log a message else we construct a transaction that collects all of them in one transaction. In the transaction we also use the function *mustValidateIn* that creates a validity interval for the transaction (84). In the real world there could be too many UTXOs to collect them in one transaction which can be only of a limited size. But in this example we forget about this case.


```

13             traceIfFalse "deadline not reached" deadlineReached
14     where
15         info :: TxInfo
16         info = scriptContextTxInfo ctx
17
18         signedByBeneficiary :: Bool
19         signedByBeneficiary = txSignedBy info $ unPaymentPubKeyHash $
                                beneficiary p
20
21         deadlineReached :: Bool
22         deadlineReached = contains (from $ deadline p) $ txInfoValidRange info
23
24     data Vesting
25     instance Scripts.ValidatorTypes Vesting where
26         type instance DatumType Vesting = ()
27         type instance RedeemerType Vesting = ()
28
29     typedValidator :: VestingParam -> Scripts.TypedValidator Vesting
30     typedValidator p = Scripts.mkTypedValidator @Vesting
31         ( $(PlutusTx.compile [| mkValidator |])
          `PlutusTx.applyCode` PlutusTx.liftCode p)
32         $(PlutusTx.compile [| wrap |])
33     where
34         wrap = Scripts.wrapValidator @() @()
35
36     validator :: VestingParam -> Validator
37     validator = Scripts.validatorScript . typedValidator
38
39     valHash :: VestingParam -> Ledger.ValidatorHash
40     valHash = Scripts.validatorHash . typedValidator
41
42     scrAddress :: VestingParam -> Ledger.Address
43     scrAddress = scriptAddress . validator
44
45     data GiveParams = GiveParams
46         { gpBeneficiary :: !PaymentPubKeyHash
47         , gpDeadline    :: !POSIXTime
48         , gpAmount      :: !Integer
49         } deriving (Generic, ToJSON, FromJSON, ToSchema)
50
51     type VestingSchema =
52         Endpoint "give" GiveParams
53         .\ Endpoint "grab" POSIXTime
54
55     give :: AsContractError e => GiveParams -> Contract w s e ()

```

```

56 give gp = do
57     let p = VestingParam
58         { beneficiary = gpBeneficiary gp
59           , deadline   = gpDeadline gp
60           }
61     tx = Constraints.mustPayToTheScript () $ Ada.lovelaceValueOf $
        gpAmount gp
62     ledgerTx <- submitTxConstraints (typedValidator p) tx
63     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
64     logInfo @String $ printf "made a gift of %d lovelace to %s
        with deadline %s"
65         (gpAmount gp)
66         (show $ gpBeneficiary gp)
67         (show $ gpDeadline gp)
68
69 grab :: forall w s e. AsContractError e => POSIXTime -> Contract w s e ()
70 grab d = do
71     now <- currentTime
72     pkh <- ownPaymentPubKeyHash
73     if now < d
74     then logInfo @String $ "too early"
75     else do
76         let p = VestingParam
77             { beneficiary = pkh
78               , deadline   = d
79             }
80         utxos <- utxosAt $ scrAddress p
81         if Map.null utxos
82         then logInfo @String $ "no gifts available"
83         else do
84             let orefs = fst <$> Map.toList utxos
85                 lookups = Constraints.unspentOutputs utxos <>
86                     Constraints.otherScript (validator p)
87             tx :: TxConstraints Void Void
88             tx = mconcat [Constraints.mustSpendScriptOutput
89                           oref unitRedeemer | oref <- orefs] <>
90                     Constraints.mustValidateIn (from now)
91             ledgerTx <- submitTxConstraintsWith @Void lookups tx
92             void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
93             logInfo @String $ "collected gifts"
94
95 endpoints :: Contract () VestingSchema Text ()
96 endpoints = awaitPromise (give' `select` grab') >> endpoints
97 where

```

```

97     give' = endpoint @"give" give
98     grab' = endpoint @"grab" grab
99
100 mkSchemaDefinitions ''VestingSchema
101
102 mkKnownCurrencies []

```

First, we add a language extension called *MultiParamTypeClasses*. Other extensions and imports are the same as in the previous *Vesting.hs* example. Then we define the vesting parameter which has basically the same structure as in our vesting example.

Next, we add our vesting parameter as an additional parameter to our validator function. The datum and the redeemer are set to unit (). Now during compilation all data should be known in advance but we can make an exception for the parameter if it is a static piece of data and not a function. We do this by calling the function *PlutusTx.liftCode* and then we combine it with the validator function that is now written in Plutus core with help of *PlutusTx.applyCode* function (31). Our type validator takes now as input the vesting parameter which can be seen in the type signature (29) and function definition (30). And additionally, to that in the wrap function the datum changes now to unit (34).

To be able for this code to work we need a lift instance for our *VestingParam* parameter (8). And if there are multiple arguments in our parameter, we need to add the language pragma from line 1. Of course, now in our validator function when we write our two conditions, we read the data from the additional parameter instead of the datum. In line 26 the type of the datum changes to unit. When we define our validator, its hash and the script address instead of calling functions upon static piece of data we have now two functions that we need to combine with the dot notation (36-43).

When we define our endpoints, we need an input parameter for grab which will be the POSIX time of the deadline. The payment public key hash we can get with the appropriate function. For the give parameter what changes compared to the previous code example is that we now provide the vesting parameter variable together with the type validator when we submit the transaction and not as the datum contained in the transaction (62).

In the grab function we now add the POSIX time as an input parameter but the actual deadline is already set by the person who performs the give action. First, we check if this parameter is larger than the current time (73) and if yes, we log a »too early« message. If not, we define our vesting parameter (76 to 78), then we get a list of all the UTXOs where we have to provide the vesting parameter as input to the script address (80). If there are any UTXOs then we construct the transaction and submit it same as in the vesting example, with the only difference that now in the lookups parameter we add the vesting variable to the validator (86). Compared to our

last example where we filtered out the suitable UTXOs with the *isSuitable* function we now provide an input parameter to the script that automatically returns the UTXOs that match the public payment key hash and the exact deadline.

In the playground we have to provide the public key hash and the deadline for the give action. We also now specify the deadline we are looking for as an argument to grab. For this reason, we cannot grab two gifts at once if they exist. We have to perform two grab actions and the posix time in the grab action has to exactly match the posix time in the give action, to be able to retrieve the funds. One thing to mention is an obvious failure of the playground.

If we try to make a gift wait until slot 10, then make a grab and wait for 2 more slots, the grab will succeed. But if the waiting time after the grab is 1 slot, then the grab will fail no matter what the waiting time after the give action is. But since we used a posix time of 10 slots as input for the give and grab action this is in contradiction with counting the time.

3.3 The Cardano testnet

Here we will show how to use the Cardano command line interface to deploy some code to the cardano test net or main net. To do this you must run a cardano node which you can find here:

<https://github.com/input-output-hk/cardano-node>

On the right side under Releases section click on the latest cardano node (Figure 11).

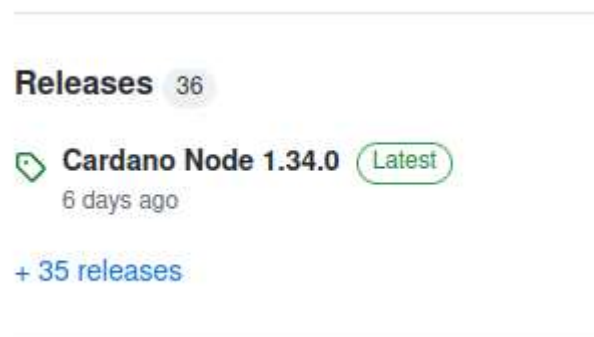


Figure 11 - Cardano node

Under the technical specifications chapter, you have the downloads section where you can click on Hydra Binaries. A web-page opens where you can download your installer for a given OS. Once the cardano node is installed you can check your version with the command:

```
$ cardano-node --version
```

You will also need some configuration files which you can find again in the technical specifications chapter in the downloads section. Download the following test-net files:

- config
- byronGenesis
- shellyGenesis
- alonzoGenesis
- topology

You can find these files also in the folder *week03/testnet/*. There you can find also the file *start-node-testnet.sh*. This bash script starts the cardano node.

```
$ ./start-node-testnet.sh
```

When you start the node for the first time you will need to wait for a couple of hours for the test net to synchronize. Beside the cardano node also the Cardano command line interface (CLI) gets installed. You can look at the help page of the client with the command:

```
$ cardano-cli --help
```

You will get a list of top-level commands. If you want to search through a command, use the same pattern and call the help page on this command to get available sub-commands:

```
$ cardano-cli address --help
```

You can repeat this procedure on a sub-command of the address command. To try out a Plutus contract we will need two keys. Use the following commands from within the *testnet* folder to create your two keys:

```
$ cardano-cli address key-gen --verification-key-file 01.vkey --signing-key-file 01.skey  
$ cardano-cli address key-gen --verification-key-file 02.vkey --signing-key-file 02.skey
```

Next you will need to generate two payment addresses which will represent two wallets.

```
$ cardano-cli address build --payment-verification-key-file 01.vkey --testnet-magic  
1097911063 --out-file 01.addr  
$ cardano-cli address build --payment-verification-key-file 02.vkey --testnet-magic  
1097911063 --out-file 02.addr
```

You can find the test-net magic number in the *testnet-shelley-genesis.json* configuration file. Now we need some test ADA to be able to make transactions on the test-net. For this we use something called the Faucet: <https://testnets.cardano.org/en/testnets/cardano/tools/faucet/>.

The faucet is a web-based service that provides test ADA to users of the test-net. On the above web-page you past in your address that you can find by viewing your 01.addr and 02.addr files and then complete the test that checks you not a robot. After that click on the Request funds button. If you want to see if the test ADA arrived you can query the blockchain:

```
$ export CARDANO_NODE_SOCKET_PATH=node.socket
$ cardano-cli query utxo --address $(cat 01.addr) --testnet-magic 1097911063
```

You should see a output of the transaction hash and transaction index which together define an UTXO and information about how much ADA was sent. If you want to add funds to your second address and don't have an API key you will have to wait 24 hours to be able to send some test ADA again from the Faucet. An easier way is to send some ADA from you first address to your second address. To send the ADA you can use the *send.sh* bash script which can be found in week03/testnet folder. You will need to change the tx-in field with your transaction hash and the index specified after the # symbol that you got from the previous query command.

```
$ ./send.sh
```

The transaction build command automatically calculates fees and creates change outputs in case your inputs are higher than the outputs. There is a block on the Cardano test-net and main-net on average every twenty seconds, so you will have to wait a bit before you check whether your transaction was processed. When it is you will have a new transaction that will produce a UTXO containing the change ADA. If we want to use Plutus code in the cardano CLI we need to serialize and write to disk various Plutus types. We do this with the *Deploy.hs* code.

```
1 {-# LANGUAGE OverloadedStrings #-}
2 {-# LANGUAGE TypeApplications #-}
3
4 module Week03.Deploy
5   ( writeJSON
6   , writeValidator
7   , writeUnit
8   , writeVestingValidator
9   ) where
10
11 import           Cardano.Api
12 import           Cardano.Api.Shelley (PlutusScript (..))
13 import           Codec.Serialise      (serialise)
14 import           Data.Aeson           (encode)
15 import qualified Data.ByteString.Lazy as LBS
16 import qualified Data.ByteString.Short as SBS
17 import           PlutusTx             (Data (..))
```

```

18 import qualified PlutusTx
19 import qualified Ledger
20
21 import           Week03.Parameterized
22
23 dataToScriptData :: Data -> ScriptData
24 dataToScriptData (Constr n xs) = ScriptDataConstructor n $
                                dataToScriptData <$> xs
25 dataToScriptData (Map xs)      = ScriptDataMap [(dataToScriptData x,
                                dataToScriptData y) | (x, y) <- xs]
26 dataToScriptData (List xs)    = ScriptDataList $ dataToScriptData <$> xs
27 dataToScriptData (I n)        = ScriptDataNumber n
28 dataToScriptData (B bs)       = ScriptDataBytes bs
29
30 writeJSON :: PlutusTx.ToData a => FilePath -> a -> IO ()
31 writeJSON file = LBS.writeFile file . encode . scriptDataToJson
                                ScriptDataJsonDetailedSchema . dataToScriptData .
                                PlutusTx.toData
32
33 writeValidator :: FilePath -> Ledger.Validator ->
                 IO (Either (FileError ()) ())
34 writeValidator file = writeFileTextEnvelope @(PlutusScript PlutusScriptV1)
                                file Nothing . PlutusScriptSerialised . SBS.toShort .
                                LBS.toStrict . serialise . Ledger.unValidatorScript
35
36 writeUnit :: IO ()
37 writeUnit = writeJSON "testnet/unit.json" ()
38
39 writeVestingValidator :: IO (Either (FileError ()) ())
40 writeVestingValidator = writeValidator "testnet/vesting.plutus" $
                                validator $ VestingParam
41                                { beneficiary = Ledger.PaymentPubKeyHash
42                                "c2ff616e11299d9094ce0a7eb5b7284b705147a822f4ffbd471f971a"
43                                , deadline   = 1643235300000
44                                }

```

The *Cardano.Api* is the Haskell library which the cardano CLI uses. It has its own data type called *ScriptData*. With the *writeJSON* function we convert some Plutus data to script data and write it in a JSON format to a file (30-31). On lines 23 to 28 we define the conversion function *dataToScriptData*. Then we define the *writeUnit* function which basically writes a unit object to the specified file (36-37). We also need our Plutus validator script that we have to convert to a script and write it to a file. We do this with the *writeValidator* function (33-34). We want to apply this function to our parameterized contract. We do this with the *writeVestingValidator* IO

action where we need to specify the beneficiary and the deadline. Note that we also use here our *validator* parameter, which is available because we imported the *Week03.Parameterized* module, which is defined in the *Parameterized.hs* file.

We get the payment public key hash of our second address with this command:

```
$ cardano-cli address key-hash --payment-verification-key-file 02.vkey --out-file 02.pkh
```

We copy the hash from the file 02.pkh and past it into the code for the IO action. We can get our deadline from epochconverter.com where we can specify the date and time and get back the POSIX time in milliseconds. We also need the actual address corresponding to the script. First we execute the function *writeVestingValidator* in the Repl so that we get the *vesting.plutus* file. Then we execute:

```
$ cardano-cli address build-script --script-file vesting.plutus --testnet-magic 1097911063 --out-file vesting.addr
```

We can give now some ADA to the vesting address we just created. You can do this with the *give.sh* bash script also found in the testnet folder. In the *--tx-in* field you have to specify the new transaction hash and index of the UTXO sitting at address 1. The transaction hash and index changed after you send some ADA to address 2, so you will need to do a query again. In the *transaction build* command, we use the *--tx-out-datum-hash-file* option that computes the hash of a datum written to a JSON file.

```
$ ./give.sh
```

After we run this script, we can again check if the ADA arrived:

```
$ cardano-cli query utxo --address $(cat vesting.addr) --testnet-magic 1097911063
```

The second wallet wants now to grab that ADA and we can use the *grab.sh* bash script to do this. Since this is a spending transaction, we need to provide the following things:

- The transaction hash and id which we provide in the *--tx-in* option.
- The actual script which we provide in the *--tx-in-script-file* option.
- The datum which we provide in the *--tx-in-datum-file* option. We get the unit datum if we load our *Deploy.hs* file in the Repl and execute the command *writeUnit*.
- The redeemer (same as the datum) which we provide in the *--tx-in-redeemer-file* option.
- In the *--tx-in-collateral* option you specify an UTXO that belongs to yourself and contains only ADA and no native tokens. It represents the collateral which must be large enough to cover the costs if validation would fail. This is a special case where you can

circumvent the validation process and you would get a failed transaction. The nodes that process your transaction have to be reimbursed for that or the system would be open to denial-of-service attacks. We can use the 02.addr for collateral.

- In the *--required-signer-hash* field we specify the public key hash of our second address.
- The validity interval which we provide in the *--invalid-before* option. It has to be provided as slots. We get the current slot of the test-net with the command:

```
$ cardano-cli query tip --testnet-magic 1097911063
```

The slot time we specify should be after our deadline.

- The protocol parameters which we provide in *--protocol-params-file* option. You get the content of the file with the command:

```
$ cardano-cli query protocol-parameters --testnet-magic 1097911063 --out-file protocol.json
```

Once we configured everything the grab should work and if we would query the second address we should see that there are two UTXOs sitting at this address.

```
$ ./grab.sh  
$ cardano-cli query utxo --address $(cat 02.addr) --testnet-magic 1097911063
```

For the main-net the procedure would be the same only that we would specify *--mainnet* option instead of *--testnet-magic*.

3.4 Homework

You can do the *Homework1.hs* and *Homework2.hs* examples by yourself. We will present here the solution and comment on it. For homework 1 we want to create a script address from which a beneficiary can retrieve his funds until a certain deadline. After the deadline we can retrieve the funds back to our own address. Here is the *Solution1.hs* code example.

```
1  import qualified Prelude          as P
2
3  data VestingDatum = VestingDatum
4    { beneficiary1 :: PaymentPubKeyHash
5    , beneficiary2 :: PaymentPubKeyHash
6    , deadline     :: POSIXTime
7    } deriving P.Show
8
9  PlutusTx.unstableMakeIsData ''VestingDatum
10
```

```

11 {-# INLINABLE mkValidator #-}
12 -- This should validate if either beneficiary1 has signed the transaction
13 -- and the current slot is before or at the deadline or if beneficiary2
14 -- has signed the transaction and the deadline has passed.
15 mkValidator :: VestingDatum -> () -> ScriptContext -> Bool
16 mkValidator dat () ctx
17   | (unPaymentPubKeyHash (beneficiary1 dat) `elem` sigs) &&
18     (to (deadline dat) `contains` range) = True
19   | (unPaymentPubKeyHash (beneficiary2 dat) `elem` sigs) &&
20     (from (1 + deadline dat) `contains` range) = True
21   | otherwise = False
22 where
23   info :: TxInfo
24   info = scriptContextTxInfo ctx
25
26   sigs :: [PubKeyHash]
27   sigs = txInfoSignatories info
28
29   range :: POSIXTimeRange
30   range = txInfoValidRange info
31
32 data Vesting
33 instance Scripts.ValidatorTypes Vesting where
34   type instance DatumType Vesting = VestingDatum
35   type instance RedeemerType Vesting = ()
36
37 typedValidator :: Scripts.TypedValidator Vesting
38 typedValidator = Scripts.mkTypedValidator @Vesting
39   $(PlutusTx.compile [| mkValidator |])
40   $(PlutusTx.compile [| wrap |])
41 where
42   wrap = Scripts.wrapValidator @VestingDatum @()
43
44 validator :: Validator
45 validator = Scripts.validatorScript typedValidator
46
47 scrAddress :: Ledger.Address
48 scrAddress = scriptAddress validator
49
50 data GiveParams = GiveParams
51   { gpBeneficiary :: !PaymentPubKeyHash
52   , gpDeadline    :: !POSIXTime
53   , gpAmount      :: !Integer
54   } deriving (Generic, ToJSON, FromJSON, ToSchema)
55
56

```

```

53 type VestingSchema =
54     Endpoint "give" GiveParams
55     .\ Endpoint "grab" ()
56
57 give :: AsContractError e => GiveParams -> Contract w s e ()
58 give gp = do
59     pkh <- ownPaymentPubKeyHash
60     let dat = VestingDatum
61         { beneficiary1 = gpBeneficiary gp
62         , beneficiary2 = pkh
63         , deadline     = gpDeadline gp
64         }
65     tx = Constraints.mustPayToTheScript dat $ Ada.lovelaceValueOf $
66         gpAmount gp
67     ledgerTx <- submitTxConstraints typedValidator tx
68     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
69     logInfo @P.String $
70     printf "made a gift of %d lovelace to %s with deadline %s"
71         (gpAmount gp)
72         (P.show $ gpBeneficiary gp)
73         (P.show $ gpDeadline gp)
74
75 grab :: forall w s e. AsContractError e => Contract w s e ()
76 grab = do
77     now <- currentTime
78     pkh <- ownPaymentPubKeyHash
79     utxos <- utxosAt scrAddress
80     let utxos1 = Map.filter (isSuitable $ \dat -> beneficiary1 dat == pkh &&
81                                     now <= deadline dat) utxos
82     utxos2 = Map.filter (isSuitable $ \dat -> beneficiary2 dat == pkh &&
83                                     now > deadline dat) utxos
84
85     logInfo @P.String $ printf "found %d gift(s) to grab"
86         (Map.size utxos1 P.+ Map.size utxos2)
87
88     unless (Map.null utxos1) $ do
89         let orefs = fst <$> Map.toList utxos1
90         lookups = Constraints.unspentOutputs utxos1 P.<>
91             Constraints.otherScript validator
92         tx :: TxConstraints Void Void
93         tx = mconcat [Constraints.mustSpendScriptOutput oref
94             unitRedeemer | oref <- orefs] P.<>
95             Constraints.mustValidateIn (to now)
96         void $ submitTxConstraintsWith @Void lookups tx
97
98     unless (Map.null utxos2) $ do
99         let orefs = fst <$> Map.toList utxos2
100         lookups = Constraints.unspentOutputs utxos2 P.<>

```



```

92             Constraints.otherScript validator
93         tx :: TxConstraints Void Void
94         tx      = mconcat [Constraints.mustSpendScriptOutput oref $
                             unitRedeemer | oref <- orefs] P.<>
95             Constraints.mustValidateIn (from now)
96         void $ submitTxConstraintsWith @Void lookups tx
97     where
98         isSuitable :: (VestingDatum -> Bool) -> ChainIndexTxOut -> Bool
99         isSuitable p o = case _ciTxOutDatum o of
100             Left _      -> False
101             Right (Datum d) -> maybe False p $ PlutusTx.fromBuiltinData d
102
103 endpoints :: Contract () VestingSchema Text ()
104 endpoints = awaitPromise (give' `select` grab') >> endpoints
105     where
106         give' = endpoint @"give" give
107         grab' = endpoint @"grab" $ const grab
108
109 mkSchemaDefinitions ''VestingSchema
110
111 mkKnownCurrencies []

```

Compared to the *Vesting.hs* example our vesting datum changes where we add another beneficiary (3-7). Then our validator function changes where we cover 2 cases for which the transaction is valid (11-27). First the case the beneficiary retrieves the funds before the deadline. Second the case that the giver retrieves the funds after the deadline. The give function is practically the same as in the vesting example with the exception that the datum contains now 2 beneficiaries. The grab function however changes now. We construct 2 UTXO lists where for the first the current time should be before the deadline and for the second after (78-79). Then we define 2 cases where the first is that the first UTXO list is not empty and the second case is when the second UTXOs list is not empty. For each case we construct the transaction and submit it. The difference in the transactions is the validity interval (87 and 96). Some of the code is then the same as in the vesting example. The *isSuitable* function also changes. Now we take in a function and a chain index transaction output and return a Bool. We provide the functions in lines 78 and 79. In the body of the *isSuitable* function we use the *maybe* function which has following type signature:

```
maybe :: b -> (a -> b) -> Maybe a -> b
```

If the third parameter is a Nothing the first parameter is returned. Else we use the function a to b on the third parameter extracted from the Maybe to produce the result of type b.

For homework 2 we want to modify the *Parameterized.hs* example in such a way that the input information is split between the additional parameter that will carry the payment public key hash and the datum that will carry the deadline. Here is the code.

```
1  import Prelude (IO, Semigroup (..), Show (..), String, undefined)
2
3  {-# INLINABLE mkValidator #-}
4  mkValidator :: PaymentPubKeyHash -> POSIXTime -> () -> ScriptContext -> Bool
5  mkValidator pkh s () ctx =
6      traceIfFalse "beneficiary's signature missing" checkSig      &&
7      traceIfFalse "deadline not reached"          checkDeadline
8  where
9      info :: TxInfo
10     info = scriptContextTxInfo ctx
11
12     checkSig :: Bool
13     checkSig = unPaymentPubKeyHash pkh `elem` txInfoSignatories info
14
15     checkDeadline :: Bool
16     checkDeadline = from s `contains` txInfoValidRange info
17
18 data Vesting
19 instance Scripts.ValidatorTypes Vesting where
20     type instance DatumType Vesting = POSIXTime
21     type instance RedeemerType Vesting = ()
22
23 typedValidator :: PaymentPubKeyHash -> Scripts.TypedValidator Vesting
24 typedValidator p = Scripts.mkTypedValidator @Vesting
25     ($$(PlutusTx.compile [| mkValidator |])) `PlutusTx.applyCode`
26     PlutusTx.liftCode p)
27     ($$(PlutusTx.compile [| wrap |]))
28 where
29     wrap = Scripts.wrapValidator @POSIXTime @()
30
31 validator :: PaymentPubKeyHash -> Validator
32 validator = Scripts.validatorScript . typedValidator
33
34 scrAddress :: PaymentPubKeyHash -> Ledger.Address
35 scrAddress = scriptAddress . validator
36
37 data GiveParams = GiveParams
38     { gpBeneficiary :: !PaymentPubKeyHash
39     , gpDeadline    :: !POSIXTime
40     , gpAmount      :: !Integer
41     } deriving (Generic, ToJSON, FromJSON, ToSchema)
```

```

41
42 type VestingSchema =
43     Endpoint "give" GiveParams
44     .\ / Endpoint "grab" ()
45
46 give :: AsContractError e => GiveParams -> Contract w s e ()
47 give gp = do
48     let p = gpBeneficiary gp
49         d = gpDeadline gp
50         tx = Constraints.mustPayToTheScript d $ Ada.lovelaceValueOf $
51             gpAmount gp
52     ledgerTx <- submitTxConstraints (typedValidator p) tx
53     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
54     logInfo @String $ printf "made a gift of %d lovelace
55                             to %s with deadline %s"
56                             (gpAmount gp)
57                             (show $ gpBeneficiary gp)
58                             (show $ gpDeadline gp)
59
60 grab :: forall w s e. AsContractError e => Contract w s e ()
61 grab = do
62     now <- currentTime
63     pkh <- ownPaymentPubKeyHash
64     utxos <- Map.filter (isSuitable now) <$> utxosAt (scrAddress pkh)
65     if Map.null utxos
66     then logInfo @String $ "no gifts available"
67     else do
68         let orefs = fst <$> Map.toList utxos
69             lookups = Constraints.unspentOutputs utxos <>
70                 Constraints.otherScript (validator pkh)
71             tx :: TxConstraints Void Void
72             tx = mconcat [Constraints.mustSpendScriptOutput oref
73                         unitRedeemer | oref <- orefs] <>
74                 Constraints.mustValidateIn (from now)
75         ledgerTx <- submitTxConstraintsWith @Void lookups tx
76         void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
77         logInfo @String $ "collected gifts"
78
79 where
80     isSuitable :: POSIXTime -> ChainIndexTxOut -> Bool
81     isSuitable now o = case _ciTxOutDatum o of
82         Left _ -> False
83         Right (Datum e) -> case PlutusTx.fromBuiltinData e of
84             Nothing -> False
85             Just d -> d <= now

```

```

83 endpoints :: Contract () VestingSchema Text ()
84 endpoints = awaitPromise (give' `select` grab') >> endpoints
85   where
86     give' = endpoint @"give" give
87     grab' = endpoint @"grab" $ const grab
88
89 mkSchemaDefinitions ''VestingSchema
90
91 mkKnownCurrencies []

```

Compared to the parameterized example we import the undefined parameter with the standard Prelude. Next, we create our validator function where the additional parameter carries the payment public key hash. There is now no need to create a vesting parameter type. The deadline is provided in the datum. For the helper functions there is a difference in the *checkSig* function (13) that now uses the *txInfoSignatories* function instead of the *txSignedBy* function. It basically gives you a list of all signatories. The instance for the vesting type also changes where the datum is now of type *POSIXTime* (20). And same holds true for the wrap function inside the type validator (28). The input parameter for the typed validator is now of type *PaymentPubKeyHash* (23) which is the same for the validator and script address. When constructing our endpoints, the grab endpoint does not take in any parameter (44). For the give function the only difference is when we are constructing the transaction, we specify the datum and input amount of lovelace (50). For the grab function we use rather the approach as in the *Vesting.hs* code example, where we filter out the UTXOs that have a suitable deadline (62). When we create the lookups, we pass to the validator our own payment public key hash as input (68). When declaring the endpoints, the grab endpoint has the *const* keyword prepended since it does not take any input parameters.

4 Monads, Traces & Contracts

In this chapter we will focus on the off-chain part of the Plutus contracts. The Plutus prelude contains functions that all have the `INLINABLE` pragma added which makes it possible to use these functions in the validation code that will then be compiled to Plutus core script. There are many Haskell libraries that weren't written taking into account the Plutus architecture so we can't use them for validation. For this reason, the code for the validation script will be relatively simple and will not have many dependencies. The off-chain part however does not get compiled and is just Haskell code so we can use much more features in this code. The wallet code (off-chain code) is written in a special monad called the contract monad.

4.1 Monads

In this chapter we will briefly explain how monads in Haskell work. If you are familiar with this you can skip this chapter. Let's first look at input and output (IO). In Haskell we say to functions that work in the IO context actions. If you have a Haskell function that is not an action you can be sure it will always produce the same result given the same input parameters. This is called referential transparency. We also refer to the term "no side effects" which means that there is nothing from the outside world of a function that could be changing the state of the function. For example, we can take a global variable that is used in a function which could possibly change for setting a global state. For this reason, a variable once it is declared it cannot be changed anymore except if we shadow it. An example of shadowing can be seen here:

```
foo = let x = 1
      in ((let x = 2 in x), x)
```

The final value of `foo` here is 2, which overwrote the value 1 set in the beginning. But such a case is rarely used in Haskell code. Haskell deals with side effects by using a IO type constructor that takes in one argument and is a member of the monad type class. This is then a IO action which is allowed to have side effects. An example of such an action is when we read some input from the terminal and compute a value from that input. The side effect here is that the input from the terminal can of course change. The monad type class implements 3 operators:

- the bind operator for which we use the `>=>` symbol
- the monad sequencing operator for which we use the `>>` symbol
- the return operator which for which we use the `return` keyword

Let's look at the type signatures of these three operators:

```
(>=>) :: m a -> (a -> m b) -> m b
(>>)  :: m a -> m b -> m b
```

```
return :: a -> m a
```

First, we explain the return operator. It simply takes a value of type `a` and puts it into a context. If we have for instance a `String`, we can put it into an `IO` context with the following command:

```
ghci> a = return "Haskell" :: IO String
ghci> :t a
a :: IO String
```

Next let's explain the sequencing operator. From its type signature we would expect it takes as input two variables in a context and throws away the first one. This is what it exactly does. This comes useful when you want to chain together function or action calls. For example, let's look at code where we put together two `putStrLn` actions:

```
myText :: IO ()
myText = putStrLn "Learning" >> putStrLn "Haskell"

ghci> myText
Learning
Haskell
```

Now let's look at the final operator called bind which is the most useful one. It takes a type parameter in a context, then a function that converts the type specified in the first parameter but without the context to another type in a context which is then the result of this operator. So, let's simply declare a variable that is of type `IO Int`, then a function that takes an `Int` and return and `IO Int` by adding 1 to it. And in the end, we combine them with the bind operator.

```
monadExample1 :: IO Int
monadExample1 = do
  let a = return 1 :: IO Int
  let func var = return (var + 1) :: IO Int
  (a >>= func)
```

This function returns an `IO 2`. In case you are wondering what the `do` keyword does, it is a simplification or so called "syntactic sugar" for the following code:

```
monadExample :: IO Int
monadExample =
  (\a ->
    a >>=
      (\var ->
        return (var + 1) :: IO Int
      )
  ) (return 1 :: IO Int)
```

This code has the exact same meaning and also returns `IO 2`. So the `do` keyword allows us to more elegantly write an expression without having to use the lambda functions to connect our

expressions. One more thing to mention is the use of the operator `<-` in the `do` notation. It allows us to take a parameter in a context and assign it to a variable without that context.

```
helloName :: IO ()
helloName = do
  putStrLn "What is your name?"
  name <- getLine
  putStrLn ("Hello " ++ name)

ghci> helloName
What is your name?
Luka
Hello Luka
```

The `getLine` function returns an *IO String*, but the `name` parameter is only of type *String*. You could rewrite this code without the `do` notation in the following form:

```
helloName :: IO ()
helloName =
  putStrLn "What is your name?" >>
  getLine >>=
  (\name -> return ("Hello " ++ name)) >>=
  putStrLn
```

What's important to note about the monad type class it supports other data constructors as well, not just *IO*. For instance, also *Maybe* and *Lists* are also members of the monad type class. Let's have a look now at the *Maybe* type constructor. Here is the type signature:

```
data Maybe a = Nothing | Just a
```

It returns either a *Nothing* or a *Just* parameterized with the type *a*. Where can this type be used for example? For instance, if we import the `readMaybe` function we can read a *String* and get a *maybe* value from it parameterized by a number type as *integer*.

```
ghci> import Text.Read (readMaybe)
ghci> readMaybe "42" :: Maybe Int
42
ghci> readMaybe "42 + text" :: Maybe Int
Nothing
```

Now we can show an example where we use the monad operators on a *Maybe* parameter. We want to write a function that takes 3 strings as input, then tries to convert them to integers and sums them together. The result will then be of type *Maybe Int*.

```
1 import Text.Read (readMaybe)
2
```

```

3  foo :: String -> String -> String -> Maybe Int
4  foo x y z = case readMaybe x of
5      Nothing -> Nothing
6      Just k   -> case readMaybe y of
7          Nothing -> Nothing
8          Just l   -> case readMaybe z of
9              Nothing -> Nothing
10             Just m  -> Just (k + l + m)
11
12  bindMaybe :: Maybe a -> (a -> Maybe b) -> Maybe b
13  bindMaybe Nothing _ = Nothing
14  bindMaybe (Just x) f = f x
15
16  foo' :: String -> String -> String -> Maybe Int
17  foo' x y z = readMaybe x `bindMaybe` \k ->
18              readMaybe y `bindMaybe` \l ->
19              readMaybe z `bindMaybe` \m ->
20              Just (k + l + m)
21
22  foo'' :: String -> String -> String -> Maybe Int
23  foo'' x y z = threeInts (readMaybe x) (readMaybe y) (readMaybe z)
24
25  threeInts :: Monad m => m Int -> m Int -> m Int -> m Int
26  threeInts mx my mz =
27      mx >>= \k ->
28      my >>= \l ->
29      mz >>= \m ->
30      let s = k + l + m in return s
31
32  threeInts' :: Monad m => m Int -> m Int -> m Int -> m Int
33  threeInts' mx my mz = do
34      k <- mx
35      l <- my
36      m <- mz
37      let s = k + l + m
38      return s

```

First, we define our *foo* function with the use of nested case statements (3-10). Then we define a second version *foo'* which uses the helper function *bindMaybe* that basically works as the bind operator and is just defined for maybe values (12-14). With the help of this function and nested lambda expressions we create the *foo'* function (16-20). Now we define a third version called *foo''* that uses the helper function called *threeInts*. It takes as input three integers in a monad context and with the help of the bind operator and nested lambda functions computes

the sum and return the result again within the monad context (25-30). We could of course use the `->` operator which is demonstrated in function *threeInts'* (32-38). Let's look now at another parameterized type called *Either*.

```
data Either a b = Left a | Right b
```

It is usually used in similar situations as *Maybe* but with a more descriptive option of returning another type instead of a *Nothing* if something goes wrong. The common way is to use the *Left* constructor to indicate the error and *Right* to return the result. We can now look at a similar code as before that uses the *Either* type instead of the *Maybe* type.

```
1  import Text.Read (readMaybe)
2
3  readEither :: Read a => String -> Either String a
4  readEither s = case readMaybe s of
5      Nothing -> Left $ "can't parse: " ++ s
6      Just a   -> Right a
7
8  foo :: String -> String -> String -> Either String Int
9  foo x y z = case readEither x of
10     Left err -> Left err
11     Right k  -> case readEither y of
12         Left err -> Left err
13         Right l  -> case readEither z of
14             Left err -> Left err
15             Right m  -> Right (k + l + m)
16
17  bindEither :: Either String a -> (a -> Either String b) -> Either String b
18  bindEither (Left err) _ = Left err
19  bindEither (Right x)  f = f x
20
21  foo' :: String -> String -> String -> Either String Int
22  foo' x y z = readEither x `bindEither` \k ->
23              readEither y `bindEither` \l ->
24              readEither z `bindEither` \m ->
25              Right (k + l + m)
26
27  foo'' :: String -> String -> String -> Either String Int
28  foo'' x y z = threeInts (readEither x) (readEither y) (readEither z)
```

With the *readEither* function we read a value from a string and return an error message if the string cannot be parsed to a value. The first version of the *foo* function uses again the nested case statements. Then we declare the *bindEither* function that has same structure as the

previous *bindMaybe* function. In our second version the *foo'* function we use *readEither*, *bindEither* and nested lambda functions similar to the maybe example. In the third version the *foo''* function has very little modifications compared to the maybe example. We replace only the read function with the either version but the *threeInts* function call stays the same. The reason for this is that it works purely with the monad type class and is not tied to a specific parameterized type as a maybe or an either type. Of course, either is also a member of the monad type class which makes it possible for this function to work with either variables. Let's look now at the *Writer.hs* example where computations can produce also log outputs.

```
1  import Control.Monad
2
3  data Writer a = Writer a [String]
4      deriving Show
5
6  number :: Int -> Writer Int
7  number n = Writer n $ ["number: " ++ show n]
8
9  tell :: [String] -> Writer ()
10 tell xs = Writer () xs
11
12 foo :: Writer Int -> Writer Int -> Writer Int -> Writer Int
13 foo (Writer k xs) (Writer l ys) (Writer m zs) =
14     let
15         s = k + l + m
16         Writer _ us = tell ["sum: " ++ show s]
17     in
18         Writer s $ xs ++ ys ++ zs ++ us
19
20 bindWriter :: Writer a -> (a -> Writer b) -> Writer b
21 bindWriter (Writer a xs) f =
22     let
23         Writer b ys = f a
24     in
25         Writer b $ xs ++ ys
26
27 foo' :: Writer Int -> Writer Int -> Writer Int -> Writer Int
28 foo' x y z = x `bindWriter` \k ->
29             y `bindWriter` \l ->
30             z `bindWriter` \m ->
31             let s = k + l + m
32             in tell ["sum: " ++ show s] `bindWriter` \_ ->
33                 Writer s []
34
```

```

35 foo'' :: Writer Int -> Writer Int -> Writer Int -> Writer Int
36 foo'' x y z = do
37     s <- threeInts x y z
38     tell ["sum: " ++ show s]
39     return s
40
41 instance Functor Writer where
42     fmap = liftM
43
44 instance Applicative Writer where
45     pure = return
46     (<*>) = ap
47
48 instance Monad Writer where
49     return a = Writer a []
50     (>>=) = bindWriter

```

First, we define our writer type and derive show. It contains the type variable and a list of strings that represents a log message. Second, we define our *number* function which transforms an integer to a *Writer* type by adding a short description. Then we define the *tell* function that creates a writer from a string. For our *foo* function we take in three logging parameters of type *Writer Int* and produce a logging parameter of the same type. The function sums up the three parameters, concatenates the log messages and adds a “sum” log at the end. Next, we write the *bindWriter* function that works as the bind operator for our *Writer* type. We can define our second version the *foo'* function with the bind function, four lambda function and the *tell* function. In the last example we use again our *threeInts* function but in order for this to work we need to make our writer type an instance of monad. Because the functor type class is a superclass of applicative and applicative is a superclass of monad, we need to make also instances for those type classes. When defining the functor and applicative instances we use functions from the *Control.Monad* module that have basically same type signatures as functions that belong to the applicative and functor type classes. The code in lines 35 to 39 we could also rewrite without the do notation and it would look like this:

```

foo'' :: Writer Int -> Writer Int -> Writer Int -> Writer Int
foo'' x y z =
    threeInts x y z >>= \s ->
    tell ["sum: " ++ show s] >>
    return s

```

4.2 The emulator trace monad

The emulator trace monad enables you to run your code from the Repl instead of running the code on the Plutus playground. The word trace here defines actions that we would normally perform on the playground (e.g. defining initial conditions, performing give and grab actions). We run an emulator trace with the *runEmulatorTrace* function (Figure 12).

```
runEmulatorTrace :: EmulatorConfig -> EmulatorTrace () -> ([EmulatorEvent], Maybe EmulatorErr, EmulatorState)
```

Run an emulator trace to completion, returning a tuple of the final state of the emulator, the events, and any error, if any.

Figure 12 - runEmulatorTrace function

It takes in as input an *EmulatorConfig* and *EmulatorTrace* parameters. The emulator config (Figure 13) is an instance of the *Default* type class so if we import *Data.Default* we can make a default parameter. The command *def :: EmulatorConfig* will return an initial distribution of 10 wallets with each of them containing a 100 ADA with a default configuration for slots and fees.

```
data EmulatorConfig
```

[# Source](#)

Constructors

EmulatorConfig

<code>_initialChainState :: InitialChainState</code>	State of the blockchain at the beginning of the simulation. Can be given as a map of funds to wallets, or as a block of transactions.
<code>_slotConfig :: SlotConfig</code>	Set the start time of slot 0 and the length of one slot
<code>_feeConfig :: FeeConfig</code>	Configure the fee of a transaction

Figure 13 – EmulatorConfig type

You can see the types SlotConfig in FeeConfig in Figure 14 and Figure 15.

```
data SlotConfig
```

Datatype to configure the length (ms) of one slot and the beginning of the first slot.

Constructors

SlotConfig

<code>scSlotLength :: Integer</code>	Length (number of milliseconds) of one slot
<code>scSlotZeroTime :: POSIXTime</code>	Beginning of slot 0 (in milliseconds)

Figure 14 - SlotConfig type

```
data FeeConfig
```

Datatype to configure the fee in a transaction.

The fee for a transaction is typically: $\text{fcConstantFee} + \text{fcScriptsFeeFactor} * \text{SIZE_DEPENDANT_SCRIPTS_FEE}$.

Constructors

FeeConfig

fcConstantFee :: Ada	Constant fee per transaction in lovelace
fcScriptsFeeFactor :: Double	Factor by which to multiply the size-dependent scripts fee

Figure 15 - FeeConfig type

In the slot config type we can define the length of one slot and the starting time for slot 0. In the fee config data type, we can define the constant fee for transactions and a factor that is used for calculating size-dependent script fees. The *EmulatorTrace* type is defined in the *Plutus.Trace.Emulator* module. It is defined with the effect *Eff* monad (Figure 16).

```
type EmulatorTrace a = Eff '[StartContract, RunContract, Assert, Waiting,  
EmulatorControl, EmulatedWalletAPI, LogMsg String, Error EmulatorRuntimeError] a
```

Source

Figure 16 - EmulatorTrace type

There is also another function called *runEmulatorTraceIO* that does not take in a config, it just uses the default configuration and it doesn't return the triple since it runs in IO. You can try to run it in the Repl with a unit emulator trace like this:

```
ghci> import Plutus.Trace.Emulator  
ghci> import Data.Default  
ghci> runEmulatorTraceIO $ return ()
```

When running this function, we get an output that first displays the genesis transaction that distributes the initial funds. Then we get a wait of 2 slots and after that we get the final balances. And because we didn't specify any transactions each of the 10 wallets has a 100 ADA. You usually specify an emulator config if you want to have more or less wallets or different initial funds. There is a variation of the IO function called *runEmulatorTraceIO'*.

```
runEmulatorTraceIO' :: TraceConfig -> EmulatorConfig -> EmulatorTrace () -> IO ()
```

It takes in two additional parameters. The trace config has the following type signature.

```
TraceConfig  
:: (Wallet.Emulator.MultiAgent.EmulatorEvent' -> Maybe String)
```

```
-> GHC.IO.Handle.Types.Handle -> TraceConfig
```

The first parameter is a function that basically filters out events that we are interested in and the second parameter is a handle with could represent the console standard output or error or it could be a file handle. We could use the file handle for displaying the output of the IO' function in a file. The *TraceConfig* data type you can see in Figure 17.

```
data TraceConfig
```

Options for how to set up and print the trace.

Constructors

TraceConfig

<code>showEvent :: EmulatorEvent' -> Maybe String</code>	Function to decide how to print the particular events.
<code>outputHandle :: Handle</code>	Where to print the outputs to. Default: <code>stdout</code>

Figure 17 - TraceConfig type

Now let's look at the *Trace.hs* file where we define a trace.

```
1 {-# LANGUAGE TypeApplications #-}
2 {-# LANGUAGE DataKinds      #-}
3
4 module Week04.Trace where
5
6 import Control.Monad.Freer.Extras as Extras
7 import Data.Default                (Default (..))
8 import Data.Functor                (void)
9 import Ledger.TimeSlot
10 import Plutus.Trace
11 import Wallet.Emulator.Wallet
12
13 import Week04.Vesting
14
15 -- EmulatorTrace a
16
17 test :: IO ()
18 test = runEmulatorTraceIO myTrace
19
20 myTrace :: EmulatorTrace ()
21 myTrace = do
22     h1 <- activateContractWallet (knownWallet 1) endpoints
23     h2 <- activateContractWallet (knownWallet 2) endpoints
24     callEndpoint @"give" h1 $ GiveParams
```

```

25     { gpBeneficiary = mockWalletPaymentPubKeyHash $ knownWallet 2
26     , gpDeadline    = slotToBeginPOSIXTime def 20
27     , gpAmount      = 10000000
28     }
29   void $ waitUntilSlot 20
30   callEndpoint @"grab" h2 ()
31   s <- waitNSlots 2
32   Extras.logInfo $ "reached " ++ show s

```

In addition to some standard Plutus modules, we import also the *Vesting.hs* example (13). Then we define the emulator trace monad with a `do` block (21). Before we can call an endpoint, we have to start the *endpoints* contract for wallet 1 and 2 which we do in lines 22 and 23. As input parameter the function *activateContractWallet* takes the wallet on which to activate a contract and the contract itself. The result we get is a contract handle. Then we call an endpoint with the *callEndpoint* function where we pass in a type level string which is defined with the `@` symbol and works because we imported the language extension *TypeApplications*. We also pass in the handle and the input parameters (24-28). Then we wait until slot 20 where we use the *void* function that transforms a result to unit (29) and then call the *grab* endpoint from the wallet 2 (30). After that we wait another 2 slots and log a message. To run this example, execute *test* from the Repl.

```
Prelude> test
```

4.3 The contract monad

The contract monad comes with four type parameters *w*, *s*, *e* and *a*. The *a* represents the result. The *w* allows the contract to write log messages. The purpose of this logging is to communicate between different contracts or pass information to the outside world. The *s* specifies what endpoints are available in the contract. The *e* specifies the type of error messages. Let's have a look now at some contract monad examples from the *Contract.hs* file.

```

1  {-# LANGUAGE OverloadedStrings #-}
2  {-# LANGUAGE TypeApplications  #-}
3  {-# LANGUAGE DataKinds         #-}
4  {-# LANGUAGE TypeOperators     #-}
5
6  module Week04.Contract where
7
8  import Control.Monad.Freer.Extras as Extras
9  import Data.Functor                (void)

```

```

10 import Data.Text                (Text, unpack)
11 import Data.Void                (Void)
12 import Plutus.Contract          as Contract
13 import Plutus.Trace.Emulator    as Emulator
14 import Wallet.Emulator.Wallet
15
16 -- Contract w s e a
17
18 myContract1 :: Contract () Empty Text ()
19 myContract1 = do
20     void $ Contract.throwError "BOOM!"
21     Contract.logInfo @String "hello from the contract"
22
23 myTrace1 :: EmulatorTrace ()
24 myTrace1 = void $ activateContractWallet (knownWallet 1) myContract1
25
26 test1 :: IO ()
27 test1 = runEmulatorTraceIO myTrace1
28
29 myContract2 :: Contract () Empty Void ()
30 myContract2 = Contract.handleError
31     (\err -> Contract.logError $ "caught: " ++ unpack err)
32     myContract1
33
34 myTrace2 :: EmulatorTrace ()
35 myTrace2 = void $ activateContractWallet (knownWallet 1) myContract2
36
37 test2 :: IO ()
38 test2 = runEmulatorTraceIO myTrace2
39
40 type MySchema = Endpoint "foo" Int .\ Endpoint "bar" String
41
42 myContract3 :: Contract () MySchema Text ()
43 myContract3 = do
44     awaitPromise $ endpoint @"foo" Contract.logInfo
45     awaitPromise $ endpoint @"bar" Contract.logInfo
46
47 myTrace3 :: EmulatorTrace ()
48 myTrace3 = do
49     h <- activateContractWallet (knownWallet 1) myContract3
50     callEndpoint @"foo" h 42
51     callEndpoint @"bar" h "Haskell"
52
53 test3 :: IO ()
54 test3 = runEmulatorTraceIO myTrace3

```



```

55
56 myContract4 :: Contract [Int] Empty Text ()
57 myContract4 = do
58     void $ Contract.waitNSlots 10
59     tell [1]
60     void $ Contract.waitNSlots 10
61     tell [2]
62     void $ Contract.waitNSlots 10
63
64 myTrace4 :: EmulatorTrace ()
65 myTrace4 = do
66     h <- activateContractWallet (knownWallet 1) myContract4
67
68     void $ Emulator.waitNSlots 5
69     xs <- observableState h
70     Extras.logInfo $ show xs
71
72     void $ Emulator.waitNSlots 10
73     ys <- observableState h
74     Extras.logInfo $ show ys
75
76     void $ Emulator.waitNSlots 10
77     zs <- observableState h
78     Extras.logInfo $ show zs
79
80 test4 :: IO ()
81 test4 = runEmulatorTraceIO myTrace4

```

For the first example (18) we choose a contract where we do not want to write any log messages so we choose unit for *w* type parameter. We also do not need any endpoints so we choose the *Empty* type. For error messages we choose the type *Text*. And we are also not interested in the result so we put a unit for type parameter *a*. First, we throw an error message with the *throwError* function for which the input is of type *Text* (20). There are two possible *throwError* functions one from *Plutus.Trace.Emulator* module and the other from *Plutus.Contract* module. So, we have to specify which one we are using. Then we log a message (21), which should not be confused with the *w* type variable which uses also a *logInfo* function from the *Control.Monad.Freer.Extras* module. This is just simple logging. The *logInfo* function is polymorphic which means it can take as input parameters various types so we specify the type of the string as *@String*. It could also be of type *Text* since we imported the *Data.Text* module and the *Overloaded* extension. We test the contract with *myTrace1* (23). There we activate the contract wallet and do not care about the result (24) and in the test statement we run the trace (26-27). For this example, if we run it the log message from line 21 will not be shown because

the contract will stop when the error on line 20 is thrown. So, the execution of a contract is stopped at the point where an exception is raised.

In our next example we show how to catch and handle exceptions. We will run the contract 1 but catch the exception. Contract 2 has the `e` variable of `Void` which means that it can't throw an error because `void` does not contain any variables in contrast to `unit` which contains one variable also called `unit`. The *handleError* function has the following type signature:

```
handleError :: (e -> Contract w s e' a) -> Contract w s e a -> Contract w s e' a
```

It takes a function and a contract as input parameters and if there is no error it returns the result of type `a`, if there is however an error the function is applied to the error type variable and the contract produced by the function will be executed. When we call *handleError* we first provide the function (31) where the *err* variable is of type `Text` and the lambda function creates a contract where we only log the error which is now of type `string`. And then we add the first contract as input parameter. If we run now the *test2* action we will catch the error, which means contract 1 will not be executed and instead the contract from the lambda function is executed. Let's look now at the third example where we take the `s` type parameter in account.

First, we define a type synonym for the schema where endpoint types are declared (40). The "foo" and "bar" in these declarations are types not strings which is possible due to the *DataKinds* language extension. We use the type operator `".\."` that enables us to combine more endpoints and for that we need the *TypeOperators* language extension. Then we define contract 3 where we use the *endpoint* function.

```
endpoint :: (a -> Contract w s e b) -> Promise w s e b
```

This function takes in a function that changes the endpoint value `a` (in our case an *Int* or a *String*) to a contract of result type `b` and then returns a promise parameterized by the same types. A promise is a blocked contract waiting to be triggered by an outside stimulus, which would be in our case when a user tries to call the endpoint from his wallet and with an integer or a string as input. So, when we call the *endpoint* function, we first tell it which endpoint to invoke i.e., what will be the type of `a` (in our case an integer or a string) and then provide it with *logInfo* function which can take an integer or string and produce a contract that just logs that variable. With the *awaitPromise* function we turn a promise into a contract (44-45).

```
awaitPromise :: Promise w s e a -> Contract w s e a
```

In the trace action we now need the handle to be able to call an endpoint (49). The *callEndpoint* function takes in a contract handle and an endpoint value and then it invokes this endpoint.

```
callEndpoint :: ContractHandle w s e -> ep -> Eff effs ()
```

First, we need to specify which endpoint to call and then we pass in our two parameters (50-51). If we look now at the off-chain code of the *Vesting.hs* contract we see how the give and grab contracts are called inside the endpoint contract. What we didn't describe in this example is the use of the *select* function that is used in the vesting endpoint contract. It takes in two contracts and returns the contract that makes progress first, discarding the other one.

In the final example 4 we look at the *w* type parameter that is responsible for communication between different contracts and the outside world. The *w* type cannot be of an arbitrary type but a type that is an instance of the type class *monoid*. We will use a list of integers which is an instance of *monoid* (56). In the body of our contract, we first define a wait for 10 slots and throw away the result (58). Then we use the *tell* function that has following type signature:

```
tell :: w -> Contract w s e ()
```

So, we provide a writer and the *tell* function creates a contract. We repeat this step two times (58-62). In the emulator trace we first activate the contract and then wait for 5 slots (66-68). We look up the state of a running contract with the *observableState* function. As argument it takes a handle of the running contract. Then we log the result we get (70). When we test this example, we see that the first log returns just an empty list [], the second log returns [1] and the third log returns [1,2]. The state in the beginning is a *mempty* monoid and then each time you call the *tell* function the *mappend* function is used to update the state. On the real blockchain the user can interact with the contract by invoking endpoints and the contract can communicate back its state with the use of the *tell* function.

4.4 Homework

For this homework we want to write an emulator trace for a simple contract that pays some ADA to a give public key. The trace should take in 2 integer parameters that represent the amount of lovelace and it should call the pay contract twice. The recipient should be wallet two. Here is the code from the *Solution.hs* file.

```
1 {-# LANGUAGE DataKinds           #-}
2 {-# LANGUAGE DeriveAnyClass      #-}
3 {-# LANGUAGE DeriveGeneric       #-}
4 {-# LANGUAGE NumericUnderscores #-}
5 {-# LANGUAGE OverloadedStrings   #-}
6 {-# LANGUAGE TypeApplications    #-}
7 {-# LANGUAGE TypeOperators       #-}
```

```

8
9  module Week04.Solution where
10
11  import Data.Aeson           (FromJSON, ToJSON)
12  import Data.Functor         (void)
13  import Data.Text            (Text, unpack)
14  import GHC.Generics         (Generic)
15  import Ledger
16  import Ledger.Ada           as Ada
17  import Ledger.Constraints   as Constraints
18  import Plutus.Contract      as Contract
19  import Plutus.Trace.Emulator as Emulator
20  import Wallet.Emulator.Wallet
21
22  data PayParams = PayParams
23    { ppRecipient :: PaymentPubKeyHash
24    , ppLovelace  :: Integer
25    } deriving (Show, Generic, FromJSON, ToJSON)
26
27  type PaySchema = Endpoint "pay" PayParams
28
29  payContract :: Contract () PaySchema Text ()
30  payContract = do
31    pp <- awaitPromise $ endpoint @"pay" return
32    let tx = mustPayToPubKey (ppRecipient pp) $ lovelaceValueOf $
33          ppLovelace pp
34    handleError (\err -> Contract.logInfo $ "caught error: " ++ unpack err) $
35      void $ submitTx tx
36    payContract
37
38  payTrace :: Integer -> Integer -> EmulatorTrace ()
39  payTrace x y = do
40    h <- activateContractWallet (knownWallet 1) payContract
41    let pkh = mockWalletPaymentPubKeyHash $ knownWallet 2
42    callEndpoint @"pay" h $ PayParams
43      { ppRecipient = pkh
44      , ppLovelace  = x
45      }
46    void $ Emulator.waitNSlots 1
47    callEndpoint @"pay" h $ PayParams
48      { ppRecipient = pkh
49      , ppLovelace  = y
50      }
51    void $ Emulator.waitNSlots 1
52

```

```
51 payTest1 :: IO ()
52 payTest1 = runEmulatorTraceIO $ payTrace 10_000_000 20_000_000
53
54 payTest2 :: IO ()
55 payTest2 = runEmulatorTraceIO $ payTrace 1000_000_000 20_000_000
```

First, we create the type *PayParams* where we specify the recipient and the amount of lovelace we want to pay (22-25). Then we define the schema which has only the pay endpoint available. Next, we define our pay contract that is parameterized by our schema and uses the Text type for the error message. In the first line of our contract, we invoke the pay endpoint by immediately returning the contract input value without any side effects (31). This means that we return the input parameters defined in the *PayParams* data type that are in a contract monad context. Next, we create a transaction where we use the *mustPayToPubKey* function which takes in a public key hash (32). Then we handle the error when we submit the transaction. In our case the error will be triggered when we try to submit a transaction with a larger amount of lovelace than the wallet has available (33). The *submitTx* function tries to balance the transaction which means it's looking for available funds in our wallet and then also constructs a transaction that returns the change amount back to our wallet. In the last line we recursively call the contract again so that it is still running and the trace can call the payment endpoint again.

The trace will take in two parameters that represent the amount of lovelace we want to pay. First, we activate the contract and then we define the payment public key hash of wallet 2 (38-39). Then we call our endpoints with the give parameters and wait for one slot after each endpoint call (40-49). In the end we create to IO actions that run our emulator traces with 2 different values. In the first run both payments should succeed but in the second run the first payment should produce an error because it tries to spend more funds as there are available. This error should be caught and the second payment should still be processed. Note that for the input values we are using numbers with underscores which is possible because we imported the *NumericUnderscores* language extension.

5 Native tokens

To be able to work with native tokens and ADA we need the modules *Plutus.V1.Ledger.Ada* and *Plutus.V1.Ledger.Value*. First, we look at the value module where the type *Value* is defined.

Constructors

Value

```
getValue :: Map CurrencySymbol (Map TokenName Integer)
```

Figure 18 - Value type

Token name and currency symbol are just wrappers for the type *BuildingByteString* that represents a byte string. These two byte strings define a native token or coin. The integer represents the amount of the currency. There is also another type called *AssetClass* (Figure 19).

Constructors

AssetClass

```
unAssetClass :: (CurrencySymbol, TokenName)
```

Figure 19 – AssetClass type

It defines an asset class which is a native token or coin. A *Value* just says how many units of an asset class are contained in it. Now let's look at some examples from the Repl.

```
ghci> import Plutus.V1.Ledger.Value
ghci> import Plutus.V1.Ledger.Ada
ghci> :set -XoverloadedStrings
ghci> :t adaSymbol
adaSymbol :: CurrencySymbol
ghci> adaSymbol

ghci> :t adaToken
adaToken :: TokenName
ghci> adaToken
""

ghci> :t lovelaceValueOf
lovelaceValueOf :: Integer -> Value
ghci> lovelaceValueOf 123
Value (Map [(,Map [("",123)])])
```

We import the two modules and activate the language extension so we can enter byte strings as literal strings. Because both currency symbol and token name implement the *IsString* class we can enter both of them as literal strings. We see that the ADA currency symbol and token

name are just an empty byte string. We can create a value of 123 lovelace with the function *lovelaceValueOf*. Next, we combine values and also create values that include tokens.

```
ghci> lovelaceValueOf 123 <> lovelaceValueOf 10
Value (Map [(,Map [("",133)])])
ghci> :t singleton
singleton :: CurrencySymbol -> TokenName -> Integer -> Value
ghci> singleton "a8ff" "ABC" 7
Value (Map [(a8ff,Map [("ABC",7)])])
ghci> singleton "a8ff" "ABC" 7 <> lovelaceValueOf 42 <> singleton "a8ff" "XYZ" 100
Value (Map [(,Map [("",42)]),(a8ff,Map [("ABC",7),("XYZ",100)])])
ghci> let v = it
ghci> :t valueOf
valueOf :: Value -> CurrencySymbol -> TokenName -> Integer
ghci> valueOf v "a8ff" "XYZ"
100
ghci> valueOf v "a8ff" "abc"
0
ghci> :t flattenValue
flattenValue :: Value -> [(CurrencySymbol, TokenName, Integer)]
ghci> flattenValue v
[(, "",42),(a8ff,"XYZ",100),(a8ff,"ABC",7)]
```

Since Value is an instance of monoid and semigroup is a superclass of monoid, we can use the *<>* operator for combining values. With the *singleton* function we can construct a value of just one asset class. For the currency symbol we can't use an arbitrary string. Instead, we must use a hexadecimal value. We can also combine values that contain different native tokens and ADA. With the *it* keywords we get the previous result. The *valueOf* function extracts the given amount of lovelace or a token. The *flattenValue* function takes a value and returns a list of triples that contain information from the value parameter.

In general, a transaction can't delete or create tokens. The inputs equal the outputs if we also take in account fees that need to be paid. Fees depend on the size of a transaction in bytes and on the scripts that need to be run to validate a transaction. The script takes more fees if it consumes more memory. The minting policies are responsible for managing tokens. The hexadecimal value that is the currency symbol represents the hash of the minting policy script. If we want to create or burn tokens the minting script has to be contained in the transaction which is then executed together with the validation script. The purpose of the minting script is to decide whether the given transaction is allowed to mint or burn tokens. And for ADA which has an empty string for the script hash, that means that there is no script which would allow minting or burning of ADA. All the ADA that exists comes from the genesis block and from monetary expansion. After each epoch rewards are paid and parts of these rewards come from monetary expansion where certain percentage of remaining reserves gets paid as rewards. The total amount of ADA in the system is fixed, it can never change.

5.1 Simple Minting Policy

So far when we looked at the *ScriptPurpose* type variable of the *ScriptContext* data type we were only concerned with the *Spending* constructor that takes as input a transaction reference. In the transaction information *TxInfo* data type we can specify a *Value* data type for the *txInfoMint* variable. Minting policies are triggered if this field contains a non-zero value. For each currency symbol defined in this field the corresponding minting policy is triggered. A minting policy has only two inputs, the redeemer and the context. The script purpose is then set to *Minting*. All of the minting policies contained in a transaction have to pass in order that the transaction passes, otherwise it fails. Let's look at a simple example now.

```
1  import      GHC.Generics      (Generic)
2  import      Plutus.Contract   as Contract
3  import      Plutus.Trace.Emulator as Emulator
4  import qualified PlutusTx
5  import      PlutusTx.Prelude   hiding (Semigroup(..), unless)
6  import      Ledger             hiding (mint, singleton)
7  import      Ledger.Constraints as Constraints
8  import qualified Ledger.Typed.Scripts as Scripts
9  import      Ledger.Value       as Value
10 import      Playground.Contract (printJson, printSchemas, stage,
                                   ensureKnownCurrencies, ToSchema)
11 import      Playground.TH       (mkKnownCurrencies,
                                   mkSchemaDefinitions)
12 import      Playground.Types    (KnownCurrency (..))
13 import      Prelude            (IO, Show (..), String)
14 import      Text.Printf        (printf)
15 import      Wallet.Emulator.Wallet
16
17 {-# INLINABLE mkPolicy #-}
18 mkPolicy :: () -> ScriptContext -> Bool
19 mkPolicy () _ = True
20
21 policy :: Scripts.MintingPolicy
22 policy = mkMintingPolicyScript $(PlutusTx.compile
                                   [| Scripts.wrapMintingPolicy mkPolicy |])
23
24 curSymbol :: CurrencySymbol
25 curSymbol = scriptCurrencySymbol policy
26
27 data MintParams = MintParams
28   { mpTokenName :: !TokenName
29   , mpAmount     :: !Integer
30   } deriving (Generic, ToJSON, FromJSON, ToSchema)
```



```

31
32 type FreeSchema = Endpoint "mint" MintParams
33
34 mint :: MintParams -> Contract w FreeSchema Text ()
35 mint mp = do
36     let val      = Value.singleton curSymbol (mpTokenName mp) (mpAmount mp)
37         lookups = Constraints.mintingPolicy policy
38         tx      = Constraints.mustMintValue val
39     ledgerTx <- submitTxConstraintsWith @Void lookups tx
40     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
41     Contract.logInfo @String $ printf "forged %s" (show val)
42
43 endpoints :: Contract () FreeSchema Text ()
44 endpoints = mint' >> endpoints
45     where
46         mint' = awaitPromise $ endpoint @"mint" mint
47
48 mkSchemaDefinitions ''FreeSchema
49
50 mkKnownCurrencies []
51
52 test :: IO ()
53 test = runEmulatorTraceIO $ do
54     let tn = "ABC"
55     h1 <- activateContractWallet (knownWallet 1) endpoints
56     h2 <- activateContractWallet (knownWallet 2) endpoints
57     callEndpoint @"mint" h1 $ MintParams
58         { mpTokenName = tn
59         , mpAmount     = 555
60         }
61     callEndpoint @"mint" h2 $ MintParams
62         { mpTokenName = tn
63         , mpAmount     = 444
64         }
65     void $ Emulator.waitNSlots 1
66     callEndpoint @"mint" h1 $ MintParams
67         { mpTokenName = tn
68         , mpAmount     = -222
69         }
70     void $ Emulator.waitNSlots 1

```

First, we create the *mkPolicy* function that represents the policy and will also be compiled to Plutus Core (35-37). We use unit for the redeemer and return a book, which is used in the typed version. Our minting policy will always return *True* no matter the context.

Then we compile the minting policy (39-40). We use the *mkMintingPolicyScript* function to create the minting policy script and with the *wrapMintingPolicy* function we convert our typed version to an un-typed version before we compile it. Because we use this function, we also need to add the *INLINABLE* pragma before the *mkPolicy* function. Next, we can get the currency symbol with the function *scriptCurrencySymbol* where we need to provide as input the compiled minting policy. The output is the hash of the script. Similar to how we parameterized validators we could also do this with minting policies. This completes the on-chain part. For the off-chain part we first declare our minting parameters where we specify the token name and the amount (45-48). If the amount is a positive number, we mint tokens and if it is a negative number, we burn tokens. Then we define the schema with only one endpoint called mint (50). The mint contract takes minting parameters as input. In the body of the contract, we first compute the Value that we want to forge (54). As lookups we specify the minting policy. The only constrain we put on our transaction is *mustMintValue* function that takes as input the previously defined Value. We can skip the redeemer since it has the value of unit. When we submit the transaction, it will automatically transfer the minted value to the wallet if it is positive. If it is negative it will try to find sufficiently many tokens in the user's wallet that will then be burned. After that we wait for confirmation and then log a message. The endpoint contract then follows the same pattern as in previous examples. In the end we define a emulator trace where we mint and burn some tokens in wallet 1 and 2 (70-88). One thing to notice is that when we try this out on the playground the UTXO that gets assigned the tokens will also get 2 ADA assigned which is the minimum UTXO value of ADA. For the actual Cardano blockchain this value can be different.

5.2 More Realistic Minting Policy

Now we will look at a minting policy that is parameterized by a payment public key hash. The minting or burning of the tokens will only be allowed if the owner of that key has signed the transaction. Here is the code from the *Signed.hs* example.

```
1 {-# INLINABLE mkPolicy #-}
2 mkPolicy :: PaymentPubKeyHash -> () -> ScriptContext -> Bool
3 mkPolicy pkh () ctx = txSignedBy (scriptContextTxInfo ctx) $
                        unPaymentPubKeyHash pkh
4
5 policy :: PaymentPubKeyHash -> Scripts.MintingPolicy
6 policy pkh = mkMintingPolicyScript $
7     $(PlutusTx.compile [|| Scripts.wrapMintingPolicy . mkPolicy |])
8     `PlutusTx.applyCode`
9     PlutusTx.liftCode pkh
```

```

10
11 curSymbol :: PaymentPubKeyHash -> CurrencySymbol
12 curSymbol = scriptCurrencySymbol . policy
13
14 data MintParams = MintParams
15     { mpTokenName :: !TokenName
16     , mpAmount     :: !Integer
17     } deriving (Generic, ToJSON, FromJSON, ToSchema)
18
19 type FreeSchema = Endpoint "mint" MintParams
20
21 mint :: MintParams -> Contract w FreeSchema Text ()
22 mint mp = do
23     pkh <- Contract.ownPaymentPubKeyHash
24     let val      = Value.singleton (curSymbol pkh) (mpTokenName mp)
25                                     (mpAmount mp)
26     lookups = Constraints.mintingPolicy $ policy pkh
27     tx      = Constraints.mustMintValue val
28     ledgerTx <- submitTxConstraintsWith @Void lookups tx
29     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
30     Contract.logInfo @String $ printf "forged %s" (show val)

```

We add only the functions and parameters that changed compared to the previous *Free.hs* example. First, we create our policy function that takes now the payment public key hash as an additional parameter (1-3). We use the *txSignedBy* function to check whether the transaction was signed with the given public key. Here is its type signature.

```
txSignedBy :: TxInfo -> PubKeyHash -> Bool
```

Next, we compile the policy function where we use now the same pattern as in our *Parameterized.hs* code (5-9). Then we create the currency symbol which now takes in the public key as a parameter (11-12). For the off-chain code we declare the minting parameters and the schema the same way as in the *Free.hs* code. And for the mint function what changes is that we first look up our own payment public key hash for which we use the function from the *Contract* module to avoid collision for the same function from another module. Then we also change the value and the lookups declaration. When defining the value, we add to the currency symbol our public payment key hash as input parameter and for the lookups we do the same for the policy parameter (24-25).



When writing Plutus code, try to compile it as often as possible to avoid multiple errors at the end when running it in the playground.

If we run now the test trace, we notice something interesting. When checking the balances of the wallets we see that the currency symbol for the token that wallet one and two got, is different. That was not the case with our first example. The token name is still the same. The reason for this is that the minting script is now longer a constant but is a function so the same is true for the hash of the script which represents the currency symbol. And because the input parameter is the wallet public payment key hash which is different for wallet 1 and wallet 2 also the currency symbol for those two wallets gets computed differently. So the wallets contain now two different tokens or asset classes.

5.3 NFT-s

Non fungible tokens can exist only once, which means there is only one token in existence. This means we have to put a constraint on our transaction and say that only one token is allowed to be minted and that the transaction that triggers the mint can be executed only once. In order to do that we need something on the Cardano blockchain that we can refer to in our minting policy and is unique i.e., it exists only in one transaction. And the trick is we use a UTXO, since it can exist only once and when consumed in a transaction it is never again available. If there is another UTXO sitting at the same address with the same value and datum it still has another ID. We identify an UTXO with the transaction ID that produced it and the index it got assigned, since a transaction can produce more than one UTXO. And transactions are unique; there can never be the same transaction again. They are unique because they spend fees which come from UTXOs and they themselves are unique. This brings us to an induction. So, the idea is that we name a specific UTXO as parameter to our minting policy and then we check that the transaction that does the minting consumes this UTXO. Let's look at the *NFT.hs* code.

```
1  -- import Playground.Contract (printJson, printSchemas,
                                ensureKnownCurrencies, stage, ToSchema)
2  -- import Playground.TH       (mkKnownCurrencies, mkSchemaDefinitions)
3  -- import Playground.Types    (KnownCurrency (..))
4  import qualified Data.Map as Map
5  import           Prelude    (IO, Semigroup (..), Show (..), String)
6
7  {-# INLINABLE mkPolicy #-}
8  mkPolicy :: TxOutRef -> TokenName -> () -> ScriptContext -> Bool
9  mkPolicy oref tn () ctx = traceIfFalse
                                "UTxO not consumed" hasUTxO &&
10                                traceIfFalse "wrong amount minted"
                                    checkMintedAmount
11  where
12      info :: TxInfo
```

```

13     info = scriptContextTxInfo ctx
14
15     hasUTxO :: Bool
16     hasUTxO = any (\i -> txInInfoOutRef i == oref) $ txInfoInputs info
17
18     checkMintedAmount :: Bool
19     checkMintedAmount = case flattenValue (txInfoMint info) of
20         [(_, tn', amt)] -> tn' == tn && amt == 1
21         -                -> False
22
23     policy :: TxOutRef -> TokenName -> Scripts.MintingPolicy
24     policy oref tn = mkMintingPolicyScript $
25         $$ (PlutusTx.compile [| \oref' tn' -> Scripts.wrapMintingPolicy $
26             mkPolicy oref' tn' |])
27             `PlutusTx.applyCode`
28             PlutusTx.liftCode oref
29             `PlutusTx.applyCode`
30             PlutusTx.liftCode tn
31
32     curSymbol :: TxOutRef -> TokenName -> CurrencySymbol
33     curSymbol oref tn = scriptCurrencySymbol $ policy oref tn
34
35     data NFTParams = NFTParams
36         { npToken    :: !TokenName
37         , npAddress  :: !Address
38         } deriving (Generic, FromJSON, ToJSON, Show)
39
40     type NFTSchema = Endpoint "mint" NFTParams
41
42     mint :: NFTParams -> Contract w NFTSchema Text ()
43     mint np = do
44         utxos <- utxosAt $ npAddress np
45         case Map.keys utxos of
46             [] -> Contract.logError @String "no utxo found"
47             oref : _ -> do
48                 let tn      = npToken np
49                 let val     = Value.singleton (curSymbol oref tn) tn 1
50                 let lookups = Constraints.mintingPolicy (policy oref tn) <>
51                     Constraints.unspentOutputs utxos
52                 let tx      = Constraints.mustMintValue val <>
53                     Constraints.mustSpendPubKeyOutput oref
54                 ledgerTx <- submitTxConstraintsWith @Void lookups tx
55                 void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
56                 Contract.logInfo @String $ printf "forged %s" (show val)

```

```

55 endpoints :: Contract () NFTSchema Text ()
56 endpoints = mint' >> endpoints
57   where
58     mint' = awaitPromise $ endpoint @"mint" mint
59
60 test :: IO ()
61 test = runEmulatorTraceIO $ do
62   let tn = "ABC"
63       w1 = knownWallet 1
64       w2 = knownWallet 2
65   h1 <- activateContractWallet w1 endpoints
66   h2 <- activateContractWallet w2 endpoints
67   callEndpoint @"mint" h1 $ NFTParams
68     { npToken    = tn
69     , npAddress  = mockWalletAddress w1
70     }
71   callEndpoint @"mint" h2 $ NFTParams
72     { npToken    = tn
73     , npAddress  = mockWalletAddress w2
74     }
75   void $ Emulator.waitNSlots 1

```

Compared to our previous code example we do not use the Playground modules and we add the Map module and add Semigroup to the Prelude module. In the *mkPolicy* function we have now 2 additional parameters, the transaction output reference and the token name (8). In the body of this function, we check that the UTXO is consumed and that we mint only 1 token. For the helper functions we first get the script context info. Then for the *hasUTxO* parameter we use the *txInInfoInputs* helper function on the *info* parameter that gives us a list of transaction inputs that are of type *TxInInfo* (15-16). This type has the following structure (Figure 20).

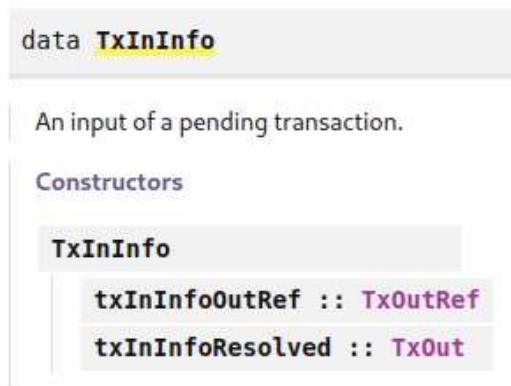


Figure 20 – TxInInfo type

The *TxOutRef* type can be seen in Figure 5. It holds the transaction ID and a reference index that together uniquely identify a UTXO. The *TxOut* data type you can see in Figure 21.

data TxOut

A transaction output, consisting of a target address, a value, and optionally a datum hash.

Constructors

TxOut

txOutAddress :: Address

txOutValue :: Value

txOutDatumHash :: Maybe DatumHash

Figure 21 – TxOut type

It holds the transaction output address and value for the same UTXO and maybe a datum hash. Maybe because the datum is only present if we have a script address but for normal public key addresses the value would be Nothing. The Address type you can see in Figure 22.

data Address

Address with two kinds of credentials, normal and staking.

Constructors

Address

addressCredential :: Credential

addressStakingCredential :: Maybe StakingCredential

Figure 22 - Address type

We can look at *Credential* type in Figure 23. It holds a public key hash and a validator hash that are of type *BuiltinByteString*.

data Credential

Source

Credential required to unlock a transaction output

Constructors

PubKeyCredential PubKeyHash

The transaction that spends this output must be signed by the private key

ScriptCredential ValidatorHash

The transaction that spends this output must include the validator script and be accepted by the validator.

Figure 23 - Credential type

Then we can look up the *StakingCredential* type (Figure 24). It holds again a *Credential* type and a staking pointer that is represented by three integers.

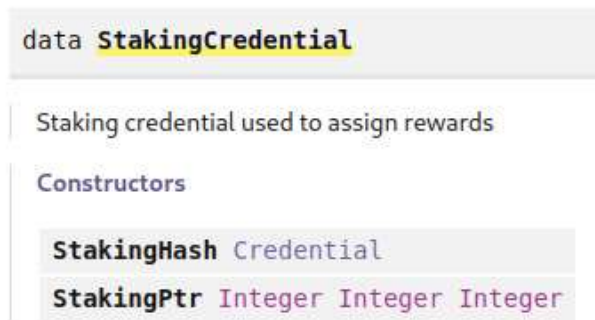


Figure 24 – StakingCredential type

For the *checkMintedAmount* parameter we check that the list we get from the minted info field contains only one triple and that the token name matches and quantity is equal to 1 (18-21). Next, we define our policy that has now the transaction output reference and token name as input parameters and same holds true for the currency symbol (23-32). For the off-chain code we create our NFT parameters type variable where we define the token name and the contract address (34-37). In the schema we define only one endpoint called mint (39). In the body of the mint contract, we first lookup all the UTXOs sitting at the given address from the input parameter (43). In the map that we get the keys that represent the *txOutRefs*. If there are none, we log an error message and if there is at least one, we take the 1st one (it could be any one). After that we get the token name and define the value, lookups and the transaction. For the currency symbol and policy, we have to provide our input parameters. For the lookups in the *unspentOutputs* function we could specify only the one UTXO that we need but we can also give a Map of UTXOs and the one we need just has to be contained in the Map (49). For the transaction we put a constraint that the given value must be minted and the transaction we are constructing spends the given output reference for our UTXO (50). This UTXO will come from our own wallet to we are allowed to spend it. This line of code ensures that the transaction can be performed only once, since we are referencing in a transaction a UTXO that will be unavailable after this transaction completes. After that we submit the transaction, wait for confirmation and log a message. Our endpoints definition follows the same pattern as before (55-58). In our emulator trace we define a token name and start the contract for wallet one and wallet two. Then we call the mint endpoint for the first and second wallet and wait for one slot that the transactions can be processed. The *mockWalletAddress* function takes as input a wallet and produces an address. This works only for playground wallets. And again, when we run the code, we end up with two different currency symbols which means the NFTs that wallet 1 and 2 contain are unique.

5.4 Homework

For the first homework we want to implement a Marry era style monetary policy. What you are allowed to do is that you can specify signatures that have to be present in the minting transaction and specify the deadline that says minting can only happen before a certain deadline. So, we want to create a minting policy that has 2 parameters, a public key hash and POSIX time. And the transaction should only succeed if it is signed by the corresponding signature and if the deadline has not passed. Let's look at the code *Solution1.hs*.

```
1  import Data.Default      (Default (..))
2  import Ledger.TimeSlot
3  import Prelude           (IO, Semigroup (..), Show (..), String)
4
5  {-# INLINABLE mkPolicy #-}
6  -- This policy should only allow minting (or burning) of tokens if the owner
7  -- of the specified PaymentPubKeyHash has signed the transaction and if the
8  -- specified deadline has not passed.
9  mkPolicy :: PaymentPubKeyHash -> POSIXTime -> () -> ScriptContext -> Bool
10 mkPolicy pkh deadline () ctx =
11     traceIfFalse "signature missing" (txSignedBy info $
12                                     unPaymentPubKeyHash pkh) &&
13     traceIfFalse "deadline missed"   (to deadline `contains`
14                                     txInfoValidRange info)
15
16 where
17     info = scriptContextTxInfo ctx
18
19 policy :: PaymentPubKeyHash -> POSIXTime -> Scripts.MintingPolicy
20 policy pkh deadline = mkMintingPolicyScript $
21     $$ (PlutusTx.compile [| \pkh' deadline' -> Scripts.wrapMintingPolicy $
22                             mkPolicy pkh' deadline' |])
23     `PlutusTx.applyCode`
24     PlutusTx.liftCode pkh
25     `PlutusTx.applyCode`
26     PlutusTx.liftCode deadline
27
28 curSymbol :: PaymentPubKeyHash -> POSIXTime -> CurrencySymbol
29 curSymbol pkh deadline = scriptCurrencySymbol $ policy pkh deadline
30
31 data MintParams = MintParams
32     { mpTokenName :: !TokenName
33     , mpDeadline  :: !POSIXTime
34     , mpAmount    :: !Integer
35     } deriving (Generic, ToJSON, FromJSON, ToSchema)
```

```

32 type SignedSchema = Endpoint "mint" MintParams
33
34 mint :: MintParams -> Contract w SignedSchema Text ()
35 mint mp = do
36     pkh <- Contract.ownPaymentPubKeyHash
37     now <- Contract.currentTime
38     let deadline = mpDeadline mp
39     if now > deadline
40     then Contract.logError @String "deadline passed"
41     else do
42         let val      = Value.singleton (curSymbol pkh deadline)
43                                     (mpTokenName mp) (mpAmount mp)
44         lookups = Constraints.mintingPolicy $ policy pkh deadline
45         tx      = Constraints.mustMintValue val <>
46                     Constraints.mustValidateIn (to $ now + 60000)
47         ledgerTx <- submitTxConstraintsWith @Void lookups tx
48         void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
49         Contract.logInfo @String $ printf "forged %s" (show val)
50
51 endpoints :: Contract () SignedSchema Text ()
52 endpoints = mint' >> endpoints
53 where
54     mint' = awaitPromise $ endpoint @"mint" mint
55
56 mkSchemaDefinitions ''SignedSchema
57
58 mkKnownCurrencies []
59
60 test :: IO ()
61 test = runEmulatorTraceIO $ do
62     let tn      = "ABC"
63     deadline = slotToBeginPOSIXTime def 100
64     h <- activateContractWallet (knownWallet 1) endpoints
65     callEndpoint @"mint" h $ MintParams
66         { mpTokenName = tn
67         , mpDeadline  = deadline
68         , mpAmount    = 555
69         }
70     void $ Emulator.waitNSlots 110
71     callEndpoint @"mint" h $ MintParams
72         { mpTokenName = tn
73         , mpDeadline  = deadline
74         , mpAmount    = 555
75         }
76     void $ Emulator.waitNSlots 1

```

Compared to the *Signed.hs* code example we make two new imports and add the Semigroup type class to the Prelude import. Then we define our policy function where we take 2 input parameters: the payment public key hash and the deadline (8). Next, we check that the correct signature is present and that the validity interval is before now (9-13). After that we compile the policy function and define the currency symbol (15-24). For the off-chain code we first define our minting parameters that include the token name, deadline and amount (26-30). For the schema we have only the mint endpoint. In the mint contract we get our own public key hash and the current POSIX time. If the current time is before the deadline we proceed with actions and if not, we just log a message. First, we define the value where the currency symbol takes in our two parameters (42). Same is true for policy parameter in the lookups variable (43). When we construct the transaction, we say which value should be minted and define the validity interval for the transaction (44). Then we submit the transaction, wait for confirmation and log a message. We define our endpoints contract as usual and create a schema definition and some currencies for the playground. In the emulator trace we define our token name and deadline. Then we activate the contract for wallet one and call our first endpoint. After that we wait for 110 slots and then call again the endpoint. This second call should fail since the deadline will already pass and we should not be able to mint any tokens. An interesting observation we can mention is that the validity interval in the mint contract is hardcoded to 60 seconds. If we change the deadline in the emulator trace to 61 slots the minting fails, but for 62 slots it succeeds. For our second homework we want to create a similar minting policy as in the *NFT.hs* example where we fix the token name to an empty byte string. Let's look at the code example from *Solution2.hs*.

```
1 {-# INLINABLE tn #-}
2 tn :: TokenName
3 tn = TokenName emptyByteString
4
5 {-# INLINABLE mkPolicy #-}
6 -- Minting policy for an NFT, where the minting transaction must consume the
7 -- given UTxO as input and where the TokenName will be the empty ByteString.
8 mkPolicy :: TxOutRef -> () -> ScriptContext -> Bool
9 mkPolicy oref () ctx = traceIfFalse "UTxO notconsumed" hasUTxO &&
10                        traceIfFalse "wrong amount minted" checkMintedAmount
11 where
12     info :: TxInfo
13     info = scriptContextTxInfo ctx
14
15     hasUTxO :: Bool
16     hasUTxO = any (\i -> txInInfoOutRef i == oref) $ txInfoInputs info
17
```

```

18     checkMintedAmount :: Bool
19     checkMintedAmount = case flattenValue (txInfoMint info) of
20         [(cs, tn', amt)] -> cs == ownCurrencySymbol ctx &&
21             tn' == tn && amt == 1
22         _ -> False
23
24 policy :: TxOutRef -> Scripts.MintingPolicy
25 policy oref = mkMintingPolicyScript $
26     $$ (PlutusTx.compile [| Scripts.wrapMintingPolicy . mkPolicy |])
27     `PlutusTx.applyCode`
28     PlutusTx.liftCode oref
29
30 curSymbol :: TxOutRef -> CurrencySymbol
31 curSymbol = scriptCurrencySymbol . policy
32
33 type NFTSchema = Endpoint "mint" Address
34
35 mint :: Address -> Contract w NFTSchema Text ()
36 mint addr = do
37     utxos <- utxosAt addr
38     case Map.keys utxos of
39     [] -> Contract.logError @String "no utxo found"
40     oref : _ -> do
41         let val = Value.singleton (curSymbol oref) tn 1
42             lookups = Constraints.mintingPolicy (policy oref) <>
43                 Constraints.unspentOutputs utxos
44             tx = Constraints.mustMintValue val <>
45                 Constraints.mustSpendPubKeyOutput oref
46         ledgerTx <- submitTxConstraintsWith @Void lookups tx
47         void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
48         Contract.logInfo @String $ printf "forged %s" (show val)
49
50 endpoints :: Contract () NFTSchema Text ()
51 endpoints = mint' >> endpoints
52 where
53     mint' = awaitPromise $ endpoint @"mint" mint
54
55 test :: IO ()
56 test = runEmulatorTraceIO $ do
57     let w1 = knownWallet 1
58         w2 = knownWallet 2
59     h1 <- activateContractWallet w1 endpoints
60     h2 <- activateContractWallet w2 endpoints
61     callEndpoint @"mint" h1 $ mockWalletAddress w1
62     callEndpoint @"mint" h2 $ mockWalletAddress w2

```

```
60 void $ Emulator.waitNSlots 1
```

First, we define our token name which is an empty byte string. Then we define our policy function that takes now as input only the transaction output reference (8). The first helper function that checks the transaction output reference is the same as in the NFT example. The second helper function however changes. There we can now compare the currency symbol, the token name and the amount (18-20). The token name is now a fixed variable rather than an input parameter. We compile our policy where we have now only one parameter and define the currency symbol (23-30). Next comes the off-chain code. When defining the schema, we take as input only the address which is also the input for the mint contract. The mint contract compared to the NFT example is very similar. What changes is the definition of the value and lookups variables where we provide for the currency symbol and the policy only one parameter (40-41). In the emulator trace we now do not need to define a token name but only pass in the address as input when calling the endpoint.

6 Deployment

In this lecture we will get to know the PAB - Plutus application backend. The PAB provides the components and an environment to help developers create and test DApps (decentralized applications), before deploying them to a live production environment. The PAB is a single Haskell library that makes it easier to write the off-chain infrastructure and the on-chain scripts [5]. We will see how to use the PAB to interact with contracts on the Cardano testnet and show an example how to mint native tokens on the testnet using the Cardano CLI and PAB.

6.1 The minting policy

In this chapter we will present the on-chain code found in the file `week06/Token/OnChain.hs`. The code is similar to the NFT example. Let's look at the code.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass       #-}
3  {-# LANGUAGE DeriveGeneric        #-}
4  {-# LANGUAGE FlexibleContexts     #-}
5  {-# LANGUAGE NoImplicitPrelude    #-}
6  {-# LANGUAGE NumericUnderscores  #-}
7  {-# LANGUAGE OverloadedStrings   #-}
8  {-# LANGUAGE ScopedTypeVariables  #-}
9  {-# LANGUAGE TemplateHaskell      #-}
10 {-# LANGUAGE TypeApplications     #-}
11 {-# LANGUAGE TypeFamilies         #-}
12 {-# LANGUAGE TypeOperators        #-}
13
14 module Week06.Token.OnChain
15   ( tokenPolicy
16   , tokenCurSymbol
17   ) where
18
19 import qualified PlutusTx
20 import          PlutusTx.Prelude           hiding (Semigroup(..), unless)
21 import          Ledger                     hiding (mint, singleton)
22 import qualified Ledger.Typed.Scripts      as Scripts
23 import          Ledger.Value               as Value
24
25 {-# INLINABLE mkTokenPolicy #-}
26 mkTokenPolicy :: TxOutRef -> TokenName -> Integer ->
27               () -> ScriptContext -> Bool
28 mkTokenPolicy oref tn amt () ctx = traceIfFalse "UTxO not consumed" hasUTxO
29                                     && traceIfFalse "wrong amount minted" checkMintedAmount
```

```

29  where
30      info :: TxInfo
31      info = scriptContextTxInfo ctx
32
33      hasUTxO :: Bool
34      hasUTxO = any (\i -> txInInfoOutRef i == oref) $ txInfoInputs info
35
36      checkMintedAmount :: Bool
37      checkMintedAmount = case flattenValue (txInfoMint info) of
38          [(_, tn', amt')] -> tn' == tn && amt' == amt
39          _                 -> False
40
41  tokenPolicy :: TxOutRef -> TokenName -> Integer -> Scripts.MintingPolicy
42  tokenPolicy oref tn amt = mkMintingPolicyScript $
43      $$ (PlutusTx.compile [| \oref' tn' amt' -> Scripts.wrapMintingPolicy $
44          mkTokenPolicy oref' tn' amt' |])
45      `PlutusTx.applyCode`
46      PlutusTx.liftCode oref
47      `PlutusTx.applyCode`
48      PlutusTx.liftCode tn
49      `PlutusTx.applyCode`
50      PlutusTx.liftCode amt
51
52  tokenCurSymbol :: TxOutRef -> TokenName -> Integer -> CurrencySymbol
53  tokenCurSymbol oref tn = scriptCurrencySymbol . tokenPolicy oref tn

```

First, we define our token policy that takes in three additional parameters: the transaction output reference, token name and the number of tokens we want to create. In the NFT example the amount was not necessary since we wanted to create only one token. In the body of the function, we check that the referenced UTXO is consumed and that the token name and quantity match (27-39). Then we compile our token policy function and create the currency symbol. All together this code represents the on-chain part.

6.2 Minting with the CLI

We will use now the command line interface to mint tokens. If we recall chapter 3 where we already used the CLI we remember that we needed the serialized script that will now be computed from our minting policy instead of a validator. If we check in the documentation for the type *MintingPolicy* we see it's just a wrapper around the *Script* type (Figure 25). Also, the type *Validator* is a wrapper around the script type. We define various helper functions in the *Utils.hs* file. There we have the *writeMintingPolicy* function that will create our serialized script.

```
newtype MintingPolicy
```

`MintingPolicy` is a wrapper around `Scripts` which are used as validators for minting constraints.

Constructors

```
MintingPolicy
```

```
getMintingPolicy :: Script
```

Figure 25 – MintingPolicy type

Now we focus on the CLI. You can use the `week06/testnet` folder where the configuration files are already included. The bash scripts we will use in this example you can find directly in the `week06` folder. First start the cardano node and let it sync to the testnet.

```
$ ./start-testnet-node.sh
```

Next, we go into the `testnet/` folder and create a key and a payment addresses. The code is the same as in chapter 3.

```
$ caradno-cli address key-gen --verification-key-file 01.vkey --signing-key-file 01.skey  
$ cardano-cli address build --payment-verification-key-file 01.vkey --testnet-magic  
1097911063 -out-file 01.addr
```

You will again need the Faucet to send some test ADA to your address. The script `env.sh` sets all important environment variables that we will use. Once you source it you can use other bash scripts. The address and wallet ID parameters will be used later in the PAB chapter.

```
$ . env.sh
```

With the `query-key1.sh` script we can get the list of UTXOs sitting at our address together with the amounts of test ADA.

```
$ ./query-key1.sh
```

	TxHash	TxIx	Amount
-----	80e62eea8e5079598e4f341c704ef8957fb4d33f9818939aeda78a959a42dbe1	0	989834279
	lovelace + TxOutDatumNone		

As we said a UTXO is defined with the transaction hash followed by the `#` symbol and then the transaction index. We can use this string and convert it to a transaction output reference with the helper function `unsafeReadTxOutRef` that is in the `Utils` module. And with the code from the file `app/token-policy.hs` we can create the minting policy script in serialized form. This module is added to the cabal file so we can run it with the following command:


```
$ cabal exec token-policy -- policy.plutus TxHash#TxIx 123456 PPP
```

You will need to provide the transaction hash and index that you got from the query command in the file TxHash#TxIx. We create an amount of 123456 tokens with name PPP. Now we can actually mint the token and we will use the *mint-token-cli.sh* script for that. It takes in 5 parameters: the transaction output reference, amount, token name, address file and the signing key file. Next it gets the protocol parameters for which we use the protocol-parameters command from the Cardano CLI. After that it creates the token policy file. Then we compute the policy id called pid and the token name in hexadecimal format which is a requirement from the Cardano CLI. The function that can convert a token name to the hexadecimal format is called *unsafeTokenNameToHex* and can be found in the Utils module. In the bash script we execute the *token-name.hs* script to get the token name in hexadecimal format. The parameter *v* represents the value we want to mint and the CLI uses the convention that we first specify the amount and then the pid and hex token name separated by a dot.

Next come the actual transaction commands. With the transaction build command, we construct our transaction where we input various data as in the example, we showed in chapter 3. The difference is that we now specify 3 mint parameters: the value, the script file and the redeemer file. For the tx-out parameter we specify some ADA which should be greater than the minimal amount of ADA required for a UTXO. What's important to notice here is that the minimal amount is not a constant but is rather calculated from size of the output in bytes. In the end we specify where to write the unsigned balanced transaction to. Then we use the CLI commands to sign and submit the transaction. We mint the token with the following command.

```
$ ./mint-token-cli.sh <TxHash#TxIx> 123456 PPP testnet/01.addr testnet/01.skey
```

After we run the command, we can wait for a few seconds and then run again the query-key1.sh script to check if the token got minted. We can also go to explorer.cardano-testnet.iohkdev.io which is a cardano blockchain explorer for the cardano testnet. There we have to input our transaction ID and we will see transaction information such as how many ada was sent from which to which address and what tokens were minted. We could do the same procedure on the main net where we would replace the Magic ID with --mainnet.

6.3 Deployment Scenarios

Let's look at the deployment models for the Plutus application backend - PAB. There are two deployment models envisioned for the PAB: Hosted and in-browser. We will use the hosted

model. It works by running a cardano node on a server, a cardano wallet backend which for example is used by the Daedalus wallet, a chain index and the PAB itself.

The chain index handles saving of the blockchain information in an SQL database. It can be used for instance to lookup a datum belonging to a datum hash, which the cardano node can't do. Our PAB should have access to our wallet so an external user of our dApp would then interact with our PAB through a user interface that has endpoints available (Figure 26).

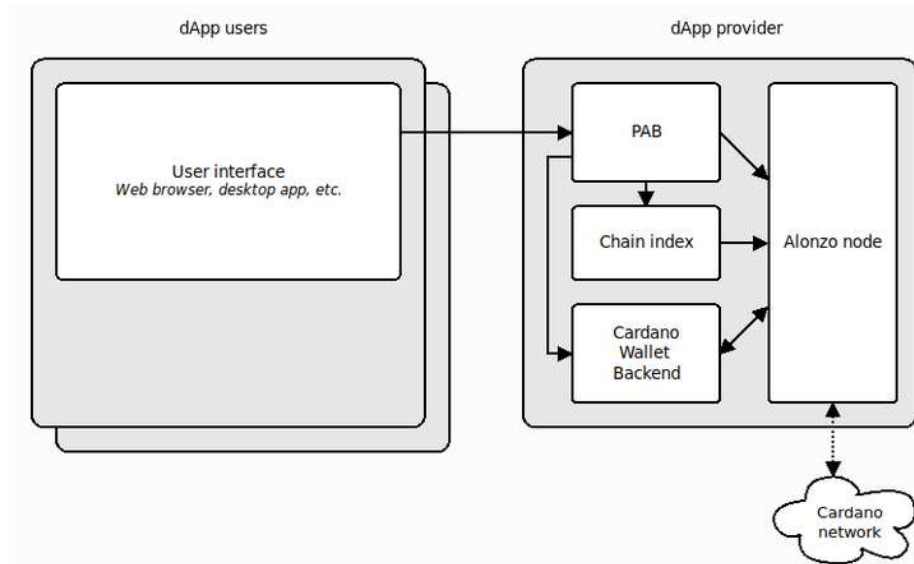


Figure 26 - dApp schema

6.4 The Contracts

We will now define some off-chain code written in a contract monad that the PAB can execute. First, we look at the *getCredentials* helper function defined in the Utils module. It takes in a Plutus Address type (Figure 22). The *getCredentials* function returns a Nothing if it the input is a script address and a Just value if it is a public key address that contains a payment public key hash and maybe a stake public key hash. Let's look now at the off-chain code contained in the *OffChain.hs* file.

```

1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass      #-}
3  {-# LANGUAGE DeriveGeneric       #-}
4  {-# LANGUAGE FlexibleContexts    #-}
5  {-# LANGUAGE NoImplicitPrelude   #-}
6  {-# LANGUAGE OverloadedStrings   #-}
7  {-# LANGUAGE ScopedTypeVariables #-}
8  {-# LANGUAGE TypeApplications   #-}
9  {-# LANGUAGE TypeFamilies        #-}
10

```

```

11 module Week06.Token.OffChain
12   ( TokenParams (..)
13     , adjustAndSubmit, adjustAndSubmitWith
14     , mintToken
15   ) where
16
17   import           Control.Monad           hiding (fmap)
18   import           Data.Aeson              (FromJSON, ToJSON)
19   import qualified Data.Map                as Map
20   import           Data.Maybe              (fromJust)
21   import           Data.OpenApi.Schema     (ToSchema)
22   import           Data.Text               (Text, pack)
23   import           Data.Void               (Void)
24   import           GHC.Generics            (Generic)
25   import           Plutus.Contract         as Contract
26   import           Plutus.Contract.Wallet  (getUnspentOutput)
27   import qualified PlutusTx
28   import           PlutusTx.Prelude       hiding (Semigroup(..), unless)
29   import           Ledger                  hiding (mint, singleton)
30   import           Ledger.Constraints      as Constraints
31   import qualified Ledger.Typed.Scripts    as Scripts
32   import           Ledger.Value            as Value
33   import           Prelude                 (Semigroup (..), Show (..),
34                                           String)
35
36   import qualified Prelude
37   import           Text.Printf              (printf)
38
39   import           Week06.Token.OnChain
40   import           Week06.Utills            (getCredentials)
41
42   data TokenParams = TokenParams
43     { tpToken      :: !TokenName
44     , tpAmount     :: !Integer
45     , tpAddress    :: !Address
46     } deriving (Prelude.Eq, Prelude.Ord, Generic, FromJSON, ToJSON,
47               ToSchema, Show)
48
49   adjustAndSubmitWith :: ( PlutusTx.FromData (Scripts.DatumType a)
50                           , PlutusTx.ToData (Scripts.RedeemerType a)
51                           , PlutusTx.ToData (Scripts.DatumType a)
52                           , AsContractError e
53                         )
54     => ScriptLookups a
55     -> TxConstraints (Scripts.RedeemerType a)
56                   (Scripts.DatumType a)

```

```

53     -> Contract w s e CardanoTx
54 adjustAndSubmitWith lookups constraints = do
55     unbalanced <- adjustUnbalancedTx <$> mkTxConstraints lookups constraints
56     Contract.logDebug @String $ printf "unbalanced: %s" $ show unbalanced
57     unsigned <- balanceTx unbalanced
58     Contract.logDebug @String $ printf "balanced: %s" $ show unsigned
59     signed <- submitBalancedTx unsigned
60     Contract.logDebug @String $ printf "signed: %s" $ show signed
61     return signed
62
63 adjustAndSubmit :: ( PlutusTx.FromData (Scripts.DatumType a)
64                    , PlutusTx.ToData (Scripts.RedeemerType a)
65                    , PlutusTx.ToData (Scripts.DatumType a)
66                    , AsContractError e
67                    )
68                => Scripts.TypedValidator a
69                -> TxConstraints (Scripts.RedeemerType a)
70                    (Scripts.DatumType a)
71                -> Contract w s e CardanoTx
72
73 adjustAndSubmit inst = adjustAndSubmitWith $
74     Constraints.typedValidatorLookups inst
75
76 mintToken :: TokenParams -> Contract w s Text CurrencySymbol
77 mintToken tp = do
78     Contract.logDebug @String $ printf "started minting: %s" $ show tp
79     let addr = tpAddress tp
80     case getCredentials addr of
81     Nothing      -> Contract.throwError $ pack $ printf
82         "expected pubkey address, but got %s" $ show addr
83     Just (x, my) -> do
84         oref <- getUnspentOutput
85         o    <- fromJust <$> Contract.txOutFromRef oref
86         Contract.logDebug @String $ printf "picked UTXO at %s with value
87             %s" (show oref) (show $ _ciTxOutValue o)
88
89         let tn      = tpToken tp
90             amt     = tpAmount tp
91             cs      = tokenCurSymbol oref tn amt
92             val     = Value.singleton cs tn amt
93             c       = case my of
94                 Nothing -> Constraints.mustPayToPubKey x val
95                 Just y  -> Constraints.mustPayToPubKeyAddress x y val
96             lookups  = Constraints.mintingPolicy
97                 (tokenPolicy oref tn amt) <>
98                 Constraints.unspentOutputs

```

```

93             (Map.singleton oref o)
94         constraints = Constraints.mustMintValue val      <>
95                     Constraints.mustSpendPubKeyOutput oref <> c
96     void $ adjustAndSubmitWith @Void lookups constraints
97     Contract.logInfo @String $ printf "minted %s" (show val)
98     return cs

```

First, we define our token parameter type that holds the token name, amount and the address. This is not the change address but the address where the tokens should be sent after the minting. The change address will then be picked by the wallet.

If we look at the *mintToken* function we see it takes in the token parameters and returns a contract parameterized with the currency symbol. We won't really need this since this was used in an oracle example from a previous lecture of the plutus pioneer program. First, we log that we are starting the mint (75) and then we define the address. Then we use the *getCredentials* helper function (77). If it returns *Nothings* which means we got a script address we raise an error. If it's not *nothing* we extract the payment public key hash and optionally the staking key hash. Next, we use the helper function *getUnspentOutput* that looks for an unspent UTXO in our wallet and returns its chain index transaction output (80). With the function *txOutFromRef* we get the actual output for which we can be sure at this moment that it exists (81).

```
txOutFromRef :: TxOutRef -> Contract w s e (Maybe ChainIndexTxOut)
```

Then we log some information about the output and its value. After that come several definitions. First, we define the token name and the amount we want to mint. Then we compute the currency symbol (86) and the value (87) we want to mint. Depending on whether we do have staking involved we compute two different constraints (88-90). For the lookups we specify the minting policy which we can get because we imported the module from the *OnChain.hs* file. And we also specify that we have to spend the UTXO that we used as input to the *tokenPolicy* script. Because there is only one unspent output we want to consume, we can use the *Map.singleton* function. Then the *constraints* parameter represents our constraints where we constrain the minting value, to which public key the minted tokens should be paid and of course which UTXO we want to spend. This makes sure there can be only one minting transaction for the given currency symbol. Now we want to submit the transaction. For that we use the helper function *adjustAndSubmitWith*. The reason for this is that we want some more extensive logging and that we want to take care of the minimal ADA requirement. We use the functions *mkTxConstraints*, *adjustUnbalancedTx*, *balanceTx* and *submitBalancedTx*.

```

mkTxConstraints :: ScriptLookups a -> TxConstraints (RedeemerType a) (DatumType a) ->
Contract w s e UnbalancedTx
adjustUnbalancedTx :: UnbalancedTx -> UnbalancedTx
balanceTx :: UnbalancedTx -> Contract w s e CardanoTx
submitBalancedTx :: CardanoTx -> Contract w s e CardanoTx

```

The function *mkTxConstraints* builds a transaction that satisfies the constraints. The function *adjustUnbalancedTx* adds the minimal ADA to the transaction. The function *balanceTx* sends an unbalanced transaction to be balanced which means that it checks if funds for the inputs are available in our wallet. The function *submitBalancedTx* sends a balanced transaction to be signed. It returns the ID of the final transaction when the transaction was submitted or throws an error if signing failed. All of this is done automatically if we use the functions *submitTx* or *submitTxConstraintsWith*. In the end of the code, we log some minimal information and return the currency symbol. There is also an emulator trace where this contract can be tested. It can be found in the *Trace.hs* file. In there we activate the contract for wallet 1 with some input parameters. And if we run it we get the newly minted token with the specified amount.

```

1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE FlexibleContexts     #-}
3  {-# LANGUAGE MultiParamTypeClasses #-}
4  {-# LANGUAGE NoImplicitPrelude    #-}
5  {-# LANGUAGE NumericUnderscores  #-}
6  {-# LANGUAGE OverloadedStrings    #-}
7  {-# LANGUAGE ScopedTypeVariables  #-}
8  {-# LANGUAGE TypeApplications     #-}
9  {-# LANGUAGE TypeFamilies         #-}
10
11  module Week06.Trace
12      ( testToken
13      ) where
14
15  import           Control.Monad           hiding (fmap)
16  import           Plutus.Contract        as Contract
17  import           Plutus.Trace.Emulator  as Emulator
18  import           PlutusTx.Prelude       hiding (Semigroup(..), unless)
19  import           Prelude                (IO)
20  import           Wallet.Emulator.Wallet
21
22  import           Week06.Token.OffChain
23
24  testToken :: IO ()
25  testToken = runEmulatorTraceIO tokenTrace
26
27  tokenTrace :: EmulatorTrace ()

```

```

28 tokenTrace = do
29     let w1 = knownWallet 1
30     void $ activateContractWallet w1 $ void $ mintToken @() @Empty
        TokenParams
31     { tpToken    = "USDT"
32       , tpAmount = 100_000
33       , tpAddress = mockWalletAddress w1
34     }

```

We can try this out in the Repl and see that we get the newly minted token.

```
Prelude> testToken
```

The *mintToken* contract does not return any information to the user and it does not have any endpoints. Let's look now at a second contract from the *week06/Monitor.hs* example. It's a long running contract that monitors an address that is in contrast to the previous example where the mint immediately returns the result. Let's have a look at the code.

```

1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE FlexibleContexts     #-}
3  {-# LANGUAGE OverloadedStrings   #-}
4  {-# LANGUAGE ScopedTypeVariables #-}
5  {-# LANGUAGE TypeApplications    #-}
6  {-# LANGUAGE TypeFamilies        #-}
7
8  module Week06.Monitor
9      ( monitor
10    ) where
11
12  import           Data.Functor      (void)
13  import qualified Data.Map          as Map
14  import           Data.Monoid       (Last (..))
15  import           Data.Text         (Text)
16  import           Plutus.Contract  as Contract
17  import           Ledger
18  import           Text.Printf       (printf)
19
20  monitor :: Address -> Contract (Last Value) Empty Text a
21  monitor addr = do
22      Contract.logInfo @String $ printf "started monitoring address %s" $
          show addr
23
24      go
25  where
26      go = do
27          utxos <- utxosAt addr

```

```

27         let v = Map.foldl' (\w o -> w <> _ciTxOutValue o) mempty utxos
28         tell $ Last $ Just v
29         void $ waitNSlots 1
30         go

```

It takes an address as its only argument and then it queries the blockchain and reports the value sitting at that address. We report this value with the use of the `w` parameter. For it we choose the *Last Value* type that is an instance of monoid type class.

```

Prelude> import Data.Monoid
Prelude Data.Monoid> :i Last
type Last :: * -> *
newtype Last a = Last {getLast :: Maybe a}

```

We see that `Last` is just a newtype wrapper around a `Maybe`. It changes the monoid instance of `Maybe a` and it always keeps the last `Just`.

```

Prelude Data.Monoid> mempty :: Last Char
Last {getLast = Nothing}
Prelude Data.Monoid> Last (Just 'a') <> Last (Just 'b')
Last {getLast = Just 'b'}
Prelude Data.Monoid> Last (Just 'a') <> Last (Just 'b') <> Last Nothing
Last {getLast = Just 'b'}

```

What we want to do in this code example is to write the last Value found at the given address in regular intervals. The `go` function calls itself recursively so it goes on forever (25-30). In each iteration we first find all UTXOs at the given address. We want to find the value contained at this address so we fold over the UTXOs to add the values from them to a single parameter. We use the accumulator `w` in the lambda function and add to it the value from a single UTXO (27). Then we tell the value and wait for one slot before repeating the process. So, every slot this will update the observable state contained in the `w` parameter value.

6.5 Minting with the PAB

We will use now the code from the `week06/PAB.hs` file to hook up our minting and monitor contracts to the PAB. Let's look at the code.

```

1 {-# LANGUAGE DeriveAnyClass    #-}
2 {-# LANGUAGE DeriveGeneric    #-}
3 {-# LANGUAGE OverloadedStrings #-}
4 {-# LANGUAGE TypeApplications #-}
5
6 module Week06.PAB
7     ( Address

```



```

8      , TokenContracts (..)
9    ) where
10
11 import      Data.Aeson                      (FromJSON, ToJSON)
12 import      Data.OpenApi.Schema             (ToSchema)
13 import      GHC.Generics                    (Generic)
14 import      Ledger                          (Address)
15 import      Plutus.PAB.Effects.Contract.Builtin (Empty, HasDefinitions
                                                (..), SomeBuiltin (..), endpointsToSchemas)
16 import      Prettyprinter                   (Pretty (..), viaShow)
17 import      Wallet.Emulator.Wallet          (knownWallet,
                                                mockWalletAddress)
18
19 import qualified Week06.Monitor               as Monitor
20 import qualified Week06.Token.OffChain        as Token
21
22 data TokenContracts = Mint Token.TokenParams | Monitor Address
23   deriving (Eq, Ord, Show, Generic, FromJSON, ToJSON, ToSchema)
24
25 instance Pretty TokenContracts where
26   pretty = viaShow
27
28 instance HasDefinitions TokenContracts where
29
30   getDefinitions      = [Mint exampleTP, Monitor exampleAddr]
31
32   getContract (Mint tp)      = SomeBuiltin $ Token.mintToken @() @Empty tp
33   getContract (Monitor addr) = SomeBuiltin $ Monitor.monitor addr
34
35   getSchema = const $ endpointsToSchemas @Empty
36
37 exampleAddr :: Address
38 exampleAddr = mockWalletAddress $ knownWallet 1
39
40 exampleTP :: Token.TokenParams
41 exampleTP = Token.TokenParams
42   { Token.tpAddress = exampleAddr
43   , Token.tpAmount   = 123456
44   , Token.tpToken    = "PPP"
45   }

```

First, we import our monitor and minting examples (19-20). In the beginning we define a data type called *TokenContracts* that represents everything a PAB can do which means it will define which contracts the PAB will expose (22). From it we derive a bunch of type classes (23). We

need to define the *Pretty* instance for our token contracts and use the *viaShow* function. Then we need to write an instance for the class *HasDefinitions* that has 3 methods. One is called *getDefinitions* that is a list of values of the token contract type. For the list we use some sample values that we define at the end of our code (37-45). The *getContract* method tells us which contract should run given a value of the token contracts type (32-33). We have to wrap our contract in the *SomeBuiltin* constructor. Finally, we provide the empty schema for both contracts where we use the Empty type (35). Now we can write an application that starts the PAB with our contracts. The code can be found in *app/token-pab.hs*.

```
1 {-# LANGUAGE DataKinds          #-}
2 {-# LANGUAGE DerivingStrategies #-}
3 {-# LANGUAGE FlexibleContexts    #-}
4 {-# LANGUAGE OverloadedStrings   #-}
5 {-# LANGUAGE RankNTypes          #-}
6 {-# LANGUAGE TypeApplications    #-}
7 {-# LANGUAGE TypeFamilies        #-}
8
9 module Main
10   ( main
11   ) where
12
13 import qualified Plutus.PAB.Effects.Contract.Builtin as Builtin
14 import          Plutus.PAB.Run                      (runWith)
15
16 import          Week06.PAB                          (TokenContracts)
17
18 main :: IO ()
19 main = do
20   runWith (Builtin.handleBuiltin @TokenContracts)
```

It's quite simple. All it does it calls the *runWith* function that takes in the token contracts parameter. This code represents the executable that will start the PAB and it will be specialized to the provided contracts. But for this to work we have to start some other applications as well. That are the applications shown in the dApp schema (Figure 26). We already know how to set up a cardano node. What we also need is a wallet and the chain index. In order to set that up we can follow the instructions for the PAB in the plutus-apps git repository:

<https://github.com/input-output-hk/plutus-apps/blob/main/plutus-pab/test-node/README.md>

We will look now at these instructions step by step. Our node should already be running. We can start it with the *start-testnet-node.sh* bash script from the week06 folder:

```
$ ./start-testnet-node.sh
```

The next step is the wallet. When we are in the nix shell `cardano-wallet` is available as a command. We can use the bash script `start-testnet-wallet.sh` to start the wallet backend from the nix shell. Before we run the command, we have to again source the `env.sh` bash script.

```
$ . env.sh
$ ./start-testnet-wallet.sh
```

The next step is to create a wallet. For that we can use the `create-wallet.sh` bash script. The script takes following input parameters: the name of the wallet, the passphrase and the file name where to write it. A file gets generated with the recovery phrase.

```
$ ./create-wallet.sh MyWallet mysecretpassphrase restore-wallet.json
```

If you look at the `restore-wallet.json` file under `mnemonic_sentence` you can find the recovery phrase with which you can import your wallet into Daedalus or Yoroi. After that we need to fund that wallet by either sending funds from an existing wallet to the new one or we use the testnet Faucet. Now we also have to inform the wallet backend about our new wallet. We can do this with the bash script `load-wallet.sh`. The wallet backend has a HTTP interface and API. The request body in the curl command is the json file we just created. So you can place the file into the `testnet/` folder and then run the bash script.

```
$ ./load-wallet.sh
```

Now the wallet backend knows about our wallet and is following the funds that come to or go from the wallet. When we run this script, we get a json object as response that contains the wallet ID under the `ID` attribute. We will later need it in the PAB, so we save it to the `env.sh` file. Next, we can start the chain index with the script `start-testnet-chain-index.sh`.

```
$ ./start-testnet-chain-index.sh
```

The chain index takes a long time to synchronize; it could be longer than the node itself. Once it has, we can start the PAB with the bash script `start-testnet-pab.sh`. There we have to use the passphrase that we choose when we created the wallet. Before we do this, we need to do some more configurations. In the `pab-config.yml` configuration file we provide in the command `developmentOptions` a parameter that allows us to specify from which block we want to start synchronizing the PAB, which can speed up the synchronization process. We can get the block id from the node where it's logged after the "new tip:" keyword every time a block gets created and the slot after the "at slot" keyword. So, we can take just the last entry. Because we make a fresh start there is no database yet and we need to migrate it before we start the PAB. We do this with the bash script `migrate-pab.sh`. After that we can run the `start-testnet-pab.sh` script.

```
$ ./migrate-pab.sh
$ ./start-testnet-pab.sh
```

From the output we will see that the PAB is available at port 9080 on the localhost. To get a nice interface we can go to localhost:9080/swagger/swagger-ui. There we have various HTTP endpoints available that we can execute and look at the results directly in this UI. With the `/api/contract/activate` endpoint we can start the contract on the PAB to do the minting or the monitoring. The cool thing about the UI is when we run an endpoint, we also get the associated curl command displayed which we can copy into a bash script and run it with our parameters. You can find the minting command in the bash script *mint-token-curl.sh*. For this script we need the address of our wallet, which we can get if we use Yoroi or Daedalus wallet where we go to the receive tab and look at our receiving address. Or we can use the wallet backend directly which is demonstrated in the script *get-address.sh*.

```
$ ./get-address.sh
```

The script generates many wallet addresses from which we can pick one and write it to our *env.hs* file. But we need to provide our addresses in Plutus format which separates the payment public key hash and the staking public key hash. We get the pkh and skh variables with use of the helper functions *payment-key-hash* and *stake-key-hash* specified at the beginning of the bash file. If we run now the *mint-token-curl.sh* script we can look at our wallet under Assets and our PPP token should appear.

```
$ ./mint-token-curl.sh 123456 PPP
```

There is also a third way to mint the tokens and it is done directly from a Haskell script. We can look at the *app/mint-token.hs* file.

```
1 {-# LANGUAGE OverloadedStrings #-}
2
3 module Main
4   ( main
5   ) where
6
7 import Control.Exception      (throwIO)
8 import Data.String            (IsString (..))
9 import Network.HTTP.Req
10 import System.Environment     (getArgs)
11 import Text.Printf            (printf)
12 import Wallet.Emulator.Wallet (WalletId (..))
13 import Wallet.Types           (ContractInstanceId (..))
14 import Week06.PAB             (TokenContracts (..))
```

```

15 import Week06.Token.OffChain      (TokenParams (...))
16 import Week06.Utills              (contractActivationArgs, unsafeReadAddress,
                                     unsafeReadWalletId)
17
18 main :: IO ()
19 main = do
20     [amt', tn', wid', addr'] <- getArgs
21     let wid = unsafeReadWalletId wid'
22         tp  = TokenParams
23             { tpToken    = fromString tn'
24             , tpAmount   = read amt'
25             , tpAddress  = unsafeReadAddress addr'
26             }
27     printf "minting token for wallet id %s with parameters %s\n"
           (show wid) $ show tp
28     cid <- mintToken wid tp
29     printf "minted tokens, contract instance id: %s\n" $ show cid
30
31 mintToken :: WalletId -> TokenParams -> IO ContractInstanceId
32 mintToken wid tp = do
33     v <- runReq defaultHttpConfig $ req
34         POST
35         (http "127.0.0.1" /: "api" /: "contract" /: "activate")
36         (ReqBodyJson $ contractActivationArgs wid $ Mint tp)
37         jsonResponse
38         (port 9080)
39     let c = responseStatusCode v
40     if c == 200
41     then return $ responseBody v
42     else throwIO $ userError $ printf "ERROR: %d\n" c

```

We recall that there were quite few steps before we got to the JSON body of our curl command from the swagger UI and wrote it into the *mint-token-curl.sh* script. In Haskell that's much easier. Our program takes in 4 parameters: the amount, the token name, the wallet ID and the address (20). Then we pass them into appropriate types (21-26). From the Utills module we can use the function *unsafeReadWalletId* that converts a string into a real wallet ID and similar *unsafeReadAddress* converts a string into a real address. Then we call the *mintToken* function (28) which takes a wallet ID and token parameters as input and makes a HTTP request. If the response code is 200, we return the response body which will be of type *ContractIntsanceID*. Else we log an error. With the bash script *mint-token-haskell.sh* we can now mint the token by using our Haskell file. As command line parameters it takes in the amount and token name.

```
$ ./mint-token-haskell.sh 1000000 Gold
```

And we can check again in our wallet if the tokens were minted. Now we can also look at the Haskell code for monitoring found in *app/monitor.hs*. But first we want to stop the PAB and clear the database with the following commands:

```
$ rm testnet/plutus-pab.db
$ ./migrate-pab.sh
$ ./start-testnet-pab.sh
```

The reason we are doing this is to remove the log messages so we have a clean start.

```
1 {-# LANGUAGE NumericUnderscores #-}
2 {-# LANGUAGE OverloadedStrings #-}
3
4 module Main
5     ( main
6     ) where
7
8 import Control.Concurrent      (threadDelay)
9 import Control.Exception      (throwIO)
10 import Control.Monad          (when)
11 import Data.Aeson             (FromJSON (..))
12 import Data.Aeson.Types       (parseMaybe)
13 import Data.Maybe             (fromMaybe)
14 import Data.Monoid             (Last (..))
15 import Data.Text              (pack)
16 import Network.HTTP.Req
17 import Plutus.PAB.Events.ContractInstanceState (PartiallyDecodedResponse
18                                                    (..))
19 import Plutus.PAB.Webserver.Types (ContractInstanceClientState
20                                    (..))
21 import Plutus.V1.Ledger.Value   (Value, flattenValue)
22 import System.Environment       (getArgs)
23 import Text.Printf              (printf)
24 import Wallet.Emulator.Wallet  (WalletId (..))
25 import Week06.PAB               (Address, TokenContracts (..))
26 import Wallet.Types             (ContractInstanceId (..))
27 import Week06.Utills            (cidToString,
28                                 contractActivationArgs,
29                                 unsafeReadAddress,
30                                 unsafeReadWalletId)
31
32 26
33 27 main :: IO ()
34 28 main = do
```

```

29     [wid', addr'] <- getArgs
30     let wid  = unsafeReadWalletId wid'
31         addr = unsafeReadAddress addr'
32     printf "monitoring address %s on wallet %s\n" (show addr) $ show wid
33     cid <- startMonitor wid addr
34     printf "started monitor-process with contract id %s\n\n" $
           cidToString cid
35     go cid mempty
36 where
37     go :: ContractInstanceId -> Value -> IO a
38     go cid v = do
39         cic <- getMonitorState cid
40         let v' = fromMaybe v $ observedValue cic
41         when (v' /= v) $
42             printf "%s\n\n" $ show $ flattenValue v'
43         threadDelay 1_000_000
44         go cid v'
45
46 startMonitor :: WalletId -> Address -> IO ContractInstanceId
47 startMonitor wid addr = do
48     v <- runReq defaultHttpConfig $ req
49         POST
50         (http "127.0.0.1" /: "api" /: "contract" /: "activate")
51         (ReqBodyJson $ contractActivationArgs wid $ Monitor addr)
52         jsonResponse
53         (port 9080)
54     let c = responseStatusCode v
55     when (c /= 200) $
56         throwIO $ userError $ printf "ERROR: %d\n" c
57     return $ responseBody v
58
59 getMonitorState :: ContractInstanceId ->
           IO (ContractInstanceClientState TokenContracts)
60 getMonitorState cid = do
61     v <- runReq defaultHttpConfig $ req
62         GET
63         (http "127.0.0.1" /: "api" /: "contract" /: "instance" /:
           pack (cidToString cid) /: "status")
64         NoReqBody
65         jsonResponse
66         (port 9080)
67     let c = responseStatusCode v
68     when (c /= 200) $
69         throwIO $ userError $ printf "ERROR: %d\n" c
70     return $ responseBody v

```

```
71
72 observedValue :: ContractInstanceClientState TokenContracts -> Maybe Value
73 observedValue cic = do
74     Last mv <- parseMaybe parseJSON $ observableState $ cicCurrentState cic
75     mv
```

The body of the main function is similar to how we did the minting. From it we call now two helper functions the *startMonitor* function that starts the monitoring and the *getMonitorState* function that gets the state of the monitor contract and some other information as the tokens we have in our wallet. We can use the *monitor.sh* script to start our code. There we input the environment variables wallet id and address. After its run it will start reporting our assets.

```
$ ./monitor.sh
```

Altogether we presented now 3 different ways how to mint our tokens: using the client, using the PAB and using Haskell functions directly. We have to note that these operations are quite resource hungry. If you have around 12GB of RAM it can still happen you will need to increase the size of your SWAP partition to 50GB to get the chain index to actually sync.

7 State Machines

In this chapter we will look at state machines (SM). They can be useful to write shorter and more concise code for both the on-chain and off-chain part. There is support for state machines in the Plutus libraries that is higher level and builds on top of the lower-level mechanisms we have seen so far. But what we do have to mention is that at the current time of writing there is a certain overhead using state machines. If you write a contract with state machines it will require more resources to run as if you wrote the same contract without state machines. Because of that, SM have not seen much use in practice yet. However, the Plutus team is permanently working on improving performance and optimizing the compiler and interpreter so we can expect state machines to be really useful in the near future.

7.1 Commit schemes

Let's imagine a game played between Alice and Bob. It's similar to rock-paper-scissors but with only two options. Instead, we have one gesture for 0 and one gesture for 1. If they both raise the same gesture then Alice wins and if they raise a different gesture then Bob wins. Let's say that Alice and Bob can't meet in person but rather play the game via email. Now we come to the problem how to make sure when Alice sends her choice to Bob that Bob does not read the email before making his choice to get an unfair advantage. There is a trick that is used in cryptographic protocols and it's about committed schemes. The idea is that Alice does not reveal her choice to Bob but rather commits to it so she later cannot change her mind. One way to make that work is using hash functions. Hashes are one-way functions because given a hash it's impossible to construct the original text or byte string from which the hash was computed. The problem we have now is that Bob will soon figure out which hash belongs to which number since there are only two choices. So, what Alice can do is that she first concatenates her choice number with some arbitrary byte string (that we call a nonce) before hashing it and then sends the hash to Bob. And when they check their choices, Alice has to send to Bob her choice and the nonce in plain text so Bob can compute the hash on his own and make sure Alice did not cheat. We will try to implement such an example in Plutus together with the code we have seen so far. The idea is Alice and Bob put down a certain amount of money that then gets processed accordingly to their choices. We also implement the possibility when Alice opens the game by posting her hash and if Bob does not reply, she can retrieve her funds after a certain amount of time. Or if Bob replies and Alice sees she has lost and does not reply, Bob can retrieve his funds after a certain amount of time has passed.

7.2 Implementation without State Machines

We will first look at the example how to implement the game in Plutus from the previous chapter without using state machines. Let's first look at the *EvenOdd.hs* file.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass       #-}
3  {-# LANGUAGE DeriveGeneric        #-}
4  {-# LANGUAGE FlexibleContexts     #-}
5  {-# LANGUAGE MultiParamTypeClasses #-}
6  {-# LANGUAGE NoImplicitPrelude    #-}
7  {-# LANGUAGE OverloadedStrings    #-}
8  {-# LANGUAGE ScopedTypeVariables  #-}
9  {-# LANGUAGE TemplateHaskell      #-}
10 {-# LANGUAGE TypeApplications     #-}
11 {-# LANGUAGE TypeFamilies          #-}
12 {-# LANGUAGE TypeOperators         #-}
13
14 module Week07.EvenOdd
15   ( Game (..)
16   , GameChoice (..)
17   , FirstParams (..)
18   , SecondParams (..)
19   , GameSchema
20   , endpoints
21   ) where
22
23 import           Control.Monad      hiding (fmap)
24 import           Data.Aeson         (FromJSON, ToJSON)
25 import qualified Data.Map           as Map
26 import           Data.Text          (Text)
27 import           GHC.Generics        (Generic)
28 import           Ledger              hiding (singleton)
29 import           Ledger.Constraints  as Constraints
30 import qualified Ledger.Typed.Scripts as Scripts
31 import           Ledger.Ada          as Ada
32 import           Ledger.Value
33 import           Playground.Contract (ToSchema)
34 import           Plutus.Contract    as Contract
35 import qualified PlutusTx
36 import           PlutusTx.Prelude   hiding (Semigroup(..), unless)
37 import           Prelude             (Semigroup (..), Show (..), String)
38 import qualified Prelude
39
40 data Game = Game
```

```

41     { gFirst      :: !PaymentPubKeyHash
42     , gSecond     :: !PaymentPubKeyHash
43     , gStake      :: !Integer
44     , gPlayDeadline :: !POSIXTime
45     , gRevealDeadline :: !POSIXTime
46     , gToken      :: !AssetClass
47     } deriving (Show, Generic, FromJSON, ToJSON, Prelude.Eq, Prelude.Ord)
48
49 PlutusTx.makeLift ''Game
50
51 data GameChoice = Zero | One
52     deriving (Show, Generic, FromJSON, ToJSON, ToSchema, Prelude.Eq,
53             Prelude.Ord)
54
55 instance Eq GameChoice where
56     {-# INLINABLE (==) #-}
57     Zero == Zero = True
58     One  == One  = True
59     _    == _    = False
60
61 PlutusTx.unstableMakeIsData ''GameChoice
62
63 data GameDatum = GameDatum BuiltinByteString (Maybe GameChoice)
64     deriving Show
65
66 instance Eq GameDatum where
67     {-# INLINABLE (==) #-}
68     GameDatum bs mc == GameDatum bs' mc' = (bs == bs') && (mc == mc')
69
70 PlutusTx.unstableMakeIsData ''GameDatum
71
72 data GameRedeemer = Play GameChoice | Reveal BuiltinByteString |
73     ClaimFirst | ClaimSecond
74     deriving Show
75
76 PlutusTx.unstableMakeIsData ''GameRedeemer
77
78 {-# INLINABLE lovelaces #-}
79 lovelaces :: Value -> Integer
80 lovelaces = Ada.getLovelace . Ada.fromValue
81
82 {-# INLINABLE gameDatum #-}
83 gameDatum :: Maybe Datum -> Maybe GameDatum
84 gameDatum md = do
85     Datum d <- md

```

```

84     PlutusTx.fromBuiltinData d
85
86 {-# INLINABLE mkGameValidator #-}
87 mkGameValidator :: Game -> BuiltinByteString -> BuiltinByteString ->
      GameDatum -> GameRedeemer -> ScriptContext -> Bool
88 mkGameValidator game bsZero' bsOne' dat red ctx =
89     traceIfFalse "token missing from input" (assetClassValueOf
      (txOutValue ownInput) (gToken game) == 1) &&
90     case (dat, red) of
91     (GameDatum bs Nothing, Play c) ->
92         traceIfFalse "not signed by second player"
      (txSignedBy info (unPaymentPubKeyHash $ gSecond game)) &&
93         traceIfFalse "first player's stake missing" (lovelaces
      (txOutValue ownInput) == gStake game) &&
94         traceIfFalse "second player's stake missing" (lovelaces
      (txOutValue ownOutput) == (2 * gStake game)) &&
95         traceIfFalse "wrong output datum"
      (outputDatum == GameDatum bs (Just c)) &&
96         traceIfFalse "missed deadline"
      (to (gPlayDeadline game) `contains` txInfoValidRange info) &&
97         traceIfFalse "token missing from output"
      (assetClassValueOf (txOutValue ownOutput) (gToken game) == 1)
98
99     (GameDatum bs (Just c), Reveal nonce) ->
100         traceIfFalse "not signed by first player"
      (txSignedBy info (unPaymentPubKeyHash $ gFirst game)) &&
101         traceIfFalse "commit mismatch"
      (checkNonce bs nonce c) &&
102         traceIfFalse "missed deadline"
      (to (gRevealDeadline game) `contains` txInfoValidRange info) &&
103         traceIfFalse "wrong stake"
      (lovelaces (txOutValue ownInput) == (2 * gStake game)) &&
104         traceIfFalse "NFT must go to first player" nftToFirst
105
106     (GameDatum _ Nothing, ClaimFirst) ->
107         traceIfFalse "not signed by first player"
      (txSignedBy info (unPaymentPubKeyHash $ gFirst game)) &&
108         traceIfFalse "too early"
      (from (1 + gPlayDeadline game) `contains` txInfoValidRange info)
      &&
109         traceIfFalse "first player's stake missing"
      (lovelaces (txOutValue ownInput) == gStake game) &&
110         traceIfFalse "NFT must go to first player" nftToFirst
111
112     (GameDatum _ (Just _), ClaimSecond) ->

```

```

113         traceIfFalse "not signed by second player"
114         (txSignedBy info (unPaymentPubKeyHash $ gSecond game)) &&
115         traceIfFalse "too early"
116         (from (1 + gRevealDeadline game) `contains`
117          txInfoValidRange info) &&
118         traceIfFalse "wrong stake"
119         (lovelaces (txOutValue ownInput) == (2 * gStake game)) &&
120         traceIfFalse "NFT must go to first player" nftToFirst
121
122     _ -> False
123 where
124     info :: TxInfo
125     info = scriptContextTxInfo ctx
126
127     ownInput :: TxOut
128     ownInput = case findOwnInput ctx of
129         Nothing -> traceError "game input missing"
130         Just i -> txInInfoResolved i
131
132     ownOutput :: TxOut
133     ownOutput = case getContinuingOutputs ctx of
134         [o] -> o
135         _ -> traceError "expected exactly one game output"
136
137     outputDatum :: GameDatum
138     outputDatum = case gameDatum $ txOutDatumHash ownOutput >=
139         flip findDatum info of
140         Nothing -> traceError "game output datum not found"
141         Just d -> d
142
143     checkNonce :: BuiltinByteString -> BuiltinByteString ->
144         GameChoice -> Bool
145     checkNonce bs nonce cSecond = sha2_256
146         (nonce `appendByteString` cFirst) == bs
147
148     where
149         cFirst :: BuiltinByteString
150         cFirst = case cSecond of
151             Zero -> bsZero'
152             One -> bsOne'
153
154     nftToFirst :: Bool
155     nftToFirst = assetClassValueOf (valuePaidTo info $ unPaymentPubKeyHash $
156         gFirst game) (gToken game) == 1
157
158 data Gaming

```

```

150 instance Scripts.ValidatorTypes Gaming where
151     type instance DatumType Gaming = GameDatum
152     type instance RedeemerType Gaming = GameRedeemer
153
154 bsZero, bsOne :: BuiltinByteString
155 bsZero = "0"
156 bsOne  = "1"
157
158 typedGameValidator :: Game -> Scripts.TypedValidator Gaming
159 typedGameValidator game = Scripts.mkTypedValidator @Gaming
160     ( $(PlutusTx.compile [| mkGameValidator |])
161       `PlutusTx.applyCode` PlutusTx.liftCode game
162       `PlutusTx.applyCode` PlutusTx.liftCode bsZero
163       `PlutusTx.applyCode` PlutusTx.liftCode bsOne)
164     $(PlutusTx.compile [| wrap |])
165   where
166     wrap = Scripts.wrapValidator @GameDatum @GameRedeemer
167
168 gameValidator :: Game -> Validator
169 gameValidator = Scripts.validatorScript . typedGameValidator
170
171 gameAddress :: Game -> Ledger.Address
172 gameAddress = scriptAddress . gameValidator
173
174 findGameOutput :: Game -> Contract w s Text (Maybe (TxOutRef,
175     ChainIndexTxOut, GameDatum))
176
177 findGameOutput game = do
178     utxos <- utxosAt $ gameAddress game
179     return $ do
180         (oref, o) <- find f $ Map.toList utxos
181         dat      <- gameDatum $ either (const Nothing) Just $
182             _ciTxOutDatum o
183         return (oref, o, dat)
184
185   where
186     f :: (TxOutRef, ChainIndexTxOut) -> Bool
187     f (_, o) = assetClassValueOf (_ciTxOutValue o) (gToken game) == 1
188
189 waitUntilTimeHasPassed :: AsContractError e => POSIXTime ->
190     Contract w s e ()
191
192 waitUntilTimeHasPassed t = do
193     s1 <- currentSlot
194     logInfo @String $ "current slot: " ++ show s1 ++
195         ", waiting until " ++ show t
196     void $ awaitTime t >> waitNSlots 1
197     s2 <- currentSlot

```

```

191     logInfo @String $ "waited until: " ++ show s2
192
193 data FirstParams = FirstParams
194     { fpSecond      :: !PaymentPubKeyHash
195     , fpStake        :: !Integer
196     , fpPlayDeadline :: !POSIXTime
197     , fpRevealDeadline :: !POSIXTime
198     , fpNonce        :: !BuiltinByteString
199     , fpCurrency     :: !CurrencySymbol
200     , fpTokenName    :: !TokenName
201     , fpChoice       :: !GameChoice
202     } deriving (Show, Generic, FromJSON, ToJSON, ToSchema)
203
204 firstGame :: forall w s. FirstParams -> Contract w s Text ()
205 firstGame fp = do
206     pkh <- Contract.ownPaymentPubKeyHash
207     let game = Game
208         { gFirst      = pkh
209         , gSecond     = fpSecond fp
210         , gStake       = fpStake fp
211         , gPlayDeadline = fpPlayDeadline fp
212         , gRevealDeadline = fpRevealDeadline fp
213         , gToken       = AssetClass (fpCurrency fp, fpTokenName fp)
214         }
215     v = lovelaceValueOf (fpStake fp) <> assetClassValue (gToken game) 1
216     c = fpChoice fp
217     bs = sha2_256 $ fpNonce fp `appendByteString` if c == Zero then
        bsZero else bsOne
218     tx = Constraints.mustPayToTheScript (GameDatum bs Nothing) v
219     ledgerTx <- submitTxConstraints (typedGameValidator game) tx
220     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
221     logInfo @String $ "made first move: " ++ show (fpChoice fp)
222
223     waitUntilTimeHasPassed $ fpPlayDeadline fp
224
225     m <- findGameOutput game
226     now <- currentTime
227     case m of
228         Nothing      -> throwError "game output not found"
229         Just (oref, o, dat) -> case dat of
230             GameDatum _ Nothing -> do
231                 logInfo @String "second player did not play"
232                 let lookups = Constraints.unspentOutputs
                    (Map.singleton oref o) <>
233                     Constraints.otherScript (gameValidator game)

```

```

234         tx'      = Constraints.mustSpendScriptOutput oref
                      (Redeemer $ PlutusTx.toBuiltinData ClaimFirst)
235         <> Constraints.mustValidateIn (from now)
236 ledgerTx' <- submitTxConstraintsWith @Gaming lookups tx'
237 void $ awaitTxConfirmed $ getCardanoTxId ledgerTx'
238 logInfo @String "reclaimed stake"
239
240 GameDatum _ (Just c') | c' == c -> do
241
242     logInfo @String "second player played and lost"
243     let lookups = Constraints.unspentOutputs
                      (Map.singleton oref o) <>
244                 Constraints.otherScript (gameValidator game)
245     tx'      = Constraints.mustSpendScriptOutput oref
                      (Redeemer $ PlutusTx.toBuiltinData $ Reveal $
246                          fpNonce fp) <>
                      Constraints.mustValidateIn (to $ now + 1000)
247     ledgerTx' <- submitTxConstraintsWith @Gaming lookups tx'
248     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx'
249     logInfo @String "victory"
250
251     _ -> logInfo @String "second player played and won"
252
253 data SecondParams = SecondParams
254     { spFirst      :: !PaymentPubKeyHash
255     , spStake      :: !Integer
256     , spPlayDeadline :: !POSIXTime
257     , spRevealDeadline :: !POSIXTime
258     , spCurrency   :: !CurrencySymbol
259     , spTokenName  :: !TokenName
260     , spChoice     :: !GameChoice
261     } deriving (Show, Generic, FromJSON, ToJSON, ToSchema)
262
263 secondGame :: forall w s. SecondParams -> Contract w s Text ()
264 secondGame sp = do
265     pkh <- Contract.ownPaymentPubKeyHash
266     let game = Game
267         { gFirst      = spFirst sp
268         , gSecond     = pkh
269         , gStake      = spStake sp
270         , gPlayDeadline = spPlayDeadline sp
271         , gRevealDeadline = spRevealDeadline sp
272         , gToken      = AssetClass (spCurrency sp, spTokenName sp)
273         }
274     m <- findGameOutput game

```



```

275     case m of
276       Just (oref, o, GameDatum bs Nothing) -> do
277         logInfo @String "running game found"
278         now <- currentTime
279         let token    = assetClassValue (gToken game) 1
280         let v        = let x = lovelaceValueOf (spStake sp) in x <> x
281                               <> token
282         c          = spChoice sp
283         lookups    = Constraints.unspentOutputs
284                               (Map.singleton oref o) <>
285                               Constraints.otherScript (gameValidator game) <>
286                               Constraints.typedValidatorLookups
287                               (typedGameValidator game)
288         tx         = Constraints.mustSpendScriptOutput oref (Redeemer $
289                               PlutusTx.toBuiltinData $ Play c) <>
290                               Constraints.mustPayToTheScript
291                               (GameDatum bs $ Just c) v <>
292                               Constraints.mustValidateIn (to now)
293         ledgerTx <- submitTxConstraintsWith @Gaming lookups tx
294         let tid = getCardanoTxId ledgerTx
295         void $ awaitTxConfirmed tid
296         logInfo @String $ "made second move: " ++ show (spChoice sp)
297
298         waitUntilTimeHasPassed $ spRevealDeadline sp
299
300         m' <- findGameOutput game
301         now' <- currentTime
302         case m' of
303           Nothing          -> logInfo @String "first player won"
304           Just (oref', o', _) -> do
305             logInfo @String "first player didn't reveal"
306             let lookups' = Constraints.unspentOutputs
307                               (Map.singleton oref' o') <>
308                               Constraints.otherScript
309                               (gameValidator game)
310             tx'         = Constraints.mustSpendScriptOutput oref'
311                               (Redeemer $ PlutusTx.toBuiltinData
312                               ClaimSecond) <>
313                               Constraints.mustValidateIn (from now') <>
314                               Constraints.mustPayToPubKey (spFirst sp)
315                               (token <> adaValueOf
316                               (getAda minAdaTxOut))
316             ledgerTx' <- submitTxConstraintsWith @Gaming
317                               lookups' tx'
318             void $ awaitTxConfirmed $ getCardanoTxId ledgerTx'

```

```

308         logInfo @String "second player won"
309
310     _ -> logInfo @String "no running game found"
311
312     type GameSchema = Endpoint "first" FirstParams .\ /
                          Endpoint "second" SecondParams
313
314     endpoints :: Contract () GameSchema Text ()
315     endpoints = awaitPromise (first `select` second) >> endpoints
316     where
317         first  = endpoint @"first"  firstGame
318         second = endpoint @"second" secondGame

```

We call this code `EvenOdd` because if the sum of the choices is even the first player wins and if it is odd the second player wins. First, we create the data type `game` that will be used as a parameter for the contract (40-47). There we define the first and second player with their public key hashes. Then we also define their stake, the playing deadline and the revealing deadline. And in the end, we define a NFT token. Since the game contains some state that is changing and UTXOs together with their datums are immutable we need to create a new UTXO every time the state of the game is changing. And to be able to connect the old UTXO with the new one we can use an NFT that exists only once and gets assigned to the new UTXO every time the state changes. We then call this token a stake token. Another reason we need this NFT is that somebody could create a UTXO at the same address with the same datum and would try to disturb the game. So, in order to uniquely be able to identify our UTXO with which we start the game we need an NFT that exists only once. The type `game choice` defines the two moves the players can make (51-52). Then we derive the Plutus equality for the game choice type (54-58). For this to work with template Haskell we need to add the `inalienable` pragma. We will use the game datum as state information for the contract (62-63). The byte string there is the hash that the first player submits and maybe game choice is the move of the second player. It's a maybe because in the beginning the second player has not yet moved. We implement also the Plutus equality for the game datum (65-67). Next, we implement the game redeemer with the options that corresponds to our player actions (71-72). `Play` means when the second player moves and makes a game choice, `reveal` is for the case that first player reveals his nonce which is the byte string argument, `claim first` is the case when the second player does not move and the first player claims back his stake and `claim second` is for the case if the first player does not reveal his nonce and the deadline passes for the second player to collect his earnings. Then we define two helper functions. The function `lovelaces` when given a value extracts the amount of lovelaces (76-78). And the function `gameDatum` given a maybe datum tries to deserialize that if it's a `Just` value to a maybe game datum (80-84).

Now we write our game validator function. The first parameter is the game parameter that we defined in the beginning. The second and third parameters are the byte strings with the digit zero and the digit one that represents the choice. Then we take in the datum, redeemer and context. Let's look first at the helper functions before we come to the main body. For the *ownInput* function we use the function *findOwnInput* (Figure 27) (123-126).

```
findOwnInput :: ScriptContext -> Maybe TxInInfo
```

Find the input currently being validated.

Figure 27 - findOwnInput function

It takes in a script context and produces a Maybe transaction input info. If a script would be used for minting this function would return Nothing. What we are interested in inside the *TxInInfo* type is the transaction output *TxOut*. The idea of our game is that with each state change we consume an UTXO and produce another one at the same address. For the *ownOutput* function we use the *getContinuingOutputs* function.

```
getContinuingOutputs :: ScriptContext -> [TxOut]
```

It takes a script context and returns a list of transaction outputs that are the new outputs of our transaction. We expect there is exactly one output sitting at our address (128-131). The output datum function should give us a game datum of our own output (133-136). We use the function *findDatum* that takes in a datum hash and a transaction info and then returns a maybe datum (Figure 28). A Maybe datum because we said that the producing transaction can optionally include the datum in the output. It is required only to include the hash of the datum.

```
findDatum :: DatumHash -> TxInfo -> Maybe Datum
```

Find the data corresponding to a data hash, if there is one

Figure 28 - findDatum function

The check nonce function is for the case that the first player has won and wants to prove it by revealing his nonce and proving that the hash submitted at the beginning of the game fits this nonce (138-144). The first argument is the hash he submitted, the second is the nonce and the third is the move that the second player made. To compute the hash, we take the nonce concatenate it with the byte string and apply the SHA 256 hash function. At the end we define the *nftToFirst* function (146-147). The idea is after the game finishes the 1st player gets back his NFT no matter who won the game. There we use the helper function *valuePaidTo*.

```
valuePaidTo :: TxInfo -> PubKeyHash -> Value
```

Get the total value paid to a public key address by a pending transaction.

Figure 29 - valuePaidTo function

It takes in a transaction info and a public key hash and returns a value that will be paid to the UTXO that will go to the specified public key hash. One thing to notice about our code at this point is we haven't spent much consideration about the staking address. In general, a user could use an address where the payment part goes to the winner but the staking part goes to the loser, since these 2 addresses could be different. So, one has to be careful when specifying the input data and take this consideration into account. Let's now look at the conditions in the body of the validator function. There is one condition that applies to all cases simultaneously. That is the input we are validating has to contain the state token (89). Then the rules after that depend on the situation. The first situation is when the 1st player has moved and 2nd player is moving now and chooses the move *c* (91-97). First, we check that the second player actually signs the transaction. Then we check that the first player has put down his stake. In the third condition we check that the second player added in this transaction his own stake. After that we check the datum from the transaction context. Then we check that the move has to happen before the first deadline. And in the end, we check that the NFT is passed to the transaction output UTXO. The second situation is both players have moved and the 1st player has won (99-104). And to prove it and collect his winnings he has to reveal his nonce. So first we check that the transaction is signed by the first player. Then the nonce must agree with the hash we submitted earlier. He must do this before the reveal deadline. The input must contain the stakes of both players. And finally, the NFT must go back to the first player. The third situation is that the 2nd player does not move and the 1st player wants his stake back (106-110). So the transaction must be signed by the first player has to have a validity interval after the deadline has passed. The first player has provided his stake and he must get back his NFT token. The last situation is both players have moved but the 1st player has lost and did not reveal his nonce (112-116). So, the transaction must be signed by the 2nd player. The reveal deadline should pass. The input must contain the stakes of both players. And the token has to go back to the first player. In all other cases we fail validation. Let's look at the rest of the on-chain code. First, we define a *ValidatorTypes* instance that bundles information what the datum and the redeemer types are (149-152). Then we define the byte strings that we use for choice 1 and choice 2. Next, we define our typed validator and compile the code with our parameters (158-166). After that we define the game validator and game address.

For the off-chain code we first define two helper functions. The *findGameOutput* function takes as input a game type parameter and then in the contract monad tries to find the UTXO (174-180). Because it could fail, we are returning a maybe type. We return the transaction output reference, the transaction output chain index and the game datum as a triple. We use the find function that has following type signature:

```
Prelude> import Data.List
Prelude Data.List> :i find
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
Prelude Data.List> find even [1 :: Int, 3, 4, 5, 6]
Just 4
```

It works like this: if it finds an element in the container that satisfies the initial condition then it returns a Just of that element. Our helper function *f* checks weather the output contains the token. With the second function *waitUntilTimeHasPassed* we take in a posix time and wait until that time has passed and then we wait for 1 more slot (185-191). Then come the two contracts for the two players with the corresponding input parameters for each contract. In the first contract we first get our public key. Then we define several parameters (207-218). First the value of the game type, second the amount of lovelace we put in as our stake plus the NFT we put in, then *c* is our choice, next we compute the hash of our nonce with the appended choice and in the end, we define our transaction. Then we submit, wait for the transaction to process and log a message. Now the second player has a chance to move but it must happen before the play deadline so as first player we wait until the deadline has passed (223). And then there are several cases. First, we check if we find the UTXO containing the NFT and if yes, we check the datum. One case is that it is Nothing so the 2nd player did not move (230). In that case as constraints, we say we must spend this UTXO we found with this redeemer and as lookups we need to provide the UTXO and the validator. Then the second case is that the second player did move and he chooses the same move as we did so we won (240). So, we have to reveal our nonce. We use the Reveal nonce game redeemer. And we need to submit this transaction before the deadline for revealing has passed. The third case is the second player won and, in that case, we don't do anything.

Now for the second contract for player 2 the input parameters are very similar (253-261). We do not need to provide the second players public key hash because we look up our own public key hash. Then we define the game value (266-273). Next, we try to find the UTXO that contains the NFT. If we find it, we continue by defining several parameters (280-287). We define the token; *v* represents the output which is twice the stake and the NFT and *c* is our choice. For the transaction we put the constraints that we must spend the existing UTXO with the redeemer Play that holds our choice. Then we create a new UTXO with the updated datum and we must

do this before the deadline passes. For the lookups we provide the UTXO. Because we are consuming the script output, we need the validator and because we are also producing, we need the script instance. After we defined our parameters we submit the transaction, wait for confirmation and log a message. Then we wait until the reveal deadline has passed so the first player can make his move. Next, we try again to find the UTXO which could be now a different one. If we do not find it the first player made his reveal and had won, so we do nothing. If we do find it we must spend the UTXO that we have found, we must do this after the deadline has passed and return the NFT to the first player. Because every UTXO has to contain some ADA we must add some ADA when returning the NFT. Then we submit the transaction and log that the 2nd player has won. In the end we define the schema and define the endpoints contract. We can test this now with the emulator trace defined in the *TestEvenOdd.hs* file.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE FlexibleContexts     #-}
3  {-# LANGUAGE MultiParamTypeClasses #-}
4  {-# LANGUAGE NoImplicitPrelude    #-}
5  {-# LANGUAGE NumericUnderscores  #-}
6  {-# LANGUAGE OverloadedStrings    #-}
7  {-# LANGUAGE ScopedTypeVariables  #-}
8  {-# LANGUAGE TypeApplications     #-}
9  {-# LANGUAGE TypeFamilies         #-}
10
11 module Week07.TestEvenOdd
12   ( test
13   , test'
14   , GameChoice (..)
15   ) where
16
17 import           Control.Monad           hiding (fmap)
18 import           Control.Monad.Freeer.Extras as Extras
19 import           Data.Default            (Default (..))
20 import qualified Data.Map                as Map
21 import           Ledger
22 import           Ledger.TimeSlot
23 import           Ledger.Value
24 import           Ledger.Ada              as Ada
25 import           Plutus.Trace.Emulator  as Emulator
26 import           PlutusTx.Prelude
27 import           Prelude                 (IO, Show (..))
28 import           Wallet.Emulator.Wallet
29
30 import           Week07.EvenOdd
31
```

```

32 test :: IO ()
33 test = do
34     test' Zero Zero
35     test' Zero One
36     test' One Zero
37     test' One One
38
39 w1, w2 :: Wallet
40 w1 = knownWallet 1
41 w2 = knownWallet 2
42
43 test' :: GameChoice -> GameChoice -> IO ()
44 test' c1 c2 = runEmulatorTraceIO' def emCfg $ myTrace c1 c2
45     where
46         emCfg :: EmulatorConfig
47         emCfg = def { _initialChainState = Left $ Map.fromList
48             [ (w1, v <> assetClassValue (AssetClass
49                 (gameTokenCurrency, gameTokenName)) 1)
50               , (w2, v)
51             ]
52         }
53
54         v :: Value
55         v = Ada.lovelaceValueOf 1_000_000_000
56
57         gameTokenCurrency :: CurrencySymbol
58         gameTokenCurrency = "ff"
59
60         gameTokenName :: TokenName
61         gameTokenName = "STATE TOKEN"
62
63 myTrace :: GameChoice -> GameChoice -> EmulatorTrace ()
64 myTrace c1 c2 = do
65     Extras.logInfo $ "first move: " ++ show c1 ++
66         ", second move: " ++ show c2
67
68     h1 <- activateContractWallet w1 endpoints
69     h2 <- activateContractWallet w2 endpoints
70
71     let pkh1      = mockWalletPaymentPubKeyHash w1
72         pkh2      = mockWalletPaymentPubKeyHash w2
73         stake     = 100_000_000
74         deadline1 = slotToBeginPOSIXTime def 5
75         deadline2 = slotToBeginPOSIXTime def 10

```

```

75     fp = FirstParams
76         { fpSecond      = pkh2
77         , fpStake        = stake
78         , fpPlayDeadline = deadline1
79         , fpRevealDeadline = deadline2
80         , fpNonce        = "SECRETNONCE"
81         , fpCurrency     = gameTokenCurrency
82         , fpTokenName    = gameTokenName
83         , fpChoice       = c1
84         }
85     sp = SecondParams
86         { spFirst      = pkh1
87         , spStake      = stake
88         , spPlayDeadline = deadline1
89         , spRevealDeadline = deadline2
90         , spCurrency   = gameTokenCurrency
91         , spTokenName  = gameTokenName
92         , spChoice     = c2
93         }
94
95     callEndpoint @"first" h1 fp
96
97     void $ Emulator.waitNSlots 3
98
99     callEndpoint @"second" h2 sp
100
101     void $ Emulator.waitNSlots 10

```

The idea is that you can test our code for all possible choices. So, we define our *test* IO action such that all possible game choices are covered (32-37). We define the two wallets (39-41). The *test'* function takes in 2 choices and runs the emulator trace where it assigns 1000 ADA to both wallets and the token to wallet 1 (43-54). Of course, the currency symbol of the token does not correspond to a hash of a real minting script but for testing it will be fine. Then we define our trace that also takes as input our two choices. We start the contract for both wallets and save the returned handle (66-67). We look up the payment public key hashes and use the stake of 100 ADA. Then we define the deadlines which are a bit short because on the real blockchain a block comes on average every 20 seconds (72-73). But for our example it will be fine. Next, we define the parameters for the first and second player. Now we can call the endpoint for the first player, wait for 3 slots, call the endpoint for the second player and wait for 10 slots.

7.3 State Machines

A state machine (SM) is a system that starts in one state and normally there are available transitions to other states. And there can be also final states from which there are no transitions left. For our game we can also draw a state diagram where all the nodes represent a state and all the arrows represent transitions (Figure 30). In the blockchain the states will be represented by UTXO sitting at the state machine script address. The state of the state machine will be the datum of the UTXO and a transition is represented by a transaction. There is special support in the Plutus libraries to implement such state machines which makes our code shorter compared to the case without using SM. The module `Plutus.Contract.StateMachine` contains all the code for using SM. Let's first look at the definition of a `StateMachine` type (Figure 32).

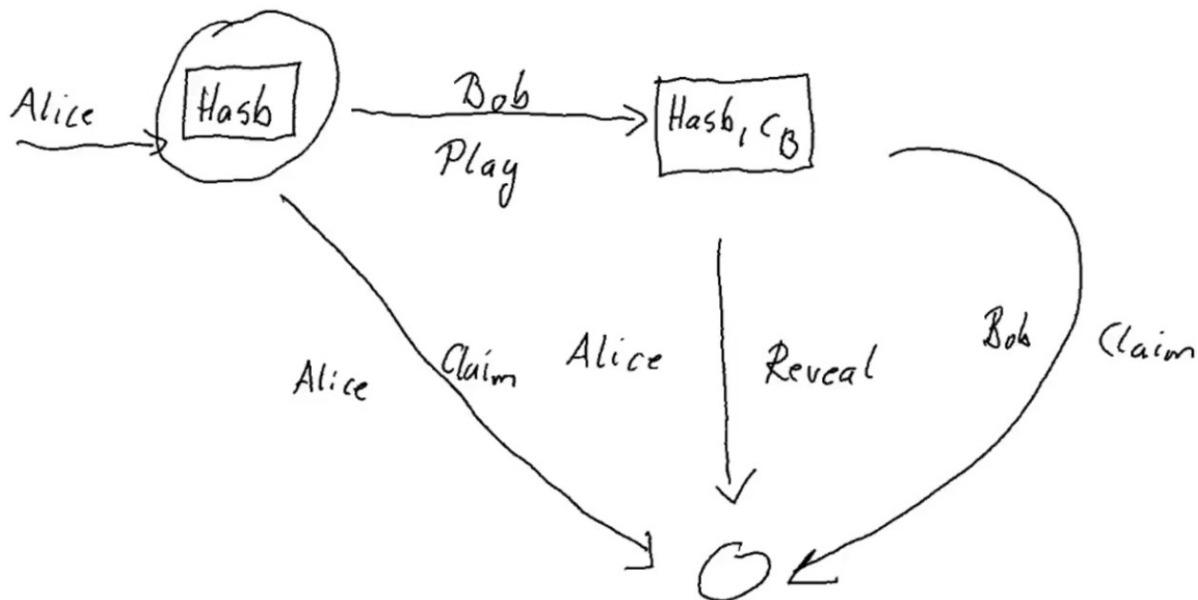


Figure 30 - Game state diagram

It takes two parameters that are state and input which correspond to datum and redeemer. It is a record type with four fields. The `smTransition` function defines from which state using which transition you can define another state. The `State` type is defined with the state data and a state value of type `Value`. If the transition is not allowed, we return a `Nothing` in the function, else we return a tuple from which the second component is the new state. And the `TxConstraints` define additional constraints that the transaction that does the transition must have. The `smFinal` function tells us whether we are transitioning to a final state. If yes then we do not produce a new UTXO and there is no value attached to the transition. The `smCheck` function takes in the datum, redeemer and context and basically makes an additional check that can't be expressed with the transaction constraints.

```
data State s
```

Constructors

State

```
stateData :: s
```

```
stateValue :: Value
```

Figure 31 - State type

```
data StateMachine s i
```

Source

Specification of a state machine, consisting of a transition function that determines the next state from the current state and an input, and a checking function that checks the validity of the transition in the context of the current transaction.

Constructors

StateMachine

```
smTransition :: State s -> i -> Maybe (TxConstraints Void Void, State s)
```

The transition function of the state machine. `Nothing` indicates an invalid transition from the current state.

```
smFinal :: s -> Bool
```

Check whether a state is the final state

```
smCheck :: s -> i -> ScriptContext -> Bool
```

The condition checking function. Can be used to perform checks on the pending transaction that aren't covered by the constraints. `smCheck` is always run in addition to checking the constraints, so the default implementation always returns true.

```
smThreadToken :: Maybe ThreadToken
```

The `ThreadToken` that identifies the contract instance. Make one with `getThreadToken` and pass it on to `mkStateMachine`. Initialising the machine will then mint a thread token value.

Figure 32 - StateMachine type

The *smThreadToken* serves the same purpose as the NFT we were using in the previous chapter to identify our UTXO for the game (Figure 33).

```
data ThreadToken
```

Constructors

```
ThreadToken TxOutRef CurrencySymbol
```

Figure 33 – ThreadToken type

If we look at the thread token it's a reference to a UTXO and a currency symbol. And the UTXO in our case will be the one that uniquely identifies the minting transaction of the NFT. We don't have to worry about the NFT, it will automatically be taken care of. So, let's look now at the code in *StateMachine.hs* that uses now state machines.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass      #-}
3  {-# LANGUAGE DeriveGeneric       #-}
4  {-# LANGUAGE FlexibleContexts    #-}
5  {-# LANGUAGE MultiParamTypeClasses #-}
6  {-# LANGUAGE NoImplicitPrelude   #-}
7  {-# LANGUAGE OverloadedStrings   #-}
8  {-# LANGUAGE ScopedTypeVariables #-}
9  {-# LANGUAGE TemplateHaskell     #-}
10 {-# LANGUAGE TypeApplications     #-}
11 {-# LANGUAGE TypeFamilies         #-}
12 {-# LANGUAGE TypeOperators        #-}
13
14 module Week07.StateMachine
15   ( Game (..)
16   , GameChoice (..)
17   , FirstParams (..)
18   , SecondParams (..)
19   , GameSchema
20   , Last (..)
21   , ThreadToken
22   , Text
23   , endpoints
24   ) where
25
26 import           Control.Monad           hiding (fmap)
27 import           Data.Aeson              (FromJSON, ToJSON)
28 import           Data.Monoid             (Last (..))
29 import           Data.Text              (Text, pack)
30 import           GHC.Generics            (Generic)
31 import           Ledger                  hiding (singleton)
32 import           Ledger.Ada              as Ada
33 import           Ledger.Constraints      as Constraints
34 import           Ledger.Typed.Tx
35 import qualified Ledger.Typed.Scripts    as Scripts
36 import           Plutus.Contract        as Contract
37 import           Plutus.Contract.StateMachine
38 import qualified PlutusTx
39 import           PlutusTx.Prelude       hiding (Semigroup(..), check,
                                         unless)
```

```

40 import Playground.Contract (ToSchema)
41 import Prelude              (Semigroup (..), Show (..),
                              String)
42 import qualified Prelude
43
44 data Game = Game
45   { gFirst      :: !PaymentPubKeyHash
46   , gSecond     :: !PaymentPubKeyHash
47   , gStake      :: !Integer
48   , gPlayDeadline :: !POSIXTime
49   , gRevealDeadline :: !POSIXTime
50   , gToken      :: !ThreadToken
51   } deriving (Show, Generic, FromJSON, ToJSON, Prelude.Eq)
52
53 PlutusTx.makeLift ''Game
54
55 data GameChoice = Zero | One
56   deriving (Show, Generic, FromJSON, ToJSON, ToSchema, Prelude.Eq,
             Prelude.Ord)
57
58 instance Eq GameChoice where
59   {-# INLINABLE (==) #-}
60   Zero == Zero = True
61   One  == One  = True
62   _    == _    = False
63
64 PlutusTx.unstableMakeIsData ''GameChoice
65
66 data GameDatum = GameDatum BuiltinByteString (Maybe GameChoice) | Finished
67   deriving Show
68
69 instance Eq GameDatum where
70   {-# INLINABLE (==) #-}
71   GameDatum bs mc == GameDatum bs' mc' = (bs == bs') && (mc == mc')
72   Finished        == Finished        = True
73   _               == _               = False
74
75 PlutusTx.unstableMakeIsData ''GameDatum
76
77 data GameRedeemer = Play GameChoice | Reveal BuiltinByteString |
                    ClaimFirst | ClaimSecond
78   deriving Show
79
80 PlutusTx.unstableMakeIsData ''GameRedeemer
81

```

```

82 {-# INLINABLE lovelaces #-}
83 lovelaces :: Value -> Integer
84 lovelaces = Ada.getLovelace . Ada.fromValue
85
86 {-# INLINABLE gameDatum #-}
87 gameDatum :: TxOut -> (DatumHash -> Maybe Datum) -> Maybe GameDatum
88 gameDatum o f = do
89     dh      <- txOutDatum o
90     Datum d <- f dh
91     PlutusTx.fromBuiltinData d
92
93 {-# INLINABLE transition #-}
94 transition :: Game -> State GameDatum -> GameRedeemer -> Maybe
95             (TxConstraints Void Void, State GameDatum)
96 transition game s r = case (stateValue s, stateData s, r) of
97     (v, GameDatum bs Nothing, Play c)
98         | lovelaces v == gStake game      ->
99             Just ( Constraints.mustBeSignedBy (gSecond game) <>
100                  Constraints.mustValidateIn (to $ gPlayDeadline game)
101                  , State (GameDatum bs $ Just c) (lovelaceValueOf $
102                  2 * gStake game) )
103
104     (v, GameDatum _ (Just _), Reveal _)
105         | lovelaces v == (2 * gStake game) ->
106             Just ( Constraints.mustBeSignedBy (gFirst game) <>
107                  Constraints.mustValidateIn (to $ gRevealDeadline game)
108                  , State Finished mempty )
109
110     (v, GameDatum _ Nothing, ClaimFirst)
111         | lovelaces v == gStake game      ->
112             Just ( Constraints.mustBeSignedBy (gFirst game) <>
113                  Constraints.mustValidateIn (from $ 1 + gPlayDeadline game)
114                  , State Finished mempty )
115
116     (v, GameDatum _ (Just _), ClaimSecond)
117         | lovelaces v == (2 * gStake game) ->
118             Just ( Constraints.mustBeSignedBy (gSecond game) <>
119                  Constraints.mustValidateIn (from $ 1 +
120                  gRevealDeadline game)
121                  , State Finished mempty )
122
123     _
124         -> Nothing
125
126 {-# INLINABLE final #-}
127 final :: GameDatum -> Bool

```

```

120 final Finished = True
121 final _        = False
122
123 {-# INLINABLE check #-}
124 check :: BuiltinByteString -> BuiltinByteString -> GameDatum ->
        GameRedeemer -> ScriptContext -> Bool
125 check bsZero' bsOne' (GameDatum bs (Just c)) (Reveal nonce) _ =
126     sha2_256 (nonce `appendByteString` if c == Zero then bsZero' else
        bsOne') == bs
127 check _ _ _ _ _ = True
128
129 {-# INLINABLE gameStateMachine #-}
130 gameStateMachine :: Game -> BuiltinByteString -> BuiltinByteString ->
        StateMachine GameDatum GameRedeemer
131 gameStateMachine game bsZero' bsOne' = StateMachine
132     { smTransition = transition game
133     , smFinal      = final
134     , smCheck      = check bsZero' bsOne'
135     , smThreadToken = Just $ gToken game
136     }
137
138 {-# INLINABLE mkGameValidator #-}
139 mkGameValidator :: Game -> BuiltinByteString -> BuiltinByteString ->
        GameDatum -> GameRedeemer -> ScriptContext -> Bool
140 mkGameValidator game bsZero' bsOne' = mkValidator $ gameStateMachine
        game bsZero' bsOne'
141
142 type Gaming = StateMachine GameDatum GameRedeemer
143
144 bsZero, bsOne :: BuiltinByteString
145 bsZero = "0"
146 bsOne  = "1"
147
148 gameStateMachine' :: Game -> StateMachine GameDatum GameRedeemer
149 gameStateMachine' game = gameStateMachine game bsZero bsOne
150
151 typedGameValidator :: Game -> Scripts.TypedValidator Gaming
152 typedGameValidator game = Scripts.mkTypedValidator @Gaming
153     ($$(PlutusTx.compile [| mkGameValidator |]))
154     `PlutusTx.applyCode` PlutusTx.liftCode game
155     `PlutusTx.applyCode` PlutusTx.liftCode bsZero
156     `PlutusTx.applyCode` PlutusTx.liftCode bsOne)
157     $$(PlutusTx.compile [| wrap |])
158 where
159     wrap = Scripts.wrapValidator @GameDatum @GameRedeemer

```

```

160
161 gameValidator :: Game -> Validator
162 gameValidator = Scripts.validatorScript . typedGameValidator
163
164 gameAddress :: Game -> Ledger.Address
165 gameAddress = scriptAddress . gameValidator
166
167 gameClient :: Game -> StateMachineClient GameDatum GameRedeemer
168 gameClient game = mkStateMachineClient $ StateMachineInstance
    (gameStateMachine' game) (typedGameValidator game)
169
170 data FirstParams = FirstParams
171     { fpSecond      :: !PaymentPubKeyHash
172     , fpStake       :: !Integer
173     , fpPlayDeadline :: !POSIXTime
174     , fpRevealDeadline :: !POSIXTime
175     , fpNonce       :: !BuiltinByteString
176     , fpChoice      :: !GameChoice
177     } deriving (Show, Generic, FromJSON, ToJSON, ToSchema)
178
179 mapError' :: Contract w s SMContractError a -> Contract w s Text a
180 mapError' = mapError $ pack . show
181
182 waitUntilTimeHasPassed :: AsContractError e => POSIXTime ->
    Contract w s e ()
183 waitUntilTimeHasPassed t = void $ awaitTime t >> waitNSlots 1
184
185 firstGame :: forall s. FirstParams -> Contract (Last ThreadToken) s Text ()
186 firstGame fp = do
187     pkh <- Contract.ownPaymentPubKeyHash
188     tt <- mapError' getThreadToken
189     let game = Game
190         { gFirst      = pkh
191         , gSecond     = fpSecond fp
192         , gStake      = fpStake fp
193         , gPlayDeadline = fpPlayDeadline fp
194         , gRevealDeadline = fpRevealDeadline fp
195         , gToken      = tt
196         }
197     client = gameClient game
198     v      = lovelaceValueOf (fpStake fp)
199     c      = fpChoice fp
200     bs     = sha2_256 $ fpNonce fp `appendByteString` if c == Zero then
        bsZero else bsOne
201     void $ mapError' $ runInitialise client (GameDatum bs Nothing) v

```

```

202     logInfo @String $ "made first move: " ++ show (fpChoice fp)
203     tell $ Last $ Just tt
204
205     waitUntilTimeHasPassed $ fpPlayDeadline fp
206
207     m <- mapError' $ getOnChainState client
208     case m of
209         Nothing      -> throwError "game output not found"
210         Just (o, _) -> case tyTxOutData $ ocsTxOut o of
211
212             GameDatum _ Nothing -> do
213                 logInfo @String "second player did not play"
214                 void $ mapError' $ runStep client ClaimFirst
215                 logInfo @String "first player reclaimed stake"
216
217             GameDatum _ (Just c') | c' == c -> do
218                 logInfo @String "second player played and lost"
219                 void $ mapError' $ runStep client $ Reveal $ fpNonce fp
220                 logInfo @String "first player revealed and won"
221
222             _ -> logInfo @String "second player played and won"
223
224 data SecondParams = SecondParams
225     { spFirst      :: !PaymentPubKeyHash
226     , spStake      :: !Integer
227     , spPlayDeadline :: !POSIXTime
228     , spRevealDeadline :: !POSIXTime
229     , spChoice     :: !GameChoice
230     , spToken      :: !ThreadToken
231     } deriving (Show, Generic, FromJSON, ToJSON)
232
233 secondGame :: forall w s. SecondParams -> Contract w s Text ()
234 secondGame sp = do
235     pkh <- Contract.ownPaymentPubKeyHash
236     let game = Game
237         { gFirst      = spFirst sp
238         , gSecond     = pkh
239         , gStake      = spStake sp
240         , gPlayDeadline = spPlayDeadline sp
241         , gRevealDeadline = spRevealDeadline sp
242         , gToken      = spToken sp
243         }
244     client = gameClient game
245     m <- mapError' $ getOnChainState client
246     case m of

```



```

247     Nothing          -> logInfo @String "no running game found"
248     Just (o, _) -> case tyTxOutData $ ocsTxOut o of
249         GameDatum _ Nothing -> do
250             logInfo @String "running game found"
251             void $ mapError' $ runStep client $ Play $ spChoice sp
252             logInfo @String $ "made second move: " ++ show (spChoice sp)
253
254             waitUntilTimeHasPassed $ spRevealDeadline sp
255
256             m' <- mapError' $ getOnChainState client
257             case m' of
258                 Nothing -> logInfo @String "first player won"
259                 Just _ -> do
260                     logInfo @String "first player didn't reveal"
261                     void $ mapError' $ runStep client ClaimSecond
262                     logInfo @String "second player won"
263
264             _ -> throwError "unexpected datum"
265
266 type GameSchema = Endpoint "first" FirstParams .\ /
267                  Endpoint "second" SecondParams
268
269 endpoints :: Contract (Last ThreadToken) GameSchema Text ()
270 endpoints = awaitPromise (first `select` second) >> endpoints
271
272 where
273     first  = endpoint @"first"  firstGame
274     second = endpoint @"second" secondGame

```

In the beginning we again define our game type (44-51). The difference to the previous example is that the token was of type asset class and now it's of type thread token. The game choice stays the same. What changes is the game datum where we add the second constructor called *Finished* that we did not need before (66-67). We need it for state machine mechanism to work. And when we make the Eq instance for the game datum we take this second constructor in account. The redeemer and the two helper functions are the same as before (77-91). Then we write the transition function for the state machine. It takes the game parameter then the state datum and the redeemer. And then we return a Nothing if the transition is not allowed or just a pair of new state and constraints on the transaction. First, we again have the case where the 1st player has already moved and the 2nd player wants to make his move with choice c. The parameter s holds the combination of datum and value. The *stateValue* parameter gives us the value of the UTXO we are consuming and *stateData* gives us the datum. So first we check that the value is the stake of the game (97). If the check passes, we return a Just where we put the constraints on the transaction for the signature and validity interval and include the new state.

Compared to our previous code where we did not use SM all conditions from the first case are also satisfied in our first SM case. The second case is when the 2nd player has moved and the 1st player realizes he has won and wants to reveal his nonce. Compared to our previous code the condition for revealing the nonce is missing here because we cannot formulate it as a constraint and we will take care of it later. We also do not return the token because it gets burned when we reached our final state. The third case is where the 2nd player does not move so the 1st player wants to reclaim his stake. If we compare the conditions, we see we satisfy all as in the non-SM example. The last case is that the 2nd player has moved and the 1st player does not reveal his nonce. Again, all the all conditions are satisfied as in the non-SM example. In the end we say for all other combinations we return nothing. All together the code gets shorter because we do not need any helper functions and do not need to take care of the NFT. What we have to add for the SM is the final state which basically says finished (118-121). Because we left out the nonce check in the transition function, we define it now (123-127).

Next, we can define our state machine that takes in the game parameter and two byte strings (129-136). In the body we provide the four fields we have just defined. In the function *mkGameValidator* we use the *mkValidator* function to transform our game state machine to a validator (138-140). We define the *gameStateMachine* where we fix the two additional parameters and we can use then this function in the off-chain code (148-149). The compilation process, computation of the validator and the address is same as in the previous example. What we additionally define is the state machine client and we need this to be able to interact with the SM from our wallet (167-168) (Figure 34).

data StateMachineClient s i

Source

Client-side definition of a state machine.

Constructors

StateMachineClient

scInstance :: StateMachineInstance s i

scChooser :: [OnChainState s i] -> Either SMContractError (OnChainState s i)

The instance of the state machine, defining the machine's transitions, its final states and its check function.

A function that chooses the relevant on-chain state, given a list of all potential on-chain states found at the contract address.

Figure 34 – StateMachineClient type

If we look at the definition, we see it has 2 fields. The state machine instance field is defined by two new fields that are the state machine and the validator code for this SM (Figure 35).

```
data StateMachineInstance s i
```

Constructors

```
StateMachineInstance
  stateMachine :: StateMachine s i           The state machine specification.
  typedValidator :: TypedValidator (StateMachine s i) The validator code for this state machine.
```

Figure 35 - State machine instance

The chooser field handles the case if we do not use the thread token mechanism and have several UTXOs sitting at our address. With it we can pick the right one. Now we come to the off-chain code. The *FirstParams* type is defined same way as in the previous example. The state machine contracts have a specific constraint on the Error type called *SMContractError*. But we want to use text for the error type so we define a function that handles the conversion (179-180). The helper function *waitUntilTimeHasPassed* is the same as in the previous code. In the *firstGame* contract we again first look up our own key and then we get our thread token (188). Then we define the game parameter (189-196) and after it we define the game client. Next three variables the stake, choice and hash are same as before. In line 201 the *runInitialise* function first mints the NFT corresponding to the thread token, then it creates the UTXO at the state machine address and we give the datum and value as input arguments. After that we log that we have made the first move and then we use the *tell* function to communicate the thread token. This is for the second player to be able to find the game. Then we wait for the 2nd player to move. In the previous example after the wait, we needed to use the helper function *findGameOutput* which we can now replace with the *getOnChainState* function (Figure 36).

```
getOnChainState :: (AsSMContractError e, FromData state, ToData state) => StateMachineClient
state i -> Contract w schema e (Maybe (OnChainState state i, Map TxOutRef ChainIndexTxOut))
```

Source

Get the current on-chain state of the state machine instance. Return Nothing if there is no state on chain. Throws an *SMContractError* if the number of outputs at the machine address is greater than one.

Figure 36 – getOnchainState function

It takes in a state machine client and returns a maybe of on-chain state and a map with transaction output references as keys and chain indexes as values. The on-chain state has 3 fields available but we use only the first two (Figure 37).

```
data OnChainState s i
```

Typed representation of the on-chain state of a state machine instance

Constructors

OnChainState

<code>ocsTxOut :: TypedScriptTxOut (StateMachine s i)</code>	Typed transaction output
<code>ocsTxOutRef :: TypedScriptTxOutRef (StateMachine s i)</code>	Typed UTXO
<code>ocsTx :: ChainIndexTx</code>	Transaction that produced the output

Figure 37 – OnChainState type

The first two fields are further displayed in Figure 38 and Figure 39. The typed script transaction output contains the transaction output and datum that is already deserialized which in our case means it would be of type *GameDatum*. And the typed script transaction output reference contains the transaction output reference and another typed script transaction output.

```
data TypedScriptTxOut a
```

A `TxOut` tagged by a phantom type: and the connection type of the output.

Constructors

```
(FromData (DatumType a), ToData (DatumType a)) => TypedScriptTxOut  
tyTxOutTxOut :: TxOut  
tyTxOutData :: DatumType a
```

Figure 38 – TypedScriptTxOutput type

```
data TypedScriptTxOutRef a
```

A `TxOutRef` tagged by a phantom type: and the connection type of the output.

Constructors

TypedScriptTxOutRef

```
tyTxOutRefRef :: TxOutRef  
tyTxOutRefOut :: TypedScriptTxOut a
```

Figure 39 – TypedScriptTxOutputRef type

Once we have our on-chain state we can use the *tyTxOutData* function to directly access the datum. And after that we have the two cases that the second player has not moved or that he has moved and we see that we won. For the first case we do the reclaim (214). And for the second case we reveal our nonce (219). In both cases we use the *runStep* function that takes in a state machine client and the redeemer and returns a contract (Figure 40). It actually creates a transaction, submits it and transitions the state machine. The *TransitionResult* type encodes whether the transition failed or succeeded. All the constraints are available to the state machine mechanism and that allows it to automatically compose the transaction that is needed. We do not need any lookups or helper functions.

runStep # Source

```

:: forall w e state schema input. (AsSMContractError e, FromData state, ToData state, ToData input)
=> StateMachineClient state input

-> input

-> Contract w schema e (TransitionResult state input)

```

Run one step of a state machine, returning the new state.

Figure 40 - Run step

The second game contract is similar to the first. We first look up our key hash and then define the game parameters and the client. With the *getOnChainState* function we again get the UTXO that represents the state machine. If we don't find the game, we just log a message but if we do find it we look up the output and the game datum. And then again, we use *runStep* to play our move. After it we wait until the reveal deadline has passed (254). We then again check the new state and if there is a Nothing the first player has won and we don't do anything. Else we claim our win (261). In order to test this, we use the code from the *TestStateMachine.hs* example.

```

1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE FlexibleContexts     #-}
3  {-# LANGUAGE MultiParamTypeClasses #-}
4  {-# LANGUAGE NoImplicitPrelude    #-}
5  {-# LANGUAGE NumericUnderscores  #-}
6  {-# LANGUAGE OverloadedStrings    #-}
7  {-# LANGUAGE ScopedTypeVariables  #-}
8  {-# LANGUAGE TypeApplications     #-}
9  {-# LANGUAGE TypeFamilies         #-}
10

```

```

11 module Week07.TestStateMachine
12   ( test
13   , test'
14   , GameChoice (..)
15   ) where
16
17 import           Control.Monad           hiding (fmap)
18 import           Control.Monad.Freeer.Extras as Extras
19 import           Data.Default             (Default (..))
20 import           Ledger.TimeSlot
21 import           Plutus.Trace.Emulator   as Emulator
22 import           PlutusTx.Prelude
23 import           Prelude                   (IO, Show (..))
24 import           Wallet.Emulator.Wallet
25
26 import           Week07.StateMachine
27
28 test :: IO ()
29 test = do
30     test' Zero Zero
31     test' Zero One
32     test' One Zero
33     test' One One
34
35 test' :: GameChoice -> GameChoice -> IO ()
36 test' c1 c2 = runEmulatorTraceIO $ myTrace c1 c2
37
38 myTrace :: GameChoice -> GameChoice -> EmulatorTrace ()
39 myTrace c1 c2 = do
40     Extras.logInfo $ "first move: " ++ show c1 ++
41                     ", second move: " ++ show c2
42
43 let w1 = knownWallet 1
44 let w2 = knownWallet 2
45
46 h1 <- activateContractWallet w1 endpoints
47 h2 <- activateContractWallet w2 endpoints
48
49 let pkh1      = mockWalletPaymentPubKeyHash w1
50     pkh2      = mockWalletPaymentPubKeyHash w2
51     stake     = 5_000_000
52     deadline1 = slotToEndPOSIXTime def 5
53     deadline2 = slotToEndPOSIXTime def 10
54
55 fp = FirstParams

```

```

55         { fpSecond          = pkh2
56         , fpStake           = stake
57         , fpPlayDeadline    = deadline1
58         , fpRevealDeadline  = deadline2
59         , fpNonce           = "SECRETNONCE"
60         , fpChoice          = c1
61         }
62
63     callEndpoint @"first" h1 fp
64
65     tt <- getTT h1
66
67     let sp = SecondParams
68         { spFirst          = pkh1
69         , spStake           = stake
70         , spPlayDeadline    = deadline1
71         , spRevealDeadline  = deadline2
72         , spChoice          = c2
73         , spToken           = tt
74         }
75
76     void $ Emulator.waitNSlots 3
77
78     callEndpoint @"second" h2 sp
79
80     void $ Emulator.waitNSlots 10
81 where
82     getTT :: ContractHandle (Last ThreadToken) GameSchema Text ->
83           EmulatorTrace ThreadToken
84     getTT h = do
85         void $ Emulator.waitNSlots 1
86         Last m <- observableState h
87         case m of
88             Nothing -> getTT h
89             Just tt -> Extras.logInfo ("read thread token " ++ show tt) >>
89                 return tt

```

It is very similar to the even odd test code. It is simpler because we do not have to specify an initial state. We define our *test'* function that takes in our choices and runs the emulator trace (35-36). The default initial state for the SM is 10 wallets with 100 ADA each. Because of that we decrease the stake from 100 to 5 ADA. What is different in this trace is that we use the *slotToEndPOSIXTime* function. We use this function to fix the issue that off-chain part should construct a transaction that has a correct time interval but the conversion between slots and

posix time does not always work when using state machines because of unfaithfulness. So, we see that the state machine approach automatically guarantees that the on-chain and off-chain code fit together. The produced transactions from it are correctly validated.

7.4 Homework

For homework we will modify the state machine code to implement the rock-paper-scissors game. Let's look at the code *RockPaperScissors.hs*.

```
1  data Game = Game
2    { gFirst      :: !PaymentPubKeyHash
3    , gSecond     :: !PaymentPubKeyHash
4    , gStake      :: !Integer
5    , gPlayDeadline :: !POSIXTime
6    , gRevealDeadline :: !POSIXTime
7    , gToken      :: !ThreadToken
8    } deriving (Show, Generic, FromJSON, ToJSON, Prelude.Eq)
9
10 PlutusTx.makeLift ''Game
11
12 data GameChoice = Rock | Paper | Scissors
13   deriving (Show, Generic, FromJSON, ToJSON, Prelude.Eq, Prelude.Ord)
14
15 instance Eq GameChoice where
16   {-# INLINABLE (==) #-}
17   Rock      == Rock      = True
18   Paper     == Paper     = True
19   Scissors  == Scissors  = True
20   _         == _         = False
21
22 PlutusTx.unstableMakeIsData ''GameChoice
23
24 {-# INLINABLE beats #-}
25 beats :: GameChoice -> GameChoice -> Bool
26 beats Rock      Scissors = True
27 beats Paper     Rock      = True
28 beats Scissors Paper     = True
29 beats _         _         = False
30
31 data GameDatum = GameDatum BuiltinByteString (Maybe GameChoice) | Finished
32   deriving Show
33
34 instance Eq GameDatum where
35   {-# INLINABLE (==) #-}
```



```

36     GameDatum bs mc == GameDatum bs' mc' = (bs == bs') && (mc == mc')
37     Finished      == Finished      = True
38     -             == -             = False
39
40     PlutusTx.unstableMakeIsData ''GameDatum
41
42     data GameRedeemer = Play GameChoice | Reveal BuiltinByteString GameChoice |
                          ClaimFirst | ClaimSecond
43     deriving Show
44
45     PlutusTx.unstableMakeIsData ''GameRedeemer
46
47     {-# INLINABLE lovelaces #-}
48     lovelaces :: Value -> Integer
49     lovelaces = Ada.getLovelace . Ada.fromValue
50
51     {-# INLINABLE transition #-}
52     transition :: Game -> State GameDatum -> GameRedeemer -> Maybe
                    (TxConstraints Void Void, State GameDatum)
53     transition game s r = case (stateValue s, stateData s, r) of
54         (v, GameDatum bs Nothing, Play c)
55             | lovelaces v == gStake game                ->
56                 Just ( Constraints.mustBeSignedBy (gSecond game) <>
57                     Constraints.mustValidateIn (to $ gPlayDeadline game)
58                     , State (GameDatum bs $ Just c) (lovelaceValueOf $ 2 *
59                         gStake game) )
56
57         (v, GameDatum _ (Just c), Reveal _ c')
58             | (lovelaces v == (2 * gStake game)) &&
59               (c' `beats` c)                            ->
60                 Just ( Constraints.mustBeSignedBy (gFirst game) <>
61                     Constraints.mustValidateIn (to $ gRevealDeadline game)
62                     , State Finished mempty )
63
64         | (lovelaces v == (2 * gStake game)) &&
65           (c' == c)                                       ->
66                 Just ( Constraints.mustBeSignedBy (gFirst game) <>
67                     Constraints.mustValidateIn (to $ gRevealDeadline game) <>
68                     Constraints.mustPayToPubKey (gSecond game)
69                     (lovelaceValueOf $ gStake game)
70                     , State Finished mempty )
71
72         (v, GameDatum _ Nothing, ClaimFirst)
73             | lovelaces v == gStake game                ->

```

```

    Just ( Constraints.mustBeSignedBy (gFirst game) <>
75         Constraints.mustValidateIn (from $ 1 + gPlayDeadline game)
76         , State Finished mempty )
77
78 (v, GameDatum _ (Just _), ClaimSecond)
79 | lovelaces v == (2 * gStake game)      ->
    Just ( Constraints.mustBeSignedBy (gSecond game) <>
80         Constraints.mustValidateIn (from $ 1 + gRevealDeadline
                                         game)
81         , State Finished mempty )
82
83     -                                     -> Nothing
84
85 {-# INLINABLE final #-}
86 final :: GameDatum -> Bool
87 final Finished = True
88 final _        = False
89
90 {-# INLINABLE check #-}
91 check :: BuiltinByteString -> BuiltinByteString -> BuiltinByteString ->
    GameDatum -> GameRedeemer -> ScriptContext -> Bool
92 check bsRock' bsPaper' bsScissors' (GameDatum bs (Just _))
    (Reveal nonce c) _ =
93     sha2_256 (nonce `appendByteString` toBS c) == bs
94 where
95     toBS :: GameChoice -> BuiltinByteString
96     toBS Rock      = bsRock'
97     toBS Paper     = bsPaper'
98     toBS Scissors  = bsScissors'
99     check _ _ _ _ _ = True
100
101 {-# INLINABLE gameStateMachine #-}
102 gameStateMachine :: Game -> BuiltinByteString -> BuiltinByteString ->
    BuiltinByteString -> StateMachine GameDatum GameRedeemer
103 gameStateMachine game bsRock' bsPaper' bsScissors' = StateMachine
104     { smTransition = transition game
105     , smFinal      = final
106     , smCheck      = check bsRock' bsPaper' bsScissors'
107     , smThreadToken = Just $ gToken game
108     }
109
110 {-# INLINABLE mkGameValidator #-}
111 mkGameValidator :: Game -> BuiltinByteString -> BuiltinByteString ->
    BuiltinByteString -> GameDatum -> GameRedeemer ->
    ScriptContext -> Bool

```

```

112 mkGameValidator game bsRock' bsPaper' bsScissors' = mkValidator $
    gameStateMachine game bsRock' bsPaper' bsScissors'
113
114 type Gaming = StateMachine GameDatum GameRedeemer
115
116 bsRock, bsPaper, bsScissors :: BuiltinByteString
117 bsRock      = "R"
118 bsPaper     = "P"
119 bsScissors  = "S"
120
121 gameStateMachine' :: Game -> StateMachine GameDatum GameRedeemer
122 gameStateMachine' game = gameStateMachine game bsRock bsPaper bsScissors
123
124 typedGameValidator :: Game -> Scripts.TypedValidator Gaming
125 typedGameValidator game = Scripts.mkTypedValidator @Gaming
126   ($$(PlutusTx.compile [| mkGameValidator |]))
127   `PlutusTx.applyCode` PlutusTx.liftCode game
128   `PlutusTx.applyCode` PlutusTx.liftCode bsRock
129   `PlutusTx.applyCode` PlutusTx.liftCode bsPaper
130   `PlutusTx.applyCode` PlutusTx.liftCode bsScissors)
131   $(PlutusTx.compile [| wrap |])
132   where
133     wrap = Scripts.wrapValidator @GameDatum @GameRedeemer
134
135 gameValidator :: Game -> Validator
136 gameValidator = Scripts.validatorScript . typedGameValidator
137
138 gameAddress :: Game -> Ledger.Address
139 gameAddress = scriptAddress . gameValidator
140
141 gameClient :: Game -> StateMachineClient GameDatum GameRedeemer
142 gameClient game = mkStateMachineClient $ StateMachineInstance
    (gameStateMachine' game) (typedGameValidator game)
143
144 data FirstParams = FirstParams
145   { fpSecond      :: !PaymentPubKeyHash
146   , fpStake       :: !Integer
147   , fpPlayDeadline :: !POSIXTime
148   , fpRevealDeadline :: !POSIXTime
149   , fpNonce       :: !BuiltinByteString
150   , fpChoice      :: !GameChoice
151   } deriving (Show, Generic, FromJSON, ToJSON)
152
153 mapError' :: Contract w s SMContractError a -> Contract w s Text a
154 mapError' = mapError $ pack . show

```

```

155
156 waitUntilTimeHasPassed :: AsContractError e => POSIXTime ->
                                Contract w s e ()
157 waitUntilTimeHasPassed t = void $ awaitTime t >> waitNSlots 1
158
159 firstGame :: forall s. FirstParams -> Contract (Last ThreadToken) s Text ()
160 firstGame fp = do
161     pkh <- Contract.ownPaymentPubKeyHash
162     tt <- mapError' getThreadToken
163     let game = Game
164         { gFirst      = pkh
165         , gSecond     = fpSecond fp
166         , gStake       = fpStake fp
167         , gPlayDeadline = fpPlayDeadline fp
168         , gRevealDeadline = fpRevealDeadline fp
169         , gToken       = tt
170         }
171     client = gameClient game
172     v      = lovelaceValueOf (fpStake fp)
173     c      = fpChoice fp
174     x      = case c of
175         Rock      -> bsRock
176         Paper     -> bsPaper
177         Scissors  -> bsScissors
178     bs      = sha2_256 $ fpNonce fp `appendByteString` x
179     void $ mapError' $ runInitialise client (GameDatum bs Nothing) v
180     logInfo @String $ "made first move: " ++ show (fpChoice fp)
181     tell $ Last $ Just tt
182
183     waitUntilTimeHasPassed $ fpPlayDeadline fp
184
185     m <- mapError' $ getOnChainState client
186     case m of
187         Nothing      -> throwError "game output not found"
188         Just (o, _) -> case tyTxOutData $ ocsTxOut o of
189
190             GameDatum _ Nothing -> do
191                 logInfo @String "second player did not play"
192                 void $ mapError' $ runStep client ClaimFirst
193                 logInfo @String "first player reclaimed stake"
194
195             GameDatum _ (Just c') | c `beats` c' || c' == c -> do
196                 logInfo @String "second player played and lost or drew"
197                 void $ mapError' $ runStep client $ Reveal (fpNonce fp) c
198                 logInfo @String "first player revealed and won or drew"

```

```

199
200         _ -> logInfo @String "second player played and won"
201
202 data SecondParams = SecondParams
203     { spFirst      :: !PaymentPubKeyHash
204     , spStake      :: !Integer
205     , spPlayDeadline :: !POSIXTime
206     , spRevealDeadline :: !POSIXTime
207     , spChoice     :: !GameChoice
208     , spToken      :: !ThreadToken
209     } deriving (Show, Generic, FromJSON, ToJSON)
210
211 secondGame :: forall w s. SecondParams -> Contract w s Text ()
212 secondGame sp = do
213     pkh <- Contract.ownPaymentPubKeyHash
214     let game = Game
215         { gFirst      = spFirst sp
216         , gSecond     = pkh
217         , gStake      = spStake sp
218         , gPlayDeadline = spPlayDeadline sp
219         , gRevealDeadline = spRevealDeadline sp
220         , gToken      = spToken sp
221         }
222     client = gameClient game
223     m <- mapError' $ getOnChainState client
224     case m of
225     Nothing      -> logInfo @String "no running game found"
226     Just (o, _) -> case tyTxOutData $ ocsTxOut o of
227         GameDatum _ Nothing -> do
228             logInfo @String "running game found"
229             void $ mapError' $ runStep client $ Play $ spChoice sp
230             logInfo @String $ "made second move: " ++ show (spChoice sp)
231
232             waitUntilTimeHasPassed $ spRevealDeadline sp
233
234             m' <- mapError' $ getOnChainState client
235             case m' of
236             Nothing -> logInfo @String "first player won or drew"
237             Just _ -> do
238                 logInfo @String "first player didn't reveal"
239                 void $ mapError' $ runStep client ClaimSecond
240                 logInfo @String "second player won"
241
242         _ -> throwError "unexpected datum"
243

```

```

244 type GameSchema = Endpoint "first" FirstParams .\ /
                          Endpoint "second" SecondParams
245
246 endpoints :: Contract (Last ThreadToken) GameSchema Text ()
247 endpoints = awaitPromise (first `select` second) >> endpoints
248   where
249     first  = endpoint @"first"  firstGame
250     second = endpoint @"second" secondGame

```

The imports and language pragmas are the same as in the SM example. Also, the game type stays the same (1-8). What changes is the game choice parameter and the Eq instance of it. Then we define the *beats* function that takes in two game choices and returns a Bool (15-20). Next, we define the game datum and its Eq instance same way as before. For the game redeemer there is a difference in the reveal option where we have to specify now also the game choice since there are 2 possibilities (42-43). One is that the game is a draw and the other one is that we have won. This will also have effect on our transition function where we will have now 5 possible cases. The difference compared to the previous code is in the second case where the 1st player chooses to reveal his choice and we consider again 2 possibilities. The first is that the 1st player has won and the second is that there is a draw in which case we put a constraint that the 2nd player gets his stake back. Now we proceed to the *check* function that takes in three byte strings for our three choices and checks weather the checksum is valid (90-99). The *gameStateMachine* function gets also updated accordingly to our three choices (101-108). Same is true for the game validator and the compiled code. The rest of the on-chain code is the same as in our previous example. In the first game contract when defining parameters what changes is that we define an additional parameter x that gives us the byte string for our choice (174-177) and we use it when computing the hash of the nonce (178). And another change is in the second case after the play deadline when the 2nd player reveals his choice and has lost or drew (195-198). Then for the second game everything stays the same except the log message on line 236 changes where we say »first player won or drew«.

8 Testing

In this chapter we will look at another example for state machines and then look at how Haskell's property testing is implemented for Plutus smart contracts.

8.1 State Machine Example: Token Sale

Here we will look at an example where a person wants to make a token sale. The idea is we lock tokens in a contract and allow users to buy them for a fixed price. Figure 41 shows a simple example of the token sale and the code follows below.

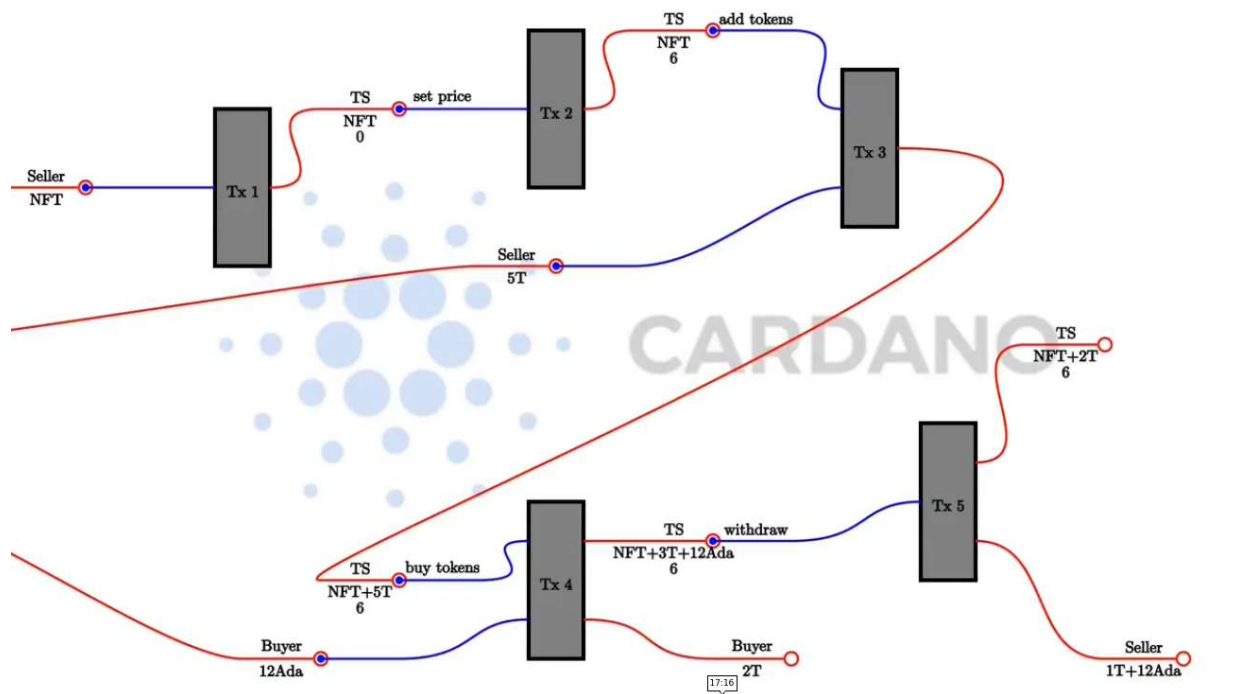


Figure 41 - Token sale workflow

```
1 {-# LANGUAGE DataKinds           #-}
2 {-# LANGUAGE DeriveAnyClass       #-}
3 {-# LANGUAGE DeriveGeneric        #-}
4 {-# LANGUAGE FlexibleContexts     #-}
5 {-# LANGUAGE MultiParamTypeClasses #-}
6 {-# LANGUAGE NoImplicitPrelude    #-}
7 {-# LANGUAGE OverloadedStrings    #-}
8 {-# LANGUAGE ScopedTypeVariables  #-}
9 {-# LANGUAGE TemplateHaskell      #-}
10 {-# LANGUAGE TypeApplications     #-}
11 {-# LANGUAGE TypeFamilies         #-}
12 {-# LANGUAGE TypeOperators        #-}
```

```

13
14 {-# OPTIONS_GHC -g -fplugin-opt PlutusTx.Plugin:coverage-all #-}
15
16 module Week08.TokenSale
17   ( TokenSale (..)
18   , TSRedeemer (..)
19   , tsCovIdx
20   , TSStartSchema
21   , TSUseSchema
22   , startEndpoint
23   , useEndpoints
24   , useEndpoints'
25   ) where
26
27   import           Control.Monad                hiding (fmap)
28   import           Data.Aeson                  (FromJSON, ToJSON)
29   import           Data.Monoid                  (Last (..))
30   import           Data.Text                    (Text, pack)
31   import           GHC.Generics                 (Generic)
32   import           Plutus.Contract              as Contract
33   import           Plutus.Contract.StateMachine
34   import qualified PlutusTx
35   import           PlutusTx.Code                (getCovIdx)
36   import           PlutusTx.Coverage            (CoverageIndex)
37   import           PlutusTx.Prelude              hiding (Semigroup(..), check,
38                                                    unless)
39
40   import           Ledger                       hiding (singleton)
41   import           Ledger.Ada                   as Ada
42   import           Ledger.Constraints           as Constraints
43   import qualified Ledger.Typed.Scripts         as Scripts
44   import           Ledger.Value
45   import           Prelude                      (Semigroup (..), Show (..),
46                                                    uncurry)
47
48   import qualified Prelude
49
50   data TokenSale = TokenSale
51     { tsSeller :: !PaymentPubKeyHash
52     , tsToken  :: !AssetClass
53     , tsTT     :: !ThreadToken
54     } deriving (Show, Generic, FromJSON, ToJSON, Prelude.Eq)
55
56   PlutusTx.makeLift "'TokenSale
57
58   data TSRedeemer =
59     SetPrice Integer

```



```

56     | AddTokens Integer
57     | BuyTokens Integer
58     | Withdraw Integer Integer
59     deriving (Show, Prelude.Eq)
60
61 PlutusTx.unstableMakeIsData ''TSRedeemer
62
63 {-# INLINABLE lovelaces #-}
64 lovelaces :: Value -> Integer
65 lovelaces = Ada.getLovelace . Ada.fromValue
66
67 {-# INLINABLE transition #-}
68 transition :: TokenSale -> State Integer -> TSRedeemer -> Maybe
69             (TxConstraints Void Void, State Integer)
69 transition ts s r = case (stateValue s, stateData s, r) of
70     (v, _, SetPrice p) | p >= 0 ->
71         Just ( Constraints.mustBeSignedBy (tsSeller ts)
72             , State p v )
73     (v, p, AddTokens n) | n > 0 ->
74         Just ( mempty
75             , State p $ v
76               <>
77               assetClassValue (tsToken ts) n )
78     (v, p, BuyTokens n) | n > 0 ->
79         Just ( mempty
80             , State p $ v
81               <>
82               assetClassValue (tsToken ts) (negate n)
83               <> lovelaceValueOf (n * p) )
84     (v, p, Withdraw n l) | n >= 0 && l >= 0 ->
85         Just ( Constraints.mustBeSignedBy (tsSeller ts)
86             , State p $ v
87               <>
88               assetClassValue (tsToken ts) (negate n)
89               <> lovelaceValueOf (negate l) )
90     _ -> Nothing
91
92 {-# INLINABLE tsStateMachine #-}
93 tsStateMachine :: TokenSale -> StateMachine Integer TSRedeemer
94 tsStateMachine ts = mkStateMachine (Just $ tsTT ts) (transition ts)
95                               (const False)

```

```

95
96 {-# INLINABLE mkTSValidator #-}
97 mkTSValidator :: TokenSale -> Integer -> TSRedeemer -> ScriptContext -> Bool
98 mkTSValidator = mkValidator . tsStateMachine
99
100 type TS = StateMachine Integer TSRedeemer
101
102 tsTypedValidator :: TokenSale -> Scripts.TypedValidator TS
103 tsTypedValidator ts = Scripts.mkTypedValidator @TS
104   ($$(PlutusTx.compile [| mkTSValidator |]))
105   `PlutusTx.applyCode` PlutusTx.liftCode ts)
106   ($$(PlutusTx.compile [| wrap |]))
107   where
108     wrap = Scripts.wrapValidator @Integer @TSRedeemer
109
110 tsValidator :: TokenSale -> Validator
111 tsValidator = Scripts.validatorScript . tsTypedValidator
112
113 tsAddress :: TokenSale -> Ledger.Address
114 tsAddress = scriptAddress . tsValidator
115
116 tsClient :: TokenSale -> StateMachineClient Integer TSRedeemer
117 tsClient ts = mkStateMachineClient $ StateMachineInstance
118   (tsStateMachine ts) (tsTypedValidator ts)
119
120
121 tsCovIdx :: CoverageIndex
122 tsCovIdx = getCovIdx $$(PlutusTx.compile [| mkTSValidator |]))
123
124 mapErrorSM :: Contract w s SMContractError a -> Contract w s Text a
125 mapErrorSM = mapError $ pack . show
126
127 startTS :: AssetClass -> Contract (Last TokenSale) s Text ()
128 startTS token = do
129   pkh <- Contract.ownPaymentPubKeyHash
130   tt <- mapErrorSM getThreadToken
131   let ts = TokenSale
132     { tsSeller = pkh
133     , tsToken = token
134     , tsTT = tt
135     }
136   client = tsClient ts
137   void $ mapErrorSM $ runInitialise client 0 mempty
138   tell $ Last $ Just ts
139   logInfo $ "started token sale " ++ show ts
140

```

```

138 setPrice :: TokenSale -> Integer -> Contract w s Text ()
139 setPrice ts p = void $ mapErrorSM $ runStep (tsClient ts) $ SetPrice p
140
141 addTokens :: TokenSale -> Integer -> Contract w s Text ()
142 addTokens ts n = void $ mapErrorSM $ runStep (tsClient ts) $ AddTokens n
143
144 buyTokens :: TokenSale -> Integer -> Contract w s Text ()
145 buyTokens ts n = void $ mapErrorSM $ runStep (tsClient ts) $ BuyTokens n
146
147 withdraw :: TokenSale -> Integer -> Integer -> Contract w s Text ()
148 withdraw ts n l = void $ mapErrorSM $ runStep (tsClient ts) $ Withdraw n l
149
150 type TSStartSchema =
151     Endpoint "start"      (CurrencySymbol, TokenName)
152 type TSUseSchema =
153     Endpoint "set price" Integer
154     .\ Endpoint "add tokens" Integer
155     .\ Endpoint "buy tokens" Integer
156     .\ Endpoint "withdraw" (Integer, Integer)
157
158 startEndpoint :: Contract (Last TokenSale) TSStartSchema Text ()
159 startEndpoint = forever
160     $ handleError logError
161     $ awaitPromise
162     $ endpoint @"start" $ startTS . AssetClass
163
164 useEndpoints' :: ( HasEndpoint "set price" Integer s
165     , HasEndpoint "add tokens" Integer s
166     , HasEndpoint "buy tokens" Integer s
167     , HasEndpoint "withdraw" (Integer, Integer) s
168     )
169     => TokenSale
170     -> Contract () s Text ()
171 useEndpoints' ts = forever
172     $ handleError logError
173     $ awaitPromise
174     $ setPrice' `select` addTokens' `select` buyTokens'
175     `select` withdraw'
176
177 where
178     setPrice' = endpoint @"set price" $ setPrice ts
179     addTokens' = endpoint @"add tokens" $ addTokens ts
180     buyTokens' = endpoint @"buy tokens" $ buyTokens ts
181     withdraw' = endpoint @"withdraw" $ Prelude.uncurry $ withdraw ts
182
183 useEndpoints :: TokenSale -> Contract () TSUseSchema Text ()

```

```
182 useEndpoints = useEndpoints'
```

First, we define the `TokenSale` that will be used for parameterizing our transition function (46-50). It contains the sellers address, the tokens we will sell and the thread token for identifying the UTXO for the token sale. For the redeemer type we provide the operations that we saw in Figure 41 (54-59). The integers represent price in lovelace and number of tokens. Then we define a helper function to extract lovelace amount from a value type (63-65). In the transition function we first put in our parameter, then the state that defines the price of the token and the remaining two parameters follow the same patten as in the previous SM example (68). In the case statement we split the state to the value and the datum (69). Next follow the four transitions. In the first case the seller wants to set the price to the new value p (70-71). We only allow that if the price is non-negative. The only constraints we put is that the seller has to sign the transaction and for the state we set now p as the new datum and v as value stays the same. In the second case we add tokens and demand that the amount is positive (73-76). We put no constraints to this action so anybody can add tokens to the contract. For the new state the price does not change but the value changes that are now the old value plus the number of tokens that are added. Third case is we define buying tokens (78-82). Also, here we put no constraints so anybody can buy tokens. The price again does not change but the value is reduced for the number of tokens that a person bought and the corresponding price in ADA is added. In the last case we define the withdrawal of tokens and lovelace (84-88). Here we insist that the seller has to sign the transaction. And the value decreases for the number of tokens and lovelace that the seller wants to withdraw. In the end we also define the case that all other transitions are illegal. Now that we have the transition function, we define the state machine where we use the smart constructor called *mkStateMachine* (92-94). It takes in 3 arguments. The first is maybe the thread token, then the transition function and the function that determines if states are final or not. And in our case, we do not have a final state so we can always return `False`. After that we can turn our state machine into a validator function (96-98) and compile it (102-107). We also do not need the SM check function. Then we define the state machine client that is used to interact with the SM in the off-chain code (115-116). What is new is that we define the coverage index (118-119). That is related to getting coverage information for tests. Next, we define our helper function that transforms the type of the error from the SM type to text (121-122). This was now the on-chain code.

For the off-chain code we first define the contract function *startTS*. It takes in one parameter and uses the writer functionality. First, we look up our own payment public key hash and get the thread token. Then we define the token sale and client parameters (128-133). Next, we initialize the client with the initial state 0 and mempty. After that we tell the token sell value so

that other contracts are able to lookup that value and participate in the token sale (135). And in the end, we log a message. Then come simple functions corresponding to actions we can take and they are guaranteed to sync with the on-chain code i.e., they create transactions that are valid and will pass validation (138-148). We now have to define 2 schemas. One to start the token sale that will have only one endpoint (150-151). And one to use the token sale that has four endpoints (152-156). For the withdraw action we first put in how many tokens we want to withdraw and then how many lovelace. The start endpoint and use endpoint bundle up the actions we have defined (158-179). The *uncurry* function takes two separate input parameters for a function call and converts them into a pair of integers in a tuple for a function call.

```
uncurry :: (a -> b -> c) -> (a, b) -> c
Prelude> :t uncurry (+)
uncurry (+) :: Num c => (c, c) -> c
```

To try now out this code we can use the emulator trace defined in the test/Spec/Trace.hs file.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE FlexibleContexts     #-}
3  {-# LANGUAGE MultiParamTypeClasses #-}
4  {-# LANGUAGE NumericUnderscores  #-}
5  {-# LANGUAGE OverloadedStrings    #-}
6  {-# LANGUAGE ScopedTypeVariables  #-}
7  {-# LANGUAGE TypeApplications     #-}
8  {-# LANGUAGE TypeFamilies         #-}
9
10 module Spec.Trace
11   ( tests
12   , testCoverage
13   , runMyTrace
14   ) where
15
16 import           Control.Exception           (try)
17 import           Control.Lens
18 import           Control.Monad              hiding (fmap)
19 import           Control.Monad.Freeer.Extras as Extras
20 import           Data.Default                (Default (..))
21 import           Data.IRef
22 import qualified Data.Map                    as Map
23 import           Data.Monoid                 (Last (..))
24 import           Ledger
25 import           Ledger.Value
26 import           Ledger.Ada                  as Ada
27 import           Plutus.Contract.Test
28 import           Plutus.Contract.Test.Coverage
```

```

29 import           Plutus.Trace.Emulator                as Emulator
30 import qualified PlutusTx.Prelude                      as Plutus
31 import           System.Exit                          (ExitCode (..))
32 import           Test.Tasty
33 import qualified Test.Tasty.HUnit                      as HUnit
34
35 import           Plutus.Contract.Test.Coverage.ReportCoverage
                 (writeCoverageReport)
36 import           Week08.TokenSale
37
38 tests :: TestTree
39 tests = checkPredicateOptions
40     myOptions
41     "token sale trace"
42     myPredicate
43     myTrace
44
45 testCoverage :: IO ()
46 testCoverage = do
47     cref <- newCoverageRef
48     e <- try $ defaultMain $ checkPredicateOptionsCoverage
49         myOptions
50         "token sale trace"
51         cref
52         myPredicate
53         myTrace
54     case e of
55         Left (c :: ExitCode) -> do
56             putStrLn $ "Tasty exited with: " ++ show c
57             report <- readCoverageRef cref
58             writeCoverageReport "TokenSaleTrace" tsCovIdx report
59         Right () -> putStrLn $ "unexpected tasty result"
60
61 myOptions :: CheckOptions
62 myOptions = defaultCheckOptions & emulatorConfig .~ emCfg
63
64 myPredicate :: TracePredicate
65 myPredicate =
66     walletFundsChange w1 (Ada.lovelaceValueOf 10_000_000 <>
67         assetClassValue token (-60) <>
68         Plutus.negate (toValue minAdaTxOut)) .&&.
69     walletFundsChange w2 (Ada.lovelaceValueOf (-20_000_000) <>
70         assetClassValue token 20) .&&.
71     walletFundsChange w3 (Ada.lovelaceValueOf (- 5_000_000) <>
72         assetClassValue token 5)

```

```

69
70 runMyTrace :: IO ()
71 runMyTrace = runEmulatorTraceIO' def emCfg myTrace
72
73 emCfg :: EmulatorConfig
74 emCfg = EmulatorConfig (Left $ Map.fromList [(knownWallet w, v) |
75                                     w <- [1 .. 3]]) def def
76
77   where
78     v :: Value
79     v = Ada.lovelaceValueOf 1_000_000_000 <> assetClassValue token 1000
80
81   currency :: CurrencySymbol
82   currency = "aa"
83
84   name :: TokenName
85   name = "A"
86
87   token :: AssetClass
88   token = AssetClass (currency, name)
89
90 myTrace :: EmulatorTrace ()
91 myTrace = do
92   h <- activateContractWallet w1 startEndpoint
93   callEndpoint @"start" h (currency, name)
94   void $ Emulator.waitNSlots 5
95   Last m <- observableState h
96   case m of
97     Nothing -> Extras.logError @String "error starting token sale"
98     Just ts -> do
99       Extras.logInfo $ "started token sale " ++ show ts
100
101       h1 <- activateContractWallet w1 $ useEndpoints ts
102       h2 <- activateContractWallet w2 $ useEndpoints ts
103       h3 <- activateContractWallet w3 $ useEndpoints ts
104
105       callEndpoint @"set price" h1 1_000_000
106       void $ Emulator.waitNSlots 5
107
108       callEndpoint @"add tokens" h1 100
109       void $ Emulator.waitNSlots 5
110
111       callEndpoint @"buy tokens" h2 20
112       void $ Emulator.waitNSlots 5
113
114       callEndpoint @"buy tokens" h3 5

```

```

113         void $ Emulator.waitNSlots 5
114
115         callEndpoint @"withdraw" h1 (40, 10_000_000)
116         void $ Emulator.waitNSlots 5
117
118     checkPredicateOptionsCoverage :: CheckOptions
119                                   -> String
120                                   -> CoverageRef
121                                   -> TracePredicate
122                                   -> EmulatorTrace ()
123                                   -> TestTree
124     checkPredicateOptionsCoverage options nm (CoverageRef ioref)
125         predicate action =
126             HUnit.testCaseSteps nm $ \step -> do
127                 checkPredicateInner options predicate action step
128                 (HUnit.assertBool nm) (\rep -> modifyIORef ioref (rep<>))

```

We run the trace with a custom emulator configuration *emCfg* (73-77). It gives an initial distribution of ADA and native tokens with the token name “A” and currency symbol “aa”. Now for the trace first we activate the start endpoint for wallet one (90). This is the wallet that will run the token sale. Then we call the start endpoint and wait for 5 slots (91-92) to give to the SM to start. Then we check the observable state that is the token sale value and in the case that it’s nothing we log an error message. Else we start the use endpoints that are parameterized by the *ts* value. Then we call various endpoints. First the wallet one sets the price of the token (103). Then wallet one adds 100 tokens (106). Next wallet two buys 20 tokens (109) and wallet three buys 5 tokens (112). In the end wallet one makes a withdraw of 40 tokens and 10 ADA.

If you want to test this code that resides in the *Spec/* folder you need to use the commands:

```

$ cabal repl plutus-pioneer-program-week08:test:plutus-pioneer-program-week08-tests
Prelude> :l test/Spec/Trace.hs
Prelude Spec.Trace> runMyTrace

```

When we get the results what’s worth noting is that wallet one will have around 1008 ADA, which is because 2 ADA had to be put to the script containing the SM contract. This is handled automatically when we construct the transaction in the start endpoint call.

8.2 Automatic testing using emulator traces

Plutus uses the Tasty test framework. You can find the tasty package with a description and example on hackage.haskell.org. Tests are of type *TestTree*. There is special support for tests in Plutus in the module *Plutus.Contract.Test*. There are various types of tests that are supported

but we will look at two of those types: one that works with emulator traces and one that uses property-based testing. By using the *checkPredicate* function we produce something that the tasty framework can understand. There is also a variation of that function called *checkPredicateOptions* that takes in an additional parameter of type *CheckOptions* (Figure 43). It has no constructors exposed but we can use various functions with it (Figure 44).

checkPredicate	
<code>:: String</code>	Descriptive name of the test
<code>-> TracePredicate</code>	The predicate to check
<code>-> EmulatorTrace ()</code>	
<code>-> TestTree</code>	
Check if the emulator trace meets the condition	

Figure 42 - checkPredicate function

checkPredicateOptions	
<code>:: CheckOptions</code>	Options to use
<code>-> String</code>	Descriptive name of the test
<code>-> TracePredicate</code>	The predicate to check
<code>-> EmulatorTrace ()</code>	
<code>-> TestTree</code>	
A version of <code>checkPredicate</code> with configurable CheckOptions	

Figure 43 – checkPredicateOptions function

The *Lens'* type is related to Optics in Haskell. We can use it to set an emulator config. To modify the *CheckOptions* without using lens we can use the *changeInitialWalletValue* function. Given a wallet and a function that updates the initial value it modifies the check options where the given wallet has the function applied to it. Going back to the *checkPredicate* function let's look at the *TracePredicate* type (Figure 45). It is a condition on a trace that represents the actual test. We see that there are also logical combinators so given a trace we can negate it or make a logical »and« or »or«. There is a wide variety of available predicates in the Assertion chapter of the *Plutus.Contract.Test* module. We will use the *walletFundsChange* predicate (Figure 46).

```
data CheckOptions
```

Options for running the

```
defaultCheckOptions :: CheckOptions
```

```
minLogLevel :: Lens' CheckOptions LogLevel
```

```
emulatorConfig :: Lens' CheckOptions EmulatorConfig
```

```
changeInitialWalletValue :: Wallet -> (Value -> Value) -> CheckOptions -> CheckOptions
```

Modify the value assigned to the given wallet in the initial distribution.

Figure 44 - checkOptions type

```
type TracePredicate = FoldM (Eff '[Reader InitialDistribution, Error EmulatorFoldErr,  
Writer (Doc Void)]) EmulatorEvent Bool # Source
```

Figure 45 – TracePredicate type

```
walletFundsChange :: Wallet -> Value -> TracePredicate
```

Check that the funds in the wallet have changed by the given amount, excluding fees.

Figure 46 – walletFundsChange function

The *walletFundsChange* checks for a given wallet after the trace is completed, that the wallet funds will have changed by this given value excluding fees. There is also a variation of this function called *walletFundsExactChange* where this fee is not automatically taken care of. If we now look again at the *test/Spec/Trace.hs* file from the previous chapter in the *tests* parameter we are using the *checkPredicateOptions* function (38-43) where we input *myOptions* that contains the emulator config (61-62). We start with the default option and add the emulator config. The operator “.” is part of Haskell optics and says that we set the *emulatorConfig* part to the given value. When defining *myPredicate* we use the logical “and” to combine three predicates and we use the *walletFundsChange* function (64-68). The fees are taken care of automatically when we use this function but not the minimal ADA deposit that we have to take care of. If we want to run the test, we have to first get into our test Repl as shown at the end of the previous sub-chapter. Then we execute the command:

```
ghci> :l test/Spec/Trace.hs
Ok, one module loaded.
ghci> import Test.Tasty
ghci> defaultMain tests
All 1 tests passed (0.34s)
```

If the test does not pass, we would get a longer log message with an error output.

8.3 Test Coverage

Plutus makes use of code coverage that enables the user to see how much code is covered by your tests. The *checkPredicateCoverage* function uses the *CoverageRef* variable (Figure 47).

checkPredicateCoverage

<code>:: String</code>	Descriptive name of the test
<code>-> CoverageRef</code>	
<code>-> TracePredicate</code>	The predicate to check
<code>-> EmulatorTrace ()</code>	
<code>-> TestTree</code>	

Figure 47 – checkPredicateCoverage function

If we look again at the code in *test/Spec/Trace.hs* we have the *checkPredicateOptionsCoverage* function (118-126). Basically, we looked at the implementation of *checkPredicateOptions* and *checkPredicateCoverage* functions and then combine the two. The *CoverageRef* type is just a newtype wrapper around a *IORef* for *CoverageReport* (Figure 48). And we can get a coverage reference with the *newCoverageRef* function.

newtype CoverageRef

Constructors

CoverageRef (IORef CoverageReport)

newCoverageRef :: IO CoverageRef

readCoverageRef :: CoverageRef -> IO CoverageReport

Figure 48 – CoverageRef type

So, using that we define the *testCoverage* IO action (45-59). First, we get the coverage reference, then we use the *checkPredicateOptionsCoverage* function but because we are in IO, we need the *defaultMain* function for this to work which always throws a default code exception at the end even if all test pass. With the *try* statement we are catching this exception. Then we provide the parameters as previously but with the new reference. In the body of the case statement, we should always get the Left constructor and if the test succeeds it should be a zero-return code and if it doesn't it should be a non-zero code. But in either case we read the report and write it to the TokenSaleTrace HTML file by using the coverage index. The *tsCovIdx* is defined in the previous *TokenSale.hs* file. We can run now the test coverage from the Repl.

```
Prelude Spec.Trace> testCoverage
```

If we open the HTML file in a web-browser the lines highlighted green represent conditions that passed for all tests and the dark highlighted lines represent conditions that were never met.

8.4 Interlude optics

The most used Haskell library for optics is called Lens. Optics are used to reach deeply into hierarchical data types to manipulate parts of them. Let's look at the example code from *Week08/Lens.hs* where we will look at a problem and show how to solve it with lens.

```
1 {-# LANGUAGE TemplateHaskell #-}
2
3 module Week08.Lens where
4
5 import Control.Lens
6
7 newtype Company = Company {_staff :: [Person]} deriving Show
8
9
10 data Person = Person
11   { _name    :: String
12   , _address :: Address
13   } deriving Show
14
15 newtype Address = Address {_city :: String} deriving Show
16
17 alejandro, lars :: Person
18 alejandro = Person
19   { _name    = "Alejandro"
20   , _address = Address {_city = "Zacateca"}
21   }
```

```

22 lars = Person
23   { _name   = "Lars"
24   , _address = Address {_city = "Regensburg"}
25   }
26
27 iohk :: Company
28 iohk = Company { _staff = [alejandro, lars] }
29
30 goTo :: String -> Company -> Company
31 goTo there c = c { _staff = map movePerson (_staff c) }
32   where
33     movePerson p = p { _address = (_address p) {_city = there} }
34
35 makeLenses ''Company
36 makeLenses ''Person
37 makeLenses ''Address
38
39 goTo' :: String -> Company -> Company
40 goTo' there c = c & staff . each . address . city .~ there

```

The company data type is just a newtype wrapper around a list of persons with the accessor called staff (7). Person is a record type with fields name and address (10-13). Address is a newtype wrapper around a string with the accessor called city (15). As an example, we define two persons (17-25) and a company where the staff consist of these two persons (27-28). The function *goTo* takes in a city name as a string and a company and changes for all the staff members the address field (30-33). For this example, we use the record update syntax. Dealing with nested record types can become quite messy because you always have to keep the old fields in place and update the new ones. And this is what optics is trying to solve by providing first class field accessors. You could say that optics provide a programmable dot, which is similar to other languages as Java where you can access an attribute with the dot notation. The lenses library provides some template Haskell features but it expects some underscore conventions which we used in our accessor names. You need to provide the types for which you want to have lenses (35-37). And the name of the lenses will be the names of the original fields without the underscore. There is a way to inspect what code the template Haskell writes at compile time. You can activate the flag »:set -ddump-splices«. If you reload your Lens module in the Repl you will get an extended output for the template Haskell definitions. Here is a short demonstration of how to use lenses in the Repl.

```

ghci> :l src/Week08/Lens.hs
ghci> import Control.Lens
ghci> lars ^. name
"Lars"

```

```
ghci> lars ^. address
Address {_city = "Regensburg"}
ghci> lars ^. address . city
"Regensburg"
ghci> lars & name .~ "LARS"
Person {_name = "LARS", _address = Address {_city = "Regensburg"}}
ghci> lars & address . city .~ "Munich"
Person {_name = "Lars", _address = Address {_city = "Munich"}}
```

The combination of lenses is done with the dot notation. The ampersand notation is used to set a record type field. There is also a different type of optics called traversals which does not only zoom into one field but into many simultaneously. If you would have a list it would zoom into all elements. Here is an example using the *each* traversable.

```
ghci> [1 :: Int, 3, 4] & each .~ 42
[42, 42, 42]
```

Various types of lenses can be combined with the dot operator. We can see such an example in our code with the *goTo'* function (39-40).

8.5 Property based testing with QuickCheck

QuickCheck is a Haskell library for property-based testing. In contrast to Unit testing where the cases of test are hardcoded we only specify a property function which defines a property of our code. It takes in a given variable and returns True if our code upholds the property for the given variable. Then variables are randomly picked by the QuickCheck library and the test passes if for all variables the property function returns True. Let's look at the code in *QuickCheck.hs*.

```
1  module Week08.QuickCheck where
2
3  prop_simple :: Bool
4  prop_simple = 2 + 2 == (4 :: Int)
5
6  -- Insertion sort code:
7
8  -- | Sort a list of integers in ascending order.
9  --
10 -- >>> sort [5,1,9]
11 -- [1,5,9]
12 --
13 sort :: [Int] -> [Int] -- not correct
14 sort []      = []
15 sort (x:xs) = insert x xs
16
17 -- | Insert an integer at the right position into an /ascendingly sorted/
```

```

18 -- list of integers.
19 --
20 -- >>> insert 5 [1,9]
21 -- [1,5,9]
22 --
23 insert :: Int -> [Int] -> [Int] -- not correct
24 insert x [] = [x]
25 insert x (y:ys) | x <= y = x : ys
26                  | otherwise = y : insert x ys
27
28 isSorted :: [Int] -> Bool
29 isSorted [] = True
30 isSorted [_] = True
31 isSorted (x : y : ys) = x <= y && isSorted (y : ys)
32
33 prop_sort_sorts :: [Int] -> Bool
34 prop_sort_sorts xs = isSorted $ sort xs
35
36 prop_sort_preserves_length :: [Int] -> Bool
37 prop_sort_preserves_length xs = length (sort xs) == length xs

```

First, we define the *sort* and the helper *insert* functions that sort a list (13-26). Next, we define the *isSorted* function that checks whether a list of integers is sorted (28-31). With the function *prop_sort_sorts* we test the sorting property of the *sort* function (33-34) and with the function *prop_sort_preserves_length* we test that the sort function preserves the length of the list (36-37). We can run our example code with the following command:

```

Prelude Week08.QuickCheck> import Test.QuickCheck
Prelude Week08.QuickCheck> quickCheck prop_sort_sorts
*** Failed! Falsified (after 8 tests and 4 shrinks):
[0,0,-1]
ghci> sort [0,0,-1]
[0,-1]

```

What the message means quick check tried out 8 examples of a list and after it found one it tried 4 times to further simplify it and returned it as an example in the Repl.



When QuickCheck checks a property, it starts with simple random arguments and then makes them more and more complex over time.

By default, quick check tries 100 arguments to test our property. For our sort function we could implement a fix as specify line 15 as follows:

```
sort (x:xs) = insert x $ sort xs
```

If we run this code and test it we see that a 100 test pass, but if we manually try the previous list it does get sorted but a zero value disappears from the list.

```
ghci> sort [0,0,-1]  
[-1,0]
```

So, it means our test is only good enough as good the *prop_sort_sorts* function is. But we can detect this with another property called *prop_sort_preserves_length* (36-37). And for this function we do get a counter example where the test now fails. And the bug is on line 25 that we can change to the following code where both tests will pass:

```
insert x (y:ys) | x <= y      = x : y : ys
```

8.6 Property based testing of Plutus contracts

The problem we encounter when writing property-based tests for Plutus contracts is how do you test code that has effect on the real world as for example our blockchain. We can explain this on the example of testing file operation with QuickCheck. The idea is you start with a model which is an idealized system. There must be some sort of relation between the real system and the model. And then what quick check does it generates a random sequence of actions so for the file system it would generate for example opening, reading and writing sequences. You can apply an action to your model and apply it to the real world. So, both progress to a new state and after that you compare the two and check whether they are still in sync.

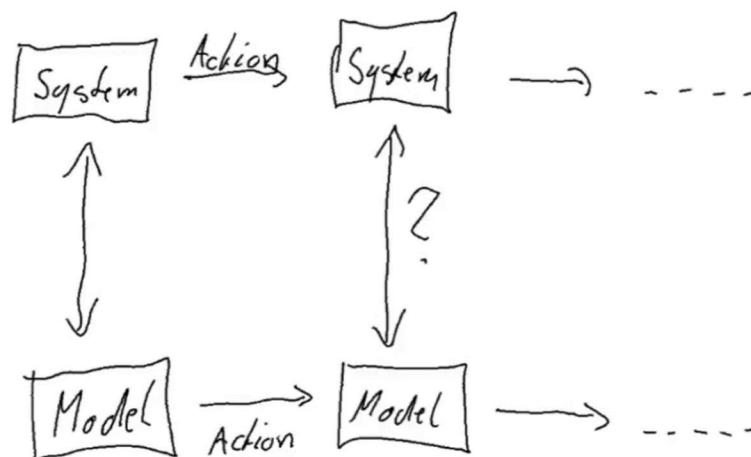


Figure 49 - Testing schema

And shrinking in this case would be if you have a list of actions, you could skip some and try if the test still fails. This is how property testing also works in Plutus. We will look at the example of our token sale contract. The code for this is in *test/Spec/Model.hs*.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass       #-}
3  {-# LANGUAGE DeriveGeneric        #-}
4  {-# LANGUAGE FlexibleContexts     #-}
5  {-# LANGUAGE FlexibleInstances    #-}
6  {-# LANGUAGE GADTs               #-}
7  {-# LANGUAGE InstanceSigs        #-}
8  {-# LANGUAGE MultiParamTypeClasses #-}
9  {-# LANGUAGE NumericUnderscores  #-}
10 {-# LANGUAGE OverloadedStrings    #-}
11 {-# LANGUAGE RankNTypes           #-}
12 {-# LANGUAGE ScopedTypeVariables  #-}
13 {-# LANGUAGE StandaloneDeriving    #-}
14 {-# LANGUAGE TemplateHaskell      #-}
15 {-# LANGUAGE TypeApplications     #-}
16 {-# LANGUAGE TypeFamilies         #-}
17 {-# LANGUAGE TypeOperators        #-}
18
19 module Spec.Model
20   ( tests
21   , test
22   , TSMModel (..)
23   ) where
24
25 import           Control.Lens      hiding
26                                     (elements)
27 import           Control.Monad     (forM_, void,
28                                     when)
29 import           Data.Map          (Map)
30 import qualified Data.Map          as Map
31 import           Data.Maybe        (isJust,
32                                     isNothing)
33 import           Data.Monoid        (Last (..))
34 import           Data.String        (IsString
35                                     (..))
36 import           Data.Text          (Text)
37 import           Plutus.Contract
38 import           Plutus.Contract.Test
39 import           Plutus.Contract.Test.ContractModel as Test
40 import           Plutus.Contract.Test.ContractModel.Symbolics
41 import           Plutus.Trace.Emulator as Trace
```

```

38 import                               Ledger                                hiding
                                                                              (singleton)
39 import                               Ledger.Ada                          as Ada
40 import                               Ledger.Value
41 import                               Test.QuickCheck
42 import                               Test.Tasty
43 import                               Test.Tasty.QuickCheck
44
45 import                               Week08.TokenSaleFixed              (TokenSale
    (...), TSStartSchema, TUseSchema, startEndpoint, useEndpoints')
46
47 type TUseSchema' = TUseSchema .\ / Endpoint "init" TokenSale
48
49 useEndpoints'' :: Contract () TUseSchema' Text ()
50 useEndpoints'' = awaitPromise $ endpoint @"init" useEndpoints'
51
52 data TSState = TSState
53   { _tssPrice      :: !Integer
54   , _tssLovelace   :: !Integer
55   , _tssToken      :: !Integer
56   } deriving Show
57
58 makeLenses ''TSState
59
60 newtype TModel = TModel {_tsModel :: Map Wallet TSState}
61   deriving Show
62
63 makeLenses ''TModel
64
65 tests :: TestTree
66 tests = testProperty "token sale model" prop_TS
67
68 instance ContractModel TModel where
69
70   data Action TModel =
71     Start Wallet
72   | SetPrice Wallet Wallet Integer
73   | AddTokens Wallet Wallet Integer
74   | Withdraw Wallet Wallet Integer Integer
75   | BuyTokens Wallet Wallet Integer
76   deriving (Show, Eq)
77
78 data ContractInstanceKey TModel w s e p where
79   StartKey :: Wallet -> ContractInstanceKey TModel
    (Last TokenSale) TSStartSchema Text ()

```

```

80         UseKey    :: Wallet -> Wallet -> ContractInstanceKey
                                   TSMModel () TSUseSchema' Text ()
81
82 instanceWallet :: ContractInstanceKey TSMModel w s e p -> Wallet
83 instanceWallet (StartKey w) = w
84 instanceWallet (UseKey _ w) = w
85
86 instanceTag :: SchemaConstraints w s e => ContractInstanceKey
                                   TSMModel w s e p ->
                                   ContractInstanceTag
87 instanceTag key = fromString $ "instance tag for: " ++ show key
88
89 arbitraryAction :: ModelState TSMModel -> Gen (Action TSMModel)
90 arbitraryAction _ = oneof
91   [ Start      <$> genWallet
92   , SetPrice   <$> genWallet <*> genWallet <*> genNonNeg
93   , AddTokens <$> genWallet <*> genWallet <*> genNonNeg
94   , BuyTokens <$> genWallet <*> genWallet <*> genNonNeg
95   , Withdraw  <$> genWallet <*> genWallet <*> genNonNeg <*> genNonNeg
96   ]
97
98 initialState :: TSMModel
99 initialState = TSMModel Map.empty
100
101 initialInstances :: [StartContract TSMModel]
102 initialInstances = [StartContract (StartKey v) () | v <- wallets]
103                   ++ [StartContract (UseKey v w) () | v <- wallets,
104                                                         w <- wallets]
105
106 precondition :: ModelState TSMModel -> Action TSMModel -> Bool
107 precondition s (Start w) = isNothing $ getTSState' s w
108 precondition s (SetPrice v _ _) = isJust $ getTSState' s v
109 precondition s (AddTokens v _ _) = isJust $ getTSState' s v
110 precondition s (BuyTokens v _ _) = isJust $ getTSState' s v
111 precondition s (Withdraw v _ _ _) = isJust $ getTSState' s v
112
113 nextState :: Action TSMModel -> Spec TSMModel ()
114 nextState (Start w) = do
115   wait 3
116   (tsModel . at w) $= Just (TSSState 0 0 0)
117   withdraw w $ Ada.toValue minAdaTxOut
118 nextState (SetPrice v w p) = do
119   wait 3
120   when (v == w) $
121     (tsModel . ix v . tssPrice) $= p

```

```

121     nextState (AddTokens v w n) = do
122         wait 3
123         started <- hasStarted v
124         -- has the token sale started?
125         when (n > 0 && started) $ do
126             bc <- actualValPart <$> askModelState (view $ balanceChange w)
127             let token = tokens Map.! v
128             when (tokenAmt + assetClassValueOf bc token >= n) $ do
129                 -- does the wallet have the tokens to give?
130                 withdraw w $ assetClassValue token n
131                 (tsModel . ix v . tssToken) $~ (+ n)
132     nextState (BuyTokens v w n) = do
133         wait 3
134         when (n > 0) $ do
135             m <- getTSState v
136             case m of
137                 Just t
138                     | t ^. tssToken >= n -> do
139                         let p = t ^. tssPrice
140                         l = p * n
141                         withdraw w $ lovelaceValueOf l
142                         deposit w $ assetClassValue (tokens Map.! v) n
143                         (tsModel . ix v . tssLovelace) $~ (+ l)
144                         (tsModel . ix v . tssToken) $~ (+ (- n))
145                     _ -> return ()
146     nextState (Withdraw v w n l) = do
147         wait 3
148         when (v == w) $ do
149             m <- getTSState v
150             case m of
151                 Just t
152                     | t ^. tssToken >= n && t ^. tssLovelace >= l -> do
153                         deposit w $ lovelaceValueOf l <>
154                         assetClassValue (tokens Map.! w) n
155                         (tsModel . ix v . tssLovelace) $~ (+ (- l))
156                         (tsModel . ix v . tssToken) $~ (+ (- n))
157                     _ -> return ()
158
159 startInstances :: ModelState TSMModel -> Action TSMModel ->
160               [StartContract TSMModel]
161 startInstances _ _ = []
162
163 instanceContract :: (SymToken -> AssetClass) -> ContractInstanceKey
164                  TSMModel w s e p -> p -> Contract w s e ()
165 instanceContract _ (StartKey _) () = startEndpoint

```

```

161     instanceContract _ (UseKey _ _) () = useEndpoints''
162
163     perform :: HandleFun TModel -> (SymToken -> AssetClass) -> ModelState
164             TModel -> Action TModel -> SpecificationEmulatorTrace ()
165     perform h _ m (Start v) = do
166         let handle = h $ StartKey v
167         withWait m $ callEndpoint @"start" handle
168             (tokenCurrencies Map.! v, tokenNames Map.! v)
169         Last mts <- observableState handle
170         case mts of
171             Nothing -> Trace.throwError $ GenericError $
172                 "starting token sale for wallet " ++ show v ++ " failed"
173             Just ts -> forM_ wallets $ \w ->
174                 callEndpoint @"init" (h $ UseKey v w) ts
175     perform h _ m (SetPrice v w p) = withWait m $ callEndpoint
176                                     @"set price" (h $ UseKey v w) p
177     perform h _ m (AddTokens v w n) = withWait m $ callEndpoint
178                                     @"add tokens" (h $ UseKey v w) n
179     perform h _ m (BuyTokens v w n) = withWait m $ callEndpoint
180                                     @"buy tokens" (h $ UseKey v w) n
181     perform h _ m (Withdraw v w n l) = withWait m $ callEndpoint
182                                     @"withdraw" (h $ UseKey v w) (n, l)
183
184 withWait :: ModelState TModel -> SpecificationEmulatorTrace () ->
185           SpecificationEmulatorTrace ()
186 withWait m c = void $ c >> waitUntilSlot ((m ^. Test.currentSlot) + 3)
187
188 deriving instance Eq (ContractInstanceKey TModel w s e p)
189 deriving instance Show (ContractInstanceKey TModel w s e p)
190
191 getTSState' :: ModelState TModel -> Wallet -> Maybe TSState
192 getTSState' s v = s ^. contractState . tsModel . at v
193
194 getTSState :: Wallet -> Spec TModel (Maybe TSState)
195 getTSState v = do
196     s <- getModelState
197     return $ getTSState' s v
198
199 hasStarted :: Wallet -> Spec TModel Bool
200 hasStarted v = isJust <$> getTSState v
201
202 wallets :: [Wallet]
203 wallets = [w1, w2]
204
205 tokenCurrencies :: Map Wallet CurrencySymbol

```

```

198 tokenCurrencies = Map.fromList $ zip wallets ["aa", "bb"]
199
200 tokenNames :: Map Wallet TokenName
201 tokenNames = Map.fromList $ zip wallets ["A", "B"]
202
203 tokens :: Map Wallet AssetClass
204 tokens = Map.fromList [(w, AssetClass (tokenCurrencies Map.! w,
    tokenNames Map.! w)) | w <- wallets]
205
206 genWallet :: Gen Wallet
207 genWallet = elements wallets
208
209 genNonNeg :: Gen Integer
210 genNonNeg = getNonNegative <$> arbitrary
211
212 tokenAmt :: Integer
213 tokenAmt = 1_000
214
215 prop_TS :: Actions TModel -> Property
216 prop_TS = withMaxSuccess 100 . propRunActionsWithOptions
217   (defaultCheckOptions & emulatorConfig . initialChainState .~ Left d)
218   defaultCoverageOptions
219   (const $ pure True)
220   where
221
222     d :: InitialDistribution
223     d = Map.fromList $ [ ( w
224                           , lovelaceValueOf 1_000_000_000 <>
225                             mconcat [assetClassValue t tokenAmt |
226                                       t <- Map.elims tokens])
227                           | w <- wallets
228                           ]
229
229 test :: IO ()
230 test = quickCheck prop_TS

```

In the import section we import the *Plutus.Contract.Test*, *Test.QuickCheck*, *Test.Tasty* and *Test.Tasty.QuickCheck* modules. The last one provides a link between the quick check and tasty libraries. We define the *TSUseSchema*' schema which adds a new init endpoint to our previous schema that takes the argument of type *TokenSale* (47). The idea is that once we start our token sale, we somehow must be able to communicate the *TokenSale* parameter to the use contract. The init endpoint simply calls the *useEndpoints*' function with the value provided in the endpoint (49-50). The reason we can add an endpoint to our existing schema is that in the

TokenSaleFixed.hs file that we import in the beginning we define *useEndpoints'* in such a way that it works for every schema that contains the four endpoints stated in the type signature. The only thing the *useEndpoints'* can do is to call the init endpoint and expose the *TokenSale* parameter. Then we define the token sale state data type that supposed to represent the state of one token sale instance (52-56). It has three fields: the current price, the current supply of lovelace and current supply of tokens in the contract UTXO. Now we define our model that we presented in Figure 49 (60-61). That is just a Map from wallet to token sale state. The idea is we will have two wallets and each of the wallet will run their token sale and the wallets will trade different tokens. We implement lenses for that (58, 63). All the logic is now in the instance of the type class contract model for the *TSMModel* type. Here we provide how our model should behave and is linked to the actual contract. First, we have a so-called associated data type (70-76). It is an associated action type which represents the actions that quick check will generate. We have one constructor for each of the endpoints and have different arguments to keep track of the wallets that are in play. There are two Wallet parameters where the first one represents the wallet that started a given contract and the second one wants to make the given action. This should work only if the wallets are owned by one owner with the exception of the buy tokens action. Then we define another associated data type called contract instance key (78-80). The idea is for each instance of the contract that we are running we want a key that identifies this instance. Here instead of just providing the constructors we write them in the form of just providing the type signatures. This is called generalized algebraic data type (GADT). GADT allow us to have different type parameters for the constructors and we need this because our contracts can have different type parameters. For the *UseKey* type the first wallet is the one running the token sale and the second wallet is the one interacting with it. There are two signatures because we had a start and a use contract. The next function *instanceWallet* tells the system how to extract the wallet that a given contract is running on by its constraint instance key (82-84). Then we implement the *instanceTag* method that takes one of these keys and turns it into a contract instance tag which implements the *isString* type class (86-87). And the contract instance tags are then used on the blockchain site, on the actual implementation site to identify running instances of contracts and interact with them by calling endpoints. We use this type class when we switch on the overloaded string extension. For the contract instance key, we are also deriving Show and because we have a GADT we have to put the name of the type after the Show keyword (181). So, it is important that this instance tag function results in a different tag for each instance that we will ever run in our simulation or in our tests. We will have one start instance for each wallet and then one-use instance for each pair of wallets. The *arbitraryAction* function is supposed to generate an arbitrary action (89-96). »oneof« is one of the combinators provided by quick check and given a list of arbitrary actions it picks one of those. The *genWallet* function generates a random wallet from wallet 1 or wallet 2. We

combine the *genWallet* then through the *fmap* operator with several wallet actions. The *genNonNeg* function generates a non-negative integer. Next comes *initialState* which is the initial state of our model that contains just an empty map which means no token sale has started yet (98-99). The *initialInstances* function gives the initial contract instances that have to run (101-103). In the body of the function, we have contracts that are supposed to run in the beginning. With them we want to start the token sale for each wallet and then the use contract for each pair of wallets. The precondition function allows us to say that some actions may not be legal in a given model state (105-110). So, for example we can say that we can only start a token sale if it hasn't already started. The precondition on the start should be it hasn't started yet and for other actions it should be that it has started. During the quick check shrinking procedure that drops some actions, this is also checked that all the preconditions are still satisfied. As input, we use the model state type that has no constructors exposed (Figure 50).

```
data ModelState state
```

The *ModelState* models the state of the blockchain. It contains,

- the contract-specific state (*contractState*)
- the current slot (*currentSlot*)
- the wallet balances (*balances*)
- the amount that has been minted (*minted*)

Figure 50 – ModelState type

But it contains a couple of functions that operate on the model state. The most important one is the *contractState* function which is a lens from model state to state (Figure 51).

```
contractState :: forall state state. Lens (ModelState state) (ModelState state)
```

```
state state
```

Source

Lens for the contract-specific part of the model state.

Spec monad update functions: *\$=* and *\$~*.

Figure 51 - contractState function

There is also a current slot optic that is a getter which means we can only look at it but cannot set it. With lenses getting and setting is possible. The *getTSSState'* function given a model state and a wallet it tries to extract the token sale that this wallet is running (183-184). With the *at* keyword, we can look up a value given a Map key and if the key does not exist, we will get a Nothing value. If we would set a given key to Nothing, we would remove it from the map. When

we explained our testing principle in the beginning, we said for each action we must know what effect that action will have on our model. And that's what the *nextState* function is for (112-154). It takes an action and results in a Spec monad that has the purpose of describing effects on our TS model. In this model there is a concept of how much every wallet owns. And in the Spec monad you can say that a given wallet earns some funds or sends them elsewhere. And you can say that time passes in between. The model and the actual simulator must keep in sync as far as slots are concerned. The first do block says when we do the start for the token sale then in the model for key w (our wallet) there now will be a *TSState* and the value of it will be 0 0 0. The $\$=$ operator comes from the Spec monad and on the left side it takes a lens. And then it will update the corresponding focus of the lens to that new value. The last statement is the effect of the funds on the wallet. We say the wallet w in question loses funds. That's because when we start the token sale, we need to put down the minimal ADA for the token sale UTXO. Similar we define other states where we deal with setting the price, adding tokens, buying tokens and make a withdrawal. In the set price action, we check whether the wallet that started the token sale is the wallet that tries to set the price and if yes, we set the price to the given value (117-120). For the add tokens action we first ask whether the token sale has started. If yes then we construct a function from model state to value change with help of the *balanceChange* getter function and apply the *askModelState* function to it (Figure 52).

```
askModelState :: GetModelState m => (ModelState (StateType m) -> a) -> m a
```

Get a component of the model state.

Figure 52 – askModelState function

Then we get that value in the spec monad which is not a Plutus value. It's something more complicated something of type symbolic value. We can extract the normal value with the *actualValPart* function so what the line 125 does is the bc is now the actual balance change of my wallet w. The purpose why we are doing this is we want to check that we actually have enough tokens to give. And if we do we make a withdraw and update the TS model where we add the tokens. This dollar tilde operator in line 129 is similar to the dollar equal, so the dollar equal sets a value and the dollar tilde applies a function to the value. For the buy tokens action if the amount we want to buy is positive we first get the token sale state and we just consider the case that the token sale is actually running. If yes, we check that there are at least n tokens available, then we look up the price and calculate how much it will cost us in lovelace. Then we make a withdraw from our wallet for this amount of lovelace and with the deposit function we gain funds to our wallet which will be the tokens that we bought. In the end we update our model with the lovelace and token amount values. The last action is withdraw. This is only

allowed for the wallet that created the token sale (146). The code is very similar to the previous action. First, we check again that the token sale is still running and that there are sufficient funds available to withdraw. If yes, we make a deposit to our wallet and we update our model again with the lovelace and token amount values we will withdraw.

Then we have the *startInstances* function (156-157). There is the possibility to start our contract instances later so given a model statement action you can give a list of contracts that are supposed to be started. But because we start them all in the beginning, we put here an empty list. Now we start providing the link between our model and the actual emulator or the blockchain and the first part of this is done by the *instanceContract* function (159-161). It takes in a function that we don't use, a key and a parameter value which in our case is always unit. And then we have to say which contract corresponds to that. If the key is a start key it's the start endpoint contract and if the key is a use key it's the use endpoint contract. And finally, the *perform* function tells us how an action is actually expressed in an actual action in the emulator on the blockchain (163-175). In the end we get the specification emulator trace monad that you can think of as an emulator trace. In the cases following after the first one we use the *withWait* function that should ensure that something takes exactly three slots (177-178). It takes in a model and an action and performs it and then waits until three slots have passed from the start of the action. And the actions are calls to various endpoints with the key parameters that correspond to the actions. This provides the links between the model and the actions with actual operations that are supposed to happen on the blockchain or in the emulator. The start action is a bit trickier because of the mechanism that when we start a token sale, we have to call the init endpoint which is now happening in this do block. The start contract actually writes the token sale into this observable state with tell which is a Maybe value (167). And if the value is something we call the init endpoint for the use contracts and we loop over all our wallets. Calling this init endpoint will actually kick off the original use contract. To tie everything together we can use now the *propRunActionsWithOptions* function (Figure 53).

<code>propRunActionsWithOptions</code>	
<code>:: ContractModel state</code>	
<code>=> CheckOptions</code>	Emulator options
<code>-> CoverageOptions</code>	Coverage options
<code>-> (ModelState state -> TracePredicate)</code>	Predicate to check at the end
<code>-> Actions state</code>	The actions to run
<code>-> Property</code>	

Figure 53 – *propRunActionsWithOptions* function

It takes check options which allow us for example to specify a fund distribution in our wallets. Then it takes coverage options but we will not use this in our example. Then it takes an additional predicate to check at the end. With the final model state, you can provide a trace predicate and then that will also be tested but we won't use this either. Then it returns something of action state to property. We use this function in the *prop_TS* function (215-227). We are composing it with the *withMaxSuccess* function that comes from quick check and we can specify how many test cases to run. We start with the default check option and add our emulator config and initial chain state and then set it to an initial distribution where every wallet should get a thousand ADA and a thousand tokens. Wallet 1 will be selling token »A« and wallet two will be selling token »B«. The *prop_TS* function returns a property check which you can think of as a Bool used by quick check. Because we saw before we can randomly generate action sequences quick check can handle our *prop_TS* function since it takes in the *Actions* type. And the property that is actually checked is that what we say in the next state function what happens to our model that is actually reflected when we perform the actions on the blockchain in our emulator traces. So, this will check that what we say should happen in the model actually does happen on the blockchain. And in the Repl we can then run our test function that calls our property test function (229-230). When we run this in the Repl we also get some statistics. We see which actions were actually tried and how many of them were rejected due to preconditions. If we would import the *TokenSale* module instead of *TokenSaleFixed* in the beginning we would get a test failure due to the fact that we could try to withdraw some ADA from the min ADA amount. Let's look now how we can grab all of this into a test suite.

```
1  module Main
2      ( main
3      ) where
4
5  import qualified Spec.Model
6  import qualified Spec.Trace
7  import          Test.Tasty
8
9  main :: IO ()
10 main = defaultMain tests
11
12 tests :: TestTree
13 tests = testGroup "token sale"
14     [ Spec.Trace.tests
15     , Spec.Model.tests
16     ]
```

This represents now the main program of our test suite. *tests* is our test tree where we just put these two tests that we wrote. And we can run the tests with the »*cabal test*« command. What we have to note is that when testing with quick check we provide the off-chain code but a user could write their own off-chain code that for example would try to steal the funds. And such scenarios are then not covered by our testing procedures. The quick check tests only check the flow of funds whether that agrees at each point with what we specified in the model but it is possible to add additional checks and it is also possible to influence these action sequences. It's also possible to do some more unit test like scenarios where we specify certain flows of actions to steer the tests into certain directions. That's called dynamic logic.

8.7 Homework

For homework we will modify the token sale contract such that it accepts an additional transition called close that can be called only by the seller to close the UTXO and collect all the remaining tokens, lovelace and the NFT. Let's look at the *Week08/TokenSaleWithClose.hs* code.

```
1  data TokenSale = TokenSale
2    { tsSeller :: !PaymentPubKeyHash
3    , tsToken  :: !AssetClass
4    , tsTT     :: !ThreadToken
5    } deriving (Show, Generic, FromJSON, ToJSON, Prelude.Eq)
6
7  PlutusTx.makeLift ''TokenSale
8
9  data TSRedeemer =
10    SetPrice Integer
11    | AddTokens Integer
12    | BuyTokens Integer
13    | Withdraw Integer Integer
14    | Close
15    deriving (Show, Prelude.Eq)
16
17  PlutusTx.unstableMakeIsData ''TSRedeemer
18
19  {-# INLINABLE lovelaces #-}
20  lovelaces :: Value -> Integer
21  lovelaces = Ada.getLovelace . Ada.fromValue
22
23  {-# INLINABLE transition #-}
24  transition :: TokenSale -> State (Maybe Integer) -> TSRedeemer -> Maybe
25    (TxConstraints Void Void, State (Maybe Integer))
26  transition ts s r = case (stateValue s, stateData s, r) of
```

```

26     (v, Just _, SetPrice p) | p >= 0                                ->
    Just ( Constraints.mustBeSignedBy (tsSeller ts)
          , State (Just p) v )
27
28
29     (v, Just p, AddTokens n) | n > 0                                ->
    Just ( mempty
          , State (Just p) $
30             v <>
31             assetClassValue (tsToken ts) n )
32
33
34     (v, Just p, BuyTokens n) | n > 0                                ->
    Just ( mempty
          , State (Just p) $
35             v <>
36             assetClassValue (tsToken ts) (negate n) <>
37             lovelaceValueOf (n * p) )
38
39
40     (v, Just p, Withdraw n l) | n >= 0 && l >= 0 &&
41     v `geq` (w <> toValue minAdaTxOut) ->
    Just ( Constraints.mustBeSignedBy (tsSeller ts)
          , State (Just p) $
42             v <>
43             negate w )
44
45
46     where
47         w = assetClassValue (tsToken ts) n <>
48             lovelaceValueOf l
49     (_, Just _, Close)                                            ->
    Just ( Constraints.mustBeSignedBy (tsSeller ts)
          , State Nothing mempty )
50
51
52     _ -> Nothing
53
54 {-# INLINABLE tsStateMachine #-}
55 tsStateMachine :: TokenSale -> StateMachine (Maybe Integer) TSRedeemer
56 tsStateMachine ts = mkStateMachine (Just $ tsTT ts)
57                               (transition ts) isNothing
58
59 {-# INLINABLE mkTSValidator #-}
60 mkTSValidator :: TokenSale -> Maybe Integer -> TSRedeemer ->
61               ScriptContext -> Bool
62 mkTSValidator = mkValidator . tsStateMachine
63
64 type TS = StateMachine (Maybe Integer) TSRedeemer

```

```

64 tsTypedValidator :: TokenSale -> Scripts.TypedValidator TS
65 tsTypedValidator ts = Scripts.mkTypedValidator @TS
66   ( $(PlutusTx.compile [| mkTSValidator |])
67     `PlutusTx.applyCode` PlutusTx.liftCode ts)
67   $(PlutusTx.compile [| wrap |])
68   where
69     wrap = Scripts.wrapValidator @(Maybe Integer) @TSRedeemer
70
71 tsValidator :: TokenSale -> Validator
72 tsValidator = Scripts.validatorScript . tsTypedValidator
73
74 tsAddress :: TokenSale -> Ledger.Address
75 tsAddress = scriptAddress . tsValidator
76
77 tsClient :: TokenSale -> StateMachineClient (Maybe Integer) TSRedeemer
78 tsClient ts = mkStateMachineClient $ StateMachineInstance
79   (tsStateMachine ts) (tsTypedValidator ts)
80
81 mapErrorSM :: Contract w s SMContractError a -> Contract w s Text a
82 mapErrorSM = mapError $ pack . show
83
84 startTS :: AssetClass -> Contract (Last TokenSale) s Text ()
85 startTS token = do
86   pkh <- Contract.ownPaymentPubKeyHash
87   tt <- mapErrorSM getThreadToken
88   let ts = TokenSale
89     { tsSeller = pkh
90     , tsToken = token
91     , tsTT = tt
92     }
93   client = tsClient ts
94   void $ mapErrorSM $ runInitialise client (Just 0) mempty
95   tell $ Last $ Just ts
96   logInfo $ "started token sale " ++ show ts
97
98 setPrice :: TokenSale -> Integer -> Contract w s Text ()
99 setPrice ts p = void $ mapErrorSM $ runStep (tsClient ts) $ SetPrice p
100
101 addTokens :: TokenSale -> Integer -> Contract w s Text ()
102 addTokens ts n = void (mapErrorSM $ runStep (tsClient ts) $ AddTokens n)
103
104 buyTokens :: TokenSale -> Integer -> Contract w s Text ()
105 buyTokens ts n = void $ mapErrorSM $ runStep (tsClient ts) $ BuyTokens n
106
107 withdraw :: TokenSale -> Integer -> Integer -> Contract w s Text ()

```

```

107 withdraw ts n l = void $ mapErrorSM $ runStep (tsClient ts) $ Withdraw n l
108
109 close :: TokenSale -> Contract w s Text ()
110 close ts = void $ mapErrorSM $ runStep (tsClient ts) Close
111
112 type TSStartSchema =
113     Endpoint "start"      (CurrencySymbol, TokenName)
114 type TSUseSchema =
115     Endpoint "set price" Integer
116     .\ / Endpoint "add tokens" Integer
117     .\ / Endpoint "buy tokens" Integer
118     .\ / Endpoint "withdraw" (Integer, Integer)
119     .\ / Endpoint "close"   ()
120
121 startEndpoint :: Contract (Last TokenSale) TSStartSchema Text ()
122 startEndpoint = forever
123     $ handleError logError
124     $ awaitPromise
125     $ endpoint @"start" $ startTS . AssetClass
126
127 useEndpoints' :: ( HasEndpoint "set price" Integer s
128     , HasEndpoint "add tokens" Integer s
129     , HasEndpoint "buy tokens" Integer s
130     , HasEndpoint "withdraw" (Integer, Integer) s
131     , HasEndpoint "close" () s
132     )
133     => TokenSale
134     -> Promise () s Text ()
135 useEndpoints' ts = setPrice' `select` addTokens' `select` buyTokens'
136     `select` withdraw' `select` close'
137 where
138     setPrice' = endpoint @"set price" $ \p ->
139         handleError logError (setPrice ts p)
140     addTokens' = endpoint @"add tokens" $ \n ->
141         handleError logError (addTokens ts n)
142     buyTokens' = endpoint @"buy tokens" $ \n ->
143         handleError logError (buyTokens ts n)
144     withdraw' = endpoint @"withdraw" $ \(n, l) ->
145         handleError logError (withdraw ts n l)
146     close' = endpoint @"close" $ \() ->
147         handleError logError (close ts)
148
149 useEndpoints :: TokenSale -> Contract () TSUseSchema Text ()
150 useEndpoints = forever . awaitPromise . useEndpoints'

```

In the *TSRedeemer* data type we define our Close action (9-15). Then in the transition function our state parameter takes in a Maybe integer. And at the end of the body, we add the case of close which closes the token sale (49-50). The type signatures for state machine, validator, TS type and ts client get updated accordingly. Also, the wrap helper function takes in the maybe integer type. When we call the *runInitialise* function in the startTS contract we have to provide now a Just 0 value. In lines 109-110 we define our close contract. And in the *TSUseSchema* we add the close endpoint. Similar we do in the type signature of the *useEndpoints'* function and in its body, we add the close endpoint call.

9 Oracles

In this lecture we will look at how to turn our code into an actual application a complete executable or several executables that even come with a little front-end. It will be a fully-fledged DApp and will run on a simulated blockchain, a so-called mockchain. An oracle in the blockchain world is a service or a way to get real world information onto the blockchain and make it usable in smart contracts. You can think of external data sources like weather data, or election results, or stock exchange rates and many more.

9.1 Workflow

There are various ways to implement oracles of various sophistication. And we want to choose a very simple approach where we have one trusted data provider that provides one feed of data. And as an example, for data, we want to use the exchange rate from ADA to USD. Of course, in real world application having a source of just one data provider can be tricky since he could provide false data or not provide the data at all due to some technical issues. What you can do is you could combine several such oracles into one, like only except the value, if all these various sources agree or only take the median or the average value of these different sources. So, let's look at the oracle example workflow (Figure 54). On the blockchain we will present the oracle data as a UTXO. Let's say that the datum is 1.75 which would be the current exchange rate. We encounter our first problem which is we can't prevent anybody from producing arbitrary outputs at the same oracle script address. We make our oracle output unique by giving it an NFT in its value. At the moment when the oracle gets created, you don't know how people might want to use the data feed provided by the oracle. The oracle must be able to work together with smart contracts that haven't even been written at the time when the oracle is created. For our example let's consider a swap contract where at the swap address, somebody can deposit ADA and then somebody else can take those ADA in exchange for USD. Let's assume that USD is represented by some native token. Let's say the oracle provider has a fee that has to be paid each time the oracle is used and for our example will be 1 ADA.

Now let's look at the swap transaction. The swap validation logic will need access to the current oracle so it will be an input to the transaction. We have a redeemer called Use and the oracle validator has to check several things. It has to check that the NFT is present in the consumed input. Next it has to check there is an output at the oracle address with the NFT, 1 ADA fee and the same datum. The transaction will also take as input the swap UTXO with the redeemer Swap and the buyer UTXO. We have also two additional outputs: the USD that goes to the seller and the buyer gets his ADA. For our example the buyer will buy 100 ADA for 175 USD. The swap validator will make sure the buyer pays the correct price. This swap contract is just an example.

The oracle should be capable of working with many different smart contracts that want to make use of the data provided by the oracle. The oracle validator, in addition to the use redeemer must be able to support another operation where the operator, the provider of the oracle can actually change the data. So, in order to update the value, we have to consume the existing oracle UTxO and produce a new one that carries the correct datum. That transaction must be signed by the oracle provider and the outputs are the oracle UTxO with the updated datum and NFT and a UTxO with ADA fees that goes to the provider.

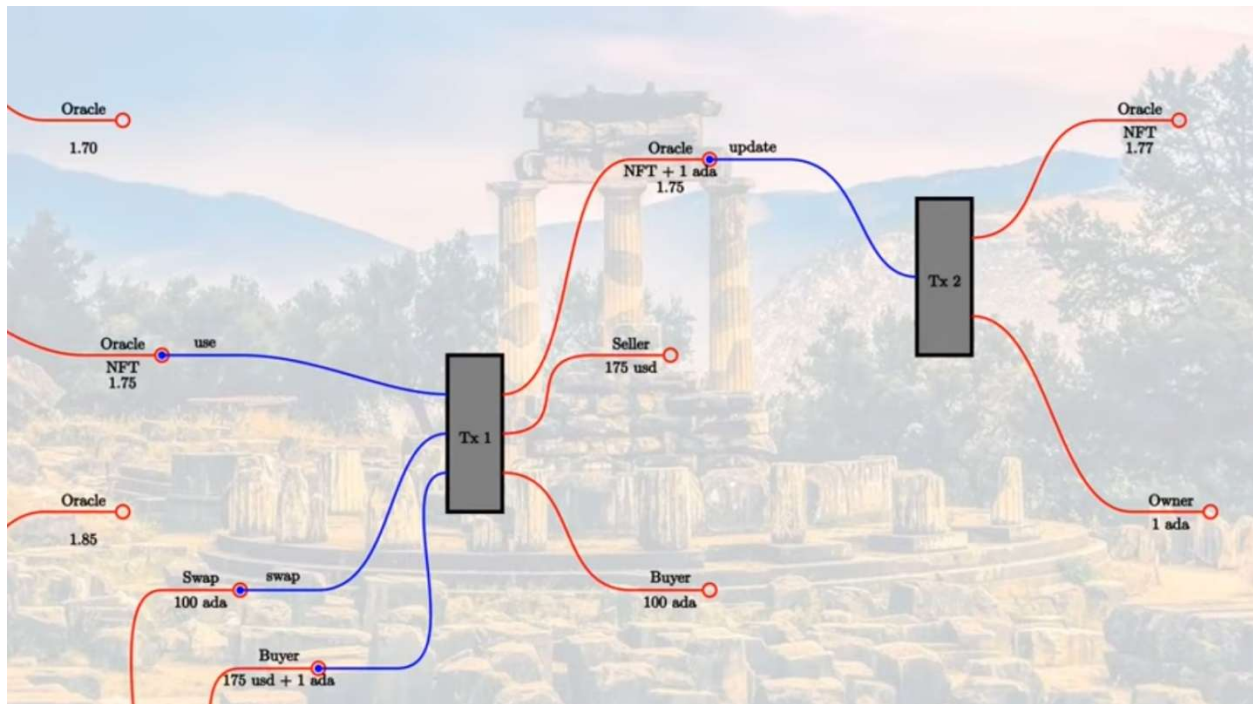


Figure 54 - Oracle workflow

9.2 Code examples

Let's look now at the code in the *Core.hs* file.

```

1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass      #-}
3  {-# LANGUAGE DeriveGeneric       #-}
4  {-# LANGUAGE FlexibleContexts    #-}
5  {-# LANGUAGE MultiParamTypeClasses #-}
6  {-# LANGUAGE NoImplicitPrelude   #-}
7  {-# LANGUAGE OverloadedStrings   #-}
8  {-# LANGUAGE ScopedTypeVariables #-}
9  {-# LANGUAGE TemplateHaskell     #-}
10 {-# LANGUAGE TypeApplications     #-}
11 {-# LANGUAGE TypeFamilies         #-}

```

```

12 {-# LANGUAGE TypeOperators          #-}
13
14 module Week06.Oracle.Core
15     ( Oracle (..)
16     , OracleRedeemer (..)
17     , oracleTokenName
18     , oracleValue
19     , oracleAsset
20     , typedOracleValidator
21     , oracleValidator
22     , oracleAddress
23     , OracleSchema
24     , OracleParams (..)
25     , runOracle
26     , findOracle
27     ) where
28
29 import           Control.Monad          hiding (fmap)
30 import           Data.Aeson             (FromJSON, ToJSON)
31 import qualified Data.Map               as Map
32 import           Data.Monoid            (Last (..))
33 import           Data.Text              (Text, pack)
34 import           GHC.Generics           (Generic)
35 import           Plutus.Contract        as Contract
36 import qualified PlutusTx
37 import           PlutusTx.Prelude      hiding (Semigroup(..), unless)
38 import           Ledger                 hiding (singleton)
39 import           Ledger.Constraints     as Constraints
40 import qualified Ledger.Typed.Scripts   as Scripts
41 import           Ledger.Value           as Value
42 import           Ledger.Ada             as Ada
43 import           Plutus.Contracts.Currency as Currency
44 import           Prelude                 (Semigroup (..), Show (..),
                                         String)
45
46 import qualified Prelude
47
48 data Oracle = Oracle
49     { oSymbol    :: !CurrencySymbol
50     , oOperator  :: !PubKeyHash
51     , oFee       :: !Integer
52     , oAsset     :: !AssetClass
53     } deriving (Show, Generic, FromJSON, ToJSON, Prelude.Eq, Prelude.Ord)
54
55 PlutusTx.makeLift ''Oracle

```

```

56 data OracleRedeemer = Update | Use
57     deriving Show
58
59 PlutusTx.unstableMakeIsData ''OracleRedeemer
60
61 {-# INLINABLE oracleTokenName #-}
62 oracleTokenName :: TokenName
63 oracleTokenName = TokenName emptyByteString
64
65 {-# INLINABLE oracleAsset #-}
66 oracleAsset :: Oracle -> AssetClass
67 oracleAsset oracle = AssetClass (oSymbol oracle, oracleTokenName)
68
69 {-# INLINABLE oracleValue #-}
70 oracleValue :: TxOut -> (DatumHash -> Maybe Datum) -> Maybe Integer
71 oracleValue o f = do
72     dh      <- txOutDatum o
73     Datum d <- f dh
74     PlutusTx.fromBuiltinData d
75
76 {-# INLINABLE mkOracleValidator #-}
77 mkOracleValidator :: Oracle -> Integer -> OracleRedeemer ->
78     ScriptContext -> Bool
79 mkOracleValidator oracle x r ctx =
80     traceIfFalse "token missing from input" inputHasToken &&
81     traceIfFalse "token missing from output" outputHasToken &&
82     case r of
83         Update -> traceIfFalse "operator signature missing"
84             (txSignedBy info $ oOperator oracle) &&
85             traceIfFalse "invalid output datum" validOutputDatum
86         Use      -> traceIfFalse "oracle value changed"
87             (outputDatum == Just x) &&
88             traceIfFalse "fees not paid" feesPaid
89
90 where
91     info :: TxInfo
92     info = scriptContextTxInfo ctx
93
94     ownInput :: TxOut
95     ownInput = case findOwnInput ctx of
96         Nothing -> throwError "oracle input missing"
97         Just i   -> txInInfoResolved i
98
99     inputHasToken :: Bool
100    inputHasToken = assetClassValueOf (txOutValue ownInput)
101        (oracleAsset oracle) == 1

```

```

97
98     ownOutput :: TxOut
99     ownOutput = case getContinuingOutputs ctx of
100         [o] -> o
101         _    -> traceError "expected exactly one oracle output"
102
103     outputHasToken :: Bool
104     outputHasToken = assetClassValueOf (txOutValue ownOutput)
105                        (oracleAsset oracle) == 1
106
107     outputDatum :: Maybe Integer
108     outputDatum = oracleValue ownOutput (`findDatum` info)
109
110     validOutputDatum :: Bool
111     validOutputDatum = isJust outputDatum
112
113     feesPaid :: Bool
114     feesPaid =
115         let
116             inVal  = txOutValue ownInput
117             outVal = txOutValue ownOutput
118             in
119                 outVal `geq` (inVal <> Ada.lovelaceValueOf (oFee oracle))
120
121 data Oracling
122 instance Scripts.ValidatorTypes Oracling where
123     type instance DatumType Oracling = Integer
124     type instance RedeemerType Oracling = OracleRedeemer
125
126 typedOracleValidator :: Oracle -> Scripts.TypedValidator Oracling
127 typedOracleValidator oracle = Scripts.mkTypedValidator @Oracling
128     ( $(PlutusTx.compile [| mkOracleValidator |])
129       `PlutusTx.applyCode` PlutusTx.liftCode oracle )
130     $$(PlutusTx.compile [| wrap |])
131 where
132     wrap = Scripts.wrapValidator @Integer @OracleRedeemer
133
134 oracleValidator :: Oracle -> Validator
135 oracleValidator = Scripts.validatorScript . typedOracleValidator
136
137 oracleAddress :: Oracle -> Ledger.Address
138 oracleAddress = scriptAddress . oracleValidator
139
140 data OracleParams = OracleParams
141     { opFees    :: !Integer

```

```

140     , opSymbol :: !CurrencySymbol
141     , opToken  :: !TokenName
142   } deriving (Show, Generic, FromJSON, ToJSON)
143
144 startOracle :: forall w s. OracleParams -> Contract w s Text Oracle
145 startOracle op = do
146   pkh <- pubKeyHash <$> Contract.ownPubKey
147   osc <- mapError (pack . show) (mintContract pkh [(oracleTokenName, 1)]
    :: Contract w s CurrencyError OneShotCurrency)
148   let cs      = Currency.currencySymbol osc
149       oracle = Oracle
150           { oSymbol   = cs
151           , oOperator = pkh
152           , oFee      = opFees op
153           , oAsset    = AssetClass (opSymbol op, opToken op)
154           }
155   logInfo @String $ "started oracle " ++ show oracle
156   return oracle
157
158 updateOracle :: forall w s. Oracle -> Integer -> Contract w s Text ()
159 updateOracle oracle x = do
160   m <- findOracle oracle
161   let c = Constraints.mustPayToTheScript x $ assetClassValue
    (oracleAsset oracle) 1
162   case m of
163     Nothing -> do
164       ledgerTx <- submitTxConstraints (typedOracleValidator oracle) c
165       awaitTxConfirmed $ txId ledgerTx
166       logInfo @String $ "set initial oracle value to " ++ show x
167     Just (oref, o, _) -> do
168       let lookups = Constraints.unspentOutputs
    (Map.singleton oref o) <>
169         Constraints.typedValidatorLookups
    (typedOracleValidator oracle) <>
170         Constraints.otherScript (oracleValidator oracle)
171       tx          = c <> Constraints.mustSpendScriptOutput oref
    (Redeemer $ PlutusTx.toBuiltinData Update)
172       ledgerTx <- submitTxConstraintsWith @Oracling lookups tx
173       awaitTxConfirmed $ txId ledgerTx
174       logInfo @String $ "updated oracle value to " ++ show x
175
176 findOracle :: forall w s. Oracle -> Contract w s Text
    (Maybe (TxOutRef, TxOutTx, Integer))
177 findOracle oracle = do
178   utxos <- Map.filter f <$> utxoAt (oracleAddress oracle)

```

```

179     return $ case Map.toList utxos of
180       [(oref, o)] -> do
181         x <- oracleValue (txOutTxOut o) $ \dh -> Map.lookup dh $
182           txData $ txOutTxTx o
183         return (oref, o, x)
184       _           -> Nothing
185   where
186     f :: TxOutTx -> Bool
187     f o = assetClassValueOf (txOutValue $ txOutTxOut o)
188           (oracleAsset oracle) == 1
189
190   type OracleSchema = Endpoint "update" Integer
191
192   runOracle :: OracleParams -> Contract (Last Oracle) OracleSchema Text ()
193   runOracle op = do
194     oracle <- startOracle op
195     tell $ Last $ Just oracle
196     go oracle
197   where
198     go :: Oracle -> Contract (Last Oracle) OracleSchema Text a
199     go oracle = do
200       x <- endpoint @"update"
201       updateOracle oracle x
202       go oracle

```

Here is the Plutus code that implements the oracle itself. It will be a parameterized contract. The data type Oracle is the parameter (47-52). The first field is the currency symbol of the NFT. As token name we will just use an empty string. The second field defines the public key hash of the owner of the oracle. Third is the fee in lovelace that has to be paid each time someone uses the oracle. Fourth is the asset class representing the USD tokens that we mentioned earlier. Then we make a lift instance for this data type. Next, we define the redeemer with the Update and Use constructors (56-57). We use template Haskell to implement *isData* for the redeemer data type. Then follow some helper definitions where we define the oracle token name, asset and value that takes in a transaction output and function from datum hash to maybe datum and returns the datum as an integer (61-74). We get a maybe value since the transaction output could be just a public key output that doesn't have a datum. Or the datum could be there but it would not be an integer. We use integer since it works with the aeson library and for this reason we then multiply the exchange rate with 1.000.000. Next, we come to the oracle validator function (76-118). We do some checks depending on the redeemer, but in the beginning, we check that the transaction input and output for the oracle UTXO both contain the NFT (79-80). If the redeemer is set to update, we first check that the signature comes from the

oracle operator (82). Next, we check that in the transaction context we have for the output a just datum written and not a nothing value (83). We use the *oracleValue* helper function we defined. If the redeemer is set to use, we first check that the datum has not changed. And then we check that the fees have been paid. So, the output value of the oracle UTXO has to be equal or greater to the input value plus the oracle fee. After the validator function we make an instance of *ValidatorTypes*. Then we compute the typed validator and the actual validator and address (125-136). This completes the on-chain code. Next follows the *OracleParams* data type where we first have the fees we want to charge and then currency symbol and token name for the USD asset. It is used in the *startOracle* contract function where we mint the oracle NFT (144-156). The minting of the NFT can take a couple of slots. And there's a currency module that provides the *mintContract* function that can be used to mint NFTs.

```
mintContract :: AsCurrencyError e => PaymentPubKeyHash -> [(TokenName, Integer)] ->
Contract w s e OneShotCurrency
```

It takes in the pub key hash of the entity that will end up with the minted coins. And it takes a list of pairs of token names and integers. So, this mint contract will create a currency symbol, which will depend on a unique UTxO. But it allows you to specify several token names with arbitrary integer amounts. In our case we want to mint only one NFT so we provide a single element list. We want a contract where we use text error messages. Text does not implement the *AsCurrencyError* class. There is a function called *mapError*.

```
mapError :: (e -> e') -> Contract w s e a -> Contract w s e' a
```

And it allows us to change the contract error type. The *CurrencyError* type implements the *AsCurrencyError* class and the *Show* class. So we can use it for the type of our error and in the *mapError* function we use (pack . show). For the result of our minting, we get the type *OneShotCurrency*. And we have the function *currencySymbol* that takes in the mentioned type and returns a currency symbol. Then we define the oracle type where we provide our own pub key hash. After that we log a message and return the oracle parameter. So, this is a contract that returns a parameter. Next we look at the *updateOracle* contract function (158-174). It deals with two cases, the case that we already have an oracle value that we want to update. And also, the case that we just started the oracle and there is no UTxO yet, so we want to create one for the very first time. It takes a Oracle parameter and an integer which is the new value we write to the datum. First, we use the helper function *findOracle* which given an Oracle parameter returns a maybe triple of a transaction output reference, transaction output transaction (TxOutTx) and the datum in form of a integer. The TxOutTx is the UTXO itself containing the transaction and transaction output data types (Figure 55).

data TxOutTx

Source

A **TxOut** along with the **Tx** it comes from, which may have additional information e.g. the full data script that goes with the **TxOut**.

Constructors

TxOutTx

txOutTxTx :: Tx

txOutTxOut :: TxOut

Figure 55 – TxOutTx type

The transaction data type contains several fields that define a transaction (Figure 56).

data Tx

A transaction, including witnesses for its inputs.

Constructors

Tx

txInputs :: Set TxIn	The inputs to this transaction.
txCollateral :: Set TxIn	The collateral inputs to cover the fees in case validation of the transaction fails.
txOutputs :: [TxOut]	The outputs of this transaction, ordered so they can be referenced by index.
txMint :: !Value	The Value minted by this transaction.
txFee :: !Value	The fee for this transaction.
txValidRange :: !SlotRange	The SlotRange during which this transaction may be validated.
txMintScripts :: Set MintingPolicy	The scripts that must be run to check minting conditions.
txSignatures :: Map PubKey Signature	Signatures of this transaction.
txRedeemers :: Redeemers	Redeemers of the minting scripts.
txData :: Map DatumHash Datum	Datum objects recorded on this transaction.

Figure 56 – Tx type

The *findOracle* function filters through all UTXOs at the oracle script address and keeps only the one that contains the specific NFT. This can fail in the case that we just started the oracle and, in this case, we return a **Nothing**. We should notice that in the current version of Plutus the *utxoAt* function does not exist in the documentation and was probably replaced by the function *utxosAt* which returns a map that contains *ChainIndexTxOut* for the keys instead of *TxOutTx*. So, this code is a bit outdated but you can still run it given you use the right commit of the plutus repository in the nix shell.

Next in the *updateOracle* function we define the constraint that we have to pay to our script address with the given datum and value that we get from the input parameters. In the first case where the oracle UTXO does not exist yet we create a transaction that produces this output at the oracle address. Then we wait for confirmation and log a message. For the second case that the UTXO does already exist we first define our lookups where we put a constraint on *unspentOutputs*, *typedValidatorLookups* and the validator script. In order to find our output that we want to spend, we must use the unspent outputs lookup, which takes in a map of the UTXOs that we want to consume. We provide the validator and the typed validator. One is to be able to consume the input. And the other one is to be able to pay to the output. And for the transaction we use the constraint *c* that we defined in the beginning and add to it the constraint *mustSpendScriptOutput* where we specify the redeemer, which is used for collecting funds at the oracle address from the owner. Then we submit the transaction, wait for confirmation and log a message. With the *@Oracling* parameter we signal what are the types for the datum and redeemer of our validator script. In the end of this code there is the function *runOracle* which combines the two contracts in one (190-200). For that we also need a schema that we define in 188. In the *runOracle* function we use tell for the oracle parameter. We need to communicate this parameter value to the outside world so that people can use our oracle. The *go* functions blocks at the update endpoint. And as soon as somebody provides an integer, which is the new value, it will call our update oracle function with this value. Let's look now at an example contract from the file *Swap.hs*.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass      #-}
3  {-# LANGUAGE DeriveGeneric       #-}
4  {-# LANGUAGE FlexibleContexts    #-}
5  {-# LANGUAGE MultiParamTypeClasses #-}
6  {-# LANGUAGE NoImplicitPrelude   #-}
7  {-# LANGUAGE OverloadedStrings   #-}
8  {-# LANGUAGE ScopedTypeVariables #-}
9  {-# LANGUAGE TemplateHaskell     #-}
10 {-# LANGUAGE TypeApplications    #-}
11 {-# LANGUAGE TypeFamilies         #-}
12 {-# LANGUAGE TypeOperators        #-}
13
14 module Week06.Oracle.Swap
15   ( SwapSchema
16   , swap
17   ) where
18
19 import           Control.Monad      hiding (fmap)
20 import           Data.List          (find)
```

```

21 import qualified Data.Map           as Map
22 import           Data.Maybe         (mapMaybe)
23 import           Data.Monoid         (Last (..))
24 import           Data.Text           (Text)
25 import           Plutus.Contract     as Contract
26 import qualified PlutusTx
27 import           PlutusTx.Prelude    hiding (Semigroup(..), (<$>), unless,
                                         mapMaybe, find)
28 import           Ledger              hiding (singleton)
29 import           Ledger.Constraints  as Constraints
30 import qualified Ledger.Typed.Scripts as Scripts
31 import           Ledger.Ada          as Ada hiding (divide)
32 import           Ledger.Value        as Value
33 import           Prelude              (Semigroup (..), Show (..), String,
                                         (<$>))
34
35 import           Week06.Oracle.Core
36 import           Week06.Oracle.Funds
37
38 {-# INLINABLE price #-}
39 price :: Integer -> Integer -> Integer
40 price lovelace exchangeRate = (lovelace * exchangeRate) `divide` 1000000
41
42 {-# INLINABLE lovelaces #-}
43 lovelaces :: Value -> Integer
44 lovelaces = Ada.getLovelace . Ada.fromValue
45
46 {-# INLINABLE mkSwapValidator #-}
47 mkSwapValidator :: Oracle -> Address -> PubKeyHash -> () ->
                    ScriptContext -> Bool
48 mkSwapValidator oracle addr pkh () ctx =
49     txSignedBy info pkh ||
50     (traceIfFalse "expected exactly two script inputs" hasTwoScriptInputs &&
51      traceIfFalse "price not paid" sellerPaid)
52
53 where
54     info :: TxInfo
55     info = scriptContextTxInfo ctx
56
57     oracleInput :: TxOut
58     oracleInput =
59         let
60             ins = [ o
61                   | i <- txInfoInputs info
62                     , let o = txInInfoResolved i

```

```

63         , txOutAddress o == addr
64     ]
65     in
66         case ins of
67             [o] -> o
68             _    -> traceError "expected exactly one oracle input"
69
70     oracleValue' = case oracleValue oracleInput (`findDatum` info) of
71         Nothing -> traceError "oracle value not found"
72         Just x   -> x
73
74     hasTwoScriptInputs :: Bool
75     hasTwoScriptInputs =
76         let
77             xs = filter (isJust . toValidatorHash . txOutAddress .
78                           txInInfoResolved) $ txInfoInputs info
79         in
80             length xs == 2
81
82     minPrice :: Integer
83     minPrice =
84         let
85             lovelaceIn = case findOwnInput ctx of
86                 Nothing -> traceError "own input not found"
87                 Just i   -> lovelaces $ txOutValue $ txInInfoResolved i
88             in
89                 price lovelaceIn oracleValue'
90
91     sellerPaid :: Bool
92     sellerPaid =
93         let
94             pricePaid :: Integer
95             pricePaid = assetClassValueOf (valuePaidTo info pkh)
96                           (oAsset oracle)
97         in
98             pricePaid >= minPrice
99
100 data Swapping
101 instance Scripts.ValidatorTypes Swapping where
102     type instance DatumType Swapping = PubKeyHash
103     type instance RedeemerType Swapping = ()
104
105 typedSwapValidator :: Oracle -> Scripts.TypedValidator Swapping
106 typedSwapValidator oracle = Scripts.mkTypedValidator @Swapping
107     ($$(PlutusTx.compile [| mkSwapValidator |]))

```

```

106     `PlutusTx.applyCode` PlutusTx.liftCode oracle
107     `PlutusTx.applyCode` PlutusTx.liftCode (oracleAddress oracle))
108     $$ (PlutusTx.compile [| wrap |])
109 where
110     wrap = Scripts.wrapValidator @PubKeyHash @()
111
112 swapValidator :: Oracle -> Validator
113 swapValidator = Scripts.validatorScript . typedSwapValidator
114
115 swapAddress :: Oracle -> Ledger.Address
116 swapAddress = scriptAddress . swapValidator
117
118 offerSwap :: forall w s. Oracle -> Integer -> Contract w s Text ()
119 offerSwap oracle amt = do
120     pkh <- pubKeyHash <$> Contract.ownPubKey
121     let tx = Constraints.mustPayToTheScript pkh $ Ada.lovelaceValueOf amt
122     ledgerTx <- submitTxConstraints (typedSwapValidator oracle) tx
123     awaitTxConfirmed $ txId ledgerTx
124     logInfo @String $ "offered " ++ show amt ++ " lovelace for swap"
125
126 findSwaps :: Oracle -> (PubKeyHash -> Bool) -> Contract w s Text
127             [(TxOutRef, TxOutTx, PubKeyHash)]
128 findSwaps oracle p = do
129     utxos <- utxoAt $ swapAddress oracle
130     return $ mapMaybe g $ Map.toList utxos
131 where
132     f :: TxOutTx -> Maybe PubKeyHash
133     f o = do
134         dh <- txOutDatumHash $ txOutTxOut o
135         (Datum d) <- Map.lookup dh $ txData $ txOutTxTx o
136         PlutusTx.fromBuiltinData d
137
138     g :: (TxOutRef, TxOutTx) -> Maybe (TxOutRef, TxOutTx, PubKeyHash)
139     g (oref, o) = do
140         pkh <- f o
141         guard $ p pkh
142         return (oref, o, pkh)
143
144 retrieveSwaps :: Oracle -> Contract w s Text ()
145 retrieveSwaps oracle = do
146     pkh <- pubKeyHash <$> ownPubKey
147     xs <- findSwaps oracle (== pkh)
148     case xs of
149         [] -> logInfo @String "no swaps found"
150         _ -> do

```

```

150         let lookups = Constraints.unspentOutputs (Map.fromList
151             [(oref, o) | (oref, o, _) <- xs]) <>
152             Constraints.otherScript (swapValidator oracle)
153         tx      = mconcat [Constraints.mustSpendScriptOutput oref $
154             Redeemer $ PlutusTx.toBuiltinData () |
155             (oref, _, _) <- xs]
156         ledgerTx <- submitTxConstraintsWith @Swapping lookups tx
157         awaitTxConfirmed $ txId ledgerTx
158         logInfo @String $ "retrieved " ++ show (length xs) ++ " swap(s)"
159
160 useSwap :: forall w s. Oracle -> Contract w s Text ()
161 useSwap oracle = do
162     funds <- ownFunds
163     let amt = assetClassValueOf funds $ oAsset oracle
164     logInfo @String $ "available assets: " ++ show amt
165
166     m <- findOracle oracle
167     case m of
168     Nothing      -> logInfo @String "oracle not found"
169     Just (oref, o, x) -> do
170         logInfo @String $ "found oracle, exchange rate " ++ show x
171         pkh <- pubKeyHash <$> Contract.ownPubKey
172         swaps <- findSwaps oracle (/= pkh)
173         case find (f amt x) swaps of
174         Nothing      -> logInfo @String "no suitable swap
175             found"
176         Just (oref', o', pkh') -> do
177             let v      = txOutValue (txOutTxOut o) <>
178                 lovelaceValueOf (oFee oracle)
179                 p      = assetClassValue (oAsset oracle) $ price
180                     (lovelaces $ txOutValue $ txOutTxOut o') x
181                 lookups = Constraints.otherScript
182                     (swapValidator oracle) <>
183                     Constraints.otherScript
184                     (oracleValidator oracle) <>
185                     Constraints.unspentOutputs
186                     (Map.fromList [(oref, o), (oref', o')])
187                 tx      = Constraints.mustSpendScriptOutput oref
188                     (Redeemer $ PlutusTx.toBuiltinData Use) <>
189                     Constraints.mustSpendScriptOutput oref'
190                     (Redeemer $ PlutusTx.toBuiltinData ()) <>
191                     Constraints.mustPayToOtherScript
192                     (validatorHash $ oracleValidator oracle)
193                     (Datum $ PlutusTx.toBuiltinData x) v
194             <>

```

```

184             Constraints.mustPayToPubKey pkh' p
185         ledgerTx <- submitTxConstraintsWith @Swapping lookups tx
186         awaitTxConfirmed $ txId ledgerTx
187         logInfo @String $ "made swap with price " ++
            show (Value.flattenValue p)
188     where
189         getPrice :: Integer -> TxOutTx -> Integer
190         getPrice x o = price (lovelaces $ txOutValue $ txOutTxOut o) x
191
192         f :: Integer -> Integer -> (TxOutRef, TxOutTx, PubKeyHash) -> Bool
193         f amt x (_, o, _) = getPrice x o <= amt
194
195     type SwapSchema =
196         Endpoint "offer"      Integer
197         .\ / Endpoint "retrieve" ()
198         .\ / Endpoint "use"    ()
199         .\ / Endpoint "funds"  ()
200
201     swap :: Oracle -> Contract (Last Value) SwapSchema Text ()
202     swap oracle = (offer `select` retrieve `select` use `select` funds) >>
        swap oracle
203
204     where
205         offer :: Contract (Last Value) SwapSchema Text ()
206         offer = h $ do
207             amt <- endpoint @"offer"
208             offerSwap oracle amt
209
210         retrieve :: Contract (Last Value) SwapSchema Text ()
211         retrieve = h $ do
212             endpoint @"retrieve"
213             retrieveSwaps oracle
214
215         use :: Contract (Last Value) SwapSchema Text ()
216         use = h $ do
217             endpoint @"use"
218             useSwap oracle
219
220         funds :: Contract (Last Value) SwapSchema Text ()
221         funds = h $ do
222             endpoint @"funds"
223             v <- ownFunds
224             tell $ Last $ Just v
225
226     h :: Contract (Last Value) SwapSchema Text () ->
        Contract (Last Value) SwapSchema Text ()

```

The idea is we have now a swap contract where someone can put down some ADA and someone else can swap it for a USDT token where 1 USD is 1.000.000 USDT. The *price* function takes in lovelace and the exchange rate and returns the equivalent amount of USD. The *lovelaces* function given a value extracts the amount of lovelace. Next comes our swap validator function (46-96). It is a validator that takes in two parameters of type Oracle and Address. For the oracle parameter we are using the code from the Core.hs file that we also import. The address parameter represents the address of the oracle. For the datum we use a pub key hash that comes from the seller that sells his ADA and receives the USDT tokens. And for the redeemer we can use unit. There are two ways to unlock the lovelace by the script address. One is if the buyer actually does this swap, gives tokens to the seller in exchange for lovelace and the other is that the seller himself retrieves the lovelace. The first condition in the validator is if the seller himself signs the transaction. Then we have a logical »or« and then two conditions follow for the swap case. The first condition is that there are two script inputs which are the oracle and the swap scripts. The buyer's input is just a regular public key address. And the second condition is that the seller gets paid. For this condition we first define the helper function *oracleInput* that returns the transaction output of the oracle UTXO. Then we define the *oracleValue'* parameter that holds the integer price exchange rate of the oracle. Next, we define the *minPrice* parameter where we use the *findOwnInput* function.

```
findOwnInput :: ScriptContext -> Maybe TxInInfo
```

From the *TxInInfo* we use the *txInInfoResolved* field to get the *TxOut*. With the *txOutValue* field we get the value and with our helper function *lovelaces* we extract the amount of lovelaces. In the end we compute the price in USD. And in the *sellerPaid* condition we use the *assetClassValueOf* function to get the quantity of the given AssetClass class in the Value.

```
assetClassValueOf :: Value -> AssetClass -> Integer
```

Then when check that the price paid, must be larger or equal to the minimal price. Next, we make an instance of the *ValidatorTypes* class (98-101). After that we compile our typed validator and then we compute the validator and the script address (103-116).

Now follows the off-chain code. The first contract is *offerSwap* which enables the seller to provide ADA for a swap (118-124). It takes a parameter of type oracle and an integer which is the amount he wants to offer. When we construct the transaction, we provide our own public key and the amount of lovelace we want to offer. Then we submit the transaction where we provide the typed validator, wait for confirmation and log a message. Next comes the helper

function *findSwaps* that finds all swaps that specify a specific predicate (126-141). Given an oracle parameter a condition function for a pub key hash it returns a list of all UTXOs (their reference, the UTXO itself and the datum) that satisfy the condition for the datum. First, we get all UTXOs sitting at this address. Then we use the *mapMaybe* function on the list we get.

```
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

It takes in a function and a list and applies the function to each element of the list. Then it throws out the Nothings and extracts out the values from the Justs. In the where clause we define the helper function *f* where we look up the datum hash and then we look up the datum and deserialize it to a pub key hash. If this would not succeed, we would get a Nothing. The *g* function takes a pair of the reference and the UTXO itself. First, we get our public key hash. Then we apply our condition function and if we get True we return a triple of UTXO information and if not we return a Nothing. The next contract *retrieveSwaps* is for the seller if he changes his mind (143-155). First, we look up our public key hash. Next, we find all UTXOs at the swap address that belong to the seller that wants to retrieve his funds. If there are none there is nothing to retrieve. If there are some, we construct a transaction that retrieves all of those. We use the data that we have gotten with the *findSwaps* function as input to our constraints. Then we submit the transaction and log a message. The next contract is called *useSwap* which is used to do the actual swap (157-193). First, we look up our own funds. We do this with the *ownFunds* helper function that comes from the imported code from Funds.hs which is below.

```
1  import           Control.Monad      hiding (fmap)
2  import qualified Data.Map           as Map
3  import           Data.Monoid        (Last (..))
4  import           Data.Text          (Text)
5  import           Plutus.Contract   as Contract
6  import           PlutusTx.Prelude  hiding (<$>)
7  import           Prelude            (Show (..), String, (<$>))
8  import           Ledger             hiding (singleton)
9  import           Ledger.Value       as Value
10
11 ownFunds :: Contract w s Text Value
12 ownFunds = do
13     pk      <- ownPubKey
14     utxos   <- utxoAt $ pubKeyAddress pk
15     let v = mconcat $ Map.elims $ txOutValue . txOutTxOut <$> utxos
16     logInfo @String $ "own funds: " ++ show (Value.flattenValue v)
17     return v
18
19 ownFunds' :: Contract (Last Value) Empty Text ()
20 ownFunds' = do
```

```

21     handleError logError $ ownFunds >=> tell . Last . Just
22     void $ Contract.waitNSlots 1
23     ownFunds'

```

In the *ownFunds* function we first get our pub key. Then we use the function *pubKeyAddress*.

```
pubKeyAddress :: PaymentPubKey -> Maybe StakePubKey -> Address
```

It gets the Address that belongs to our key. And we can use it for the *utxoAt* function to get all UTXOs at our address. Then we get the combined value of those UTXOs, we log a message and return this value. Next, we check in our funds how many of the USDT tokens we have and we log a message. Then we find the oracle with the *findOracle* function. If we get a Nothing value, we just log a message else we pattern match the output reference, UTXO and datum. Next, we log a message, look up our own pub key hash and find swap contracts that do not belong to us. Then we use the find function that returns the first element of a list that satisfies a predicate, or Nothing, if there is no such element.

```
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
```

With the helper functions *getPrice* and *f* we check that the price we would have to pay for that specific swap is at most as high as the tokens we own. We compare USDT how much we have and how much a swap could give us ADA in USDT value. In case we do not find a suitable UTXO we just log a message else we pattern match the output reference, the UTXO itself and the pub key hash from the datum. Then we define the value we need to pay to the oracle and the price we need to pay to the swap contract. Now we define the lookups. Since we are using two scripts as input, we have to specify the validators here. And we have to provide the unspent outputs of the oracle and swap UTXOs that we found. Then we define the transaction where we define that we have to spend the script output of the oracle and swap UTXOs with the redeemers Use and unit. We add a constraint that we have to pay to the oracle address the value we computed with the unchanged datum. And we have to pay to the public key from the swap datum the amount of USDT we also computed. Then we submit the transaction, wait for confirmation and log a message.

Now that we have our contracts, we define the schema for our 4 endpoints. We have an »offer« endpoint where we offer a swap, then a »retrieve« endpoint to retrieve our offered funds, next a »use« endpoint for using the swap and in the end the »funds« endpoint to query our funds. And we define the swap contract (201-226). We offer all the endpoints and whichever of these components makes progress first will be executed. These 4 endpoints are wrappers of the contracts we defined before. What we do is we block with the endpoint until

someone provides a parameter and then we execute it. The `h` is an error handler that just logs the error in case there is one. Let's look at the code in `Test.hs` file.

```
1 {-# LANGUAGE DataKinds           #-}
2 {-# LANGUAGE DeriveAnyClass      #-}
3 {-# LANGUAGE DeriveGeneric       #-}
4 {-# LANGUAGE FlexibleContexts    #-}
5 {-# LANGUAGE MultiParamTypeClasses #-}
6 {-# LANGUAGE NoImplicitPrelude   #-}
7 {-# LANGUAGE NumericUnderscores  #-}
8 {-# LANGUAGE OverloadedStrings   #-}
9 {-# LANGUAGE ScopedTypeVariables #-}
10 {-# LANGUAGE TemplateHaskell     #-}
11 {-# LANGUAGE TypeApplications    #-}
12 {-# LANGUAGE TypeFamilies        #-}
13 {-# LANGUAGE TypeOperators       #-}
14
15 module Week06.Oracle.Test where
16
17 import           Control.Monad           hiding (fmap)
18 import           Control.Monad.Freeer.Extras as Extras
19 import           Data.Default            (Default (..))
20 import qualified Data.Map                as Map
21 import           Data.Monoid              (Last (..))
22 import           Data.Text                (Text)
23 import           Ledger
24 import           Ledger.Value             as Value
25 import           Ledger.Ada              as Ada
26 import           Plutus.Contract         as Contract
27 import           Plutus.Trace.Emulator   as Emulator
28 import           PlutusTx.Prelude        hiding (Semigroup(..), unless)
29 import           Prelude                  (IO, Semigroup(..), Show (..))
30 import           Wallet.Emulator.Wallet
31
32 import           Week06.Oracle.Core
33 import           Week06.Oracle.Funds
34 import           Week06.Oracle.Swap
35
36 assetSymbol :: CurrencySymbol
37 assetSymbol = "ff"
38
39 assetToken :: TokenName
40 assetToken = "USDT"
41
42 test :: IO ()
```

```

43 test = runEmulatorTraceIO' def emCfg def myTrace
44   where
45     emCfg :: EmulatorConfig
46     emCfg = EmulatorConfig $ Left $ Map.fromList [(Wallet i, v) |
47       i <- [1 .. 10]]
48     v :: Value
49     v = Ada.lovelaceValueOf 100_000_000 <>
50       Value.singleton assetSymbol assetToken 100_000_000
51
52 checkOracle :: Oracle -> Contract () Empty Text a
53 checkOracle oracle = do
54   m <- findOracle oracle
55   case m of
56     Nothing      -> return ()
57     Just (_, _, x) -> Contract.logInfo $ "Oracle value: " ++ show x
58     Contract.waitNSlots 1 >> checkOracle oracle
59
60 myTrace :: EmulatorTrace ()
61 myTrace = do
62   let op = OracleParams
63       { opFees = 1_000_000
64       , opSymbol = assetSymbol
65       , opToken = assetToken
66       }
67
68   h1 <- activateContractWallet (Wallet 1) $ runOracle op
69   void $ Emulator.waitNSlots 1
70   oracle <- getOracle h1
71
72   void $ activateContractWallet (Wallet 2) $ checkOracle oracle
73
74   callEndpoint @"update" h1 1_500_000
75   void $ Emulator.waitNSlots 3
76
77   void $ activateContractWallet (Wallet 1) ownFunds'
78   void $ activateContractWallet (Wallet 3) ownFunds'
79   void $ activateContractWallet (Wallet 4) ownFunds'
80   void $ activateContractWallet (Wallet 5) ownFunds'
81
82   h3 <- activateContractWallet (Wallet 3) $ swap oracle
83   h4 <- activateContractWallet (Wallet 4) $ swap oracle
84   h5 <- activateContractWallet (Wallet 5) $ swap oracle
85
86   callEndpoint @"offer" h3 10_000_000

```

```

87     callEndpoint @"offer" h4 20_000_000
88     void $ Emulator.waitNSlots 3
89
90     callEndpoint @"use" h5 ()
91     void $ Emulator.waitNSlots 3
92
93     callEndpoint @"update" h1 1_700_000
94     void $ Emulator.waitNSlots 3
95
96     callEndpoint @"use" h5 ()
97     void $ Emulator.waitNSlots 3
98
99     callEndpoint @"update" h1 1_800_000
100    void $ Emulator.waitNSlots 3
101
102    callEndpoint @"retrieve" h3 ()
103    callEndpoint @"retrieve" h4 ()
104    void $ Emulator.waitNSlots 3
105    where
106    getOracle :: ContractHandle (Last Oracle) OracleSchema Text ->
        EmulatorTrace Oracle
107    getOracle h = do
108        l <- observableState h
109        case l of
110            Last Nothing      -> Emulator.waitNSlots 1 >> getOracle h
111            Last (Just oracle) -> Extras.logInfo (show oracle) >>
                return oracle

```

The test module tests the contracts we have written with the emulator trace monad. In order to test it we need to make up some currencies. So, we define an arbitrary currency symbol which is not the hash of a script but same as in the playground we can do this in the emulator trace and we define our token name USDT (36-40). Then we are using the *runEmulatorTraceIO'* function that gives us more configuration control. The first input parameter determines how exactly the various log messages are displayed. With the second parameter we configure the initial distribution and we specify that everybody has a 100 ADA and 100 USDT. Then we define a helper contract *checkOracle* which should permanently check the oracle value and log it (52-58). We use the *findOracle* function and in case we find a value we log it and recourse after 1 slot. Then we define our trace (60-111). First, we define the oracle parameters. Then we start the oracle with these parameters for wallet 1. Then we use the *getOracle* helper function that looks up the state of the oracle contract. If we do not find it we wait 1 slot and try again. Now that we have the oracle value, we start the *checkOracle* function that prints the oracle value every slot. Then we initialize the oracle to the value 1.5 which is 1500000 USDT per 1 ADA and

wait for 3 slots. And then we call the *ownFunds'* function for all 5 wallets. It is a variation of the *ownFunds* function where we every slot tell the value of the funds we have. Next, we start the swap contract for wallets 3 to 5. Then we try out some scenarios. Wallet 3 offers 10 ADA for swap and wallet 4 offers 20 ADA. Now wallet 5 uses the swap. It picks the one that it finds first. After that wallet 1 updated the oracle value to 1.7 and wallet 5 tries again to do a swap. So now it will grab the remaining swap. Then we try to retrieve all the remaining swaps but that should not do anything because we already used up the funds. In the Repl we can run the test.

```
Prelude> test
```

In the output you will get the results for all 5 wallets with the final balances.

9.3 Using the PAB

Now we will use the PAB to turn our application into an executable that actually runs the contracts. For that we need the code in *PAB.hs* which is basically just one type definition.

```
1  {-# LANGUAGE DeriveAnyClass      #-}
2  {-# LANGUAGE DeriveGeneric      #-}
3
4  module Week06.Oracle.PAB
5      ( OracleContracts (..)
6      ) where
7
8  import           Data.Aeson          (FromJSON, ToJSON)
9  import           Data.Text.Prettyprint.Doc (Pretty (..), viaShow)
10 import           GHC.Generics         (Generic)
11 import           Ledger
12
13 import qualified Week06.Oracle.Core    as Oracle
14
15 data OracleContracts = Init | Oracle CurrencySymbol | Swap Oracle.Oracle
16     deriving (Eq, Ord, Show, Generic, FromJSON, ToJSON)
17
18 instance Pretty OracleContracts where
19     pretty = viaShow
```

So, we have various contracts and now we define the data type where each value of the data type corresponds to a contract we eventually want to run. This init is nothing we have written until now, but this basically corresponds to what we did in the emulator trace monad to give initial funds. The Oracle constructor corresponds to the *runOracle* contract and the Swap

constructor corresponds to *swap* contract that allows us to call various endpoints. In the cabal file we have the executable oracle-pab which runs the *oracle-pab.hs* file that starts a simulated wallet and initialize all the contracts and start a set up a web server that allows the outside world to interact with these contracts. And then we have two more executable. The oracle-client interacts with the run oracle contract and actually fetches exchange rates from the internet and feeds them into the system. And then the swap-client executable would be run by the clients that want to make use of the swap contract. Let's look at the code in the *oracle-pab.hs* file.

```

1 {-# LANGUAGE DataKinds           #-}
2 {-# LANGUAGE DeriveAnyClass      #-}
3 {-# LANGUAGE DerivingStrategies  #-}
4 {-# LANGUAGE FlexibleContexts    #-}
5 {-# LANGUAGE LambdaCase         #-}
6 {-# LANGUAGE NumericUnderscores #-}
7 {-# LANGUAGE OverloadedStrings   #-}
8 {-# LANGUAGE RankNTypes          #-}
9 {-# LANGUAGE TypeApplications    #-}
10 {-# LANGUAGE TypeFamilies        #-}
11 {-# LANGUAGE TypeOperators       #-}
12
13 module Main
14     ( main
15     ) where
16
17 import Control.Monad                (forM_, void, when)
18 import Control.Monad.Freer         (Eff, Member,
19                                     interpret, type (~>))
20 import Control.Monad.Freer.Error   (Error)
21 import Control.Monad.Freer.Extras.LogMsg (LogMsg)
22 import Control.Monad.IO.Class      (MonadIO (..))
23 import Data.Aeson                  (FromJSON, Result (..),
24                                     fromJSON)
25 import Data.Default                (Default (..))
26 import Data.Monoid                 (Last (..))
27 import Data.Text                   (Text, pack)
28 import Ledger                      (Ledger.Constraints
29                                   as Value
30                                   as Contract
31                                   as ContractEffect (..)
32                                   as Builtin, SomeBuiltin
33                                   as endpointsToSchemas,
34                                   handleBuiltin)

```

```

32 import          Plutus.PAB.Monitoring.PABLogMsg      (PABMultiAgentMsg)
33 import          Plutus.PAB.Simulator                 (SimulatorEffectHandler
s)
34 import qualified Plutus.PAB.Simulator                as Simulator
35 import          Plutus.PAB.Types                     (PABError (..))
36 import qualified Plutus.PAB.Webserver.Server         as PAB.Server
37 import qualified Plutus.Contracts.Currency            as Currency
38
39 import          Wallet.Emulator.Types                (Wallet (..),
walletPubKey)
40 import          Wallet.Types                         (ContractInstanceId
(..))
41
42 import qualified Week06.Oracle.Core                  as Oracle
43 import          Week06.Oracle.PAB                   (OracleContracts (..))
44 import qualified Week06.Oracle.Swap                  as Oracle
45
46 main :: IO ()
47 main = void $ Simulator.runSimulationWith handlers $ do
48     Simulator.logString @(Builtin OracleContracts) "Starting Oracle PAB
webserver. Press enter to exit."
49     shutdown <- PAB.Server.startServerDebug
50
51     cidInit <- Simulator.activateContract (Wallet 1) Init
52     cs      <- waitForLast cidInit
53     _      <- Simulator.waitUntilFinished cidInit
54
55     cidOracle <- Simulator.activateContract (Wallet 1) $ Oracle cs
56     liftIO $ writeFile "oracle.cid" $ show $ unContractInstanceId cidOracle
57     oracle <- waitForLast cidOracle
58
59     forM_ wallets $ \w ->
60         when (w /= Wallet 1) $ do
61             cid <- Simulator.activateContract w $ Swap oracle
62             liftIO $ writeFile ('W' : show (getWallet w) ++ ".cid") $ show
$ unContractInstanceId cid
63
64     void $ liftIO getLine
65     shutdown
66
67 waitForLast :: FromJSON a => ContractInstanceId -> Simulator.Simulation t a
68 waitForLast cid =
69     flip Simulator.waitForState cid $ \json -> case fromJSON json of
70         Success (Last (Just x)) -> Just x
71         _                        -> Nothing

```



```

72
73 wallets :: [Wallet]
74 wallets = [Wallet i | i <- [1 .. 5]]
75
76 usdt :: TokenName
77 usdt = "USDT"
78
79 oracleParams :: CurrencySymbol -> Oracle.OracleParams
80 oracleParams cs = Oracle.OracleParams
81   { Oracle.opFees    = 1_000_000
82   , Oracle.opSymbol  = cs
83   , Oracle.opToken   = usdt
84   }
85
86 handleOracleContracts ::
87   ( Member (Error PABError) effs
88   , Member (LogMsg (PABMultiAgentMsg (Builtin OracleContracts))) effs
89   )
90   => ContractEffect (Builtin OracleContracts)
91   ~> Eff effs
92 handleOracleContracts = handleBuiltin getSchema getContract where
93   getSchema = \case
94     Init      -> endpointsToSchemas @Empty
95     Oracle _  -> endpointsToSchemas @Oracle.OracleSchema
96     Swap _    -> endpointsToSchemas @Oracle.SwapSchema
97   getContract = \case
98     Init      -> SomeBuiltin  initContract
99     Oracle cs -> SomeBuiltin $ Oracle.runOracle $ oracleParams cs
100    Swap oracle -> SomeBuiltin $ Oracle.swap oracle
101
102 handlers :: SimulatorEffectHandlers (Builtin OracleContracts)
103 handlers =
104   Simulator.mkSimulatorHandlers @(Builtin OracleContracts) def []
105   $ interpret handleOracleContracts
106
107 initContract :: Contract (Last CurrencySymbol) Empty Text ()
108 initContract = do
109   ownPK <- pubKeyHash <$> ownPubKey
110   cur   <-
111     mapError (pack . show)
112     (Currency.mintContract ownPK [(usdt, fromIntegral
113                                     (length wallets) * amount)]
114   :: Contract (Last CurrencySymbol) Empty Currency.CurrencyError
115               Currency.OneShotCurrency)
116   let cs = Currency.currencySymbol cur

```

```

115     v = Value.singleton cs usdt amount
116   forM_ wallets $ \w -> do
117     let pkh = pubKeyHash $ walletPubKey w
118     when (pkh /= ownPK) $ do
119       tx <- submitTx $ mustPayToPubKey pkh v
120       awaitTxConfirmed $ txId tx
121   tell $ Last $ Just cs
122 where
123   amount :: Integer
124   amount = 100_000_000

```

The part from line 86 to 105 is something you always need and is used to hook up the data type we defined in *PAB.hs*, hook that up with the correspondent schemas and contracts that we defined earlier. Init won't have any schema. Then we define the *initContract* (107-124). There we use again the mint contract where we mint 100.000.000 USDT for each wallet except our own wallet that already has the tokens. Now let's look at the beginning of the code. We make use of another monad called simulator monad. From it you can start contracts on wallets, inspect the log and the state and you can call endpoints. In the simulator monad you can do IO operations in contrast to the emulator trace monad. If you have some arbitrary IO action that you can do in Haskell, then by applying *liftIO* to it, you can move it into the simulator monad.



At the time of writing the *liftIO* function is already outdated.

So first we log that we will start the PAB server. With *startServerDebug* we start the PAB and the return value is an action that we can later use to shut down the server (49). Then we activate the init contract for wallet 1 (51). We get the currency symbol with the helper function *waitForLast* that is defined in 67-71. The *waitForState* function given a contract instance ID and a predicate that takes a json expression and returns a maybe and then returns a simulation. So, this waits until the state of the contract has told a just value and then returns that value. And the *waitUntilFinished* function waits until the contract has finished. Next step is we start the oracle on wallet 1 (55). If we want to use the web interface later to talk to the contract, we need this cid so we write it into a file called *oracle.cid* (56). And we use again the *waitForLast* function to get the oracle value (57). And we need it because the swap contract takes it in as a input parameter. Then we loop over all wallets except wallet 1 and activate the swap contract for each wallet. And then we write the contract instance IDs to files. Then we block until the user presses enter and then we shut down the server (64-65). We can now run the PAB.

```
$ cabal run oracle-pab
```

And we get log outputs similar to the trace example, just that we have now a running executable that sets up a live server. And until we press enter the server will be running. The server creates a REST API on port 8080 that has several endpoints to query. In the *oracle-client.hs* file we query the endpoints on the PAB server. Here is the code.

```
1 {-# LANGUAGE NumericUnderscores #-}
2 {-# LANGUAGE OverloadedStrings #-}
3
4 module Main
5   ( main
6   ) where
7
8 import Control.Concurrent
9 import Control.Monad      (when)
10 import Control.Monad.IO.Class (MonadIO (..))
11 import Data.ByteString    (ByteString)
12 import Data.ByteString.Char8 (unpack)
13 import Data.Proxy         (Proxy (..))
14 import Data.Text          (pack)
15 import Data.UUID
16 import Network.HTTP.Req
17 import Text.Regex.TDFA
18
19 main :: IO ()
20 main = do
21   uuid <- read <$> readFile "oracle.cid"
22   putStrLn $ "oracle contract instance id: " ++ show uuid
23   go uuid Nothing
24   where
25     go :: UUID -> Maybe Integer -> IO a
26     go uuid m = do
27       x <- getExchangeRate
28       let y = Just x
29       when (m /= y) $
30         updateOracle uuid x
31       threadDelay 5_000_000
32       go uuid y
33
34 updateOracle :: UUID -> Integer -> IO ()
35 updateOracle uuid x = runReq defaultHttpConfig $ do
36   v <- req
37     POST
38     (http "127.0.0.1" /: "api" /: "new" /: "contract" /: "instance" /:
39       pack (show uuid) /: "endpoint" /: "update")
39   (ReqBodyJson x)
```

```

40     (Proxy :: Proxy (JsonResponse ()))
41     (port 8080)
42     liftIO $ putStrLn $ if responseStatusCode v == 200
43     then "updated oracle to " ++ show x
44     else "error updating oracle"
45
46 getExchangeRate :: IO Integer
47 getExchangeRate = runReq defaultHttpConfig $ do
48     v <- req
49     GET
50     (https "coinmarketcap.com" /: "currencies" /: "cardano")
51     NoReqBody
52     bsResponse
53     mempty
54     let priceRegex      = "priceValue__11gHJ \">>\$([\.\0-9]*)" ::
                           ByteString
55     (_, _, _, [bs]) = responseBody v =~ priceRegex ::
                           (ByteString, ByteString, ByteString, [ByteString])
56     d                = read $ unpack bs :: Double
57     x                = round $ 1_000_000 * d
58     liftIO $ putStrLn $ "queried exchange rate: " ++ show d
59     return x

```

We use the *req* library to make the HTTP requests. So, in the main program we read the *oracle.cid* file. Then we call the *go* function that loops forever. What it does it looks up the current exchange rate with help of the *getExchangeRate* function which goes to coinmarketcap to lookup the exchange rate between USD and ADA (46-59). Then it checks whether the value has changed compared to the previous function call. For this reason, in the beginning, we provide a Nothing value. And if it does change, we call the *updateOracle* function (34-44). Then we recurse after 5 seconds. In the update oracle function, we make a POST request. Then we must provide the request body in json format of the value we want to update which is our exchange rate. We can try this code out. The oracle-pab has to be running.

```
$ cabal run oracle-client
```

We will see the current exchange rate regularly outputted to the terminal. Let's look now at the code in *swap-client.hs*.

```

1  {-# LANGUAGE NumericUnderscores #-}
2  {-# LANGUAGE OverloadedStrings  #-}
3  {-# LANGUAGE ScopedTypeVariables #-}
4
5  module Main

```

```

6      ( main
7      ) where
8
9      import Control.Concurrent
10     import Control.Exception
11     import Control.Monad.IO.Class      (MonadIO (..))
12     import Data.Aeson                  (Result (..), fromJSON)
13     import Data.Monoid                  (Last (..))
14     import Data.Proxy                   (Proxy (..))
15     import Data.Text                    (pack)
16     import Data.UUID
17     import Ledger.Value                 (flattenValue)
18     import Network.HTTP.Req
19     import Plutus.PAB.Events.ContractInstanceState (PartiallyDecodedResponse
20                                                         (..))
21
22     import Plutus.PAB.Webserver.Types
23     import System.Environment            (getArgs)
24     import System.IO
25     import Text.Read                     (readMaybe)
26
27     import Week06.Oracle.PAB             (OracleContracts)
28
29     main :: IO ()
30     main = do
31       [i :: Int] <- map read <$> getArgs
32       uuid        <- read <$> readFile ('W' : show i ++ ".cid")
33       hSetBuffering stdout NoBuffering
34       putStrLn $ "swap contract instance id for Wallet " ++ show i ++
35         ": " ++ show uuid
36
37       go uuid
38     where
39       go :: UUID -> IO a
40       go uuid = do
41         cmd <- readCommand
42         case cmd of
43           Offer amt -> offer uuid amt
44           Retrieve  -> retrieve uuid
45           Use       -> use uuid
46           Funds     -> getFunds uuid
47         go uuid
48
49     readCommand :: IO Command
50     readCommand = do
51       putStr "enter command (Offer amt, Retrieve, Use or Funds): "
52       s <- getLine

```

```

49         maybe readCommand return $ readMaybe s
50
51 data Command = Offer Integer | Retrieve | Use | Funds
52     deriving (Show, Read, Eq, Ord)
53
54 getFunds :: UUID -> IO ()
55 getFunds uuid = handle h $ runReq defaultHttpConfig $ do
56     v <- req
57     POST
58     (http "127.0.0.1" /: "api" /: "new" /: "contract" /: "instance" /:
59      pack (show uuid) /: "endpoint" /: "funds")
60     (ReqBodyJson ())
61     (Proxy :: Proxy (JsonResponse ()))
62     (port 8080)
63     if responseStatusCode v /= 200
64     then liftIO $ putStrLn "error getting funds"
65     else do
66         w <- req
67         GET
68         (http "127.0.0.1" /: "api" /: "new" /: "contract" /:
69          "instance" /: pack (show uuid) /: "status")
70         NoReqBody
71         (Proxy :: Proxy (JsonResponse (ContractInstanceClientState
72          OracleContracts)))
73         (port 8080)
74         liftIO $ putStrLn $ case fromJSON $ observableState $
75             cicCurrentState $ responseBody w of
76             Success (Last (Just f)) -> "funds: " ++ show (flattenValue f)
77             _ -> "error decoding state"
78     where
79         h :: HttpException -> IO ()
80         h _ = threadDelay 1_000_000 >> getFunds uuid
81
82 offer :: UUID -> Integer -> IO ()
83 offer uuid amt = handle h $ runReq defaultHttpConfig $ do
84     v <- req
85     POST
86     (http "127.0.0.1" /: "api" /: "new" /: "contract" /: "instance" /:
87      pack (show uuid) /: "endpoint" /: "offer")
88     (ReqBodyJson amt)
89     (Proxy :: Proxy (JsonResponse ()))
90     (port 8080)
91     liftIO $ putStrLn $ if responseStatusCode v == 200
92     then "offered swap of " ++ show amt ++ " lovelace"
93     else "error offering swap"

```

```

89     where
90         h :: HttpException -> IO ()
91         h _ = threadDelay 1_000_000 >> offer uuid amt
92
93     retrieve :: UUID -> IO ()
94     retrieve uuid = handle h $ runReq defaultHttpConfig $ do
95         v <- req
96             POST
97             (http "127.0.0.1" /: "api" /: "new" /: "contract" /: "instance" /:
98              pack (show uuid) /: "endpoint" /: "retrieve")
99             (ReqBodyJson ())
100            (Proxy :: Proxy (JsonResponse ()))
101            (port 8080)
102         liftIO $ putStrLn $ if responseStatusCode v == 200
103             then "retrieved swaps"
104             else "error retrieving swaps"
105     where
106         h :: HttpException -> IO ()
107         h _ = threadDelay 1_000_000 >> retrieve uuid
108
109     use :: UUID -> IO ()
110     use uuid = handle h $ runReq defaultHttpConfig $ do
111         v <- req
112             POST
113             (http "127.0.0.1" /: "api" /: "new" /: "contract" /: "instance" /:
114              pack (show uuid) /: "endpoint" /: "use")
115             (ReqBodyJson ())
116            (Proxy :: Proxy (JsonResponse ()))
117            (port 8080)
118         liftIO $ putStrLn $ if responseStatusCode v == 200
119             then "used swap"
120             else "error using swap"
121     where
122         h :: HttpException -> IO ()
123         h _ = threadDelay 1_000_000 >> use uuid

```

The idea is we want to offer a very simple console interface. We offer the 4 command line possibilities: offer an amount, retrieve funds, use the swap and lookup the funds. When we start this client, we provide the wallet number as a command line parameter. Then we read out the cid file for the given wallet and we go into a loop where the user is prompted to type in one of the 4 commands. If the command is invalid the user is prompted again. For the *getFunds* function we make the first request to write the funds into the observable state of the contract

and then the second request for the status endpoint to read this information out. Let's start this client for wallet 2 and wallet 3. We have to do this in two separate terminals.

```
$ cabal run swap-client -- 2  
$ cabal run swap-client -- 3
```

We can now query our funds and then offer 10 ADA from wallet 2.

```
Funds  
Offer 10000000
```

If we look at the PAB console, we will see that something happened there. If we go now to wallet 3, we can exchange those ADA for our USDT tokens and check our funds.

```
Use  
Funds
```

It will take some seconds before the funds will be updated. We should then see a larger amount of ADA as in the initial funds and some smaller amount of USDT. This completes our example. In real life different instances of wallets would be running different PAB servers.

Appendix A. Code examples

In this chapter we will present some code examples. The first code example is a simple auction contract that was presented in lecture 1 of the plutus pioneer program. And the second example is a Plutus version of the popular Uniswap contract from Ethereum. This was presented in the 10th lecture of the 2nd iteration of the plutus pioneer program found here:

<https://www.youtube.com/watch?v=7Lfj2mGIPLQ>

A.1. Token sale

The idea of the token sale is that Alice wants to auction a NFT that exists only once. The auction will be parameterized by the owner of the token and the token itself, then the minimal bid and the deadline before which all the bids have to arrive. You can see the schema in Figure 57.

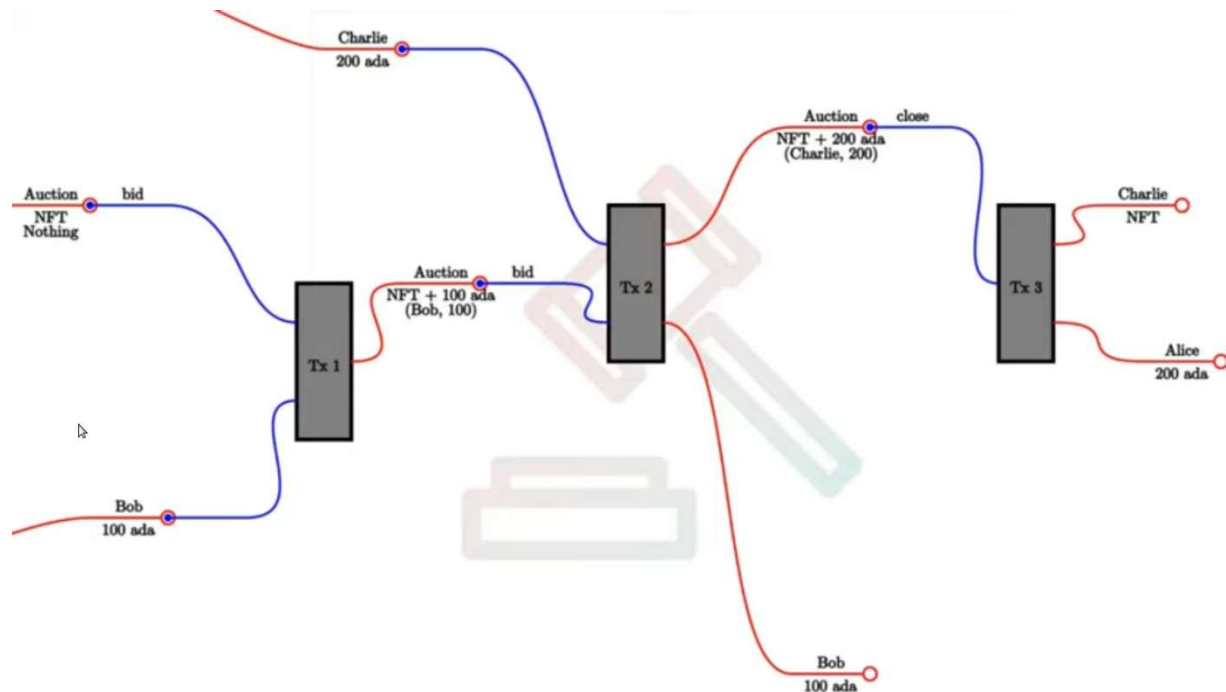


Figure 57 – Auction workflow

So, let's say that Alice creates a UTXO at the auction script. And the value of this UTXO is just the NFT and the datum at the moment is Nothing. Later it will be the highest bidder and the highest bid. Then we say that Bob wants to bid 100 ADA. He creates a transaction with 2 UTXOs as input (his bid and the auction UTXO) and 1 as output which is again sitting at the auction script but the value and the datum have changed compared to the previous UTXO. The auction script checks that the bid happens before the deadline and is high enough. Next Charlie makes a bid of 200 ADA. The transaction he creates has 2 inputs and outputs. One output is a UTXO

that will go to Bob and will contain his bid. Now the script will have to check that the deadline has not been reached, that the new bid is higher than the old one, that the new auction UTXO gets correctly updated and that the previous highest bidder Bob gets his bid back. Finally let's assume that the auction will be closed from Alice. This transaction will have only one input the auction UTXO with the redeemer close and two outputs that go to Charlie and Alice. Another example we can have is that nobody makes a bid. Then there has to be a mechanism for Alice to get back her NFT. Now let's look at the code in *EnglishAuction.hs*.

```
1  {-# LANGUAGE DataKinds           #-}
2  {-# LANGUAGE DeriveAnyClass       #-}
3  {-# LANGUAGE DeriveGeneric        #-}
4  {-# LANGUAGE DerivingStrategies   #-}
5  {-# LANGUAGE FlexibleContexts     #-}
6  {-# LANGUAGE GeneralizedNewtypeDeriving #-}
7  {-# LANGUAGE LambdaCase          #-}
8  {-# LANGUAGE MultiParamTypeClasses #-}
9  {-# LANGUAGE NoImplicitPrelude    #-}
10 {-# LANGUAGE OverloadedStrings    #-}
11 {-# LANGUAGE RecordWildCards      #-}
12 {-# LANGUAGE ScopedTypeVariables   #-}
13 {-# LANGUAGE TemplateHaskell      #-}
14 {-# LANGUAGE TypeApplications     #-}
15 {-# LANGUAGE TypeFamilies         #-}
16 {-# LANGUAGE TypeOperators        #-}
17
18 module Week01.EnglishAuction
19   ( Auction (..)
20   , StartParams (..), BidParams (..), CloseParams (..)
21   , AuctionSchema
22   , start, bid, close
23   , endpoints
24   , schemas
25   , ensureKnownCurrencies
26   , printJson
27   , printSchemas
28   , registeredKnownCurrencies
29   , stage
30   ) where
31
32 import           Control.Monad      hiding (fmap)
33 import           Data.Aeson         (ToJSON, FromJSON)
34 import           Data.List.NonEmpty (NonEmpty (..))
35 import           Data.Map           as Map
36 import           Data.Text          (pack, Text)
```

```

37 import      GHC.Generics      (Generic)
38 import      Ledger            hiding (singleton)
39 import qualified Ledger.Constraints as Constraints
40 import qualified Ledger.Typed.Scripts as Scripts
41 import      Ledger.Value      as Value
42 import      Ledger.Ada        as Ada
43 import      Playground.Contract (IO, ensureKnownCurrencies,
44                                printSchemas, stage, printJson)
45 import      Playground.TH      (mkKnownCurrencies,
46                                mkSchemaDefinitions)
47 import      Playground.Types   (KnownCurrency (...))
48 import      Plutus.Contract
49 import qualified PlutusTx      as PlutusTx
50 import      PlutusTx.Prelude   hiding (unless)
51 import qualified Prelude      as P
52 import      Schema              (ToSchema)
53 import      Text.Printf         (printf)
54
55 minLovelace :: Integer
56 minLovelace = 2000000
57
58 data Auction = Auction
59   { aSeller    :: !PaymentPubKeyHash
60   , aDeadline  :: !POSIXTime
61   , aMinBid    :: !Integer
62   , aCurrency :: !CurrencySymbol
63   , aToken     :: !TokenName
64   } deriving (P.Show, Generic, ToJSON, FromJSON, ToSchema)
65
66 instance Eq Auction where
67   {-# INLINABLE (==) #-}
68   a == b = (aSeller a == aSeller b) &&
69           (aDeadline a == aDeadline b) &&
70           (aMinBid a == aMinBid b) &&
71           (aCurrency a == aCurrency b) &&
72           (aToken a == aToken b)
73
74 PlutusTx.unstableMakeIsData ''Auction
75 PlutusTx.makeLift ''Auction
76
77 data Bid = Bid
78   { bBidder :: !PaymentPubKeyHash
79   , bBid     :: !Integer
80   } deriving P.Show

```

```

80 instance Eq Bid where
81     {-# INLINABLE (==) #-}
82     b == c = (bBidder b == bBidder c) &&
83             (bBid b == bBid c)
84
85 PlutusTx.unstableMakeIsData ''Bid
86 PlutusTx.makeLift ''Bid
87
88 data AuctionAction = MkBid Bid | Close
89     deriving P.Show
90
91 PlutusTx.unstableMakeIsData ''AuctionAction
92 PlutusTx.makeLift ''AuctionAction
93
94 data AuctionDatum = AuctionDatum
95     { adAuction      :: !Auction
96     , adHighestBid :: !(Maybe Bid)
97     } deriving P.Show
98
99 PlutusTx.unstableMakeIsData ''AuctionDatum
100 PlutusTx.makeLift ''AuctionDatum
101
102 data Auctioning
103 instance Scripts.ValidatorTypes Auctioning where
104     type instance RedeemerType Auctioning = AuctionAction
105     type instance DatumType Auctioning = AuctionDatum
106
107 {-# INLINABLE minBid #-}
108 minBid :: AuctionDatum -> Integer
109 minBid AuctionDatum{..} = case adHighestBid of
110     Nothing      -> aMinBid adAuction
111     Just Bid{..} -> bBid + 1
112
113 {-# INLINABLE mkAuctionValidator #-}
114 mkAuctionValidator :: AuctionDatum -> AuctionAction -> ScriptContext -> Bool
115 mkAuctionValidator ad redeemer ctx =
116     traceIfFalse "wrong input value" correctInputValue &&
117     case redeemer of
118         MkBid b@Bid{..} ->
119             traceIfFalse "bid too low" (sufficientBid bBid) &&
120             traceIfFalse "wrong output datum" (correctBidOutputDatum b) &&
121             traceIfFalse "wrong output value" (correctBidOutputValue bBid) &&
122             traceIfFalse "wrong refund" correctBidRefund &&
123             traceIfFalse "too late" correctBidSlotRange
124         Close ->

```

```

125         traceIfFalse "too early" correctCloseSlotRange &&
126         case adHighestBid ad of
127             Nothing      ->
128                 traceIfFalse "expected seller to get token"
129                     (getValue (aSeller auction) $ tokenValue <>
130                     Ada.lovelaceValueOf minLovelace)
129             Just Bid{..} ->
130                 traceIfFalse "expected highest bidder to get token"
131                     (getValue bBidder $ tokenValue <>
132                     Ada.lovelaceValueOf minLovelace) &&
131                 traceIfFalse "expected seller to get highest bid"
132                     (getValue (aSeller auction) $ Ada.lovelaceValueOf bBid)
132
133     where
134         info :: TxInfo
135         info = scriptContextTxInfo ctx
136
137         input :: TxInInfo
138         input =
139             let
140                 isScriptInput i = case (txOutDatumHash . txInInfoResolved) i of
141                     Nothing -> False
142                     Just _  -> True
143                 xs = [i | i <- txInfoInputs info, isScriptInput i]
144             in
145                 case xs of
146                     [i] -> i
147                     _   -> traceError "expected exactly one script input"
148
149         inVal :: Value
150         inVal = txOutValue . txInInfoResolved $ input
151
152         auction :: Auction
153         auction = adAuction ad
154
155         tokenValue :: Value
156         tokenValue = Value.singleton (aCurrency auction) (aToken auction) 1
157
158         correctInputValue :: Bool
159         correctInputValue = inVal == case adHighestBid ad of
160             Nothing      -> tokenValue <> Ada.lovelaceValueOf minLovelace
161             Just Bid{..} -> tokenValue <> Ada.lovelaceValueOf (minLovelace+bBid)
162
163         sufficientBid :: Integer -> Bool
164         sufficientBid amount = amount >= minBid ad

```

```

165
166 ownOutput    :: TxOut
167 outputDatum  :: AuctionDatum
168 (ownOutput, outputDatum) = case getContinuingOutputs ctx of
169     [o] -> case txOutDatumHash o of
170         Nothing    -> traceError "wrong output type"
171         Just h -> case findDatum h info of
172             Nothing    -> traceError "datum not found"
173             Just (Datum d) -> case PlutusTx.fromBuiltinData d of
174                 Just ad' -> (o, ad')
175                 Nothing  -> traceError "error decoding data"
176     _      -> traceError "expected exactly one continuing output"
177
178 correctBidOutputDatum :: Bid -> Bool
179 correctBidOutputDatum b = (adAuction outputDatum == auction)    &&
180                           (adHighestBid outputDatum == Just b)
181
182 correctBidOutputValue :: Integer -> Bool
183 correctBidOutputValue amount =
184     txOutValue ownOutput == tokenValue <>
185         Ada.lovelaceValueOf (minLovelace + amount)
186
187 correctBidRefund :: Bool
188 correctBidRefund = case adHighestBid ad of
189     Nothing    -> True
190     Just Bid{..} ->
191         let
192             os = [ o
193                 | o <- txInfoOutputs info
194                   , txOutAddress o == pubKeyHashAddress bBidder Nothing
195                 ]
196         in
197             case os of
198                 [o] -> txOutValue o == Ada.lovelaceValueOf bBid
199                 _    -> traceError "expected exactly one refund output"
200
201 correctBidSlotRange :: Bool
202 correctBidSlotRange = to (aDeadline auction) `contains`
203     txInfoValidRange info
204
205 correctCloseSlotRange :: Bool
206 correctCloseSlotRange = from (aDeadline auction) `contains`
207     txInfoValidRange info
208
209 getsValue :: PaymentPubKeyHash -> Value -> Bool

```

```

207     getsValue h v =
208         let
209             [o] = [ o'
210                     | o' <- txInfoOutputs info
211                       , txOutValue o' == v
212                     ]
213         in
214             txOutAddress o == pubKeyHashAddress h Nothing
215
216 typedAuctionValidator :: Scripts.TypedValidator Auctioning
217 typedAuctionValidator = Scripts.mkTypedValidator @Auctioning
218   $$ (PlutusTx.compile [| mkAuctionValidator |])
219   $$ (PlutusTx.compile [| wrap |])
220 where
221     wrap = Scripts.wrapValidator @AuctionDatum @AuctionAction
222
223 auctionValidator :: Validator
224 auctionValidator = Scripts.validatorScript typedAuctionValidator
225
226 auctionHash :: Ledger.ValidatorHash
227 auctionHash = Scripts.validatorHash typedAuctionValidator
228
229 auctionAddress :: Ledger.Address
230 auctionAddress = scriptHashAddress auctionHash
231
232 data StartParams = StartParams
233   { spDeadline :: !POSIXTime
234   , spMinBid   :: !Integer
235   , spCurrency :: !CurrencySymbol
236   , spToken    :: !TokenName
237   } deriving (Generic, ToJSON, FromJSON, ToSchema)
238
239 data BidParams = BidParams
240   { bpCurrency :: !CurrencySymbol
241   , bpToken     :: !TokenName
242   , bpBid       :: !Integer
243   } deriving (Generic, ToJSON, FromJSON, ToSchema)
244
245 data CloseParams = CloseParams
246   { cpCurrency :: !CurrencySymbol
247   , cpToken     :: !TokenName
248   } deriving (Generic, ToJSON, FromJSON, ToSchema)
249
250 type AuctionSchema =
251     Endpoint "start" StartParams

```

```

252     .\ Endpoint "bid"    BidParams
253     .\ Endpoint "close" CloseParams
254
255 start :: AsContractError e => StartParams -> Contract w s e ()
256 start StartParams{..} = do
257     pkh <- ownPaymentPubKeyHash
258     let a = Auction
259         { aSeller    = pkh
260         , aDeadline  = spDeadline
261         , aMinBid    = spMinBid
262         , aCurrency  = spCurrency
263         , aToken     = spToken
264         }
265     d = AuctionDatum
266         { adAuction    = a
267         , adHighestBid = Nothing
268         }
269     v = Value.singleton spCurrency spToken 1 <>
        Ada.lovelaceValueOf minLovelace
270     tx = Constraints.mustPayToTheScript d v
271     ledgerTx <- submitTxConstraints typedAuctionValidator tx
272     void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
273     logInfo @P.String $ printf "started auction %s for token %s"
        (P.show a) (P.show v)
274
275 bid :: forall w s. BidParams -> Contract w s Text ()
276 bid BidParams{..} = do
277     (oref, o, d@AuctionDatum{..}) <- findAuction bpCurrency bpToken
278     logInfo @P.String $ printf "found auction utxo with datum %s" (P.show d)
279
280     when (bpBid < minBid d) $
281         throwError $ pack $ printf "bid lower than minimal bid %d"
            (minBid d)
282
283     pkh <- ownPaymentPubKeyHash
284     let b = Bid {bBidder = pkh, bBid = bpBid}
285     d' = d {adHighestBid = Just b}
286     v = Value.singleton bpCurrency bpToken 1 <>
        Ada.lovelaceValueOf (minLovelace + bpBid)
287
288     r = Redeemer $ PlutusTx.toBuiltinData $ MkBid b
289
290     lookups = Constraints.typedValidatorLookups typedAuctionValidator
291     P.<> Constraints.otherScript auctionValidator
292     P.<> Constraints.unspentOutputs (Map.singleton oref o)
293
294     tx = case adHighestBid of
295         Nothing    -> Constraints.mustPayToTheScript d' v <>

```



```

293         Constraints.mustValidateIn
                (to $ aDeadline adAuction) <>
294         Constraints.mustSpendScriptOutput oref r
295         Just Bid{..} -> Constraints.mustPayToTheScript d' v <>
296         Constraints.mustPayToPubKey bBidder
                (Ada.lovelaceValueOf bBid) <>
297         Constraints.mustValidateIn
                (to $ aDeadline adAuction) <>
298         Constraints.mustSpendScriptOutput oref r
299 ledgerTx <- submitTxConstraintsWith lookups tx
300 void $ awaitTxConfirmed $ getCardanoTxId ledgerTx
301 logInfo @P.String $ printf "made bid of %d lovelace
                        in auction %s for token (%s, %s)"
302                        bpBid
303                        (P.show adAuction)
304                        (P.show bpCurrency)
305                        (P.show bpToken)
306
307 close :: forall w s. CloseParams -> Contract w s Text ()
308 close CloseParams{..} = do
309     (oref, o, d@AuctionDatum{..}) <- findAuction cpCurrency cpToken
310     logInfo @P.String $ printf "found auction utxo with datum %s" (P.show d)
311
312     let t      = Value.singleton cpCurrency cpToken 1
313         r      = Redeemer $ PlutusTx.toBuiltinData Close
314         seller = aSeller adAuction
315
316     lookups = Constraints.typedValidatorLookups typedAuctionValidator
317             P.<> Constraints.otherScript auctionValidator
318             P.<> Constraints.unspentOutputs (Map.singleton oref o)
319     tx      = case adHighestBid of
320         Nothing      -> Constraints.mustPayToPubKey seller
                        (t <> Ada.lovelaceValueOf minLovelace)<>
321         Constraints.mustValidateIn
                        (from $ aDeadline adAuction) <>
322         Constraints.mustSpendScriptOutput oref r
323         Just Bid{..} -> Constraints.mustPayToPubKey bBidder
                        (t <> Ada.lovelaceValueOf minLovelace)<>
324         Constraints.mustPayToPubKey seller
                        (Ada.lovelaceValueOf bBid) <>
325         Constraints.mustValidateIn
                        (from $ aDeadline adAuction) <>
326         Constraints.mustSpendScriptOutput oref r
327 ledgerTx <- submitTxConstraintsWith lookups tx
328 void $ awaitTxConfirmed $ getCardanoTxId ledgerTx

```

```

329     logInfo @P.String $ printf "closed auction %s for token (%s, %s)"
330         (P.show adAuction)
331         (P.show cpCurrency)
332         (P.show cpToken)
333
334 findAuction :: CurrencySymbol
335             -> TokenName
336             -> Contract w s Text (TxOutRef, ChainIndexTxOut, AuctionDatum)
337 findAuction cs tn = do
338     utxos <- utxosAt $ scriptHashAddress auctionHash
339     let xs = [ (oref, o)
340               | (oref, o) <- Map.toList utxos
341               , Value.valueOf (_ciTxOutValue o) cs tn == 1
342               ]
343     case xs of
344     [(oref, o)] -> case _ciTxOutDatum o of
345         Left _      -> throwError "datum missing"
346         Right (Datum e) -> case PlutusTx.fromBuiltinData e of
347             Nothing -> throwError "datum has wrong type"
348             Just d@AuctionDatum{..}
349                 | aCurrency adAuction == cs && aToken adAuction == tn ->
350                     return (oref, o, d)
351                 | otherwise ->
352                     throwError "auction token mismatch"
353     _ -> throwError "auction utxo not found"
354
355 endpoints :: Contract () AuctionSchema Text ()
356 endpoints = awaitPromise (start' `select` bid' `select` close') >> endpoints
357     where
358         start' = endpoint @"start" start
359         bid'   = endpoint @"bid" bid
360         close' = endpoint @"close" close
361
362 mkSchemaDefinitions "'AuctionSchema
363
364 myToken :: KnownCurrency
365 myToken = KnownCurrency (ValidatorHash "f") "Token" (TokenName "T" :| [])
366
367 mkKnownCurrencies ['myToken]

```

First we define the minimum lovelace amount that a UTXO has to have and for the testnet is 2 ADA. Then we define the auction data type that holds all necessary information (56-62). Then we make this data type an instance of the equal type class. Since we will use that data type as part of the datum we have to make it an instance of the *ToData* type class (72). Next we define

the Bid data type that contains the bid and the bidder's public key hash (75-78). We also make it an instance of the `equal` and `ToData` type classes. Then we define our action data type that represents our redeemer. It has only two constructors, the bid and the close option. After that we define the datum that contains the auction and bid data types (94-97). We make also an instance for the `Scripts.ValidatorTypes` class to define our datum and redeemer. We need this because our datum and redeemer are custom data types. Next, we define the helper function `minBid` that takes in the datum and returns the minimal bid a bidder has to make to replace the current bid (107-111). Now we come to our auction validator function. First, we check if we have the correct input value. For this we use the helper function `input` that checks all the transaction inputs that represent a script and in case there is more than one raises an error (137-147). Else it returns the one transaction input. We can check whether a UTXO belongs to a script address by checking if the datum hash exists. With the `inVal` function we get the Value sitting at the returned transaction input. And with the `correctInputValue` function we check whether the value of the transaction input we filtered out matches the value in the provided datum. After we checked that the input value is correct, we further decide to do checks depending on the redeemer. If the redeemer is `Close`, we check that the validity interval of the transaction falls after the bid deadline. If it does, we further check the `adHighestBid` parameter. If it is `Nothing` it means that no bid was made and, in this case, we check that the NFT and minimal ADA amount go back to the seller. The `getValue` function compares the address from the script context information and the datum and returns a `Bool`. But if the bid was made, we check that the highest bidder gets the token and the seller gets the highest bid. And that is it for the `Close` action. For the `MkBid` action we check several conditions. First, we check that the new bid is high enough. Then we check that the correct datum information is contained in the script context information by comparing it to the redeemer information. After it we check that the correct value will be contained in the output UTXO for the auction address. Then we check that in case there was a previous bid that we are outbidding the bid goes back to the original bidder. To get the correct address we use the function `pubKeyHashAddress` that takes a payment pub key hash and a maybe stake pub key hash and returns an address belonging to that key.

```
pubKeyHashAddress :: PaymentPubKeyHash -> Maybe StakePubKeyHash -> Address
```

In the end we check that the bid is made before the auction deadline. And that is the entire validator function. Then we compile the typed auction validator and we compute the auction hash and address. This completes the on-chain code. Next follows the off-chain code where we first define the data types for the three endpoints we will have (232-248). Then we define the auction schema (250-253). After that we define the start contract that takes in the start parameter. First, we look up our own payment public key hash. Then we define the auction

datum for the start of the auction. We compute the value that holds the NFT and minimal ADA and construct the transaction. In the end we submit the transaction, wait for confirmation and log a message. Next follows the bid contract that takes in the bid parameter. First, we use the *findAuction* function defined in 334-351 that for the given token name and currency symbol finds the UTXO at the auction script address and returns the output reference, output and datum. If it would not find the UTXO it would throw an error. Then we define following parameters: the bid, the new datum, the new value the auction UTXO will contain and the redeemer. Next, we define the lookups where we use for the first time the constraint *typedValidatorLookups*. In addition, we define 2 other constraints as usual. With the *Map.singleton* function we construct a map of output references and chain index outputs. Then we construct the transaction. We put constraints that we must pay to the script with value and datum that we computed. Then we set the validity interval for the transaction. Next, we specify which output reference we want to spend and with what redeemer. In the case that we are not the first bidder we put the additional constraint that we pay the current bid to the previous bidder back. Then we submit the transaction, wait for confirmation and log a message. The third contract is the close contract. We again use the *findAuction* function that gives us the output reference, chain index output and datum for the auction UTXO. Then we log a message. Next, we define our parameters: the token value, the redeemer that we set to Close and the payment public key of the seller. We define our lookups same as in the previous example. And we define our transaction depending on whether any bids have been made or not. If not, we say that the token must go back to the seller and if yes, we say the token goes to the bidder that is written inside the datum and the bid goes to the seller. In both cases we define the validity interval and the script output we want to spend. Then we submit the transaction, wait for confirmation and log a message. In the end we define our endpoints contract, create a schema definition and create currencies where we use the *myToken* parameter that defines our token. In the playground we define 3 wallets with a 100 ADA for each wallet.

The screenshot displays the 'Wallets' interface in a simulation environment. At the top, there's a tab labeled 'Simulation 1' with a plus icon to add more. Below this, the title 'Wallets' is followed by a subtitle: 'Add some initial wallets, then click one of your function calls inside the wallet to begin a chain of actions.' Three wallet cards are shown side-by-side:

- Wallet 1:** Opening Balances: Lovelace (100000000), T (1). Available functions: bid, close, start, Pay to Wallet.
- Wallet 2:** Opening Balances: Lovelace (100000000), T (0). Available functions: bid, close, start, Pay to Wallet.
- Wallet 3:** Opening Balances: Lovelace (100000000), T (0). Available functions: bid, close, start, Pay to Wallet.

Figure 58 – Wallets

Then we define our workflow that we discussed in the begging of that chapter (Figure 59).

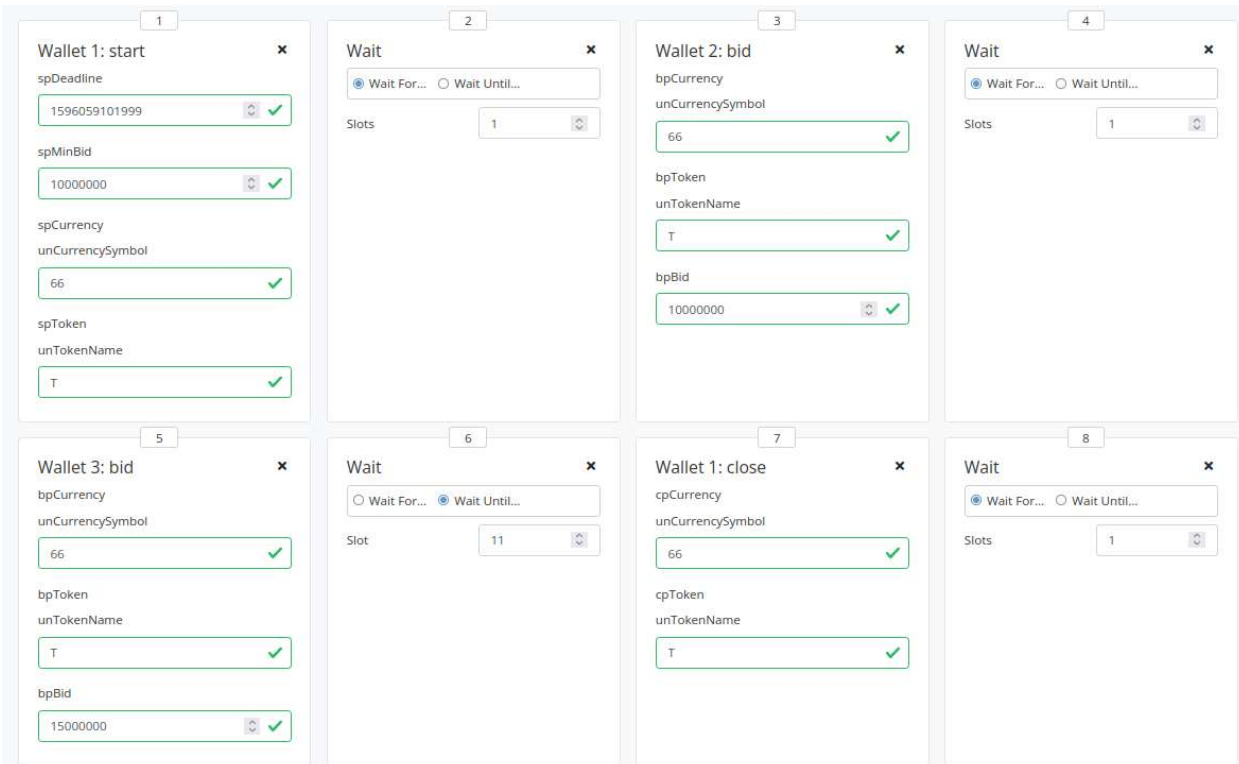


Figure 59 - Playground workflow

First wallet 1 starts the bid. For the currency symbol that we defined as »f« in the code we need to use the number 66 which is some kind of conversion. In the Repl we use the function *slotToEndPOSIXTime* to get the deadline for the auction that we set to 10 slots.

```
Prelude> import Data.Default
Prelude> import Ledger.TimeSlot
Prelude> slotToEndPOSIXTime def 10
POSIXTime {getPOSIXTime = 1596059101999}
```

Then wallet 2 and 3 make a bid. And in the end wallet 1 calls the close endpoint. Wallet 3 makes a higher bid so it should get the token in the end. When you evaluate the simulation, you should get 5 transactions (Figure 60). The first is the genesis transaction that distributes 100 ADA to each of the 3 wallets. The second is the transaction where wallet 1 sends the minimal ADA and NFT to the auction script address. In the third transaction wallet 2 sends a bid of 10 ADA to the auction script address. In the fourth transaction wallet 3 sends a bid of 15 ADA to the auction script address and wallet 2 should receive 10 ADA as part of the output transaction. In the fifth transaction wallet 1 should receive 15 ADA and wallet 3 receives the NFT and the 2 ADA that were the minimal deposit. And the auction finishes.

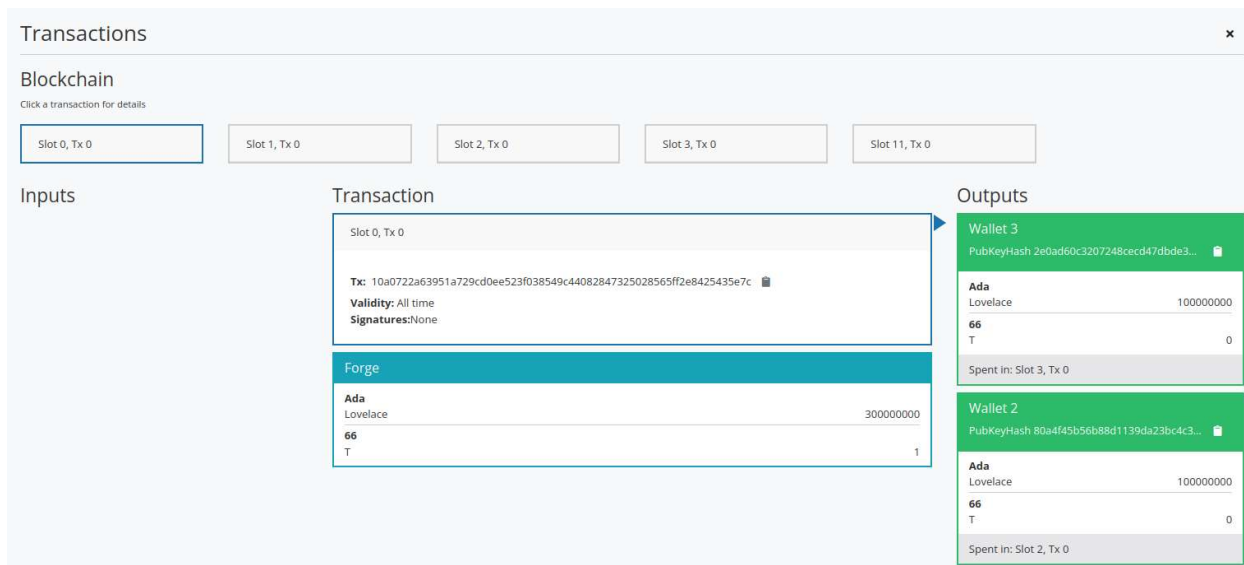


Figure 60 - Auction transactions

A.2. Uniswap contract

Uniswap is a decentralized finance application (DeFi) that allows swapping of tokens. In the case of Ethereum its ERC20 tokens without any central authority. Everything is governed by smart contracts and works fully automatically on the blockchain. It uses an automatic price discovery system. The idea is that people can create so called liquidity pools. If they want other users to be able to swap 2 different tokens then someone can create a liquidity pool and put certain amount of those tokens in this pool. And then the creator of the pool will receive so called liquidity tokens that are specific to this one pool. If people use this pool to swap, they take some amount of one of the tokens out in exchange for putting an amount of the other token back in. In addition, people can also add tokens to the pool and receive liquidity tokens or they can also burn liquidity tokens in exchange for tokens in the pool. So, we will demonstrate these features on the Cardano blockchain. It starts by somebody setting up the whole system that wants to offer this uniswap service (Figure 61). It starts with a transaction that creates a UTXO at this script address that is called factory for Uniswap factory. It contains an NFT that identifies the factory and as datum it will contain a list of all liquidity pools. In the beginning that list will be empty. Let's assume that Alice wants to create a liquidity pool for tokens A and B. She provides a 1000 A and 2000 B number of tokens. She believes that the ratio for trading will be 1 : 2 for A and B. She creates a transaction with 2 inputs and 3 outputs. The inputs are here number of tokens and the Uniswap factory invoked with the Create redeemer. And the outputs will be the newly created pool that we call Pool AB. It contains Alice's tokens and a freshly minted NFT called AB. The datum will be the number 1415 that represents the amount of liquidity tokens Alice receives in return. That is the square root of the product of 1000 and

2000. The second output is the Uniswap factory again that has now an updated datum that contains AB pool name. Finally, there is the output for Alice where she receives the freshly minted liquidity tokens called AB. Let's now assume that Bob wants to swap 100 A against B. He will create a transaction that has 2 inputs and outputs. The inputs are the 100 A and the pool with the Swap redeemer. And the outputs are 181 B he gets in return and the updated pool where the datum is still the same. So, price discovery in Uniswap works with the rule that the product of the amounts of the two tokens must never decrease. So, the reason that Bob gets less than 200 B, which would reflect the original ratio, are that the number of tokens in the liquidity pool is never allowed to go to zero. The more of one sort you get out the more expensive it gets. For this reason, Bob lowered the ration since he took out the B tokens. It automatically accounts for supply and demand. Another reason that Bob gets a bit less out are fees. So, the incentive for Alice to set up a pool is that she wants to earn on swaps that people make. This original product formula is modified a bit to insist that the product increases by a certain percentage depending on how much people swap. It's around 0.3%. The next operation we look at is the add operation where somebody supplies the pool with additional liquidity. So Let's say that Charlie also believes that ration should be 1 : 2 and he contributes 400 A and 800 B. The ration reflects his believe in the true relative value of the tokens. He creates a transaction with 2 inputs and outputs. The inputs are the pool with the redeemer Add and his contribution. The outputs are the updated pool where the datum changes to 1982 and the 567 liquidity tokens that go to Charlie. The formula to calculate this is a bit more complicated. But you take in account how many tokens have already been minted. It also ensures that Alice profits from the fees that Bob paid with the swap and Charlie doesn't. If people add tokens after a time, they shouldn't profit from the fees that were paid before that. The next operation we look at is called remove that allows owners of liquidity tokens of a pool to burn some of them. Let's assume that Alice wants to burn all of her liquidity tokens. She creates a transaction with 2 inputs and outputs. The inputs are her 1415 liquidity tokens and the pool with the redeemer Remove. The outputs are the tokens from the pool she receives in return. She would get 1078 A and 1869 B. And the updated pool is the second output where the datum changes to 567. And the formula for how many tokens Alice gets when she burns her liquidity tokens is again somewhat complicated but it takes into account the current ration of the liquidity tokens and the actual tokens that are in the pool. The last operation is close and it is for completely closing a pool and removing it. And this can only happen when the last remaining liquidity tokens are burnt. In our example Charlie creates a transaction with three inputs and two outputs. The first input is the factory with the Close redeemer, second input is the pool also with the Close redeemer and third input are Charlie's liquidity tokens. The outputs are the updated factory that has now again an empty list for the datum and Charlie's tokens. We have

to note that during the existence of the AB pool nobody else could open a duplicated pool with the same tokens.

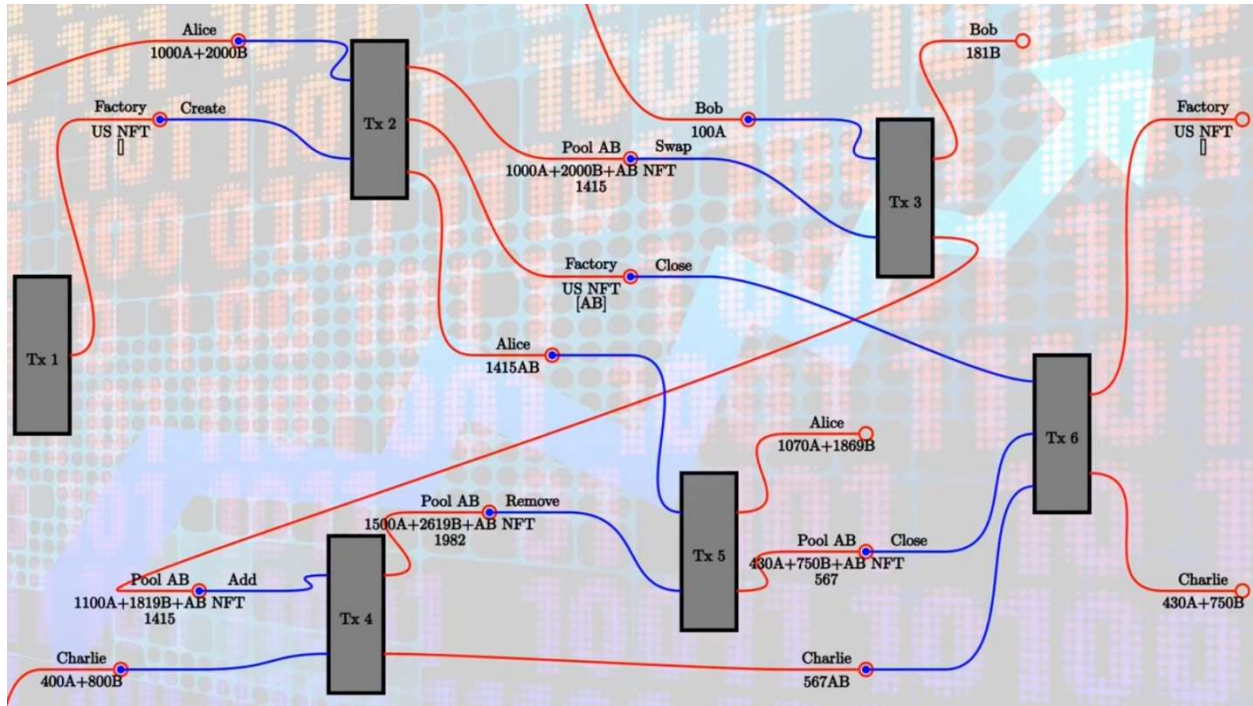


Figure 61 - Uniswap workflow

The code for Uniswap is part of the Plutus repository and it's contained in the `plutus-use-cases` package. It's in the `Plutus.Contract.Uniswap` module. It re-exports 5 modules that contain all the necessary code. In the `Types` module `U` represents the uniswap coin that identifies the factory A and B that are used for pool operations. `PoolState` is the NFT that identifies the pool. And `Liquidity` is used for the liquidity tokens. In the datum a `Coin` type is simply an asset class. `Amount` is just a wrapper around an integer and is used for not confusing amounts of tokens A and B. Then we have some helper functions:

- `valueOf` gives a value type for a given coin and amount
- `unitValue` creates one amount of the given coin
- `isUnity` checks whether this coin is contained in the value exactly once
- `amountOf` checks how often a coin is contained in the value
- `mkCoin` turns a currency symbol and a token name into a coin

Then we have the `Uniswap` type which identifies the instance of the uniswap system we are running. This is used if somebody sets up a competing uniswap system with another factory. We have a type for liquidity pools that is just the two coins in there and the order of the coins

does not matter. Then we have the `UniswapAction` type that contains the actions we have seen in our example. The `UniswapDatum` contains the datum that has the `Factory` and `Pool` constructors which are used for the factory UTXO and the pool UTXOs.

Next let's look at the `Pool` module that contains the business logic. The *`calculateInitialLiquidity`* function gets the initial amount of tokens A and B put into the pool and returns the liquidity tokens that are returned in exchange. The *`calculateAdditionalLiquidity`* function calculates how many liquidity tokens will be additionally minted if someone adds tokens to the pool. The *`calculateRemoval`* function is for the opposite case. The *`checkSwap`* function calculates the swap so it checks for the initial and final amounts of A and B tokens if the swap is valid and returns a `Bool`. It also makes sure that none of the amounts ever drop to zero. The *`lpTicker`* function given a liquidity pool computes a token name for the pool. Let's look now at the on-chain part. Only two functions are exported to make the validator for the Uniswap both factory and pools, because they share the same script address. They address distinguished by the datum and by the coins that identify them and validate liquidity forging. So that's the monetary policy script for the liquidity tokens. Let's look at the off-chain code. We defined two different schemas. The idea is that one is for the entity that creates the Uniswap factory. And that only has one endpoint start and no parameters. And then once that is created a second schema for people that make use of this Uniswap system, and all the contracts in here will be parameterized by the uniswap instance that this first action creates. There are also tests for Uniswap contained in the previous mentioned plutus-use-cases library.

Let's look now at the Plutus PAB and how you can write a front-end for Uniswap. In the `plutus/plutus-pab/examples/uniswap` folder there is code that contains the simulation monad. We take this as the basis and slightly modify it. If we look at the cabal file for the 10th project, we see there are two executables. One Uniswap PAB, which will run the PAB server, and then one Uniswap client, which is a simple console based front-end for the Uniswap application. And both executables have Uniswap listed under other modules. Let's look at this code now.

```
1 {-# LANGUAGE DataKinds           #-}
2 {-# LANGUAGE DeriveAnyClass      #-}
3 {-# LANGUAGE DeriveGeneric       #-}
4 {-# LANGUAGE DerivingStrategies  #-}
5 {-# LANGUAGE FlexibleContexts    #-}
6 {-# LANGUAGE LambdaCase         #-}
7 {-# LANGUAGE OverloadedStrings   #-}
8 {-# LANGUAGE RankNTypes         #-}
9 {-# LANGUAGE TypeApplications    #-}
10 {-# LANGUAGE TypeFamilies        #-}
11 {-# LANGUAGE TypeOperators       #-}
12
```

```

13 module Uniswap where
14
15 import           Control.Monad                (forM_, when)
16 import           Data.Aeson                  (FromJSON, ToJSON)
17 import qualified Data.Semigroup               as Semigroup
18 import           Data.Text.Prettyprint.Doc    (Pretty (..), viaShow)
19 import           GHC.Generics                 (Generic)
20 import           Ledger
21 import           Ledger.Constraints
22 import           Ledger.Value                 as Value
23 import           Plutus.Contract
24 import qualified Plutus.Contracts.Currency    as Currency
25 import qualified Plutus.Contracts.Uniswap     as Uniswap
26 import qualified Plutus.PAB.Effects.Contract.Builtin as Builtin
27 import           Wallet.Emulator.Types        (Wallet (..),
                                                walletPubKey)
28
29 data UniswapContracts =
30     Init
31   | UniswapStart
32   | UniswapUser Uniswap.Uniswap
33   deriving (Eq, Ord, Show, Generic)
34   deriving anyclass (FromJSON, ToJSON)
35
36 instance Pretty UniswapContracts where
37     pretty = viaShow
38
39 instance Builtin.HasDefinitions UniswapContracts where
40     getDefinitions = [Init, UniswapStart]
41     getSchema = \case
42         UniswapUser _ -> Builtin.endpointsToSchemas
43                             @Uniswap.UniswapUserSchema
44         UniswapStart -> Builtin.endpointsToSchemas
45                             @Uniswap.UniswapOwnerSchema
46         Init          -> Builtin.endpointsToSchemas @Empty
47     getContract = \case
48         UniswapUser us -> Builtin.SomeBuiltin $ Uniswap.userEndpoints us
49         UniswapStart   -> Builtin.SomeBuiltin Uniswap.ownerEndpoint
50         Init           -> Builtin.SomeBuiltin initContract
51
52 initContract :: Contract (Maybe (Semigroup.Last Currency.OneShotCurrency))
53               Currency.CurrencySchema Currency.CurrencyError ()
54
55 initContract = do
56     ownPK <- pubKeyHash <$> ownPubKey
57     cur   <- Currency.mintContract ownPK [(tn, fromIntegral (length wallets))

```

```

* amount) | tn <- tokenNames]
54   let cs = Currency.currencySymbol cur
55   v = mconcat [Value.singleton cs tn amount | tn <- tokenNames]
56   forM_ wallets $ \w -> do
57     let pkh = pubKeyHash $ walletPubKey w
58     when (pkh /= ownPK) $ do
59       tx <- submitTx $ mustPayToPubKey pkh v
60       awaitTxConfirmed $ txId tx
61     tell $ Just $ Semigroup.Last cur
62   where
63     amount = 1000000
64
65   wallets :: [Wallet]
66   wallets = [Wallet i | i <- [1 .. 4]]
67
68   tokenNames :: [TokenName]
69   tokenNames = ["A", "B", "C", "D"]
70
71   cidFile :: Wallet -> FilePath
72   cidFile w = "W" ++ show (getWallet w) ++ ".cid"

```

First, we need some data type that captures the various instances we can run for the wallets. And in our *UniswapContract* data type we have three constructors and define the API (29-34). *Init* is just used to create some example tokens and distribute them in the beginning. Then *UniswapStart* is used to setting up the whole system. And *UniswapUser* corresponds to various endpoints for the user to interact with the system and it is parameterized by the *Uniswap* value which is the result of starting. Then we have the *initContract* that distributes the initial funds (50-63). It makes use of the *forge* contract. And it produces 1000000 tokens with token names A, B, C and D for each token. And we repeat this process 4 times and sent the tokens to four different wallets. The *cidFile* helper function produces a file name for a given wallet (71-72). The *HasDefinitions* instance provides links (39-48). So, the *getSchema* links the *UniswapContract* type constructors with the corresponding schemas. And the *getContracts* links this type against the actual contracts. Now let's look at the *uniswap-pab.hs* file.

```

1 {-# LANGUAGE DataKinds          #-}
2 {-# LANGUAGE DeriveAnyClass     #-}
3 {-# LANGUAGE DerivingStrategies #-}
4 {-# LANGUAGE FlexibleContexts   #-}
5 {-# LANGUAGE LambdaCase        #-}
6 {-# LANGUAGE OverloadedStrings  #-}
7 {-# LANGUAGE RankNTypes         #-}
8 {-# LANGUAGE TypeApplications  #-}
9 {-# LANGUAGE TypeFamilies       #-}

```

[illegible]

```

49
50   cidStart <- Simulator.activateContract (Wallet 1) UniswapStart
51   us       <- flip Simulator.waitForState cidStart $ \json -> case
        (fromJSON json :: Result (Monoid.Last (Either Text
            Uniswap.Uniswap))) of
52       Success (Monoid.Last (Just (Right us))) -> Just us
53       _                                           -> Nothing
54   logString @(Builtin UniswapContracts) $ "Uniswap instance created: "
        ++ show us
55
56   forM_ wallets $ \w -> do
57     cid <- Simulator.activateContract w $ UniswapUser us
58     liftIO $ writeFile (cidFile w) $ show $ unContractInstanceId cid
59     logString @(Builtin UniswapContracts) $ "Uniswap user contract
        started for " ++ show w
60
61   void $ liftIO getLine
62
63   shutdown
64
65   handlers :: SimulatorEffectHandlers (Builtin UniswapContracts)
66   handlers =
67     Simulator.mkSimulatorHandlers @(Builtin UniswapContracts) def def
68     $ interpret
69     $ Builtin.contractHandler
70     $ Builtin.handleBuiltin @UniswapContracts

```

In the simulator monad we execute certain things. First, we start the server and get the handle to shut it down again. In the end we wait that the user types a key and then we shut it down again. So, first thing we do is wallet 1 activates the init contract (41). We use the *waitForState* function until init returns and we get back the currency symbol (42-44). And what init will do is it will write the currency symbol of the forged example tokens into the state. Then we wait until the init contract has finished (45). And then we write the currency symbol into a file called *symbol.son* (47). Then we write a log message. Then again for wallet 1 we start the Uniswap system (50). And again, we use *waitForState* to get the result of the *UniswapStart*. The value we get will be of type *Uniswap* and we will need it to parameterize the user contracts (51-53). After the uniswap system is running we can start the user instances for all the wallets (56-59). We get the contract instance IDs - *cid* handles and in order to communicate from the front end with the server we need these handles (57). We write them into a file (58). Then we just wait until the user types a key and then we shut down the server. We can try this out.

```
$ cabal run uniswap-pab
```

We will see that all the Uniswap user contracts will be started for the 4 wallets. The `symbol.json` file that gets created holds the currency symbol of the example tokens we created. And we will have `W1.cid` to `W4.cid` files. In order to find the correct HTTP endpoints to communicate with them we will need these contract instance IDs. Let's look at the *uniswap-client.hs* file next.

[illegible]

```

34 main :: IO ()
35 main = do
36     w <- Wallet . read . head <$> getArgs
37     cid <- read <$> readFile (cidFile w)
38     mcs <- decode <$> LB.readFile "symbol.json"
39     case mcs of
40         Nothing -> putStrLn "invalid symbol.json" >> exitFailure
41         Just cs -> do
42             putStrLn $ "cid: " ++ show cid
43             putStrLn $ "symbol: " ++ show (cs :: CurrencySymbol)
44             go cid cs
45     where
46         go :: UUID -> CurrencySymbol -> IO a
47         go cid cs = do
48             cmd <- readCommandIO
49             case cmd of
50                 Funds -> getFunds cid
51                 Pools -> getPools cid
52                 Create amtA tnA amtB tnB -> createPool cid $ toCreateParams cs
53                                         amtA tnA amtB tnB
54                 Add amtA tnA amtB tnB -> addLiquidity cid $ toAddParams cs
55                                         amtA tnA amtB tnB
56                 Remove amt tnA tnB -> removeLiquidity cid $ toRemoveParams
57                                         cs amt tnA tnB
58                 Close tnA tnB -> closePool cid $ toCloseParams
59                                         cs tnA tnB
60                 Swap amtA tnA tnB -> swap cid $ toSwapParams
61                                         cs amtA tnA tnB
62             go cid cs
63
64 data Command =
65     Funds
66     | Pools
67     | Create Integer Char Integer Char
68     | Add Integer Char Integer Char
69     | Remove Integer Char Char
70     | Close Char Char
71     | Swap Integer Char Char
72     deriving (Show, Read, Eq, Ord)
73
74 readCommandIO :: IO Command
75 readCommandIO = do
76     putStrLn "Enter a command: Funds, Pools, Create amtA tnA amtB tnB, Add
77     amtA tnA amtB tnB, Remove amt tnA tnB, Close tnA tnB, Swap amtA tnA tnB"
78     s <- getLine

```

```

73     maybe readCommandIO return $ readMaybe s
74
75 toCoin :: CurrencySymbol -> Char -> US.Coin c
76 toCoin cs tn = US.Coin $ AssetClass (cs, fromString [tn])
77
78 toCreateParams :: CurrencySymbol -> Integer -> Char -> Integer ->
    Char -> US.CreateParams
79 toCreateParams cs amtA tnA amtB tnB = US.CreateParams (toCoin cs tnA)
    (toCoin cs tnB) (US.Amount amtA)
    (US.Amount amtB)
80
81 toAddParams :: CurrencySymbol -> Integer -> Char -> Integer ->
    Char -> US.AddParams
82 toAddParams cs amtA tnA amtB tnB = US.AddParams (toCoin cs tnA)
    (toCoin cs tnB) (US.Amount amtA)
    (US.Amount amtB)
83
84 toRemoveParams :: CurrencySymbol -> Integer -> Char ->
    Char -> US.RemoveParams
85 toRemoveParams cs amt tnA tnB = US.RemoveParams (toCoin cs tnA)
    (toCoin cs tnB) (US.Amount amt)
86
87 toCloseParams :: CurrencySymbol -> Char -> Char -> US.CloseParams
88 toCloseParams cs tnA tnB = US.CloseParams (toCoin cs tnA) (toCoin cs tnB)
89
90 toSwapParams :: CurrencySymbol -> Integer -> Char -> Char -> US.SwapParams
91 toSwapParams cs amtA tnA tnB = US.SwapParams (toCoin cs tnA) (toCoin cs tnB)
    (US.Amount amtA) (US.Amount 0)
92
93 showCoinHeader :: IO ()
94 showCoinHeader = printf "\n currency symbol token name amount\n\n"
95
96 showCoin :: CurrencySymbol -> TokenName -> Integer -> IO ()
97 showCoin cs tn = printf "%64s %66s %15d\n" (show cs) (show tn)
98
99 getFunds :: UUID -> IO ()
100 getFunds cid = do
101     callEndpoint cid "funds" ()
102     threadDelay 2_000_000
103     go
104 where
105     go = do
106         e <- getStatus cid
107         case e of
108             Right (US.Funds v) -> showFunds v

```



```

109         -> go
110     -
111     showFunds :: Value -> IO ()
112     showFunds v = do
113         showCoinHeader
114         forM_ (flattenValue v) $ \(cs, tn, amt) -> showCoin cs tn amt
115         printf "\n"
116
117     getPools :: UUID -> IO ()
118     getPools cid = do
119         callEndpoint cid "pools" ()
120         threadDelay 2_000_000
121         go
122     where
123         go = do
124             e <- getStatus cid
125             case e of
126                 Right (US.Pools ps) -> showPools ps
127                 -                    -> go
128
129     showPools :: [((US.Coin US.A, US.Amount US.A),
130                    (US.Coin US.B, US.Amount US.B))] -> IO ()
131     showPools ps = do
132         forM_ ps $ \(US.Coin (AssetClass (csA, tnA)), amtA),
133                     (US.Coin (AssetClass (csB, tnB)), amtB)) -> do
134             showCoinHeader
135             showCoin csA tnA (US.unAmount amtA)
136             showCoin csB tnB (US.unAmount amtB)
137
138     createPool :: UUID -> US.CreateParams -> IO ()
139     createPool cid cp = do
140         callEndpoint cid "create" cp
141         threadDelay 2_000_000
142         go
143     where
144         go = do
145             e <- getStatus cid
146             case e of
147                 Right US.Created -> putStrLn "created"
148                 Left err'       -> putStrLn $ "error: " ++ show err'
149                 -                -> go
150
151     addLiquidity :: UUID -> US.AddParams -> IO ()
152     addLiquidity cid ap = do
153         callEndpoint cid "add" ap

```

```

152     threadDelay 2_000_000
153     go
154 where
155     go = do
156         e <- getStatus cid
157         case e of
158             Right US.Added -> putStrLn "added"
159             Left err'      -> putStrLn $ "error: " ++ show err'
160             _              -> go
161
162 removeLiquidity :: UUID -> US.RemoveParams -> IO ()
163 removeLiquidity cid rp = do
164     callEndpoint cid "remove" rp
165     threadDelay 2_000_000
166     go
167 where
168     go = do
169         e <- getStatus cid
170         case e of
171             Right US.Removed -> putStrLn "removed"
172             Left err'        -> putStrLn $ "error: " ++ show err'
173             _                 -> go
174
175 closePool :: UUID -> US.CloseParams -> IO ()
176 closePool cid cp = do
177     callEndpoint cid "close" cp
178     threadDelay 2_000_000
179     go
180 where
181     go = do
182         e <- getStatus cid
183         case e of
184             Right US.Closed -> putStrLn "closed"
185             Left err'        -> putStrLn $ "error: " ++ show err'
186             _                 -> go
187
188 swap :: UUID -> US.SwapParams -> IO ()
189 swap cid sp = do
190     callEndpoint cid "swap" sp
191     threadDelay 2_000_000
192     go
193 where
194     go = do
195         e <- getStatus cid
196         case e of

```

```

197         Right US.Swapped -> putStrLn "swapped"
198         Left err'        -> putStrLn $ "error: " ++ show err'
199         _                -> go
200
201 getStatus :: UUID -> IO (Either Text US.UserContractState)
202 getStatus cid = runReq defaultHttpConfig $ do
203     liftIO $ printf "\nget request to
204                     127.0.1:9080/api/contract/instance/%s/status\n" (show cid)
205
206     w <- req
207     GET
208     (http "127.0.0.1" /: "api" /: "contract" /: "instance" /:
209       pack (show cid) /: "status")
210     NoReqBody
211     (Proxy :: Proxy (JsonResponse (ContractInstanceClientState
212                                     UniswapContracts)))
213     (port 9080)
214
215     case fromJSON $ observableState $ cicCurrentState $ responseBody w of
216     Success (Last Nothing) -> liftIO $ threadDelay 1_000_000 >>
217       getStatus cid
218     Success (Last (Just e)) -> return e
219     _                      -> liftIO $ ioError $ userError
220       "error decoding state"
221
222 callEndpoint :: ToJSON a => UUID -> String -> a -> IO ()
223 callEndpoint cid name a = handle h $ runReq defaultHttpConfig $ do
224     liftIO $ printf "\npost request to
225                     127.0.1:9080/api/contract/instance/%s/endpoint/%s\n"
226                     (show cid) name
227
228     liftIO $ printf "request body: %s\n\n" $ B8.unpack $ encode a
229
230     v <- req
231     POST
232     (http "127.0.0.1" /: "api" /: "contract" /: "instance" /:
233       pack (show cid) /: "endpoint" /: pack name)
234     (ReqBodyJson a)
235     (Proxy :: Proxy (JsonResponse ()))
236     (port 9080)
237
238     when (responseStatusCode v /= 200) $
239       liftIO $ ioError $ userError $ "error calling endpoint " ++ name
240
241     where
242       h :: HttpException -> IO ()
243       h = ioError . userError . show

```

In the main program we expect just one command line parameter, which is the number of 1 to 4 so the main program knows for which wallet it is running. Then we read the corresponding cid

file (37). And we read the `symbol.json` file to get the currency symbol of the example tokens (38). And then we check whether there was an error and if not, we invoke the `go` function to which we provide the `cid` and the currency symbol. And the `go` function is just a loop that reads in a command and depending on the command it executes various helper functions. The commands exactly correspond to the endpoints we have. So, we can carry our funds, we can look for existing pools, we can create a pool, we can add liquidity to a pool, we can remove liquidity from a pool, we can close a pool and we can swap. The `readCommandIO` functions tries to parse out a command from the user input and if it fails it just recurses again (69-73). Then there are various helper functions that convert something of type `Command` into the corresponding parameter types, like `createParams` or `addParams` from the `Uniswap` module that we looked at earlier (75-91). The `showCoinHeader` and `showCoin` functions are just to make it look a bit prettier when we query the funds or the pools. After that we have the various endpoints which all make use of the helper functions `getStatus` and `callEndpoint` (99-199). The `getStatus` function we need in order to get something back from the contracts (201-213). We use a GET request. In the end we just check the state if it is empty, which happens right in the beginning because before in anything else has told anything to the state. Then we wait a second and recurse. And if there's a state it's just `e`, then we know that this is of type *Either Text UserContractState*. If something went wrong, we end in the third case (213). With the `callEndpoint` function we just call an endpoint where we need to convert the URL to type `Text` so we use the `pack` function. In the end we check whether we get a 200-status code or not. Let's look now at the `getFunds` endpoint function (99-115). First, we use the call the endpoint »funds« and the request body for it is just `unit`. Then we wait for 2 seconds and we use the `getStatus` helper function and if we get a `Right` result, we show the funds. Else we recurse. The next endpoint function is `getPools` (117-134). It's more or less the same as the previous just instead of funds we have pools now. In the `createPool` endpoint function we again call the endpoint and wait for 2 seconds. Now something could go wrong for instance if we want to create a pool with coins that already exist in another pool or that we do not have sufficient funds. So, in the case we get an error we just log it to the console. Let's try now out this code in the console. Let's start three instances for wallets one, two, three, and try to recreate the scenario from the diagrams in the beginning.

```
$ cabal run uniswap-client -- 1
$ cabal run uniswap-client -- 2
$ cabal run uniswap-client -- 3
```

We need to perform these commands in three different shells so we can then interact with the pools for each wallet individually. We can use the »Funds« command to query our funds and we will see that we have 1 million for each of the tokens A, B, C and D and a 100.000 ADA. Let's

say now that wallet 1 will be Alice, wallet 2 will be Bob and wallet 3 will be Charlie. Alice first creates a pool with 1000 of tokens A and 2000 of tokens B. After it we query the pools.

```
Create 1000 'A' 2000 'B'  
Pools
```

The next step is that Bob swaps 100 A for Bs. And then we check how many funds Bob has.

```
Swap 100 'A' 'B'  
Funds
```

And we see the numbers from the initial schema. Next Charlie added liquidity: 400 A and 800 B. After it we check the pools to see if the funds got updated.

```
Add 400 'A' 800 'B'  
Pools
```

Now we go back to Alice. She wants to remove her entire liquidity 1415 liquidity tokens. We first query her funds and after the removal we do this again.

```
Funds  
Remove 1415 'A' 'B'  
Funds
```

The last step is that Charlie closes the pool. And then we check if any pools are present.

```
Close 'A' 'B'  
Pools
```

Finally, we want to show how to do this entire workflow without Haskell and instead use curl. So in the week10 folder you find various shell scripts with curl commands. The *status.sh* script expects one argument, that's the wallet. If we try it for wallet 1 we get back the pool state.

```
$ ./status.sh 1
```

We see the result of the last endpoint call which was for wallet 1 the Pools endpoint and at that time the AB pool was still open so we get this result. The *funds.sh* script calls the funds endpoint. Again, it takes just the wallet parameter.

```
$ ./funds.sh 1
```

If we call afterwards the status script again, we see the result changes to the funds call. We can now use the *create.sh* script to open up again a pool with tokens A and B for wallet 1.

```
$ ./create.sh 1 1000 A 2000 B
```

We can call now the pools script and then again, the status script to get the pool status.

```
$ ./pools.sh 1  
$ ./status.sh 1
```

And that is it. Basically you can find the GET and POST HTTP commands that we use in these curl scripts also in the helper functions *getStatus* and *callEndpoint* from the *uniswap-client.hs* file we looked at before.

10 Resources

[1] The Plutus Pioneer Program git repository

<https://github.com/input-output-hk/plutus-pioneer-program>

[2] The Cardano documentation

<https://docs.cardano.org/plutus/learn-about-plutus>

<https://docs.cardano.org/plutus/eutxo-explainer>

[3] The Extended UTXO Ledger Model paper

<https://hydra.iohk.io/build/12982960/download/1/extended-utxo-specification.pdf>

[4] The Plutus apps repository

<https://github.com/input-output-hk/plutus-apps>

[5] Plutus application backend blog post

<https://iohk.io/en/blog/posts/2021/10/28/plutus-application-backend-pab-supporting-dapp-development-on-cardano/>